Accelerating the Inference Era with AI-Driven, Globally Optimized HW/SW Co-Design

Miria Feng
Electrical Engineering
Stanford University
miria0@stanford.edu

Fangzhao Zhang
Electrical Engineering
Stanford University
zfzhao@stanford.edu

Adrian Lafuente
Computer Science
Stanford University
agamarra@stanford.edu

Mert Pilanci
Electrical Engineering
Stanford University
pilanci@stanford.edu

Azalia Mirhoseini
Computer Science
Stanford University
azalia@stanford.edu

Abstract

Deep learning inference demand is projected to grow $10{,}000\times$ over the next five years, a trajectory that general-purpose accelerators cannot match. Custom accelerators offer a viable path forward, but designing them requires simultaneous co-optimization of hardware datapaths, software schedules, and compiler transforms across a combinatorial search space exceeding $\mathcal{O}(10^{2300})$. While co-design is becoming increasingly essential, existing methodologies rely on decoupled, sequential pipelines that miss critical cross-stage trade-offs. We introduce CHEETAH: a globally optimized, AI-driven framework for inference accelerator co-design. To our knowledge, this is the first co-design framework to combine *joint* tiling-fusion, dynamic placement, and rematerialization in a single LP-based formulation. Key contributions include: (1) time-indexed variables for dynamic memory management, (2) rematerialization variables to enable cheap recomputation over costly DRAM reloads, and (3) joint tiling-fusion optimization via McCormick linearization. The hardware is not a fixed context that a schedule is solved against: MAC count, on-chip memory, on-package capacity, clock frequency, and memory technology are decision variables of the same linear program, and the cost coefficients are functions of them, so compute time is charged against the reciprocal of the MAC count, memory time against the reciprocal of the delivered bandwidth, and die area and power enter as rows over those same variables. Sizing stays continuous where the physics is continuous, while bandwidth and the discrete menus (precision, sparsity, memory technology, array aspect) are carried as one-hot selections whose coefficients are disaggregated exactly at integral selections, which keeps the program linear on each branch and avoids the relaxation leak that a raw bytes-over-bandwidth product introduces. This inner-loop precision is steered by an LLM-guided outer loop that performs causal interventions based on specific deployment profiles and Lagrangian sensitivity signals. The same formulation extends to the memory-bound decode regime of LLM serving, co-sizing on-chip memory against a bounded, quantized KV cache to shrink the decode memory wall. In less than 6 hours, CHEETAH discovers designs on the Pareto frontier achieving an average of $\sim 22.07\times$ raw speedup across 11 workloads, reaching up to $\sim 40.1\times$ speedup on a single workload over a TPU-v3 baseline, unlocking efficiency inaccessible to decoupled sequential pipelines.

40th Conference on Neural Information Processing Systems (NeurIPS 2026).

1 Introduction

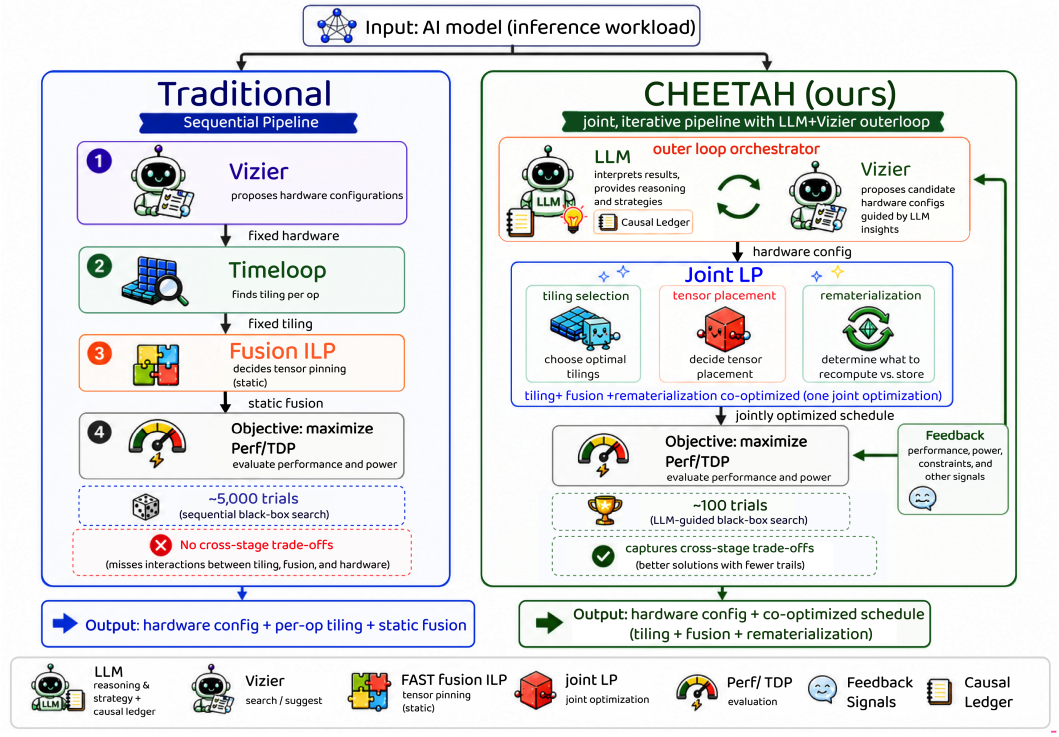


Figure 1: Sequential vs Our Unified Optimization Paradigm. (Left) Baseline methods decouple hardware search, operation-level tiling, and static fusion; precluding cross-stage trade-offs. (Right) CHEETAH presents a unified LLM-driven LP-solver-based framework that jointly optimizes tiling, placement, and rematerialization while learning from a causal ledger.

The AI inference demand is expected to grow by $10,000\times$ over the next 5 years [Computing Research Association, 2026]. General-purpose accelerators leave significant efficiency gains unrealized, which motivates the design of targeted custom accelerators. Co-design, which jointly optimizes hardware-software (HW/SW), and scheduling for target workloads, offers a path to unlock performance efficiency for the upcoming inference era. However, the prevailing approach relies on sequential pipelines of heuristic tools, a slow, human-driven process navigating a combinatorial search space exceeding $\mathcal{O}(10^{2300})$ [Zhang et al., 2022]. State-of-the-art frameworks like FAST [Zhang et al., 2022] improve on this through an automated flow (a hardware proposer feeds an operation-level tiling agent, which feeds a downstream fusion solver), but remain fundamentally decoupled: a tiling that slightly reduces compute efficiency may shrink the working set enough to eliminate a DRAM bottleneck, yet sequential pipelines cannot discover such cross-stage trade-offs.

Unifying the design space into a single convex relaxation yields previously infeasible global optima. Specifically, we introduce CHEETAH: a custom inference accelerator design framework that unlocks globally optimized designs with a dual-loop. Our inner-loop is a novel large-scale Linear Program (LP) to precisely mathematically model strict design decisions and constraints that must not be violated. The LP captures trade-offs for targeted workloads through time-indexed placement and rematerialization, thus allowing tensors to be dynamically pinned, evicted, recomputed across inference timesteps. We further extend our reformulation to internalize tiling selection into the LP, by introducing Pareto-pruned tiling variables and the McCormick linearization (a convex relaxation used for optimization of bilinear problems [McCormick, 1976]). By exploiting the structural invariance of our matrices across hardware trials, we also adapt a GPU-accelerated primal-dual solver in JAX [Bradbury et al., 2018] that can resolve millions of variables to rapidly reduce trial time from days to hours. This allows the LP solver to jointly reason about hardware configuration, activation placement, and weight pinning in a single pass, while providing feedback signal via Lagrangian duals.

One program, many architectures. A natural question is how a single program can co-design accelerators for workloads as different as EfficientNet, the Llama family, DeepSeek-V3's mixture-of-experts, and the Stable Diffusion UNet. The LP is *not* per-architecture: it is defined over a generic graph intermediate representation, a directed acyclic graph of operations, each carrying a Timeloop-derived tile-configuration Pareto set and a roofline, connected by edges with residency. A depthwise convolution, a dense GEMM, an attention matmul, and an expert feed-forward are all simply "operations with shapes"; the architecture-specific content lives entirely in the coefficients (operation shapes, Pareto data, and edge topology), never in the structure of the program. Architecture-specific decisions (a KV-cache budget, mixture-of-experts expert-residency, diffusion cross-step reuse) are optional constraint families that switch on only when the relevant operation type is present and are inert otherwise. One program therefore covers every architecture the way a single register allocator compiles every source language: it operates on the graph, not the workload.

Our outer-loop framework stacks a black-box optimizer (Google Vizier [Golovin et al., 2017]) to propose candidate hardware configurations defined by a set of hyperparameters (e.g., processing element grid dimensions, systolic array sizes, DRAM size, etc), while a LLM is used for efficient search-space steering. The LLM acts as a causal architect, interpreting a compact trial ledger of roofline checks and LP feedback diagnostics to propose constrained architectural search input for Vizier. This prevents the search-space gaming typical of black-box optimizers in wide design regions, and reduces the need for thousands of sequential trials via Bayesian search strategies in Vizier to learn good vs bad neighborhoods in search topology. Our results demonstrate that the LLM Architect framework further reduces invalid hardware trials from 24.5% under vanilla Vizier to 5.9%, to reach the Pareto frontier faster than the next strongest baseline. We validate all results with Model Flop Utilization (MFU) and roofline lower-bound audits, ensuring predicted latencies are physically grounded in compute and memory throughput limits. Figure 1 outlines the comparative difference between the standard sequential workflow and our joint optimization paradigm.

Contributions. We reformulate the fundamental HW/SW co-design problem into a unified framework in CHEETAH, combining principled mathematical rigor of the LP formulation to solve strict design decisions optimally, with the powerful aggregation and causal learning of an LLM Architect outer loop for meta-optimization. Our open-source repository for full reproducibility is at https://anonymous.4open.science/r/CHEETAH-B268/.

1. **CHEETAH-V0 - Dynamic Placement with Rematerialization:** We introduce time-indexed LP variables $p^k_{i,t}$ and rematerialization variables $r_{i,t}$ to enable cheap recompute operations instead of paying DRAM reload costs, supporting skip connections and multi-fanout nodes.

2. **CHEETAH-V1 and V3 - Joint Tiling-Fusion via Classical Convex Modeling:** We unify tiling and fusion into a single LP problem by introducing tiling selection variables $y_{i,c}$, and employ **McCormick envelopes** to exactly linearize the bilinear interaction between tiling and tensor pinning; V3 then promotes the chip's MAC count and bandwidth to decisions and replaces the aggregate roofline with a **disaggregated perspective epigraph** that is the exact convex hull of the per-tile roofline disjunction. Both are classical tools of mathematical programming (McCormick's 1976 envelopes and epigraph/perspective reformulation) that the accelerator co-design literature does not use, because its pipelines are built from simulators and heuristics rather than from programs. Applied here they buy two things at once: a parameter-free relaxation that is exact at every integer vertex, so the LP is simultaneously an optimizer and a valid lower bound, and a coupling that internalizes the tiling-placement trade-off to discover design points that sequential pipelines systematically miss.

3. **VM-One-Armed-Rescue Solver:** The McCormick linearization that makes V1 exact also *blows up* the constraint matrix (for DeepSeek-V3, $\sim$2.5M variables, $\sim$3.0M constraints, and $\sim$6.1M nonzeros) and leaves it severely ill-conditioned, with coefficients spanning $\sim$10 orders of magnitude. Stock first-order GPU solvers (cuPDLP) and CPU simplex/barrier (HiGHS) stall or diverge, with the primal or dual side lagging. We introduce **VM-One-Armed-Rescue**, a matrix-free JAX solver that augments restarted PDHG with a diagonal *variable-metric* rescaling and a one-sided "rescue" restart, resolving these LPs to dual convergence in seconds. To our knowledge it is the first solver to make the McCormick-

linearized joint tiling–fusion–placement co-design LP tractable at LLM scale: the enabling step that collapses a days-long search into hours (Section 6).

4. **LLM-guided Causal Architect:** The LLM outer loop steers hardware search by interpreting *converged Lagrangian dual signals* from the inner LP together with a Causal Ledger of past trials to perform targeted subspace interventions, transforming black-box search into a structured, diagnostic-driven process. To our knowledge, CHEETAH is the **first** to steer an LLM-guided architecture search with the converged Lagrangian dual variables of a co-design LP: the duals *indicate* where to search while the LP retains every optimality certificate; the LLM proposes, the solver never loses rigor. The Architect guides Vizier through widened search spaces, reducing invalid trials by $\sim 4\times$ ($24.5\% \rightarrow 5.9\%$) while preserving the global optimal score, and enables profile-specific designs (speech models may emphasize latency, while data-synthesis models may prioritize bandwidth).
5. **Certificates, not counters:** Every CHEETAH schedule ships with an LP optimality certificate: the converged dual bound against which the realized schedule is measured, with the relaxation gap reported per workload and the integral optimum certified exactly on chain-structured models. A cycle-accurate RTL replay then confirms, decision by decision (tile choice, residency, eviction, rematerialization), that the plan executed as planned with byte-exact memory occupancy. This is a stronger guarantee than the "did the optimization fire" checks that agentic kernel optimizers use to validate hardware-aware code [Tschand et al., 2026]: a counter confirms a feature was used, a certificate bounds how far the result can be from optimal.
6. **Speedup:** Our method demonstrates an average speedup of $\sim 22.07\times$ over the baseline across a wide range of 12 workloads (11 vision and language models plus the Stable Diffusion 1.5 UNet), reaching up to $\sim 40.1\times$ on a single workload, as shown in Figure 2 below.

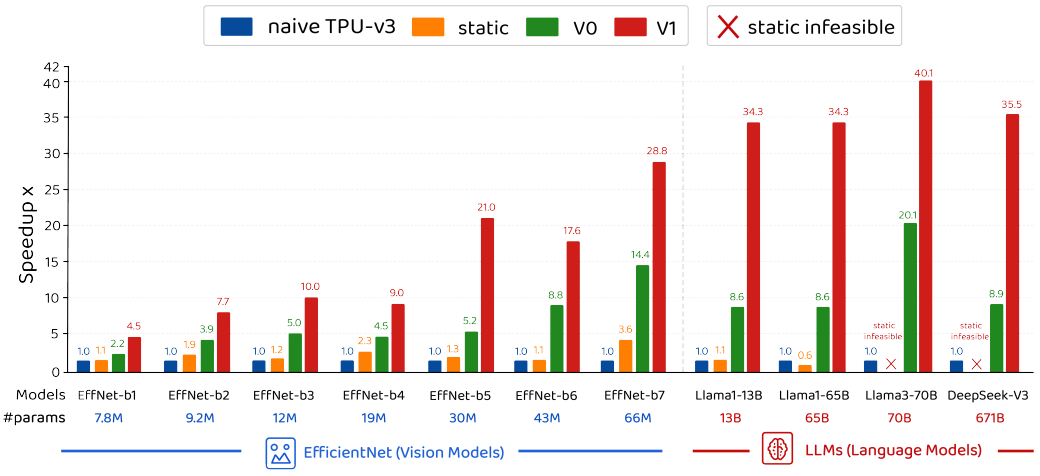


Figure 2: Speedup over modeled TPU-v3 baseline for each workload and formulation: CHEETAH-V1's joint tiling-fusion optimization achieves up to $\sim 40.1\times$ acceleration, while static fusion is infeasible on large-scale LLMs. Progressive gains are visible across formulations: V0's dynamic placement improves over static by 2–20×, while V1's joint tiling-fusion adds another 2–4× on top. See Section 7 for full analysis and wall-clock time breakdowns.

CHEETAH reframes accelerator co-design from a sequential pipeline of heuristics into a single certified optimization. Its inner LP provably *contains* the map-then-fuse pipelines of prior work (Theorem 1) and returns a certified lower bound on schedule quality, so the field's usual "$N\times$ over a baseline" becomes "within a certified gap of the optimum"; its outer loop couples this rigor with an LLM that reads the LP's own Lagrangian duals to steer the hardware search. The consequences are concrete: a $\sim 22\times$ average (up to $\sim 40\times$) speedup over a modeled TPU-v3 across 12 vision, language, and diffusion workloads, every design validated by cycle-accurate RTL, six physical-feasibility gates, and roofline lower-bound audits, and a co-design loop that runs in *hours* rather than months. The

human's role shifts accordingly: they write constraints (area, power, capacity, and the cost model), not designs, and a new chip or a new workload enters the same program as new columns and rows rather than as a new search. We view this as a step-change in *how* accelerators are designed: not a faster point inside an existing search, but a reformulation that makes the global optimum (and a proof of how close a design sits to it) attainable for the inference era.

Beyond prefill: shrinking the decode memory wall. The speedups above are largest in the compute-bound *prefill* phase, where scheduling has the most room. Realistic LLM *serving*, however, is dominated by autoregressive *decode*, where generating each token reloads the full weights and a growing KV cache from DRAM: the bottleneck shifts from compute to memory, and on fixed hardware no schedule can move it because the bytes are schedule-invariant. CHEETAH's co-design attacks this wall directly and jointly, by *sizing on-chip global memory against a bounded, quantized KV cache* so that the cache is co-designed to fit on chip and its per-token read never touches DRAM. The structural insight the program exploits is that on-chip memory should hold the KV cache, not the weights: model weights (13–70 GB) cannot fit and must stream, but the KV cache can be bounded by eviction and shrunk by quantization until it fits, and it is re-read every decode step, so every byte of on-chip memory spent on it pays off once per generated token. Sizing the global buffer, the eviction budget, and the KV precision in one program is what shrinks the decode memory wall.

Why the decode wall does not dissolve under load: sparse routing defeats batching. For a dense model the decode memory wall is, in principle, a low-batch problem: a larger serving batch amortizes each weight load perfectly, because the same weights serve every token in the batch, so at high enough throughput the wall recedes on its own. A sparse mixture of experts breaks this argument, and this is what makes the co-design necessary rather than merely convenient. Under top-k routing the set of experts a decode step must load is the *union* of the experts its tokens select, so a larger batch touches *more* experts: the expected number of distinct experts follows $n(1-(1-k/n)^B)$, which for DeepSeek-V3 ($n=256$, $k=8$) rises from 8 experts at $B=1$ to 163 at $B=32$ and 252 at $B=128$. The reload therefore *grows* with batch, from 19 to 598 GB per step across that range, and although the per-token cost does fall, it falls only from 19 to 4.7 GB, a $4\times$ amortization bought with a $128\times$ increase in batch, and it saturates once essentially every expert is swept. The wall is thus a structural property of sparse serving rather than an artifact of small batches, and we confirm the consequence empirically in Section 7.5: the speculative width the program selects rises rather than collapses as batch grows, doubling with every fourfold increase in batch, and the certified serving advantage rises monotonically from $33.1\times$ at $B=1$ to $48.6\times$ at $B=128$. Two mechanisms that are usually described as competing amortizers, batching and speculation, are complementary here, precisely because batching amortizes so poorly under sparse routing.

Why this program is hard to solve. The difficulty is numerical rather than conceptual, and it is worth stating concretely before the formulation. A single captured DeepSeek-V3 MoE request expands to 15,826,069 logical invocations. Written over the event axis, the master program for that workload reaches 4.92×10^8 columns and 8.93×10^8 rows, which we never form: it is carried as 113,552 row recipes over 50,106 axes in ten row families, and a candidate solution is checked against the original rows without materializing the matrix. Even the far smaller per-workload tiling instance is badly scaled rather than merely large. For DeepSeek-V3 it has 111,070 columns, 1,527,769 nonzeros at a density of 3.0×10^{-5}, 10,079 equality rows, and a maximum row degree of 481 against a maximum column degree of 141—a few shared hardware columns touching nearly every row. Its coefficients span 1.25×10^{-10} to 5.58×10^7, a dynamic range of 4.5×10^{17} within one matrix, because the same rows carry $0/1$ structural incidences, byte counts, and reciprocal hardware rates, and because the reciprocal envelopes and product relaxations that linearize rate $\times$ traffic contribute nearly parallel constraints. A problem of this shape defeats off-the-shelf solvers: the scaling degrades the factorizations an interior-point method depends on, and assembling the constraint matrix is impossible at the full event horizon. We therefore solve it with a custom variant of OCS, an operator-splitting solver that touches the constraint operator only through matrix–vector products and never assembles it, and we certify the recovered point by exact rational verification against the original row families.

We present the work as a progression, so that each formulation's contribution can be read on its own terms. Section 2 positions CHEETAH against prior mapping, co-design, and compilation systems (Table 1). Section 5 builds the formulation in rungs: CHEETAH-V0 (Section 5.1) adds time-indexed placement and rematerialization to the static fusion ILP; V1 (Section 5.2) brings tiling inside the

program, using Pareto pruning to compress the raw tile space and McCormick envelopes to flatten the resulting bilinear coupling into a linear program; V2 adds memory sizing and tile-dependent residency; and V3 (Section 5.2) promotes the hardware itself to a decision under a disaggregated perspective epigraph, with the solver (Section 6) and the LLM-guided outer loop closing the loop. Section 7 reports the progressive gains across these rungs together with the same-silicon fixed-hardware study, each result grounded by roofline, MFU, and manufacturability validators; Section 9 replays the schedules on cycle-accurate RTL; Section 8 reads the outcome in FAST's own cost model; and the appendices carry the containment theory, the solver, and the deficit and sensitivity analyses.

2 Related Work

Hardware and software design exploration has been studied through tools that optimize datapath configurations and operation mappings, including Eyeriss [Chen et al., 2019], Timeloop [Parashar et al., 2019], MAESTRO [Kwon et al., 2020], Interstellar [Yang et al., 2020], CoSA [Huang et al., 2021], and ZigZag [Mei et al., 2021]. For LLM serving specifically, LLMCompass [Zhang et al., 2024] provides a fast analytical hardware-evaluation framework for design-space exploration, but treats mapping and scheduling as enumeration over a hardware template rather than as decisions inside an optimization program. FAST [Zhang et al., 2022] advanced the field with a full-stack search over datapath, schedule, and compiler passes, achieving $3.7\times$ Perf/TDP over TPU-v3, but its sequential pipeline (Vizier$\rightarrow$Timeloop$\rightarrow$ILP) requires $\sim$5,000 trials and precludes cross-stage trade-offs. Recent work has begun addressing joint tiling-fusion: TileFlow [Zheng et al., 2023b] explores fusion dataflows via tree-based analysis, and FADiff [Jia et al., 2025] formulates a differentiable joint mapping-fusion objective. Our formulation differs in targeting HW/SW co-design with a large-scale LP that provides valid lower bounds via tight McCormick relaxation exact at integral tiling/placement decisions, and additionally handles rematerialization and dynamic memory scheduling. Polyhedral compilation [Feautrier, 1992, Bondhugula et al., 2008, Baghdadi et al., 2019] reasons jointly about tiling and fusion as affine loop transformations for fixed hardware, and fusion-conflict-graph methods [Acharya et al., 2020] cast fusion partitioning as graph optimization; none model time-varying buffer residency, eviction, or rematerialization as scheduling decisions, and hardware parameters are never decision variables. Conversely, execution *reordering*, the core strength of the polyhedral model, is the one axis we fix: the topological order is an input (Definition 1), and making order a decision above the LP is future work. Our joint LP is enabled by McCormick envelopes [McCormick, 1976], and scalable first-order LP solvers descended from PDLP [Applegate et al., 2022]. For outer-loop search, we draw on LLMs as informed mutation operators such as FunSearch [Romera-Paredes et al., 2024], AlphaEvolve [Novikov et al., 2025], and EvoPrompting [Chen et al., 2023], to propose hardware configurations guided by workload bottleneck analysis. A full discussion appears in Appendix A.

3 Preliminaries

This section provides background and defines notation. Section 3.1 outlines the HW/SW co-design problem, and Section 3.2 introduces the setting for linear programs and their large-scale solvers.

3.1 The Sequential Co-Design Pipeline

The standard pipeline iterates for a minimum of ~ 5000 times through three sequential stages:

Stage 1: Hardware proposal. Vizier proposes a hardware configuration $\mathbf{h}$ consisting of 16 hyperparameters: PE grid dimensions, systolic array sizes, vector unit width, buffer configurations at L1/L2/Global levels, DRAM channels, and batch size. We defer a full picture of how Vizier's parameters specify a general accelerator chip template to Figure 11 in the appendix.

Stage 2: Schedule optimization. For each operation i in the neural network's XLA HLO graph, Timeloop finds the best tiling and mapping onto the hardware defined by $\mathbf{h}$. This produces execution time estimates $T_i^{\max}$ (all tensors streamed from DRAM) and $T_i^{\min}$ (all tensors on-chip), as well as per-tensor DRAM access costs t_i^k for $k \in \{I, O, W\}$ (input activations, output activations, weights).

Stage 3: Fusion ILP. Given fixed per-operation timing from Stage 2, an ILP decides which tensors to pin in the Global Memory to minimize total execution time. The ILP uses binary placement variables $p_i^k \in \{0, 1\}$ indicating whether tensor k of layer i resides in Global Memory.

The result (predicted Perf/TDP) is returned to Vizier, which proposes the next $\mathbf{h}$. This repeats for $\sim$5,000 trials. See Appendix F.2 for details.

Limitations and sequential information loss. Each stage commits to decisions that are irreversible by downstream stages. Timeloop optimizes each operation's tiling to minimize that operation's execution time *in isolation*, without considering how tiling affects the feasibility or quality of fusion decisions. The fusion ILP receives fixed working-set sizes and cannot request alternative tiling configurations that might enable better global solutions. This sequential decomposition systematically misses trade-offs of the form: *accept a tiling that reduces compute efficiency by δ but shrinks the working set enough to pin an additional tensor, removing a DRAM bottleneck worth $\gg \delta$.*

3.2 Linear Programs and Solvers

Standard LPs cannot directly express the nonlinear interaction between tiling, fusion, and memory. We address this via McCormick envelopes [McCormick, 1976], which provide a tight convex relaxation of bilinear terms, ensuring that the gap between the continuous LP solution and the final rounded hardware specification remains minimal. Appendix C.1 provides further details.

3.3 Capability comparison with prior co-design and mapping systems

Table 1 positions CHEETAH against ten prior accelerator co-design, mapping, and compilation systems along the decision decisions that a joint co-design formulation can expose. Each cell is filled from the system's own paper, and marked conservatively: ✓ for a decision the system searches or optimizes, ○ for one it supports only in a restricted or staged form, and × for one it does not model.

Read down the columns, prior systems each own a slice of the problem. Per-op mappers (Timeloop, ZigZag, CoSA) search tilings but hold nothing across layers; CoSA is a per-layer tiling MIP whose optimality certificate is confined to that per-layer subspace. Fusion-aware systems (DeFiNES, Fast-and-Fusiest) add cross-layer fusion, and the Fast-and-Fusiest mapper is provably optimal over a comprehensive fused mapspace, but only for fixed hardware, with no hardware co-search, no rematerialization, and no time-indexed eviction. The co-design systems (DOSA, Gemini, CODEBench, FAST) reach the hardware knobs, yet each drops a scheduling decision: DOSA and Gemini do not model operator fusion; CODEBench cannot search dataflow at all, fixing an output-stationary mapping; and FAST runs as a sequential pipeline (Vizier $\rightarrow$ Timeloop $\rightarrow$ fusion ILP) that takes the per-operator execution order $o(i)$ as fixed input from Timeloop and returns a heuristic Perf/TDP with no certificate. Crucially, *none* of the ten carries a certified schedule-quality bound over the joint space: the subspace guarantees of CoSA and Fast-and-Fusiest do not bound the joint hardware + tiling + fusion + placement problem, and the remaining systems are heuristic or local. CHEETAH is the only system that unifies all of these decisions (per-op tiling, fusion, time-indexed placement and eviction, rematerialization, dataflow search, and hardware sizing; execution order is the one decision it takes as input) in a single joint program whose LP relaxation is a certified lower bound on latency (tight to the exact integral optimum on chain-structured workloads). It is, by this accounting, the most comprehensive formulation of the accelerator co-design problem to date.

4 What the program contains and how it is scored

4.1 Evaluating a design without a proposal loop

Vizier proposed hardware configurations; the performance model and execution evaluator determined their scores. Removing the proposer does not remove that evaluator. The formulation minimizes modeled latency under hardware constraints, including area and power budgets, with compute tiles, bandwidth, SRAM and vector lanes as shared LP variables; discrete physical choices still require a supported hardware menu and a recovery step. For each recovered chip and schedule we report the quantities in Table 2.

Table 1: Capability matrix over prior accelerator co-design, mapping, and compilation systems. Columns are grouped into the schedule decisions a joint program can expose, the search it performs, and the guarantee it provides. ✓ = searched/optimized, ○ = partial or restricted (see notes), × = not modeled. CHEETAH carries every decision except execution order, which it takes as input (note f). Systems: Timeloop [Parashar et al., 2019], CoSA [Huang et al., 2021], DeFiNES [Mei et al., 2023], Fast-and-Fusiest [Andrulis et al., 2026], ZigZag [Mei et al., 2021], DOSA [Hong et al., 2023], Gemini [Cai et al., 2024], CODEBench [Tuli et al., 2023], FAST [Zhang et al., 2022], Polyhedral compilers (Pluto, Tiramisu) [Bondhugula et al., 2008, Baghdadi et al., 2019].

	Schedule decisions					**Search**		**Guarantee**	
System	Per-op tiling	Operator fusion	Time-indexed placement	Rematerialization	Exec. order as variable	Dataflow / mapping	Hardware sizing	Certified bound	Single joint program
Per-op mappers									
Timeloop	✓	×	×	×	×	✓	×	×	×
CoSA	✓	×	×	×	×	✓	×	○ [a]	×
Fusion-aware mappers									
DeFiNES	✓	✓	○ [b]	×	×	✓	×	×	×
Fast-and-Fusiest	✓	✓	○ [b]	×	×	✓	×	○ [a]	×
Hardware design-space exploration									
ZigZag	✓	×	×	×	×	✓	✓	×	○ [c]
DOSA	✓	×	×	×	×	✓	✓	×	✓ [d]
Gemini	✓	×	×	×	×	✓	✓	×	○ [c]
CODEBench	○ [e]	×	×	×	×	×	✓	×	○ [c]
FAST	✓	✓	○ [b]	×	×	✓	✓	×	×
Loop compilers									
Polyhedral (Pluto, Tiramisu)	✓	✓	×	×	✓	×	×	×	○ [g]
CHEETAH (ours)	✓	✓	✓	✓	○ [f]	✓	✓	✓	✓

Notes. [a] Optimal only within a restricted subspace (CoSA: a per-layer tiling MIP; Fast-and-Fusiest: an optimal mapping within a fused mapspace at fixed hardware), not a bound over the joint hardware + schedule problem. [b] Static or fusion-induced residency only (FAST pins tensors in Global Memory over consecutively-executed operators, and DeFiNES / Fast-and-Fusiest hold intermediates on-chip within a fused group), but eviction timing is not an independent, time-indexed decision variable. [c] Joint over its own space via staged or surrogate search (ZigZag design-space exploration; Gemini simulated annealing; CODEBench active learning) rather than a single mathematical program. [d] DOSA co-optimizes hardware and mapping in one differentiable "one-loop" search, but returns a local optimum with no certificate. [e] CODEBench sizes accelerator resources (PEs, buffers) under a fixed output-stationary dataflow; tiling is not an explicit search variable and dataflow is not searched. [f] The topological execution order is an input to the LP, not a decision variable; time-indexed residency, eviction, and rematerialization are optimized exactly for any given order, and order search is a decision above the LP (Section 2). [g] Affine scheduling is itself a linear or integer program (Feautrier; Pluto), but over loop transformations for fixed hardware, with no residency, eviction, rematerialization, or hardware variables.

Table 2: Quantities evaluated for each recovered chip and schedule.

Metric	**Evaluation**
Prefill	Prompt execution latency and time to first output
Decode	Remaining generation latency and time per output token
Total	Actual full-task makespan
Throughput	Completed output tokens divided by elapsed time
Perf/TDP	Throughput divided by the chip's TDP
Speedup	Matched baseline latency divided by candidate latency

For a fixed amount of work these compose exactly,

$$\frac{(\mathrm{Perf}/\mathrm{TDP})_{\mathrm{design}}}{(\mathrm{Perf}/\mathrm{TDP})_{\mathrm{baseline}}} = \frac{T_{\mathrm{baseline}}}{T_{\mathrm{design}}} \frac{\mathrm{TDP}_{\mathrm{baseline}}}{\mathrm{TDP}_{\mathrm{design}}}, \tag{1}$$

so a latency objective does not by itself maximize the ratio. We obtain a performance/power tradeoff by solving across power budgets or hardware branches and comparing the recovered designs, rather than by optimizing Eq. (1) directly.

Three limits apply to every number reported this way. These are analytical execution results: physical timing and power require calibration against implementations, simulation or measurement. TDP is not average consumed power, so energy per inference needs a separate power calculation. And the

Table 3: Formulations represented in the master program. Rows marked *extension* are implemented and tested but not activated in the serial-policy experiment.

Feature	What it represents
Original paper V1	Supplied complete configuration, placement, retention and rematerialization programs, preserving their original costs
Independent mapping and tiling	Separate implementation choices for logical invocations, including different decode positions
Buffering/dataflow modes	Choices supplied by each qualified implementation menu
Batching	Compatible operations may share a batch implementation while retaining their individual identities
Fusion	Supported fusion groups and fused kernel atoms, including SwiGLU variants
Temporal scheduling	Start, work, finish and active state over event labels
Dependency enforcement	Producer completion, operand readiness and control dependencies
Routing causality	Expert work becomes available only after the relevant router dependency
Time-dependent placement	Host, DRAM and SRAM presence throughout execution
Retention and eviction	Keep useful data between consumers or release its storage
Explicit transfers	Charged host↔DRAM and DRAM↔SRAM copies with source and destination obligations
Weight residency/streaming	Retained weights versus supported streamed-weight implementations
Streamed inputs/outputs	Qualified tile staging instead of requiring every operand/output to be fully resident
KV across timesteps	KV production, persistence, subsequent reads, growth and declared terminal backing
KV spill/restore	Host backing, reservations, transfer completion and readiness *(extension)*
Rematerialization	Recompute declared deterministic values, charging inputs, execution and temporary storage
Typed representations/conversions	Distinct stored/runtime representations, conversion work, destination allocation and readiness *(extension)*
Capacity accounting	Live tensors, incoming copies, workspace, output reservations and conversion temporaries
Shared resource accounting	Compute, vector, local-memory, bandwidth and host-link service
Shared hardware co-design	One common hardware allocation for the execution
Hardware constraints	Bounds plus area, peak-power and power-density proxies
Linear relaxations	Reciprocal envelopes and product relaxations connecting hardware rates to service time

accepted prefill result remains our reference point — new total-inference claims require a matched workload and numerical contract, and decode's share must come out of the evaluated schedule rather than be imposed as a target fraction.

4.2 Features represented in the master

One implementation distinction matters for reading the results that follow. The master architecture carries several branches and extensions, and the serial-policy solver experiment of Section 7 does not activate all of them together. Table 3 is an inventory of the available formulations, not a claim that every row is exercised in every reported solve.

The program-sheet inventory and the conversion and spill/restore extension records document each row; Table 6 gives the row families that realize them.

5 Methodology

We present the **CHEETAH** (**C**o-designed **H**ardware **E**xploration via **E**fficient **T**hroughput and **H**ybrid-search) framework in three parts: Section 5.1 introduces CHEETAH-V0, which extends the static fusion ILP with time-indexed placement and rematerialization. Section 5.2 presents CHEETAH-V1, which jointly optimizes tiling and fusion via McCormick linearization. Section 5.4 describes the LLM-Architect outer loop. Appendix B provides theoretical analysis on the exactness of our mathematical modeling, and Appendix B.3 provides details on structured constraint matrices, and Appendix G presents the LLM Architect outer loop design details.

What the program optimizes over: static structure, dynamic dimensions. A single model's compute graph is *structurally static*: the layers and dependencies of, say, Llama-70B do not change from request to request. What varies at inference is the *data and the dimensions*: sequence length, batch size, mixture-of-experts routing, and the number of decode steps. CHEETAH is therefore a *design-time* program, solved offline over the static structure at a representative operating point (context, batch, generation length) together with workload statistics (per-expert routing frequency f_e, the prefix cache-hit profile). The chip is designed once for the operating envelope, not re-solved per request. Data-dependent routing enters as a distribution: the expected number of distinct experts touched at the deployed batch (a balls-in-bins occupancy model), with load-balancing and token-drop caps bounding the tail. When a deployment spans a wide range of shapes, the same program is solved over a distribution of operating points (a few weighted $(\text{context}, \text{batch}, \text{routing})$ samples) to yield a design robust across the dynamic range rather than overfit to one point; such a stochastic objective is

the weighted sum $\sum_k w_k T^{(k)}$ over the samples, and the robust one the epigraph $T \geq T^{(k)}$ for every sample; both are linear rows of the same LP. Per-request scheduling of one realized dynamic graph is a separate problem (a fixed-hardware compiler), not the co-design program studied here.

5.1 CHEETAH-V0: Multi-Period Placement with Rematerialization

Our first extension models the neural network's (NN) execution as a sequence of T discrete timesteps (one per operation in execution order). This decides at which time, which tensor is pinned, evicted, or recomputed to optimally minimize cycle counts. The key features are:

Time-indexed placement. Instead of a single binary p_i^k per tensor, we introduce $p_{i,t}^k \in [0,1]$ for each tensor k of layer i at each timestep t. A tensor can be pinned at timestep $t = 5$, and evicted at $t = 10$. This models a dynamic memory manager within a single inference pass.

Rematerialization. Rather than always paying DRAM bandwidth to reload an evicted tensor, we introduce variables $r_{i,t} \in [0,1]$ indicating that layer i is recomputed at timestep t to restore its output to the Global Buffer. This is particularly valuable for models such as EfficientNet, where depthwise convolutions account for only $\sim$5% of total FLOPs but produce large activation tensors. Recomputing a depthwise layer to avoid re-reading 6 MiB from DRAM at 256 GB/s can be faster than the DRAM reload itself.

Relaxed skip-connection handling. CHEETAH-V0 lifts the baseline's restriction to consecutive operations by using edge-indexed placement variables $p_{e,t}$ for each edge $e \in \mathcal{E}$, correctly modeling multi-fanout skip connections where consumers independently benefit from on-chip data.

The CHEETAH-V0 LP formulation is:

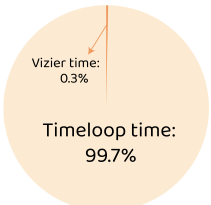
$$
\begin{aligned}
\min \quad & \sum_{i \in \mathcal{N}} T_i + \sum_{i \in \mathcal{N}} \sum_{t \in \mathcal{T}} c_i^{\text{remat}}\, r_{i,t} \\
\text{s.t.} \quad & T_i \geq T_i^{\max} - \sum_{k \in \{I,O,W\}} t_i^k \cdot p_{i,o(i)}^k && \forall\, i \in \mathcal{N} \\
& T_i \geq T_i^{\min} && \forall\, i \in \mathcal{N} \\
& \sum_{i \in \mathcal{N}} \sum_{k} d_i^k\, p_{i,t}^k \leq C_{\text{GM}} && \forall\, t \in \mathcal{T} \\
& p_{i,t}^k \leq p_{i,t-1}^k + r_{i,t} && \forall\, i,t,k \in \{I,O\} \\
& p_{i,t}^W \leq p_{i,t-1}^W && \forall\, i,t \\
& r_{i,t} = 0 && \forall\, i,t \leq o(i) \\
& 0 \leq p_{i,t}^k, r_{i,t} \leq 1
\end{aligned}
\tag{V0}
$$

where $o(i)$ is the execution order of operation i, d_i^k is the byte size of tensor k for layer i, and c_i^{remat} is the recomputation cost.

Variable and constraint digest. The objective minimizes total execution time $\sum_i T_i$ plus rematerialization costs. Per-operation latency T_i is lower-bounded by the on-chip floor $T_i^{\min}$ and the off-chip time $T_i^{\max}$ adjusted for DRAM savings from pinned tensors. Persistence constraints enforce monotone eviction for weights and allow recomputation-based restoration for activations; rematerialization is only permitted post-execution and requires on-chip availability of both inputs and weights. For DeepSeek-V3 ($L \approx 6{,}300$), this formulation yields a large-scale program with approximately 3.3M variables and 3.7M constraints. See Appendix 3.2 for a detailed breakdown.

5.2 CHEETAH-V1: Joint Tiling, Fusion, and Hardware Sizing Inside the Program

Despite V0's improvements, it inherits the sequential dependence on external tiling tools (e.g., Timeloop) because it lacks a linear expression for nonlinear tiling-fusion interactions. As shown in Figure 3, Timeloop accounts



for over 99% of per-trial latency, creating a prohibitive bottleneck for multi-thousand-trial searches. To resolve this, CHEETAH-V1 internalizes tiling by precomputing Pareto-pruned configurations and encoding them as decision variables. This removes Timeloop from the critical path, collapsing each trial to a single Vizier-proposal and LP-solve step. We start with introducing the preprocessing stage below before presenting the full formulation.

Pareto pruning pre-pass. The raw tiling search space for a 7-dimensional convolution loop with 3 memory levels is $\mathcal{O}(10^7)$ configurations per operation. We reduce this to a tractable Pareto-optimal subset $\mathcal{C}_i$ for each operation i via a pre-pass for (1) L1 feasibility where configurations whose tiled working set exceeds L1 scratchpad capacity C_{L1} are discarded; (2) Pareto optimality where a configuration c is retained only if no other configuration achieves both lower $T_i^{\max}(c)$ *and* smaller working-set size in the Global Buffer.

In practice, Pareto pruning reduces $\mathcal{O}(10^7)$ raw configurations to $|\mathcal{C}_i| \approx 10^2$ to 10^3 per operation. Timeloop is run once per (operation, configuration) pair, producing the triple $(T_i^{\min}(c), T_i^{\max}(c), t_i^k(c))$ for each retained configuration $c \in \mathcal{C}_i$. These are precomputed constants: the LP solver makes no Timeloop calls at solve time. Appendix D provides more details.

Time-invariant Pareto fronts. A key observation is that the Pareto front shape is invariant to memory time: configurations are ranked by compute cycles and working-set bytes, neither of which depends on bandwidth. Changing bandwidth rescales the DRAM access cost $t_i^k(c)$ uniformly. This means the Pareto-pruned configuration sets can be precomputed *once* and reused across all Vizier trials that share the same compute configuration class, amortizing the expensive Timeloop precomputation.

From fixed constants to decision variables. V1 brings the tiling choice *inside* the LP by introducing **tiling selection variables** $y_{i,c} \in [0,1]$, one per Pareto-optimal configuration $c \in \mathcal{C}_i$ from the pre-pass, with the constraint $\sum_c y_{i,c} = 1$ (exactly one configuration is selected per operation). The Timeloop-precomputed constants $T_i^{\max}(c)$, $T_i^{\min}(c)$, $t_i^k(c)$ now become *per-configuration coefficients* indexed by c, so the DRAM savings term becomes $t_i^k(c) \cdot y_{i,c} \cdot p_{\text{assoc}(i,k),\, o(i)}$, a bilinear product that we linearize below.

Linearization via McCormick envelopes. We linearize this product exactly via the classical McCormick substitution [McCormick, 1976]. For a bilinear term whose factors live in the unit box, the McCormick system is not merely *a* linear relaxation but the convex hull of the product's graph $\{(y, p, w) : w = y\,p,\ 0 \le y, p \le 1\}$: no tighter linear description exists without lifting to additional variables, and it is exact whenever either factor is integral. Relaxation looseness can therefore enter only through fractional (y, p) pairs, a quantity our rounding-and-replay audits measure per workload. Specifically, we introduce **auxiliary variables** $w_{i,c,k} := y_{i,c} \cdot p_{\text{assoc}(i,k),\, o(i)}$ that represent the joint effect of selecting tiling c *and* pinning tensor k, constrained by:

$$\begin{aligned} w_{i,c,k} &\le y_{i,c}, \qquad & w_{i,c,k} &\le p_{\text{assoc}(i,k),\, o(i)} \\ w_{i,c,k} &\ge y_{i,c} + p_{\text{assoc}(i,k),\, o(i)} - 1, \qquad & w_{i,c,k} &\ge 0 \end{aligned} \tag{2}$$

Since both factors lie in $[0,1]$, these four constraints force $w_{i,c,k} = y_{i,c} \cdot p$ at any integer vertex, requiring no manually tuned constants. Geometrically (Figure 4), the two upper inequalities are the concave envelope of the bilinear term $w = y\,p$ over the unit box and the two lower ones its convex envelope: the tightest concave over-estimator and convex under-estimator of the product. Because the product is bilinear over a box, these envelopes coincide with the convex hull and are exact at the integer corners, so an integral tile choice is credited with exactly its true DRAM savings, whereas a fractional blend of tiles can be over-credited by up to one quarter of the savings coefficient (at $y = p = \frac{1}{2}$). This over-credit is one of the two mechanisms behind the relaxation gap in V1 and V2, the other being the aggregate roofline row that Section 5.2 disaggregates, and it is exactly what the rounding-and-replay audits measure. Together, $y_{i,c}$ and $w_{i,c,k}$ constitute the majority of V1's decision variables (Table 5).

The Joint CHEETAH-V1 LP Formulation. Combining tiling selection, time-indexed placement, rematerialization, and McCormick linearization yields the full V1 program. We do not display it

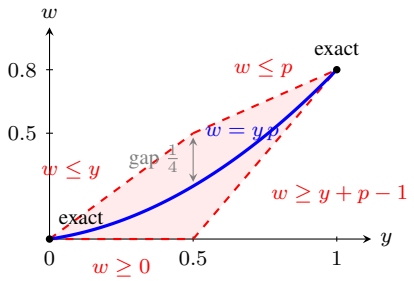


Figure 4: McCormick envelope of the bilinear term $w = y\,p$ on the unit box, drawn along the slice $p = 0.2 + 0.6\,y$ so that all four inequalities of (2) are distinct lines: the upper pair $w \le y$, $w \le p$ forms the roof and the lower pair $w \ge 0$, $w \ge y + p - 1$ the floor of the shaded feasible region, and the true product (blue) lies inside, touching the envelope only at the integer corners $y \in \{0, 1\}$. The largest over-credit is $\frac{1}{4}$, attained at $y = p = \frac{1}{2}$.

separately: it is the special case of the flagship program (V1) with the hardware reciprocals v, u pinned to the proposed chip, the CGM one-hot h_{GM} pinned to its memory, and the tile-dependent families z^{act}, z^{W} dropped; it in turn contains the fixed-chip program (V0), and (V1) is displayed in full at the end of Section 5.2.

Tile-dependent residency. Extending V1's linearization, V2 introduces two additional McCormick families for the bilinear products that couple tile choice to on-chip residency:

$$\begin{aligned} z^{\mathrm{act}}_{e,c,t} \equiv y_{\mathrm{dst}(e),c}\,p_{e,t}: \quad & z^{\mathrm{act}}_{e,c,t} \le y_{\mathrm{dst}(e),c}, \quad z^{\mathrm{act}}_{e,c,t} \le p_{e,t}, \\ & z^{\mathrm{act}}_{e,c,t} \ge y_{\mathrm{dst}(e),c} + p_{e,t} - 1, \quad z^{\mathrm{act}}_{e,c,t} \ge 0, \\ z^{W}_{i,c,t} \equiv y_{i,c}\,p^{W}_{i,t}: \quad & z^{W}_{i,c,t} \le y_{i,c}, \quad z^{W}_{i,c,t} \le p^{W}_{i,t}, \\ & z^{W}_{i,c,t} \ge y_{i,c} + p^{W}_{i,t} - 1, \quad z^{W}_{i,c,t} \ge 0. \end{aligned} \tag{3}$$

V2 also promotes the CGM size itself to an LP decision. Let $\{2^k \text{ MiB} : k = 0, \ldots, K-1\}$ be a discrete size grid and $h_{\mathrm{GM},k} \in [0, 1]$ a one-hot selector; the chip's on-chip global-buffer capacity is then $\sum_k 2^k\, h_{\mathrm{GM},k}$ bytes.

Hardware sizing inside the same program. The rows above optimize the schedule for a *fixed* chip proposed by the outer loop. CHEETAH-V1 closes the final decoupling in the same program: it promotes the chip's compute and bandwidth to LP decisions and replaces the aggregate roofline with its exact convex hull. Fixing the hardware knobs and collapsing the disaggregated variables recovers the fixed-chip case exactly, so this single formulation subsumes CHEETAH-V0 and is our flagship program.

Hardware as decision variables. Alongside the one-hot CGM selector $h_{\mathrm{GM},k}$, the flagship program (V1) sizes the MAC array and DRAM bandwidth *inside* the program. Writing the reference-normalized reciprocals $v = M_{\mathrm{ref}}/\mathrm{MACs}$ and $u = \mathrm{BW}_{\mathrm{ref}}/\mathrm{BW}$ (so a tile's compute time is $T_i^{\min}(c)\,v$ and its memory time is $T_i^{\max}(c)\,u$), the chip must respect the same-silicon area and power envelopes

$$\kappa_{\mathrm{MAC}}\,\mathrm{MACs} + \kappa_{\mathrm{SRAM}} \sum_k 2^k h_{\mathrm{GM},k} + \kappa_{\mathrm{BW}}\,\mathrm{BW} \;\le\; A_{\mathrm{cap}}, \qquad \pi_{\mathrm{MAC}}\,\mathrm{MACs} \;\le\; P_{\mathrm{cap}}. \tag{4}$$

The reciprocals are convex epigraphs, and the program is a linear program. Bandwidth *is* a finite menu: BW is a one-hot selection ν_ℓ over the gate-feasible legal points BW_ℓ (8 of the 12 catalogue points on DeepSeek-V3 pass the pin gate $\mathrm{BW}/0.8 \le 4096$ pins), so $u = 1/\mathrm{BW} = \sum_\ell \nu_\ell/\mathrm{BW}_\ell$ is exact at integral ν and the LP relaxation of the menu is a valid lower bound. The compute reciprocal $v \ge 1/m$ is bracketed by tangent rows on a geometric grid of m (a valid relaxation, hence a certified lower bound) and by chord rows on the same grid (an inner approximation); with 89 grid rows each envelope is within 10^{-3} of $1/m$, so the bracket is at most 2×10^{-3} wide, and no auxiliary variable is needed. Every product of a menu selector with a continuous quantity is written as its convex hull (disaggregated copies with a linking equality), which is exact at integral selectors; McCormick

envelopes are used only in that hull form. The chord (inner) LP yields an achievable design only once every menu selector is pinned to an integral value and the pinned solution passes an exact-lift check (reciprocals and products set to their true values, every original row re-verified); with relaxed selectors it is neither a lower nor an upper bound, so every "exact" below means exact at integral selectors. This internalizes the hardware proposal the outer loop previously made, so a single solve sizes chip *and* schedule jointly under a hard area/TDP budget.

Disaggregated perspective epigraph. The aggregate roofline row $T_i \geq \max\big(\sum_c T_i^{\min}(c)\, y_{i,c}\, v,\ \sum_c (T_i^{\max}(c) - \text{save})\, y_{i,c}\, u\big)$, the form used before the per-tile split below, is loose once y is fractional: it lets the solver blend a compute-bound tile against a bandwidth-bound tile and hide *both* bottlenecks (the source of a large relaxation gap on weight-heavy models). The flagship program (V1) replaces it with the disaggregated perspective form

$$\begin{aligned} &\textstyle\sum_c v_{i,c} = v, \quad \sum_c u_{i,c} = u; \qquad \underline{v}\, y_{i,c} \leq v_{i,c} \leq \overline{v}\, y_{i,c}, \quad \underline{u}\, y_{i,c} \leq u_{i,c} \leq \overline{u}\, y_{i,c}; \\ &\tau_{i,c} \geq T_i^{\min}(c)\, v_{i,c}, \quad \tau_{i,c} \geq \big(T_i^{\max}(c) - \text{save}_{i,c}\big)\, u_{i,c}, \quad T_i \geq \textstyle\sum_c \tau_{i,c}, \end{aligned} \tag{5}$$

which is the closed convex hull of the per-op "pick one tile, pay its roofline maximum" disjunction: it enforces $T_i \geq \sum_c \max(\cdot,\cdot)$ rather than the weaker $\max(\sum, \sum)$, tightening the certified lower bound at no cost to feasibility and preserving exactness at every integer vertex.

Certified column generation. The Pareto tile set $\mathcal{C}_i$ is a column *pool*: a candidate tiling/mapping enters only when its reduced cost against the converged LP dual variables is negative. The native mapper (or the LLM-Architect outer loop, which reads exactly these Lagrangian duals) can therefore *propose* columns while the LP *scores and certifies* them. The oracle can reshape where the search looks; it can never weaken the optimality certificate. This is the mechanism that makes "the LLM proposes, the solver certifies" rigorous, and it is what the outer loop of the next subsection exploits.

The joint CHEETAH-V1 LP formulation. Collecting the pieces, the full V1 program takes the fixed-chip rows above with the aggregate roofline row replaced by the disaggregated perspective epigraph, the reciprocals v, u and the memory one-hot promoted to decisions under the same-silicon caps, and one further hull family $X_{i,c,\ell}$, the bandwidth-menu copies of the memory-time numerator $X_{i,c} = T_i^{\max}(c)\, y_{i,c} - \sum_k t_i^k(c)\, w_{i,c,k}$, so that the DRAM savings scale with bandwidth exactly at integral ν. Writing $m = \text{MACs}/M_{\text{ref}}$ and $\beta = \text{BW}/\text{BW}_{\text{ref}}$, the program is (V1). The CHEETAH-

V1 LP formulation is:

$$
\begin{aligned}
\min \quad & \sum_{i\in\mathcal{N}} T_i \;+\; \sum_{i\in\mathcal{N}}\sum_{t\in\mathcal{T}} c_i^{\text{remat}}\, r_{i,t} \;+\; \varepsilon_{\text{CGM}} \sum_{k=0}^{K-1} k\, h_{\text{GM},k} \\
\text{s.t.} \quad & \sum_{c\in\mathcal{C}_i} y_{i,c} = 1 \;\; \forall\, i, \qquad \sum_{k=0}^{K-1} h_{\text{GM},k} = 1 \\
& v \;\geq\; \tfrac{2}{m_j} - \tfrac{m}{m_j^2} \quad \forall\, j \in \mathcal{J} \quad \text{(tangent rows; chord rows for the inner LP)} \\
& \beta = \textstyle\sum_\ell \nu_\ell\, \beta_\ell, \qquad u = \textstyle\sum_\ell \nu_\ell/\beta_\ell, \qquad \textstyle\sum_\ell \nu_\ell = 1 \quad \text{(bandwidth menu)} \\
& \kappa_{\text{MAC}}\,\text{MACs} + \kappa_{\text{SRAM}} \sum_k 2^k h_{\text{GM},k} + \kappa_{\text{BW}}\,\text{BW} \leq A_{\text{cap}}, \qquad \pi_{\text{MAC}}\,\text{MACs} \leq P_{\text{cap}} \\
& \sum_{c\in\mathcal{C}_i} v_{i,c} = v, \qquad \sum_{c\in\mathcal{C}_i} u_{i,c} = u \quad \forall\, i \\
& \underline{v}\, y_{i,c} \leq v_{i,c} \leq \overline{v}\, y_{i,c}, \qquad \underline{u}\, y_{i,c} \leq u_{i,c} \leq \overline{u}\, y_{i,c} \quad \forall\, i, c \\
& \tau_{i,c} \geq T_i^{\min}(c)\, v_{i,c}, \qquad \tau_{i,c} \geq \textstyle\sum_\ell X_{i,c,\ell}/\beta_\ell, \qquad T_i \geq \textstyle\sum_{c\in\mathcal{C}_i} \tau_{i,c} \quad \forall\, i, c \\
& \text{Envelopes (2) for } w_{i,c,k};\; X_{i,c} = T_i^{\max}(c)\, y_{i,c} - \textstyle\sum_k t_i^k(c)\, w_{i,c,k} = \textstyle\sum_\ell X_{i,c,\ell} \quad \forall\, i, c \\
& \underline{X}_{i,c}\, \nu_\ell \leq X_{i,c,\ell} \leq T_i^{\max}(c)\, \nu_\ell \quad \forall\, i, c, \ell \\
& \text{(3) for } z^{\text{act}}_{e,c,t}, z^{W}_{i,c,t} \\
& \sum_{e\in\mathcal{E}_{\text{live}}(t)} \sum_c d^{\text{act}}_{e,c}\, z^{\text{act}}_{e,c,t} + \sum_i \sum_c d^{W}_{i,c}\, z^{W}_{i,c,t} \leq \sum_{k=0}^{K-1} 2^k\, h_{\text{GM},k} \quad \forall\, t \\
& p_{e,t} \leq p_{e,t-1} + r_{\text{src}(e),\,t} \;\; \forall\, e,\, t > o(\text{src}(e)); \qquad p^W_{i,t} \leq p^W_{i,t-1} \;\; \forall\, i,\, t > o(i) \\
& r_{i,t} = 0 \;\; \forall\, t \leq o(i); \qquad r_{i,t} \leq \min\!\big(p_{e,t},\, p^W_{i,t}\big) \;\; \forall\, e \in \text{in}(i),\, t > o(i) \\
& T_i,\, \tau_{i,c},\, v_{i,c},\, u_{i,c} \geq 0; \quad y, p, p^W, r, w, z, h_{\text{GM}}, \nu \in [0,1]; \quad v \in [\underline{v}, \overline{v}],\; u \in [\underline{u}, \overline{u}]
\end{aligned}
\tag{V1}
$$

Here $\underline{X}_{i,c} = \min\{0,\, T_i^{\max}(c) - \sum_k t_i^k(c)\}$: in the measured tile caches the per-tensor byte terms can exceed $T_i^{\max}(c)$, so the numerator box is not sign-restricted, and the bandwidth-menu rows $(X_{i,c,\ell},\, \nu_\ell)$ are exact at integral ν. Pinning (v, u, h_{GM}) to a proposed chip makes the cap, tangent, and link rows inert, turns the menu copies $X_{i,c,\ell}$ into the constant multiple $X_{i,c}/\beta$, and leaves (V1) up to the tightening of Corollary 2; this is the precise sense in which the flagship program subsumes V0. When an achievable certified sub-problem is required, the compute tangent rows are replaced by the chord rows with every menu selector pinned (the inner LP), whose solution is a design once the exact-lift check passes; bandwidth is already a menu and needs no replacement. This is the form used by the column-generation oracle.

5.3 The Comprehensive CHEETAH-V1: Serving-Regime Co-Design Decisions

Program (V1) sizes compute, bandwidth, and on-chip memory jointly with the schedule. The memory-bound decode regime of LLM, MoE, and diffusion serving admits four further decisions, each of which reduces the bytes moved or sets a hardware capacity, hence is a linear (or McCormick-linearizable) addition to the flagship program (V1) rather than an outer-loop guess. We fold them in below. Every added block is *inert when pinned*: fixing its selector to a baseline policy leaves (V1), and with it the V0 and V1 chain, exactly intact, so the subsumption property is preserved. Decisions that only rescale coefficients (the decode batch N, the speculative-decode width K) enter as workload parameters that reshape the per-op dimensions and thus arithmetic intensity, and the flagship program (V1) evaluates any (N, K) without deciding them; genuinely discrete or non-convex choices (draft model, eviction policy, denoising-step schedule) remain columns the outer loop proposes and the LP

evaluates against its duals. Accuracy is never linearized: each precision menu below is restricted to configurations the quantization literature certifies as lossless-enough.

Precision as a decision (per phase, per expert). Let $\mathcal{Q}$ be an accuracy-vetted precision menu, each point ρ carrying weight-, activation-, and KV-bit fractions $(b_\rho^W, b_\rho^A, b_\rho^{\mathrm{KV}})$ relative to the 8-bit baseline. We write a menu entry as $\mathrm{W}w\mathrm{A}a$, meaning w-bit weights and a-bit activations, with a trailing $/\mathrm{KV}k$ when the KV cache is additionally held at k bits. The menu used here is

$$\rho \in \{\ \underbrace{\mathrm{W8A8}}_{\text{8-bit weights, 8-bit activations}},\ \underbrace{\mathrm{W4A8}}_{\text{4-bit weights, 8-bit activations}},\ \underbrace{\mathrm{W4A8/KV4}}_{\text{as W4A8, plus a 4-bit KV cache}},\ \underbrace{\mathrm{W4A16}}_{\text{4-bit weights, 16-bit activations}}\ \},$$

each admissible under AWQ, GPTQ and KVQuant [Lin et al., 2024, Frantar et al., 2023, Hooper et al., 2024]. Narrower weights reduce the bytes a decode step must stream, which is the dominant cost; narrower activations additionally reduce multiplier switching energy, and Section 9.2 reports the measured saving for each entry. Per phase $\phi \in \{\mathrm{pre}, \mathrm{dec}\}$ a one-hot selector chooses the operating precision, inducing byte-scales:

$$\sum_\rho q_{\phi,\rho} = 1, \qquad s_\phi^W = \sum_\rho b_\rho^W\, q_{\phi,\rho}, \qquad s_\phi^{\mathrm{KV}} = \sum_\rho b_\rho^{\mathrm{KV}}\, q_{\phi,\rho}, \qquad q_{\phi,\rho} \in \{0,1\}. \tag{6}$$

The weight and KV byte footprints $d_{i,c}^W$ and d^{KV} in the capacity row and the DRAM-savings term are replaced by $s_\phi^W\, d_{i,c}^W$ and $s_\phi^{\mathrm{KV}} d^{\mathrm{KV}}$; the resulting selector-times-variable products are written as hull disaggregations of the per-step byte aggregates over the precision menu (exact at integral q). For a mixture-of-experts layer the phase index is refined to an *expert* index: each expert e carries its own one-hot $q_{e,\rho}$ and byte-scale $s_e^W = \sum_\rho b_\rho^W q_{e,\rho}$, so the solver may place the lowest-bit points on the hottest experts, where weight-reload traffic dominates.

Energy-aware same-silicon cap. Precision helps under a fixed power budget only if bandwidth actually costs power, so the cap of (4) is extended to charge DRAM traffic at π_{bit} joules per bit:

$$\pi_{\mathrm{MAC}}\,\mathrm{MACs} \;+\; \pi_{\mathrm{bit}}\,\varrho_{\mathrm{DRAM}} \;\le\; P_{\mathrm{cap}}, \qquad \varrho_{\mathrm{DRAM}} = \sum_{\text{class}\,\in\,\{W,A,\mathrm{KV}\}} s_\phi^{\text{class}}\,\beta_{\text{class}}, \tag{7}$$

where ϱ_{DRAM} is the precision-scaled DRAM byte-rate over the weight, activation, and KV classes; each β_{class} is a workload constant (the bytes of that class streamed at the reference operating point), so the row is linear in the precision selectors. Setting $\pi_{\mathrm{bit}} = 0$ recovers (4); a positive π_{bit} is precisely what makes provisioned bandwidth "cost power" in the certified program, so the byte-reducing decisions (precision, KV budget, expert residency) relax the same cap that raw bandwidth tightens.

MoE expert residency (HBM pinning). Let C_{HBM} be the fast-memory capacity, a hardware decision entering the area cap, and let $f_e \in [0,1]$ be expert e's measured routing frequency (a workload statistic). A pin variable g_e is the resident fraction of expert e's weights: residency is knapsack-constrained, and an offloaded expert pays a per-visit reload:

$$\sum_e g_e\, s_e^W\, W_e \;\le\; C_{\mathrm{HBM}}, \qquad \Delta_{\mathrm{dram}}^{\mathrm{moe}} = \sum_e f_e\,(1-g_e)\, s_e^W\, W_e, \qquad g_e \in [0,1], \tag{8}$$

with W_e the expert weight bytes and $\Delta_{\mathrm{dram}}^{\mathrm{moe}}$ the expert-reload traffic added to the decode memory term. Pinning the hottest experts removes their reload; the dual variable λ_{HBM} of the capacity row is the marginal value of fast memory the outer loop reads when sizing C_{HBM}.

KV budget (eviction as a resource cap). A retained-cache budget $B^{\mathrm{KV}} \in [B_{\min}, L_{\mathrm{ctx}}]$ replaces the growing context length in the attention-input traffic, with $B_{\min}$ the quality floor of the eviction method (external). With per-token KV footprint d^{KV} and an on-chip residency indicator $\sigma_{\mathrm{KV}} \in \{0,1\}$, the budget is disaggregated over the joint one-hot $\pi_{\rho,s}$ of decode precision ρ and residency branch $s \in \{\mathrm{on}, \mathrm{off}\}$ (with $\sum_{\rho,s}\pi_{\rho,s} = 1$, $\sum_s \pi_{\rho,s} = q_{\mathrm{dec},\rho}$, $\sum_\rho \pi_{\rho,\mathrm{on}} = \sigma_{\mathrm{KV}}$),

$$\begin{aligned} &B^{\mathrm{KV}} = \textstyle\sum_{\rho,s} B_{\rho,s}, \qquad B_{\min}\,\pi_{\rho,s} \le B_{\rho,s} \le L_{\mathrm{ctx}}\,\pi_{\rho,s}, \\ &\Delta_{\mathrm{dram}}^{\mathrm{KV}} = d^{\mathrm{KV}} \textstyle\sum_\rho b_\rho^{\mathrm{KV}}\, B_{\rho,\mathrm{off}}, \qquad d^{\mathrm{KV}} \textstyle\sum_\rho b_\rho^{\mathrm{KV}}\, B_{\rho,\mathrm{on}} \;\le\; \sum_k 2^k h_{\mathrm{GM},k}, \end{aligned} \tag{9}$$

a two-branch hull with no big-M, exact at integral $(q, \sigma_{\mathrm{KV}})$, so that a small enough budget on the on-chip branch ($\sigma_{\mathrm{KV}} = 1$) lets the KV read be served on chip and leave the DRAM term, coupling the eviction budget directly to CGM sizing through λ_{CGM}.

Diffusion cross-step reuse (DeepCache). For an S-step denoiser, a per-block reuse variable $\chi_b \in [0,1]$ caches block b's slow-changing feature map across a window of W steps instead of recomputing it each step:

$$\widehat{\text{cost}}_b = \big(1 - \chi_b(1 - \tfrac{1}{W})\big)\,\text{cost}_b, \qquad \sum_b \chi_b\, F_b \;\le\; \textstyle\sum_k 2^k h_{\text{GM},k}, \tag{10}$$

where cost_b is block b's per-step compute and feature traffic, F_b its cached-feature footprint, and the window W and cacheable set are workload statistics. This gives the otherwise fusion-inert diffusion workload a genuine cross-step memory decision, the temporal analog of the KV cache.

The remaining decisions are drawn from production serving and kernel systems (vLLM, SGLang, ThunderKittens, TensorRT-LLM); each enriches the *software* side of the model and, crucially, ties it more tightly to a *hardware* knob, so the LP models how the chip and the kernel react to one another.

Prefill–decode interleaving. Serving co-schedules compute-bound prefill chunks with memory-bound decode steps; because their bottlenecks are complementary, an interleaved window fills both rooflines at once. An interleave fraction $\theta \in [0, \theta_{\max}]$ (bounded by the prefill backlog and a per-token latency SLA) folds prefill into the decode window, and the additive $T^{\text{pre}} + N\,T^{\text{dec}}$ is replaced by a mixed-window epigraph,

$$T_{\text{serve}} \ge \theta\, C^{\text{pre}} + N\, C^{\text{dec}}, \qquad T_{\text{serve}} \ge \theta\, M^{\text{pre}} + N\, M^{\text{dec}}, \tag{11}$$

with C^{ph} the lifted compute streams (linear in the per-branch copies of v), M^{ph} the bandwidth-menu memory streams (linear in the copies $X_{i,c,\ell}$), and θ a scheduler constant rather than a decision, so $T_{\text{serve}} \ge \max(\cdot,\cdot)$ over the combined stream and the optimal (v,u) ratio now tracks the *combined* arithmetic intensity rather than either phase alone [Kwon et al., 2023].

Load-engine vs. compute-worker datapath split. Following the load/store-vs-compute worker split of tile kernels [Spector et al., 2024], the die is partitioned between MAC units and data-movement (DMA/async-copy) engines, and the provisioned bandwidth is realizable only if enough load engines back it. Adding a load-engine throughput B_{LD} (area cost κ_{LD}) to the area cap,

$$\kappa_{\text{MAC}}\text{MACs} + \kappa_{\text{LD}} B_{\text{LD}} + \kappa_{\text{SRAM}} \sum_k 2^k h_{\text{GM},k} + \kappa_{\text{BW}}\text{BW} \le A_{\text{cap}}, \qquad \text{BW}_{\text{eff}} \le \text{BW}, \;\; \text{BW}_{\text{eff}} \le B_{\text{LD}}, \tag{12}$$

and the perspective reciprocal reads $u = \text{BW}_{\text{ref}}/\text{BW}_{\text{eff}}$: a memory-bound decode buys load engines (λ_{LD} large), a compute-bound prefill buys MACs, a split the solver makes per phase.

Prefix and KV cache reuse (RadixAttention). Shared prompt prefixes (system prompts, few-shot exemplars, chat history, repeated images) let requests reuse cached KV, cutting both prefill compute and KV traffic [Zheng et al., 2024, Kwon et al., 2023]. With a cache capacity C_{cache} (a hardware decision in the area cap) and a saturating hit-rate profile $\eta_s \in [0,1)$ measured under cache-aware (longest-shared-prefix-first) scheduling for the workload's sharing structure, a one-hot over cache sizes $s \in \mathcal{S}$ scales the reused terms:

$$\sum_s \rho_s = 1, \quad \eta_{\text{hit}} = \sum_s \eta_s\, \rho_s, \quad \widehat{C}^{\text{pre}} = (1 - \eta_{\text{hit}})\, C^{\text{pre}}, \quad \widehat{M}^{\text{pre}} = (1 - \eta_{\text{hit}})\, M^{\text{pre}}, \tag{13}$$

so the solver trades cache area against the prefill recompute and traffic it removes. The credit applies to the prefill streams only: the decode KV traffic $\Delta^{\text{KV}}_{\text{dram}}$ is not credited, because a shared prefix is re-read at every decode step whether or not it was cached at prefill (the convention of Section 7.9). The $\eta_{\text{hit}}\, C^{\text{pre}}$ and $\eta_{\text{hit}}\, M^{\text{pre}}$ products are exact by disaggregating the prefill streams over the cache-size menu ρ_s; the LP relaxation of that menu is the chord hypograph of the measured hit-rate curve.

Array shape and native tile granularity. Fixing MAC *count* leaves the array *shape* free, and utilization is acutely shape-dependent (a square array idles on skinny decode GEMVs and depthwise convs, the very cause of the mapper underutilization behind our analytic compute floor). A one-hot α_a over aspect ratios / native-MMA tiles $a \in \mathcal{A}$ (16×16, WGMMA, TCGEN05) carries a precomputed per-op utilization $\eta_{i,a} \in (0,1]$; disaggregating $v_{i,c}$ over a as in the perspective epigraph,

$$\textstyle\sum_a v^a_{i,c} = v_{i,c}, \quad 0 \le v^a_{i,c} \le \overline{v}\,\alpha_a, \quad \tau_{i,c} \ge \sum_a \frac{T^{\min}_i(c)}{\eta_{i,a}}\, v^a_{i,c}, \tag{14}$$

which co-designs the array to the workload's matmul shapes and directly serves LLM/MoE decode GEMVs and EfficientNet depthwise convs [Spector et al., 2024]. The lower bound $\underline{v}\, y_{i,c} \le v_{i,c}$ lives on the tile selector in (5) and is inherited by the copies through the linking sum, not placed on the aspect copies: since $\sum_a \alpha_a = 1$, a positive lower bound on every copy would force $v_{i,c} \ge \underline{v}$ on every non-selected tile and make every integral tile choice infeasible (this was found and fixed in the implementation on 2026-09-20). The DVFS copies $v^f_{i,c}$ of (15) carry the same one-sided bounds, $\sum_f v^f_{i,c} = v_{i,c}$ and $0 \le v^f_{i,c} \le \overline{v}\, \phi_{\mathrm{ph},f}$.

Per-phase clock and voltage (DVFS). Compute-bound prefill benefits from a high clock; memory-bound decode does not, so per-phase frequency is a decision. A per-phase one-hot $\phi_{\mathrm{ph},f}$ over a frequency menu (throughput scale γ_f, power factor π_f/π_0) scales the phase compute time and enters the power cap:

$$\sum_f \phi_{\mathrm{ph},f} = 1, \qquad \tau^{\mathrm{ph}}_{i,c} \ge \sum_f \frac{T^{\min}_i(c)}{\gamma_f}\, v^f_{i,c}, \qquad \pi_{\mathrm{MAC}}\mathrm{MACs}\sum_f \frac{\pi_f}{\pi_0}\phi_{\mathrm{dec},f} + \pi_{\mathrm{bit}}\varrho_{\mathrm{DRAM}} \le P_{\mathrm{cap}}, \tag{15}$$

so the chip runs fast in prefill and slow in decode, freeing power for bandwidth in the phase where compute is idle (the $\mathrm{MACs}\cdot\phi$ term is the hull $\mathrm{MACs} = \sum_f \mathrm{MACs}_f$, $M_{\min}\phi_f \le \mathrm{MACs}_f \le M_{\max}\phi_f$, exact at integral ϕ).

Constrained and speculative decoding (parameters). Constrained decoding over a compressed grammar FSM [Zheng et al., 2024] and speculative/draft verification emit $K > 1$ tokens per forward pass on deterministic or accepted spans, raising decode arithmetic intensity toward the compute-bound regime. Like the batch N, the effective width K (and the deterministic or accepted fraction) is a *workload parameter* the program evaluates, not a decision; the draft model and grammar remain outer-loop inputs.

The comprehensive program and its constraint signals. The comprehensive CHEETAH-V1 program is (V1) augmented with the precision, capacity, datapath, and scheduling families (6)–(15): the interleaved perspective epigraph now reads the precision-scaled and cache-reduced bytes, the reload/eviction deltas $\Delta^{\mathrm{moe}}_{\mathrm{dram}} + \Delta^{\mathrm{KV}}_{\mathrm{dram}}$, and the shape-, frequency-, and load-engine-dependent compute in place of the fixed footprints. Because every block is inert under a pinned policy, the subsumption chain (V0)$\subseteq$(V1)$\subseteq$(comprehensive program) holds by construction. Its converged duals are exactly the resource-dual vector the LLM-Architect outer loop consumes: λ_{BW} (large in decode) values a GB/s, $\lambda_{\mathrm{MAC}} \approx 0$ in decode makes added multipliers visibly worthless (the anti-gaming property as a certificate, not a claim), and $\lambda_{\mathrm{HBM}}, \lambda_{\mathrm{CGM}}, \lambda_{\mathrm{LD}}, \lambda_{\mathrm{cache}}$ value capacity for expert pinning, KV residency, load engines, and prefix reuse. The outer loop then proposes only the genuinely discrete and non-convex axes (interconnect radix, HBM-stack count, array aspect ratio, whether to spend area on low-bit MAC units, draft model, denoising-step schedule) as columns; the LP evaluates each by reduced cost and the same-silicon caps make inflation infeasible, so the search is a dual-steered, certified column generation rather than a blind sweep.

Variable and constraint digest. Each tier of the hierarchy inherits every variable of the tiers before it: V0 contributes latency, residency, and rematerialization; V1 adds tile selection $y_{i,c} \in [0,1]$ (constrained by $\sum_c y_{i,c} = 1$ over $|\mathcal{C}_i|$ Pareto-optimal configurations) with its McCormick products; V2 adds memory sizing and tile-dependent residency products; V3 adds hardware sizing and the disaggregated epigraph. Table 4 collects the full digest; Appendix C.1 describes variables and constraints in detail. Each configuration defines compute costs $T^{\max}_i(c)$, $T^{\min}_i(c)$, and per-tensor DRAM savings $t^k_i(c)$; the resulting bilinear DRAM-savings term is linearized exactly via McCormick auxiliary variables $w_{i,c,k}$, which enforce $w_{i,c,k} = y_{i,c}\cdot p_{\mathrm{assoc}(i,k),\,o(i)}$ at integer vertices without a relaxation gap.

Per-operation latency T_i is lower-bounded by the weighted configuration floor $\sum_c T^{\min}_i(c)\, y_{i,c}$ and the weighted off-chip cost $\sum_c T^{\max}_i(c)\, y_{i,c}$ minus achievable savings $\sum_{c,k} t^k_i(c)\, w_{i,c,k}$. This coupling enables the solver to accept sub-optimal local tiling (higher $T^{\max}_i$) if its smaller memory footprint unlocks global fusion gains that outweigh the compute-utilization loss. Capacity and persistence constraints carry over from V0, utilizing edge-indexed placement $p_{e,t}$ to handle multi-fanout nodes. For DeepSeek-V3, the V1 LP scales to approximately 2.5M variables, 3.0M constraints

Table 4: Decision-variable digest for the CHEETAH hierarchy. Each tier inherits all variables of the tiers above it in the table.

Variable	Index set	Tier	Role
T_i	op i	V0	per-operation latency (epigraph value)
$p_{e,t}$	edge e, step t	V0	activation residency in on-chip memory
$p^W_{i,t}$	op i, step t	V0	weight residency in on-chip memory
$r_{i,t}$	op i, step t	V0	rematerialization trigger
$y_{i,c}$	op i, config $c \in \mathcal{C}_i$	V1	Pareto tile-config selection, $\sum_c y_{i,c} = 1$
$w_{i,c,k}$	op i, config c, tensor k	V1	McCormick product $y_{i,c} \cdot p_{\mathrm{assoc}(i,k),o(i)}$
$h_{\mathrm{GM},k}$	size level k	V1	one-hot on-chip (CGM) capacity selection
$z^{\mathrm{act}}_{e,c,t}$, $z^W_{i,c,t}$	edge/op, config, step	V1	tile-dependent capacity products
MACs, BW	chip scalars	V1	compute/BW sizing via reciprocals v, u (4)
$v_{i,c}$, $u_{i,c}$, $\tau_{i,c}$	op i, config c	V1	disaggregated perspective epigraph (5)
$X_{i,c,\ell}$, ν_ℓ	op i, config c, menu point ℓ	V1	bandwidth-menu copies; menu selector

Table 5: Decision variables of the merged CHEETAH-V1 program and its size at two scales. The upper block lists every variable family: the schedule variables inherited from V0, the joint tiling and fusion variables, and the hardware-sizing variables that V1 promotes into the same program. The lower block gives the resulting problem size for DeepSeek-V3 both per operation and after shape aggregation (Appendix K), which is what makes a single monolithic solve tractable.

Variable	**Index set**	**Role**
Schedule (inherited from V0)		
T_i	op i	per-operation latency (epigraph value)
$p_{e,t}$	edge e, step t	activation residency on chip
$p^W_{i,t}$	op i, step t	weight residency on chip
$r_{i,t}$	op i, step t	rematerialization trigger
Joint tiling and fusion		
$y_{i,c}$	op i, config c	Pareto tile-config selection, $\sum_c y_{i,c} = 1$
$w_{i,c,k}$	op i, config c, tensor k	McCormick product $y_{i,c}\, p_{\mathrm{assoc}(i,k),o(i)}$
$z^{\mathrm{act}}_{e,c,t}$, $z^W_{i,c,t}$	edge/op, config, step	tile-dependent capacity products
Hardware sizing inside the program		
MACs, BW	chip scalars	compute and bandwidth sizing
v, u	chip scalars	reciprocals $1/m$ (tangent/chord rows), $1/\beta$ (bandwidth menu)
$h_{\mathrm{GM},k}$	size level k	one-hot on-chip capacity selection
$v_{i,c}$, $u_{i,c}$, $\tau_{i,c}$	op i, config c	disaggregated perspective epigraph
$X_{i,c,\ell}$, ν_ℓ	op i, config c, menu point ℓ	bandwidth-menu copies of $X_{i,c}$; menu selector
DeepSeek-V3 problem size		
Per operation ($N = 903$ ops)	2,504,019 vars, 2,983,450 rows, 6,094,156 nnz	
Shape-aggregated ($S = 13$ shapes)	11,166 vars, 32,230 rows, $\sim$103k nnz	

and 6.1M nonzeros. Table 5 provides a representative breakdown; full per-model sizes are in Table 23 and 24 in Appendix C.3.

5.4 Strategic Intelligence: LLM-Guided Outer Search

We use Google Vizier as a black-box Bayesian optimizer to propose hardware configurations. While effective, Vizier requires thousands of iterations to identify suboptimal regions without leveraging structural hardware-workload knowledge. To address this, we introduce an LLM-guided outer-loop controller based on SkyDiscovery that reasons about co-design decisions. This integration is enabled by the CHEETAH-V1 formulation, which removes the Timeloop bottleneck to reduce trial times from days to hours.

In this hybrid loop, the LLM acts as a *Causal Architect*: it reads a compact *Causal Ledger* of past trials (containing hardware parameters, validity, Perf/TDP, LP duals/slacks, MFU, and roofline diagnostics) and emits a *campaign patch* that constrains Vizier's search to a workload-appropriate

Table 6: The ten row families of the realized master, with instance counts for the DeepSeek-V3 structure. Counts are recipe instances, each standing for a range over the event axis.

Family	Count	Enforces
`independent_exact_cover`	410	each logical invocation covered by exactly one implementation
`lifetime_and_mass`	43,940	start/work/finish mass equals the chosen selector; active interval recurrence
`readiness_and_control`	57,570	operand residency, producer completion and routing causality at the firing event
`copy_state_and_generation`	5,406	tier-presence and birth recurrences; per-invocation completion front
`copy_source_backing_and_exit`	6,140	copy sources valid, eviction keeps backing, terminal data exits to host or DRAM
`capacity_and_destination_reservations`	9	live bytes, in-flight copy reservations, workspace and output allocations per tier
`traffic_and_work_ownership`	3	per-event demand equals the transfers and work charged to it
`shared_service`	29	per-event duration dominates compute, vector, bandwidth and port service
`shared_resource_reciprocals`	42	reciprocal envelopes tying hardware rates to service time
`shared_physical_hardware`	3	area, peak-power and power-density budgets over one shared allocation

architectural neighborhood (e.g., adjusting Global Memory floors, DRAM bandwidth ranges, or systolic shapes). A deterministic *Campaign Validator* clips or rejects each patch to enforce physical constraints (TDP caps, area limits, MAC/bandwidth coupling), preventing the LLM from proposing infeasible regions. Vizier then performs local Bayesian optimization within the approved patch, and the V1 LP evaluates each concrete candidate. A *Roofline Auditor* rejects any result whose predicted latency falls below the physical lower bound $\mathrm{LB} = \max(t_{\text{compute}}, t_{\text{memory}})$. The ledger records each intervention and outcome, preventing redundant exploration. The online Architect is implemented as Qwen2.5-7B-Instruct [Qwen Team et al., 2025] with a LoRA [Hu et al., 2022] adapter trained on past trial logs; a larger teacher model (DeepSeek-V3.2) was used offline for campaign label generation. The full loop details are in Appendix G.

5.5 The realized master: row families and scale

The formulation above is realized as a row-recipe compiler over an event axis, and it is worth naming what that compiler emits, because the reported solves are solves of *these* rows. Every decision in Table 3 lands in one of ten families (Table 6). The compiler stores one recipe per family instance rather than one row per event, so the event axis is never expanded: for the DeepSeek-V3 structure reported in Section 1 the program is 113,552 recipes over 50,106 axes, standing for 8.93×10^8 rows over 4.92×10^8 columns at an event horizon of $E = 11{,}909$. A recovered point is verified against those original rows directly, without materializing the matrix.

Two properties of this realization matter for reading the results. The objective is latency alone, so hardware is constrained by the budget rows rather than priced in the objective; and the service rows carry the reciprocal and product relaxations of Section 6.1, so a solved value is a bound on the modeled latency of the recovered schedule, which we then replay exactly.

6 The McCormick Blow-Up and the VM-One-Armed-Rescue Solver

The expressive power of the joint program is not free. Turning the tiling choice into a decision variable (Section 5.2) makes the DRAM-savings accounting *bilinear*; linearizing it exactly through McCormick envelopes [McCormick, 1976] then multiplies the size of the constraint matrix and drives its conditioning across roughly ten orders of magnitude. We describe this blow-up (Section 6.1) and the custom first-order GPU solver that renders the resulting linear program tractable at LLM scale

(Section 6.2). The two are inseparable: we adopt the exact linearization *only because* we have a solver engineered to consume its output.

6.1 The McCormick Blow-Up

A bilinear coupling. In the sequential pipeline the per-configuration timing $T_i^{\max}(c)$, $T_i^{\min}(c)$ and per-tensor DRAM cost $t_i^k(c)$ are fixed constants. V1 instead selects the tiling inside the LP through one-hot variables $y_{i,c} \in [0,1]$ with $\sum_{c \in \mathcal{C}_i} y_{i,c} = 1$ (Section 5.2). The savings realized when tensor k of operation i is pinned on chip are only credited if that tensor is *both* selected under configuration c *and* resident, so the DRAM-savings term becomes a product of two decision variables,

$$t_i^k(c) \underbrace{y_{i,c}}_{\text{tiling selection}} \cdot \underbrace{p_{\text{assoc}(i,k),\, o(i)}}_{\text{residency}}, \tag{16}$$

where $p_{e,t} \in [0,1]$ is the edge-residency variable and $o(i)$ the execution order of i. The bilinear factor $y_{i,c}\, p$ is nonconvex and cannot be placed in a linear program directly.

The McCormick box. For each such product we introduce an auxiliary variable $w_{i,c,k} := y_{i,c}\, p_{\text{assoc}(i,k),\, o(i)}$ and bound it inside a box cut out by four linear inequalities [McCormick, 1976],

$$w_{i,c,k} \le y_{i,c}, \qquad w_{i,c,k} \le p_{\text{assoc}(i,k),\, o(i)}, \qquad w_{i,c,k} \ge y_{i,c} + p_{\text{assoc}(i,k),\, o(i)} - 1, \qquad w_{i,c,k} \ge 0. \tag{17}$$

Geometrically, the four planes *bracket* the bilinear surface $w = y\, p$ over the unit box $[0,1]^2$: $w \le y$ and $w \le p$ form an over-estimating upper envelope, $w \ge 0$ and $w \ge y + p - 1$ an under-estimating lower envelope, and the feasible w is squeezed between them. Because both factors lie in $[0,1]$, this box is the *tight convex hull* of the graph of $y\, p$: the smallest convex relaxation over the interior, and at each of the four integer vertices $(y,p) \in \{0,1\}^2$ the upper and lower planes pinch shut so that $w_{i,c,k} = y_{i,c}\, p$ holds *exactly*, with no relaxation gap. Since $y_{i,c}$ is one-hot and residency rounds to $\{0,1\}$, the solution lands only on these gap-free corners, so (17) introduces no tuning constants and no bias. The same construction handles the two tile-to-residency families $z^{\text{act}}_{e,c,t}$, $z^{W}_{i,c,t}$ of the tile-dependent LP (V1).

A matrix that explodes. Exactness is bought with size. Each bilinear product spends one auxiliary variable and four inequalities (three coupling rows plus a nonnegativity bound), replicated over every $(\text{operation}, \text{tile config}, \text{tensor})$ triple. Summed over the network this dominates the program. For DeepSeek-V3 the V1 LP reaches roughly $2{,}504{,}019$ variables, $2{,}983{,}450$ constraints, and $6{,}094{,}156$ nonzeros, of which the McCormick auxiliaries and their selection partners are the large majority. Worse than its size, the matrix is severely *ill-conditioned*: the objective and constraint coefficients span about ten orders of magnitude, because Timeloop per-tile cycle counts run from 10^3 to 10^{10} while the residency and byte terms are $O(1)$ and the roofline memory term scales as $1/\text{BW}$; the products injected by (17) sit at the extreme ends of this range simultaneously. State it plainly: this blow-up is acceptable *only* because we have a solver that can handle a multi-million-variable, ten-orders-ill-conditioned LP in seconds. Standard tools cannot. The remainder of this section is that solver.

6.2 The Variable-Metric-One-Armed-Rescue Solver

We solve the McCormick-blown-up LP with a custom, matrix-free, first-order GPU solver written in JAX, which we call **VM-One-Armed-Rescue**. Its backbone is restarted *primal-dual hybrid gradient* (PDHG) [Chambolle and Pock, 2011] in the practical PDLP lineage [Applegate et al., 2025, 2026]: every iteration is a pair of sparse matrix-vector products and elementwise projections costing $\mathcal{O}(\text{nnz})$, with no factorization ever formed. This is what lets a two-million-variable LP live entirely on a single GPU. Three ingredients turn the generic restarted PDHG of GPU solvers such as cuPDLP [Lu and Yang, 2024, Lu et al., 2025] into a method that actually converges on our instances:

- **Diagonal variable-metric rescaling.** Rather than a fixed Euclidean metric, we precondition primal and dual iterates with a diagonal variable metric, initialized by Ruiz-style equilibration [Ruiz, 2001] of the coefficient matrix and then driven online by a smoothed estimate of $\sqrt{r_{\text{dual}}/r_{\text{primal}}}$, the square-root ratio of dual to primal residual. This rebalances the two sides of the saddle point that the ten-orders coefficient spread would otherwise pull apart.

- **One-sided "rescue" restart.** Stock PDLP restarts the primal and dual symmetrically; on our ill-conditioned LPs that heuristic overshoots and the duality gap oscillates without settling. VM-One-Armed-Rescue instead restarts *asymmetrically*, re-anchoring only the side (the "one arm") whose residual is lagging, so the trailing side is rescued without destabilizing the side that is already converging.
- **Halpern anchoring.** Restarts use Halpern-style anchoring [Halpern, 1967] toward the last restart point, which contracts the iterates and yields the stable, monotone convergence that the bare restart heuristic lacks.

A JIT-compiled `lax.scan` batches $\sim 10^5$ inner iterations onto the accelerator at a few microseconds each and fuses the equilibration loop into a single kernel.

Why it matters. On the V1-scale co-design LP, off-the-shelf solvers do not deliver a usable solution: CPU simplex and barrier (HiGHS) exhaust factorization memory or stall on the badly scaled matrix, and stock first-order GPU solvers (cuPDLP) leave the duality gap oscillating rather than closing it. VM-One-Armed-Rescue solves the same instances to primal–dual convergence in *seconds* on one GPU (the eight production blueprints report the custom JAX PDHG solving the V1 LP in 1.7–6.7 s), landing within a bounded, reported rounding gap of the mature HiGHS optimum wherever HiGHS can reach one. To our knowledge this is the first solver to make the McCormick-linearized *joint* tiling-plus-fusion-plus-placement co-design LP tractable at LLM scale, and that tractability is precisely what makes the inner LP cheap enough to sit inside a thousands-of-trials outer search rather than a one-off solve.

Scope of the claim. We state the solver claim exactly where it holds. VM-One-Armed-Rescue is engineered for, and validated on, the V1 and tile-dependent LLM-scale LPs of the $\sim$2.5M-variable class characterized above (the LPs that emit the blueprints), and it is on those instances that the contrast with HiGHS above is measured. We do *not* claim it solves the strictly larger monolith that additionally lifts the chip choice into a single program: the Tier-1 hardware-menu LP for DeepSeek-V3 reaches roughly 20.7M variables and 33.3M constraints, and on that instance VM-One-Armed-Rescue did not converge; neither did HiGHS, which stalled at the time limit without reaching primal feasibility. That both a GPU first-order method and a mature simplex/barrier code fail on the menu monolith is exactly what motivates decomposing it rather than solving it whole; the $\sim$2.5M-variable per-workload LP is the tractable unit the outer search is built on.

7 Experiments

Experiments span full CHEETAH V0 and V1 pipelines and the LLM Architect across 12 workloads: 11 vision and language models plus the Stable Diffusion 1.5 UNet. We benchmark against TPU-v3 because it is the production deep learning accelerator with the most publicly documented microarchitecture, process, and power figures [Jouppi et al., 2017, 2021], which lets every comparison be normalized to a disclosed chip and stay consistent with prior work [Zhang et al., 2022]. For the fixed-hardware study we add a second, deliberately generic hardware point, a compact edge NPU (Section 7.3), so that the scheduling gains are shown on both ends of the provisioning spectrum. Section 7.1 introduces our experimental setting, sanity checks, and physical manufacturability validation gates. Section 7.2 examines progressive unlocked speedup gains between LP formulations, Section 7.4 provides the breakdown of accelerator performance, Section 10.3 summarizes our custom systems solver design, and Section 10.4 provides discussion on CHEETAH custom accelerator blueprint analysis.

7.1 Experimental Setup

We benchmark against the static FAST framework [Zhang et al., 2022] using a simulated TPU-v3 [Jouppi et al., 2021] baseline normalized to sub-10nm process technology (within $8.2 \pm 2.7\%$ of profiled TPU-v3 performance). We use a custom adapted JAX PDLP solver for large models (Llama-70B, DeepSeek-V3) and HiGHS [Huangfu and Hall, 2018] for smaller models, reporting speedup over the TPU-v3 baseline in order to be consistent with prior work. We evaluate across 12 workloads: EfficientNet-B1 [Tan and Le, 2019] through B7 (stressing low-intensity depthwise convolutions), Llama-1 13B/65B and Llama-3 70B [Grattafiori et al., 2024], DeepSeek-V3-MoE

Table 7: Manufacturability gate definitions. Each candidate hardware configuration must pass all six gates before being accepted as a feasible design, rather than a theoretical Pareto-frontier point that exists only in LP space. These are not post-hoc screens applied to a finished design: every one of them is a row of the program itself, so the optimizer cannot return a design that violates them. The pin-count limit, for instance, enters as a linear constraint on bandwidth, which removes the illegal bandwidth branches from the menu before any design is scored. A gate can therefore only be reported as satisfied, never as repaired.

Gate	What it checks	Threshold
1. Pin count	HBM3-class signal pins at 0.8 GB/s/pin	$\leq 4{,}096$ pins
2. Reticle area	Linear area model on 7 nm	$\leq 600\,\mathrm{mm}^2$
3. Logic depth	MAC-tree combinational depth	≤ 35 levels
4. NoC routing	$n_{\mathrm{PEs}} \times n_{\mathrm{banks}}$ cells	$\leq 5 \times 10^6$
5. Power density	$P_{\mathrm{dyn}}/A_{\mathrm{die}}$	$\leq 2.0\,\mathrm{W/mm}^2$
6. SRAM macros	64 KiB banks; aspect ratio $\in$ [1:4, 4:1]	$\leq 4{,}096$ banks

[DeepSeek-AI, 2024] (stressing memory capacity and bandwidth), and the Stable Diffusion 1.5 UNet, a 346-operation diffusion backbone with long skip connections (stressing bandwidth-bound activation traffic). Each workload is optimized independently under 100 trials per method. For Static and V0 methods, the pipeline is Vizier→Timeloop→LP; for the V1 method, the pipeline simplifies to Vizier→LP. The LLM Architect outer loop proposes search-space interventions for Vizier using Lagrangian dual feedback, and is only permitted to change Vizier's search neighborhood while the strict LP inner loop remains unchanged.

Physical Design Validation. Since modeling simulations ultimately present a gap with real world physical performance, we validate all results through a five-level audit process: (i) Roofline Auditor, (ii) Campaign Validator, (iii) Causal Ledger, (iv) solver diagnostics, and (v) six pre-RTL gates. Each CHEETAH design must pass all five audit tiers before it is deemed feasible and reported.

The roofline auditor confirms the basic and necessary condition that the reported LP latency is not physically impossible: no result is faster than either the compute or memory lower bound. All reported CHEETAH results are roofline-feasible with zero lower-bound violations. The pre-RTL manufacturability gates aim to ensure that CHEETAH-designed chips can fit a single TSMC N7 reticle, has a wirable pin-out for four HBM3 stacks, meets 1 GHz timing, has a wirable NoC, fits an AIO cooler envelope, and decomposes into standard SRAM macros. This seeks to close the gap between simulation at research-level, and a physical design chip that could plausibly be taped out. Table 7 summarizes the six gate definitions, with detailed numerical reports and discussion provided in Appendix D.

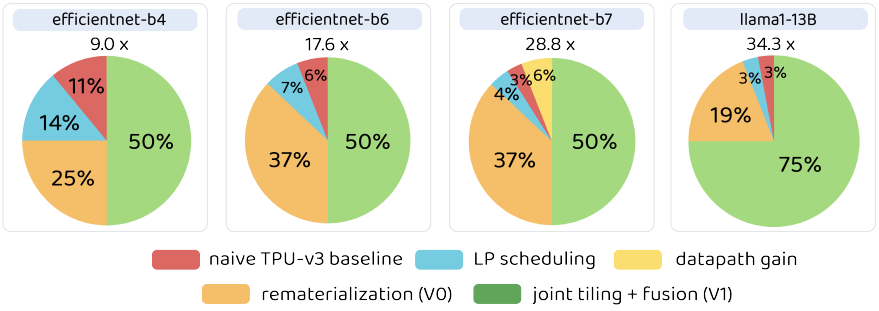


Figure 5: Speedup decomposition over naive TPU-v3 baseline. Each pie shows the relative contribution of four optimization sources to the total speedup. CHEETAH-V1 joint tiling + fusion dominates on all workloads, especially on larger models where memory management is the primary bottleneck.

7.2 Static Baseline vs. CHEETAH-V0 vs. CHEETAH-V1 Unlock Progressive Gains

Figure 5 decomposes the total speedup over a naive TPU-v3 baseline into four sources: LP-based scheduling and fusion, datapath gain from hardware search, dynamic rematerialization (CHEETAH-V0), and joint tiling + fusion (CHEETAH-V1). Table 25 in Appendix C.4 gives side-by-side structural comparison of the V0 and V1 formulation matrices in greater detail.

The dominant contributor across all workloads is V1's joint tiling-fusion optimization. V1's share grows with model size: from 50% on B1–B4 to 75–88% from B5 onward and 83–87% on Llama, reflecting that memory management increasingly dominates as models scale. On smaller models, LP scheduling and datapath gains split the remainder; V0 rematerialization contributes $\sim$25% on B3–B4 where recomputing large depthwise activations outperforms DRAM reloads.

Critically, Figure 2 shows the static fusion method is *infeasible* on modern large models (Llama-70B, and DeepSeek-V3), because their large activation tensors cannot fit under a static pinning policy. The V0 formulation resolves feasibility on large models via dynamic placement, while V1 further improves by jointly selecting memory-efficient tiling configurations alongside placement, achieving feasibility across the entire workload suite. This demonstrates that joint optimization is not merely an incremental improvement but a qualitative capability unlock for large-scale models. Additional optimization details and observations are provided in Appendix D. We further provide two interesting examples of optimal tiling solutions the V1 formulation proposed in Appendix H.

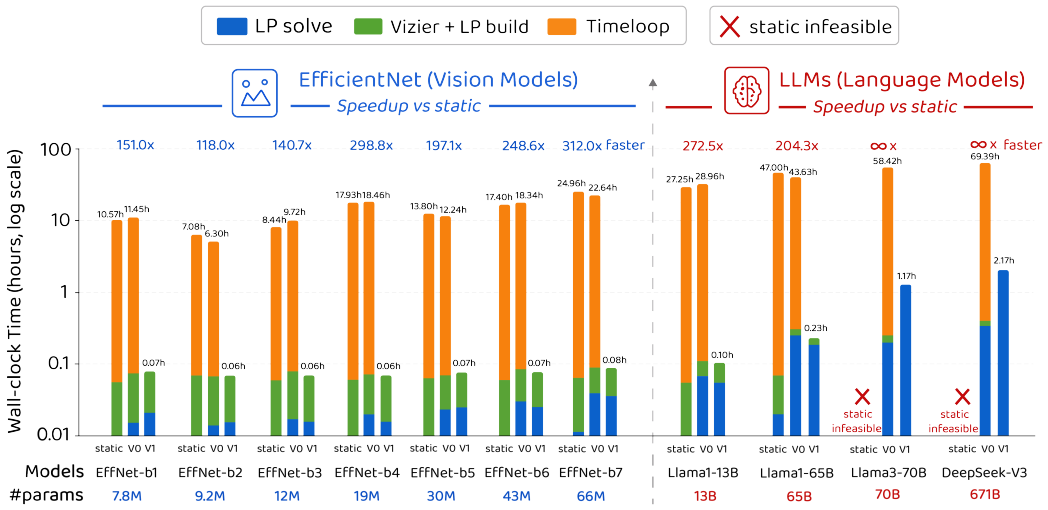


Figure 6: Wall-clock time decomposition and performance acceleration across formulations: CHEETAH-V1's internalized tiling eliminates the Timeloop bottleneck, achieving up to 100–300$\times$ faster search than the classical static method. Upper bars show wall-clock hours (log scale); annotations show speedup vs. static.

7.3 Same-Silicon Fixed-Hardware Speedup

The speedup decomposition of Figure 5 folds hardware search into the reported gains. To isolate CHEETAH's scheduling contribution from silicon provisioning, we hold the chip *fixed* and compare the joint CHEETAH-V1 schedule against a naive streaming schedule on the *same* silicon. Because the identical chip appears in both the numerator and the denominator of every ratio, these speedups are equal-area and equal-TDP by construction and require no resource normalization. We evaluate two fixed-hardware baselines: the simulated TPU-v3 (900 GB/s, 32 MiB on-chip) and a compact edge NPU representative of Jetson-Orin-Nano-class parts (32,768 INT8 MACs at 1 GHz, $\sim$65 TOPS, 64 GB/s LPDDR5, 16 MiB on-chip). We additionally include Stable Diffusion 1.5 UNet (SD1.5-UNet), a 346-operation diffusion UNet with long skip connections, to stress a bandwidth-bound regime distinct from the CNN and LLM suites.

Table 8 reports $T_{\text{naive}}/T_{\text{joint}}$ per cell, where the naive denominator is a $\sum T_{\max}$ streaming schedule computed under the same cost-model convention on each chip's own per-layer cache. On the TPU-

Table 8: Same-silicon fixed-hardware speedup: the joint CHEETAH-V1 schedule vs. a naive streaming schedule on the *identical* chip. Speedup is $T_{\text{naive}}/T_{\text{joint}}$ under the same cost-model convention on that chip's own per-layer cache (naive $= \sum T_{\max}$ streaming); MFU is model-MFU (cache compute-floor over latency within one convention), not measured silicon utilization. Ratios are equal-area/TDP by construction. Bold marks the best speedup per chip.

	TPU-v3 (900 GB/s, 32 MiB)			**Compact NPU** (64 GB/s, 16 MiB)		
Workload	$\text{MFU}_{\text{naive}}$	$\text{MFU}_{\text{joint}}$	Speedup	$\text{MFU}_{\text{naive}}$	$\text{MFU}_{\text{joint}}$	Speedup
Llama-1 13B	0.467	1.000	2.14×	0.022	0.694	**31.4×**
Llama-3 70B	0.399	1.000	**2.51×**	0.043	0.330	7.66×
DeepSeek-V3	0.451	1.000	2.22×	0.043	0.330	7.8×
SD1.5-UNet	0.643	0.963	1.50×	0.038	0.223	5.89×
Geomean			**2.06×**			**10.3×**

v3 the joint schedule delivers 1.50–2.51× (geomean 2.06×); on the compact NPU the gains are markedly larger, 5.89–31.4× (geomean 10.3×). This gap is a direct consequence of bandwidth: at 64 GB/s the naive streaming schedule is severely memory-starved, running at a model-MFU of only 0.02–0.04, so the placement and residency structure of the joint LP recovers a far larger fraction of idle time. SD1.5-UNet exemplifies this sensitivity: its 1.50× TPU-v3 gain rises to 5.89× on the edge NPU, tracking the same inverse-bandwidth trend as the LLMs because the diffusion UNet is bandwidth-bound once on-chip capacity is tight. All model-MFU values are the cache compute-floor divided by LP latency within a single convention, not measured silicon utilization: on the TPU-v3 the joint schedule recovers model-MFU from a naive 0.40–0.64 to 0.96–1.00 (a near-pure utilization recovery, not additional compute), while on the bandwidth-starved edge NPU the joint model-MFU (0.22–0.69) is the physical roofline, honestly below one.

7.4 Wall-Clock Acceleration and LLM Outer Loop Search Efficiency

Figure 6 reveals a striking practical advantage of V1: by internalizing tiling into the LP, it removes Timeloop from the per-trial critical path, collapsing each trial to a single Vizier-proposal and LP-solve step. On EfficientNet models, V1 completes 50 trials 151–312× faster than the static pipeline. On LLMs the acceleration is even more dramatic: V0 requires 58.42 hours on Llama-70B and 69.39 hours on DeepSeek-V3 for 50 trials, whereas V1 requires only 1.17 hours and 2.17 hours respectively, a ∼93% average reduction. This speedup arises because Timeloop accounts for over 99% of per-trial latency in the sequential pipeline (Figure 3), and V1 eliminates this bottleneck entirely via precomputed Pareto tiling sets that are reused across all Vizier trials sharing the same compute configuration class. The combination of higher speedup *and* dramatically lower search cost makes V1 strictly dominant: it explores a larger feasible design space in a fraction of the wall-clock time.

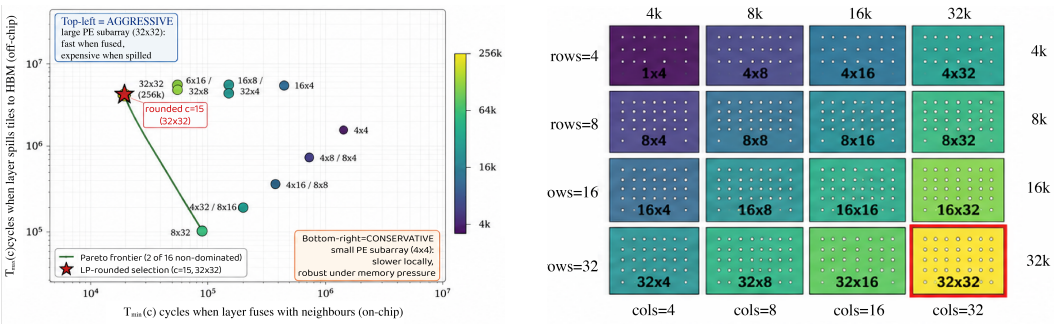


Figure 7: Example per-layer tiling cache for EfficientNet-B7 layer L4. Left: Timeloop-evaluated candidates plotted by fused/on-chip cost $T_{\min}(c)$ and spilled/off-chip cost $T_{\max}(c)$; the frontier shows non-dominated choices and the star is the LP-rounded selection. Right: the 16 tiling choices; labels such as 4×4, 8×8, and 32×32 denote active tile rows/columns inside the fixed accelerator, white dots indicate active PE/MAC sites. These are schedule choices, not different chips.

Per-layer joint candidate tiling advantage. For each operation, CHEETAH-V1 precomputes a small set of Timeloop-evaluated tile mapping configurations and exposes them to the V1 LP through the selection variables $y_{i,c}$. Figure 7 illustrates this for one layer: each point is one candidate mapping, with $T_{\min}(c)$ on the x-axis representing the fused/on-chip execution floor and $T_{\max}(c)$ on the y-axis representing the spilled/off-chip cost. Aggressive mappings (upper left) are fast when fusion succeeds but expensive, while conservative mappings (lower right) are slower locally but more robust under memory pressure. The LP relaxation assigns fractional mass to two configurations in this example, $y_{15} = 0.71$ and $y_{11} = 0.29$, and the rounded replay selects $c = 15$ (the 32×32 mapping), with only a 3% gap to the LP lower bound.

This illustrates one core advantage of the CHEETAH-V1 LP: tiling is no longer fixed before fusion. Instead, tile choice, placement, and rematerialization are jointly decided, and the solver is free to choose a locally non-obvious mapping whenever doing so improves the global schedule. Concretely, the V1 LP introduces $y_{i,c}$ alongside McCormick auxiliary variables $w_{i,c,k}$ to make the tiling selection interact directly with tensor placement and rematerialization decisions, with Timeloop running once per retained configuration to supply the triple $\left(T_i^{\min}(c),\, T_i^{\max}(c),\, t_i^k(c)\right)$ for each candidate c.

LLM Architect vs. Optimizer. Modeled on SkyDiscovery [Liu et al., 2026], the LLM Architect prunes the outer search loop by proposing campaign patches rather than predicting raw performance. It analyzes a Causal Ledger containing invalid rates, MFU, and solver diagnostics to suggest constrained hardware neighbourhoods for Vizier to explore locally. The V1 LP remains the ground-truth evaluator, supported by a Roofline Auditor that rejects physically impossible results. In a widened search space, the LLM loop reduces invalid evaluations from 24.5% to 5.9% on average to yield a $4.1\times$ reduction in wasted trials (Figure 8), while reaching the same optimal Perf/TDP scores as full-space Vizier. This demonstrates that the LLM's role is steering Bayesian optimization toward feasible, workload-appropriate subspaces rather than replacing the solver. We provide exact prompts in Appendix G.2 for full replication of results, as well as two specific case studies of real-world use cases.

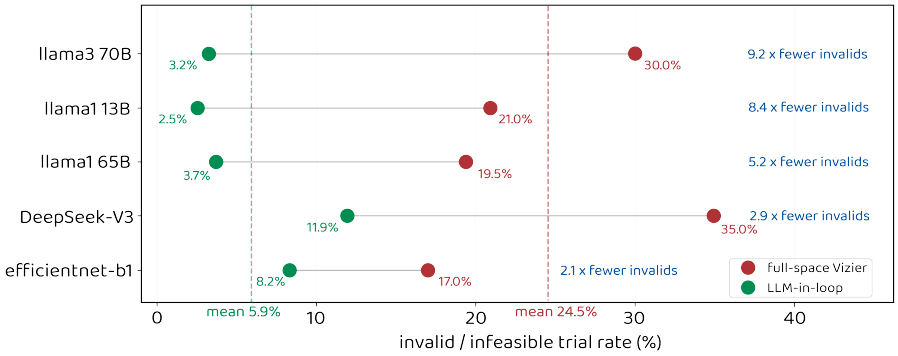


Figure 8: Invalid/infeasible trial rate (%) for full-space Vizier vs. LLM-in-loop across five workloads. The LLM Architect reduces the mean invalid rate from 24.5% to 5.9% ($4.1\times$ reduction), with the largest gains on LLM workloads where the hardware search space is most prone to infeasible configurations.

7.5 Sensitivity to serving batch size

The serving results above are reported at a decode batch of 32. Batch is a workload parameter rather than a decision variable, but it is not a neutral one: it reshapes the program's coefficients in four places at once, namely the mixture-of-experts reload traffic through the number of distinct experts a step touches, the decode compute, the KV read, and the array utilization of the decode operator. We therefore rebuild the program at $B \in \{1, 8, 32, 128\}$ and re-certify, comparing in each case against a TPU-v3 baseline evaluated at the *same* batch, so that the comparison is like for like at every operating point.

Table 9 reports the result. Two things follow, and the second is specific to mixture-of-experts serving.

The advantage is not an artifact of the operating point, and $B = 32$ is conservative. The certified serving speedup is $33.1\times$ at $B = 1$ and rises monotonically to $48.6\times$ at $B = 128$. The claim of a large serving advantage therefore does not depend on a favorable choice of batch: it holds from single-stream interactive decoding to throughput-oriented batch serving, and the batch we report elsewhere sits below the most favorable point rather than at it. The increase with batch has a simple cause: the fixed baseline must stream a reload that grows with batch across a bandwidth it cannot change, whereas the co-designed chip provisions bandwidth and applies the byte-reducing decisions, so the gap widens exactly where the baseline is most starved.

Batching is a weak amortizer for a sparse mixture of experts, which is why speculation retains its value. For a dense model, a larger batch amortizes each weight load perfectly: the same weights serve every token in the batch. Top-k routing breaks this. The expected number of distinct experts touched per step follows $n(1 - (1 - k/n)^B)$, so a larger batch touches *more* experts and the reload grows: from 19 GB per step at $B = 1$ to 598 GB at $B = 128$ in Table 9. Per token the reload does fall, from 19 to 4.7 GB, but this is a $4\times$ amortization for a $128\times$ increase in batch, and it saturates once essentially every expert is swept (251.6 of 256 at $B = 128$). The consequence is that the decode memory wall is not a low-batch phenomenon that disappears under load, and the optimal speculative width does not collapse as batch grows but *increases* with it: $K^\star$ doubles with every fourfold increase in batch, from 2 at $B = 1$ through 8 and 16 to 32 at $B = 128$. The reason is visible in the same table: because the reload grows with batch, each verification pass has more traffic to amortize, so wider speculation keeps paying. Speculation and batching are commonly described as competing amortizers; for a sparse mixture of experts they are better described as complementary, since batching amortizes the reload far less effectively than it does for a dense model and thereby leaves the work that speculation does.

Table 9: Certified serving speedup against a TPU-v3 baseline evaluated at the same batch ($p = 0.75$, 1 GB diffusion draft). $\mathbb{E}$[experts] is the expected number of distinct routed experts touched per decode step, out of 256. Best-of-K is the certified achievable design at that batch. All entries, including the baselines, come from one search setting (Appendix O): a 24-point grid on the compute reciprocal, an integral bandwidth branch, and full enumeration of the precision, sparsity, memory-technology and array-aspect selectors.

B	$\mathbb{E}$[experts]	Reload (GB/step)	K=2	K=8	K=16	K=32	Best
1	8.0	19.0	**33.13**×	30.12×	23.28×	15.69×	33.13× at $K^\star$=2
8	57.4	136.6	28.35×	**36.76**×	32.18×	24.32×	36.76× at $K^\star$=8
32	163.3	388.5	28.56×	43.03×	**44.85**×	36.01×	44.85× at $K^\star$=16
128	251.6	598.5	29.02×	46.11×	48.39×	**48.64**×	48.64× at $K^\star$=32

7.6 Performance per TDP

Speedup alone does not settle whether a design is worth building: a chip that is fast because it is large is not obviously an advance. FAST [Zhang et al., 2022] therefore searches on performance per unit power, and reports a headline of about $3.7\times$ over its baseline. We evaluate the same quantity on the designs of Table 9, using one power model for both sides (Appendix O) so that the ratio is convention-consistent: $\mathrm{Perf/TDP} = \mathrm{speedup}/(\mathrm{TDP}_{\mathrm{design}}/\mathrm{TDP}_{\mathrm{baseline}})$, with the TPU-v3 baseline at 41.9 W.

Table 10 reports the result. Performance per TDP rises from $2.73\times$ at $B = 1$ to $4.83\times$ at $B = 128$, exceeding the FAST headline for batches of 32 and above and approaching it at $B = 8$. Two points are worth making precisely. First, the designs are selected by *serving latency*, not by performance per TDP, so this is the efficiency of a latency-optimal design rather than the best efficiency the program can reach; searching the same certified frontier on performance per TDP directly moves the figure by less than 2% at $B \le 8$ and selects the identical design at $B = 32$, which indicates that the speedup is not being bought with disproportionate power. Second, the efficiency improves with batch for the same structural reason the speedup does: the fixed baseline must stream a reload that grows with batch across bandwidth it cannot change, while the co-designed chip holds a single $24.4\times$-MAC, 3200 GB/s configuration across $B \in \{8, 32, 128\}$ and simply serves more tokens per unit of that fixed power.

Table 10: Performance per TDP of the certified designs of Table 9, in the FAST convention (speedup normalized by the TDP ratio; TPU-v3 baseline 41.9 W). FAST reports approximately 3.7×.

B	$K^\star$	Speedup	MACs	BW (GB/s)	TDP (W)	Perf/TDP
1	2	33.13×	31.1×	3200	509	2.73×
8	8	36.76×	24.4×	3200	422	3.65×
32	16	44.85×	24.4×	3200	422	**4.45**×
128	32	48.64×	24.4×	3200	422	**4.83**×

7.7 Certified bounds and sensitivity to draft acceptance

Every design reported above is a certified achievable point: a concrete, feasibility-checked configuration whose serving time we can exhibit. Solving the same program with the physically integral selectors relaxed gives the complementary quantity, a bound that no design in this model can beat. Together they bracket the unknown optimum, and the width of the bracket is a certified statement about how much performance could still be left on the table. This is the quantity a point estimate cannot provide, and we report it under one search configuration so that both ends come from the same search.

Table 11 gives the bracket across batch and draft acceptance. Three readings follow. First, *the bracket tightens where throughput serving lives*: the gap falls from 63.8% at $B = 1$ to 13.5% at $B = 128$ with $p = 0.60$, so in the high-batch regime the optimum is pinned to within about a seventh of the achievable value. Second, *acceptance matters unevenly*. At $B = 1$ the three acceptance columns are identical, because $K^\star = 2$ consumes almost no acceptance; at $B = 128$ the achievable spans 34.6× to 66.5× as p moves from 0.60 to 0.85. The parameter we state rather than measure is therefore nearly inert in the latency regime and dominant in the throughput regime, which is where a measured acceptance rate would sharpen the result most. Third, *the relaxation loosens as acceptance rises* (from +40.0% to +59.0% at $B = 8$): a higher acceptance rate gives the fractional relaxation more room to exploit, so the bound degrades in exactly the regime where the achievable improves, and we report both rather than only the favourable half.

We note that the operating point used elsewhere in this paper, $p = 0.75$, is the middle of the swept range: the $p = 0.85$ column is materially higher at every batch, so the headline figures are conservative on the acceptance axis as well as on the batch axis.

Table 11: Certified bracket on the serving optimum under one search configuration. "Achv" is the certified achievable design, "bound" the relaxed-LP bound that no design in this model can beat, and "gap" their separation. Both ends are computed by the same search, and the $B = 32$, $p = 0.75$ entry reproduces Table 9 exactly.

		$p = 0.60$			$p = 0.75$			$p = 0.85$		
B	$K^\star$	Achv	Bound	Gap	Achv	Bound	Gap	Achv	Bound	Gap
1	2	32.93×	53.96×	63.8%	33.13×	54.27×	63.8%	33.13×	54.27×	63.8%
8	8	31.32×	43.86×	40.0%	36.76×	55.28×	50.4%	40.72×	64.74×	59.0%
32	16	33.46×	39.91×	19.3%	44.85×	57.26×	27.7%	56.85×	78.38×	37.9%
128	32	34.60×	39.26×	**13.5**%	48.64×	58.37×	20.0%	66.49×	86.13×	29.5%

7.8 Does sparse routing cause the effect?

The explanation offered above is causal: we claim that the widening of $K^\star$ and the growth of the advantage with batch are consequences of top-k routing, which makes the reload grow with batch instead of amortizing. That claim is testable by ablation. We rebuild the identical program with one line changed, pinning the expected distinct-expert count to its $B = 1$ value so that a larger batch reuses the same expert set and weight loads amortize perfectly, as they do for a dense model. Model dimensions, KV traffic, decode compute, the decision menus, the speculative-width menu, the search setting and the baseline convention are untouched, so any difference isolates routing and nothing else. The prediction is sharp: under dense-equivalent routing both effects should disappear.

Table 12 reports the comparison. They disappear. With the reload pinned at 19.0 GB per step, the selected speculative width is $K^\star = 2$ at every batch and the certified speedup is flat near $33\times$, against $K^\star = 2, 8, 16, 32$ and $33.1\times$ to $48.6\times$ under real routing. The ablation also supplies its own positive control: at $B = 1$ sparse and dense routing coincide by construction, and the two arms agree to four significant figures ($33.13\times$, $K^\star = 2$ in both), which confirms that the control differs from the treatment in the intended variable only.

We therefore state the mechanism as measured rather than argued. Speculation scales with batch in sparse serving because each verification pass has a larger reload to amortize; remove that growth and the optimal width collapses to its low-batch value. The practical reading is that a serving stack tuned on dense models will systematically under-speculate on a mixture of experts, and that the error grows with the batch size it was tuned at.

Table 12: Controlled ablation of sparse routing. The dense-control column pins the distinct-expert count to its $B = 1$ value so weight loads amortize perfectly with batch; everything else is identical, including the search setting. Both signature effects vanish, and the two arms agree exactly at $B = 1$, where the ablation is a no-op.

	Sparse (top-k routing)		Dense control	
B	Reload (GB)	$K^\star$ (speedup)	Reload (GB)	$K^\star$ (speedup)
1	19.0	2 (33.13×)	19.0	2 (33.13×)
8	136.6	8 (36.76×)	19.0	2 (33.08×)
32	388.5	16 (44.85×)	19.0	2 (33.06×)
128	598.5	32 (48.64×)	19.0	2 (33.24×)

7.9 Where the decode speedup comes from

Table 29 splits the prefill speedup into a scheduling factor on fixed silicon and a provisioning factor to the co-designed chip. The decode phase admits the same treatment, and the answer is different in kind: prefill is compute-bound, so its gain is dominated by provisioning, while decode is bound by the bytes each step must stream, so most of its gain is bought before any silicon changes.

Table 13 reports the decomposition. Each step is a certified integral design found by the same enumeration, the arms are cumulative, and every arm is compared against the same fixed TPU-v3 evaluated at the same batch, context, and generation length. Decode time is $(G/A(K))\,(T_{\text{draft}} + T_{\text{pass}})$, so a step that raises the per-pass cost but accepts more tokens per pass still shows as a gain.

Table 13: Cumulative decode speedup decomposition, DeepSeek-V3 at decode batch 32, acceptance $p = 0.75$, $G = 512$ generated tokens, against a fixed TPU-v3 at the same operating point. $K^\star$ is the selected speculative width, where $K^\star$=1 means speculation is off and no draft model is paid for. Steps 1 and 2 hold the silicon at TPU-v3 (65,536 MACs, 900 GB/s); step 4 frees it. Conventions: lossless KV (the budget is pinned to the full history), no prefix-cache credit against decode traffic, per-step KV over four context bins, and attention compute charged over the history.

			$c = 4{,}096$		$c = 32{,}768$	
Step	**What it adds**	$K^\star$	Cumul.	Step	Cumul.	Step
D0	fixed TPU-v3, W8A8-KV8, dense	1	1.00×		1.00×	
D1	weight bytes: W4, 2:4 sparsity, expert pinning	1	3.87×	3.87×	3.21×	3.21×
D2	KV co-design: KV4, on-chip hot window, prefix reuse	1	4.04×	1.04×	4.23×	1.32×
D3	speculation: width K and its draft cost	4 / 2	7.62×	1.89×	4.32×	1.02×
D4	hardware: MACs, bandwidth, memory technology	16 / 8	**35.79**×	4.70×	**32.74**×	7.59×

The KV term grows with context, and it is the only term that does. At $c = 4{,}096$ the KV read is 4.9 GB of the 422 GB a baseline step streams, so KV co-design is worth $1.04\times$ and the decode bottleneck is almost entirely the mixture-of-experts weight reload. At $c = 32{,}768$ the same read is 37.1 GB, and because step D1 has already cut the weight bytes in half it is 26% of what the step streams, so the same lever is now worth $1.32\times$. The expert reload cannot grow past the point where a step touches all 256 experts, while KV traffic grows linearly in batch times context,

so the KV share rises with both. This is why the KV window is a co-design decision rather than a detail: time-multiplexing the on-chip window by layer buys enough resident capacity to hold 31,858 positions per layer at $c = 32{,}768$, which is what converts the growing KV read back into an on-chip one.

Speculation is a lever, not an assumption. $K{=}1$ with no draft model is in the menu at every step, so the program is free to decline speculation, and at $c = 32{,}768$ on fixed silicon it nearly does: $K^\star$ falls to 2 and the step is worth only $1.02\times$, because a wider window re-reads a longer KV history for every draft token it verifies. Once the silicon is free the balance moves back, and $K^\star$ rises to 8 at $c = 32{,}768$ and 16 at $c = 4{,}096$. A fixed speculative width would have reported the 4,096 answer at both contexts.

8 Total Cost of Ownership and Return on Investment

A co-designed accelerator only pays for itself if the recurring efficiency it buys eventually amortizes its one-time silicon cost. To evaluate this we adopt FAST's total-cost-of-ownership (TCO) methodology [Zhang et al., 2022] *verbatim* (the same cost model, the same non-recurring-engineering (NRE) decomposition, and the same return-on-investment (ROI) break-even analysis; their Eq. 1–2 and Sec. 5.1), so that our deployment economics are read on FAST's own terms and are directly comparable to their Table 4. We stress at the outset that this is the weakest-grounded part of our evaluation: absolute TCO depends on proprietary fleet data neither we nor FAST possess, several cost constants are assumption-flagged, and the speedup that drives the ROI depends strongly on which baseline it is measured against. We therefore do not report a single S: Section 8 stratifies every Perf/TDP figure over three denominators of increasing strength, and we carry the ROI through on the design that clears the strongest of them (Section 8), stating plainly that the throughput-provisioned blueprints do not. We report ROI as an order-of-magnitude plausibility check, not a precise claim, and we state every caveat inline.

FAST's cost model (what we mirror). FAST models the three-year cost of a fleet that offloads n baseline accelerator-units of traffic onto a co-designed chip as

$$\mathrm{TCO}_{\mathrm{old}}(n) \;=\; C_{\mathrm{cap}}(n) \;+\; t_D\, C_{\mathrm{op}}(n), \tag{18}$$

capital cost plus lifetime ($t_D = 3$ yr) operating cost, and defines the ROI of committing the co-design NRE as

$$\mathrm{ROI} \;=\; \frac{\mathrm{TCO}_{\mathrm{old}}\,(S-1)}{\left(t_{\mathrm{design}}\, C_{\mathrm{eng}} + C_{\mathrm{mask}} + C_{\mathrm{IP}}\right) S}, \tag{19}$$

where S is the per-unit efficiency gain of the co-designed chip over the baseline; $\mathrm{ROI} > 1$ means the deployment is profitable. Because per-chip TCO is confidential, FAST uses $S =$ Perf/TDP as a proxy for Perf/TCO (the two are reported to be highly correlated for datacenter accelerators [Jouppi et al., 2017, 2021]) and anchors the baseline unit to a commodity DGX-A100 accelerator. We reuse this proxy, this DGX-anchored baseline unit, and FAST's engineering-NRE constant ($t_{\mathrm{design}}\, C_{\mathrm{eng}} =$ 65 engineer-years $\times$ \$240K $\times$ 1.65 overhead $=$ \$25.74M) unchanged. Inverting Eq. (19) at a target ROI multiple R gives the break-even deployment volume

$$n^\star(R) \;=\; \frac{R \cdot \mathrm{NRE} \cdot S}{(S-1) \cdot \mathrm{TCO}_{\mathrm{unit}}}, \qquad \mathrm{NRE} = t_{\mathrm{design}}\, C_{\mathrm{eng}} + C_{\mathrm{mask}} + C_{\mathrm{IP}}, \tag{20}$$

the number of baseline units whose traffic must be offloaded before the NRE is repaid. All arithmetic below is a pure evaluation of Eqs. (18) to (20) on the eight co-designed blueprints of Appendix I; no solver is involved.

Break-even under FAST parity. Under FAST's Perf/TDP proxy, our co-designed chips retain $S = 1.15$–$1.79\times$ Perf/TDP over a modeled TPU-v3 after normalizing away raw MAC scale, measured against the c0-naive denominator (D1) of Section 8. These are *prefill* blueprints under that convention; the 2.73–$4.83\times$ reported in Section 7.6 are *serving* designs under the FAST TDP-ratio convention, so the two sets are not comparable and neither supersedes the other. Both are superseded, as the figure a reader should carry, by the iso-resource result of Section 8, which is the only one of the three measured against a well-scheduled baseline. For the best chip (the DeepSeek-V3 [DeepSeek-AI, 2024] $V1$

Table 14: FAST-parity break-even volumes (convention F: S = Perf/TDP proxy, DGX-A100-anchored baseline unit), computed from Eq. (20). NRE is \$50.7M for the N7 ($V1$) family and \$65.7M for the N5+HBM3 ($V2$) family. For reference, FAST's Table 4 reports 1×-ROI volumes of 2,164–3,534 units at $S = 1.84$–3.91. Cost constants are assumption-flagged; see Section 8.

Chip (blueprint)	**Process / memory**	**S**	**1×**	**2×**	**4×**
DeepSeek-V3 $V1$	N7 / LPDDR5X	1.79	**4,410**	8,821	17,641
Llama1-65B $V1$	N7 / LPDDR5X	1.74	4,591	9,182	18,363
Llama3-70B $V1$	N7 / LPDDR5X	1.54	5,528	11,057	22,114
Llama1-13B $V2$	N5 / HBM3	1.57	6,920	13,841	27,681
Llama1-13B $V1$	N7 / GDDR6X	1.42	6,641	13,283	26,566
DeepSeek-V3 $V2$	N5 / HBM3	1.24	13,083	26,166	52,332
Llama1-65B $V2$	N5 / HBM3	1.20	15,079	30,158	60,317
Llama3-70B $V2$	N5 / HBM3	1.15	19,237	38,473	76,947

Figure 9: FAST-parity 1×-ROI break-even deployment volume per co-designed blueprint (convention F), from Eq. (20). The shaded band marks FAST's own reported range (2,164–3,534 units); our best chip (DeepSeek-V3 $V1$, 4,410 units) lands just above it, and drops to ∼3,650 units (*inside* FAST's range) when evaluated at FAST's implied ∼\$42M total NRE. The N7+commodity-memory ($V1$) family is uniformly ROI-preferred over N5+HBM3 ($V2$).

design on TSMC N7 with LPDDR5X, $S = 1.79$), Eq. (20) gives a 1×-ROI break-even of **4,410** units (Table 14, Fig. 9). This sits at the *same order of magnitude* as FAST's own reported 2,164–3,534 units (their Table 4, at $S = 1.84$–3.91), and well within the thousands-to-tens-of-thousands scale of modern accelerator fleets. As Eq. (20) makes explicit and Fig. 9 shows, deployment volume dominates ROI: the ranking of chips by break-even volume is driven jointly by S and by NRE, and the N7-plus-commodity-memory family ($V1$) is uniformly ROI-preferred over the N5-plus-HBM3 family ($V2$), which carries ∼\$15M more mask NRE, 1.7–2.5× the unit TCO, and (except on the 13B) *lower* Perf/TDP, pushing its break-even to 13,000–19,000 units.

Reconciling the NRE. The NRE terms in Eq. (20) carry the largest uncertainty, and we flag them as assumptions [A] rather than measured quantities: the mask-set cost (\$15M at N7, \$30M at N5), the IP/PHY-licensing cost (\$10M, ASIC-Clouds-class [Magaki et al., 2016]), the defect density D_0 used in the die-yield model, and the PUE all follow public industry-class estimates, not vendor data. FAST published only its engineering-NRE constant, so rather than assert NRE parity we *reconcile*: back-computing from FAST's own Table 4 break-even volumes and reported S implies a FAST total NRE of ∼\$42.0M. Our best chip carries a larger, deliberately conservative \$50.7M; re-evaluating Eq. (20) at FAST's implied ∼\$42M moves our best-chip break-even from 4,410 to ∼**3,650** units, now *inside* FAST's own 2,164–3,534 range. The Perf/TDP proxy S is independent of these [A] constants except through NRE, so the headline break-even row is robust to them: doubling the mask cost alone moves the 1×-ROI volume only to ∼5,700. We also disclose the honest error bar from the proxy itself: a fully modeled TCO on both sides (convention M) inflates break-even to ∼79,000–108,000 units (an ∼18× spread), which is precisely why FAST, and we, headline Perf/TDP and cite the Perf/TCO correlation rather than claim TCO precision.

The zero-NRE deployment option. A distinct and strictly cleaner deployment story requires *no silicon at all*. Because CHEETAH's scheduler is a drop-in software layer, applying the joint LP schedule to the *existing* TPU-v3 fleet (same chip, same TDP) delivers $S = 2.14$–2.51× Perf/TDP on the LLM workloads (2.21× on EfficientNet-B7 [Tan and Le, 2019], 1.50× on the Stable-Diffusion U-Net) at software-only engineering cost, with $C_{\text{mask}} = C_{\text{IP}} = 0$ (Fig. 10). Under Eq. (19) the NRE collapses to a software term and the ROI exceeds any silicon option at every volume. This is a genuine, low-risk path to fleet-wide efficiency that does not depend on any of the [A]-flagged silicon constants above.

Baseline stratification: three denominators, never mixed. Our S and FAST's S are *not measured against the same baseline*: CHEETAH's speedup is measured against a naive streaming schedule on the reference hardware, whereas FAST's is measured against a production-calibrated simulator.

Figure 10: Perf/TDP (FAST's TCO proxy) of the two deployment options against a TPU-v3 baseline of 1.0. Co-designed silicon retains 1.15–1.79× after normalizing away MAC scale; deploying CHEETAH *scheduling* on the existing TPU-v3 fleet delivers 2.14–2.51× at zero silicon NRE. The dashed band marks the 0.46–0.84× obtained when the blueprints are measured against a *joint-scheduled* TPU-v3 (denominator (D3)) rather than a naive one, and the marker at 3.00–3.21× is CHEETAH-iso against that same denominator (Section 8).

Table 15: CHEETAH-iso (65,536 MACs = TPU-v3, 0.82× area, 128 GB/s) Perf/TDP against the three denominators of Section 8, with the 1×-ROI break-even volume from Eq. (20) computed at the (D3) figure: the strongest baseline, a TPU-v3 running CHEETAH's own joint schedule. NRE is held at the N7 family's \$50.7M even though the die is smaller, which is conservative. FAST's Table 4 reports 2,164–3,534 units. Richer TDP convention ($\mathrm{TDP} = \mathrm{MACs} \cdot 2.3\times10^{-4} + 0.07\,\mathrm{W}/(\mathrm{GB/s}) \cdot \mathrm{BW}$) applied to both sides.

Workload	(D1) c0	(D2) stream	(D3) joint	1× break-even	In FAST range?
EfficientNet-B7	6.96×	3.36×	**3.21×**	2,827	yes
DeepSeek-V3	4.95×	3.05×	**3.00×**	2,919	yes

Rather than carry a single S and a caveat, we report Perf/TDP against three denominators of increasing strength and tag every number with the one it uses:

(D1) **c0-naive.** TPU-v3 running its deployed per-operation configuration. This is the FAST-comparable "TPU-v3 as deployed" convention.

(D2) **stream-naive.** The per-operation streaming optimum $\sum_i \min_c \max(C_{i,c}, \alpha B_{i,c})$: the strongest capacity-unaware analytic baseline.

(D3) **joint-scheduled.** A TPU-v3 running *CHEETAH's own* joint LP schedule. This is the well-scheduled denominator, and it is the correct hurdle rate for any silicon proposal, because Section 8 shows that schedule is available on the existing fleet at zero silicon NRE.

Against (D3) the co-designed *blueprints* of Appendix I do not clear the bar, and we state the arithmetic rather than a rounded caveat. Their $S = 1.15$–$1.79\times$ is measured against (D1), while the same schedule on unmodified TPU-v3 already buys 2.14–2.51×; the blueprints therefore sit at 0.46–0.84× against (D3), $S - 1 < 0$, and Eq. (20) admits *no break-even at any deployment volume*. That is the correct reading of those chips, and it is not a defect of the formulation: the blueprints are provisioned for *throughput*, which Appendix E.2 shows is where 20.0–26.9× of their headline comes from, and raw throughput is not what a Perf/TDP proxy rewards. The blueprints' deployment case rests on per-chip throughput, latency, and the consequent reduction in fleet size and footprint, mirroring FAST's own observation that latency and model-enablement can justify even break-even ROI. It does not rest on Perf/TDP, and we no longer claim that it does.

The iso-resource design does clear the bar. Optimizing Perf/TDP directly, under hard caps at TPU-v3's own area and TDP budget, selects a different chip. **CHEETAH-iso** carries the *same* 65,536 MACs as TPU-v3 at 0.82× its area (73.83 mm^2 against an 89.83 mm^2 cap), and provisions 128 GB/s of GDDR6 where TPU-v3 provisions 900 GB/s of HBM. Because it has no additional MACs, the provisioning factor f_2 of Appendix E.2 is eliminated *by construction*, so what it reports is attributable to the optimization rather than to silicon scale. Table 15 evaluates it against all three denominators.

Two properties make this the number we would have a reader carry. First, it survives the hurdle rate: at $S = 3.00$–$3.21\times$ against (D3), Eq. (20) gives a 1×-ROI break-even of **2,827–2,919** units, *inside* FAST's own reported 2,164–3,534 range, and we hold NRE at the larger family's \$50.7M rather than crediting the smaller die. Second, the design was recovered independently by multiple search arms (all four on DeepSeek-V3; arms 1, 2, and 4 on EfficientNet-B7), so it does not depend on the LLM-guided arm whose attribution we bound in Appendix G.1.

Where the iso-resource win comes from, and how fragile it is. The mechanism is bandwidth provisioning, not compute: with the same MAC count and both sides joint-scheduled, the performance ratio is ≈1 and essentially all of the 3.21× is the TDP denominator (78.07 W → 24.03 W, a 3.25×

ratio). The joint schedule raises arithmetic intensity enough that 128 GB/s suffices where TPU-v3 provisions 900 GB/s, and the memory power follows. This makes the result sensitive to one [A]-flagged constant, the $0.07\,\mathrm{W}/(\mathrm{GB/s})$ memory-power coefficient, so we report its sensitivity rather than assert it: the TDP ratio falls to $2.80\times$ at 0.05, $2.38\times$ at 0.035, $1.88\times$ at 0.02, and $1.47\times$ at 0.01, reaching parity only as the coefficient reaches zero. The conclusion that CHEETAH-iso beats a joint-scheduled TPU-v3 on Perf/TDP is therefore robust to a $7\times$ error in that constant; the specific break-even volume is not, and should be read as an order-of-magnitude figure.

9 Implementation-Level Validation: Cycle-Accurate RTL

The results so far certify *model-level* quantities computed inside one cost-model convention; as we note in Appendix D.2, a real measured utilization would require an executable implementation of the selected hardware/schedule pair. This section supplies that check. We take the committed schedule and the datapath geometry the LP sizes, express them as *synthesizable* SystemVerilog, simulate them cycle-accurately with Verilator 5.050 [Snyder and contributors, 2025], and compare against a reference model written in C++. We call this *analytical-model-to-RTL cycle validation*: the RTL confirms that the LP's schedule is physically realizable, that its time-indexed memory constraints hold byte-for-byte in hardware, and that the utilization *mechanisms* the LP charges are the ones a real datapath exhibits. We do *not* treat RTL cycle counts as absolute ground truth (the analytical per-layer cache floors are uniformly conservative by $192\times$ to $2475\times$, quantified below), and we make no tape-out claim. Section 9.1 then closes the remaining gap on the cost-model side, taking the sized datapath through logic synthesis and placement at a 7 nm predictive PDK to test the area and power coefficients themselves.

Experiment 1: schedule tape replay (realizability and memory legality). We compile the full committed DeepSeek-V3 [DeepSeek-AI, 2024] schedule at a TPU-v3-class configuration ($C_{\mathrm{GM}} = 8$ MiB) into a 1,383-instruction event tape over the vocabulary {EXECUTE, LOAD, FREE, EVICT, RELOAD, REMATERIALIZE}, and execute it on a cycle-accurate SystemVerilog event engine (a Global-Buffer allocator, a bandwidth-limited DMA model, and a compute model) under Verilator. Each instruction is cross-checked against the analytical residency model layer-by-layer, and a deliberately over-committed tape confirms the capacity checker is live (it flags exactly one violation). Three pre-stated criteria pass (Table 17), and Table 16 reports, decision by decision, that every planned event executed. *Memory legality:* the tape runs to completion with **zero** capacity violations, and the RTL peak on-chip occupancy equals the analytical prediction *exactly* (7,602,176 B $= 7.25/8$ MiB, 9.4% headroom). *Cycles:* the total RTL cycle count lands within $+0.000001\%$ of the rounded LP objective, the entire gap being the 16,431 decoder/DMA sequencing cycles (10^{-8} of the total); i.e., the rounded schedule realizes with *no hidden serialization or stall* beyond the model's accounting. *Two-tape ratio:* against a stream-everything realization of the same operation order, the RTL-measured latency ratio is $1.0723\times$, matching the analytical ratio to 8 significant digits. This ratio is scoped to the pin/reload channel alone (both tapes hold the per-operation-optimal tiling fixed, so joint tile selection, the dominant channel elsewhere, is deliberately held equal). Together these establish that the LP's time-indexed residency model (Section 5.1) is not merely roofline-feasible but executes byte- and cycle-faithfully on hardware that enforces the capacity constraint at every step.

Experiment 2: datapath utilization mechanism. We next build one weight-stationary INT8 systolic PE cluster in synthesizable SystemVerilog (4×4 MAC array, INT8×INT8→INT32, with L1 buffers and a tile-loop FSM; plus a 16×16 variant), golden-verified against a plain-C++ reference convolution ($8/8$ runs bit-exact). Simulated on EfficientNet-B7 [Tan and Le, 2019] layers, the RTL reproduces the two utilization mechanisms the cost model charges for. First, the *depthwise structural ceiling*: on a broadcast systolic array a depthwise convolution can engage only 4 of 16 MAC lanes, and the RTL measures a steady-state depthwise utilization of 24.9% against the 25% analytic ceiling (6.13% vs. 6.25% on the 16×16 variant), while dense 1×1 layers hold 97–99% on the identical RTL. Second, the predicted *geometry contrasts*: the measured lane law reproduces the $4.00\times$ systolic-vs-grid contrast to $4.057\times$, and extends the cache's $18\times$ depthwise contrast to within $1.22\times$. Crucially, the same runs show the analytical cache cycle *floors* to be $192\times$ to $2475\times$ conservative (the RTL executes the identical work that much faster), which is precisely why RTL-derived claims are scoped to these utilization *contrasts* and to the schedule replay above, never to absolute cycle agreement.

Table 16: Did each decision fire? Per-decision plan-versus-execution counters from the DeepSeek-V3 tape replay ($C_{\text{GM}} = 8$ MiB). Every event the LP plans is executed by the RTL engine exactly once, the capacity row is checked in hardware at every step, and the decisions the LP chose not to use at this cell are confirmed absent while their opcodes are proven live on a synthetic tape. The bound says how good the plan is; the counters say the plan is what ran.

Decision	LP plan	RTL executed	Check
Tile choice (y)	903 EXECUTE	903	✓ ids match 903/903
Residency (p, p^W)	180 LOAD, 299 FREE, 2,709 pins	180, 299	✓ 2,709/2,709 pins
Fusion (operand kept resident)	418 of 903 ops	418	✓
Eviction / reload	0 / 0	0 / 0	✓ opcodes live
Rematerialization (r)	0	0	✓ opcode live
Capacity row	1,083 hardware checks	0 violations	✓ injected overflow flags 1
Peak occupancy	7,602,176 B	7,602,176 B	✓ byte-exact, trace within 4 B
Total cycles	1.6437×10^{12}	+16,431 cycles	✓ +0.000001%, accounted

Table 17: Analytical-model-to-RTL cycle validation. Synthesizable SystemVerilog simulated cycle-accurately with Verilator [Snyder and contributors, 2025] and checked against a C++ reference. The RTL confirms schedule realizability, byte-exact memory legality, and the LP's utilization mechanisms; it is *not* used as an absolute cycle oracle.

Quantity (RTL vs. analytical)	Measured	Verdict
Exp. 1: DeepSeek-V3 schedule replay (1,383 instr., $C_{GM} = 8$ MiB)		
Capacity violations	0	pass
Peak on-chip occupancy	7,602,176 B (exact)	byte-exact
Total cycles vs. rounded LP obj.	+0.000001%	no hidden serialization
Two-tape ratio (pin/reload only)	1.0723× (8-digit match)	pass
Exp. 2: EfficientNet-B7 INT8 systolic cluster (golden 8/8)		
Depthwise MAC-lane utilization	24.9% vs. 25% ceiling	mechanism confirmed
Systolic-vs-grid geometry contrast	4.057× vs. 4.00×	mechanism confirmed
Absolute cache cycle floors	192–2475× conservative	scope guard

The mechanism is what matters here: the utilization losses the LP models (Section 5) are the ones the real datapath exhibits.

9.1 Physical-design validation: post-synthesis area and the linearity of the cost model

Experiments 1 and 2 validate the *schedule* the program commits to. A second, independent question is whether the *cost model* that evaluates the hardware is itself faithful: the program sizes a chip through the coefficients k_{MAC} (area per MAC) and π_{MAC} (power per MAC), and every manufacturability gate and the area and power caps are expressed in them. If those coefficients are wrong, the certified designs are certified against the wrong physics. We therefore take the datapath the program sizes, write it as synthesizable RTL, and push it through a full open-source RTL-to-placement flow at an advanced predictive node.

Setup. We implement a parameterized weight-stationary INT8 systolic tile (an $N \times N$ array of processing elements, each holding a resident 8-bit weight, streaming 8-bit activations east and 32-bit partial sums south, one MAC per PE per cycle in steady state) and synthesize, floorplan, place, and timing-repair it with OpenROAD [Ajayi et al., 2019] against the ASAP7 7 nm predictive PDK [Clark et al., 2016] at a 1 GHz target, for $N \in \{16, 32, 64\}$, that is 256 to 4,096 MACs. We report post-detailed-placement extracted areas. We do not run clock-tree synthesis or routing, and we make no tape-out claim: the quantity under test is the area coefficient and its scaling law, both of which are fixed at placement.

Result 1: the linear area model is structurally correct. Table 18 reports extracted cell area at the three sizes. A linear fit gives $A_{\text{cell}} = 82.08\,\text{MACs} - 206.5\ \mu\text{m}^2$ with $R^2 = 0.999999$, and the extracted area *per MAC* varies by only 0.46% across a 16× range in MAC count. This is a stronger statement than agreement on a single number: the program assumes area is *linear* in the MAC count,

with the same slope at every chip size, and that assumption is exactly what a placed datapath exhibits. The linear area row in the program is therefore not a convenient approximation but a measured property of the datapath it sizes.

Result 2: the area coefficient is anchored and conservative. The fitted slope is 8.21×10^{-5} mm^2/MAC of standard-cell area. Placed designs occupy a core larger than their cell area; our floorplans sit at 41% placement utilization, giving 2.0×10^{-4} mm^2/MAC of core area. The value the program uses, $k_{\mathrm{MAC}} = 1.5 \times 10^{-4}$ mm^2/MAC, lies between these two and corresponds to the cell area at 54.7% placement utilization, a typical density for a datapath of this kind. The coefficient is thus bracketed by extracted silicon area from below and by our own conservative floorplan from above.

Result 3: the power coefficient, and why we anchor it to silicon rather than to this flow. Power at the same operating point is instructive in a different way. Reporting placement-level power on this tile with flat random switching activity yields 2.0 to 6.5 pJ per MAC operation as the assumed input activity rises from 5% to 50%, against the 0.40 pJ/MAC-op implied by π_{MAC}. Rather than adjudicate this within the flow, we test it against an external physical bound. Shipped accelerators deliver INT8 arithmetic at 0.35 pJ/op (H100, 4 nm), 0.64 pJ/op (A100, 7 nm), and 1.27 pJ/op (TPU-v4i, 7 nm), and these are *whole-chip* figures that include memory, cache, and control. Our number is for the multiplier array alone, at the same node, and exceeds an entire A100 per operation by 3.2–10.1×; a bare array cannot consume more energy per operation than the complete system that contains it. The measurement, not the coefficient, is therefore out of range: with a generic synthesized multiplier, no clock gating or operand isolation, and flat random activity in place of correlated INT8 data, this flow reports an upper bound rather than a representative figure. The program's $\pi_{\mathrm{MAC}} = 0.40$ pJ/MAC-op sits where an array-only coefficient should, at the same order as and slightly below whole-chip shipped silicon, and we anchor it there (Appendix O).

Synthesis of the three hardware verification results. Design-space-exploration frameworks are conventionally validated in three ways: against measured taped-out chips, against post-synthesis extraction, and against other frameworks [Mei et al., 2021]. The Timeloop-derived Pareto data places this work in the third category and the silicon-anchored coefficients in the first; this section supplies the second. Concretely, it converts the area side of every gate-legality claim in this paper, and of the certified designs in Section 7, from a literature-shaped constant into a quantity extracted from RTL-to-placement at 7 nm, together with a validated scaling law; and it leaves the power side anchored to shipped silicon, with an explicit account of why this particular flow overstates it.

Table 18: Post-placement extracted area of the INT8 systolic tile at ASAP7 7 nm. Area per MAC is constant to within 0.46% over a $16\times$ range in MAC count; the linear fit has $R^2 = 0.999999$.

N	MACs	Cell area (μm^2)	Cell area/MAC (mm^2)	Core area/MAC (mm^2)
16	256	20,981	8.196×10^{-5}	1.986×10^{-4}
32	1,024	83,629	8.167×10^{-5}	1.986×10^{-4}
64	4,096	336,049	8.204×10^{-5}	1.998×10^{-4}
Linear fit ($R^2 = 0.999999$)			8.208×10^{-5}	
Program value k_{MAC}			1.5×10^{-4} (cell area at 54.7% utilization)	

9.2 Measuring the compute-energy coefficients

The area result above validates k_{MAC}. The program also charges for compute *energy*, and one property of that dual-guided scoring deserves a measurement rather than an assumption. Reducing arithmetic precision is usually modelled as a reduction in bytes moved, and in an earlier form of our program precision affected memory traffic alone. That is incomplete: a narrower multiplier also switches less, so precision should reduce compute energy as well. We add that term, and because a multiplier's energy is often argued to follow its partial-product count, we could have set the ratio analytically. Instead we measured it.

Method. We synthesize and place six variants of the tile at ASAP7: three operand-width combinations (W8A8, W4A8, W4A4) crossed with two power-management levels (ungated, and clock-gated

with operand isolation). Power is reported at a matched input activity so that the ratios are comparable. As a check on the flow, the W8A8 ungated variant reproduces the independently synthesized tile of Table 18 to within 0.1% despite a different source file, module name and container.

Result: the analytic ratio is wrong in both directions. Table 19 reports the measurements. The partial-product model predicts energy ratios of 0.50 for W4A8 and 0.25 for W4A4 against the W8A8 baseline. The measured ratios are 0.376 and 0.358. Narrowing the *weight* saves more than the model predicts, because switching concentrates in the multiplier tree; narrowing the *activation* saves far less, because the 32-bit accumulator and the partial-sum pipeline registers do not narrow with the operands, so the second halving buys only a further 5%. We therefore charge compute energy with the measured ratios rather than the analytic ones. The $\mathrm{W4A16}$ entry is not directly measured and is held at a conservative 0.5 rather than extrapolated in our own favour.

A negative result on gating, and why we do not include it. The same matrix was intended to characterize a clock-clock-gating decision, and it cannot. Under a flat synthetic activity model the gated variants measure as *more* expensive than the ungated ones ($1.40\times$ the power at W8A8), which is not physical. The cause is that a flat activity assumption charges for the gating cells while never crediting the idle cycles they exist to skip; the benefit of gating is visible only under a realistic idle pattern, which requires switching activity captured from a simulation of the actual workload. The area cost of gating is measured and real ($+22\%$ at W4A8, $+20\%$ at W4A4), but without a credible power benefit to weigh against it we do not introduce the decision. We report the attempt because the failure is informative: it delimits what this class of measurement can and cannot establish.

Scope. These measurements sharpen a coefficient; they do not change any reported speedup, and the reason is visible in the dual variables (Section 10.2): the power constraint is slack at every design we report, so by complementary slackness a change that relieves it cannot move the optimum. The value of the measurement is provenance. The best measured variant consumes 1.22 pJ per MAC operation against the 0.40 implied by π_{MAC}, a factor of three rather than the order of magnitude that an ungated tile driven by random data suggested, with the residual attributable to predictive-PDK pessimism, a generic rather than optimized multiplier, and synthetic activity. We continue to anchor π_{MAC} to shipped silicon.

Table 19: Measured compute energy and area of the tile at ASAP7 7 nm ($N = 16$, 1 GHz, matched input activity). Ratios are against the W8A8 ungated variant. The analytic partial-product model predicts 0.50 and 0.25 for the W4A8 and W4A4 energy ratios; both are wrong.

Variant	Power (W)	pJ/MAC-op	Energy ratio	Area ratio
W8A8, ungated	0.872	3.41	1.000	1.001
W4A8, ungated	0.328	1.28	**0.376**	0.726
W4A4, ungated	0.312	1.22	**0.358**	0.625
W8A8, gated	1.220	4.77	1.399	–
W4A8, gated	0.347	1.36	0.398	0.887
W4A4, gated	0.312	1.22	0.358	0.752

10 Discussion

10.1 Which mechanism is doing the work

The speedups above are produced by three mechanisms acting together: sizing the hardware, reducing bytes through the precision, sparsity and memory-technology decisions, and amortizing the reload through speculation. Because each is a separate certified design in our formulation, we can decompose the result rather than attribute it: we solve the same program with the mechanisms enabled cumulatively, under one search configuration, and read off what each contributes. Table 20 reports the decomposition, and its totals agree with Table 9 to within $0.02\times$ at every batch.

The dominant mechanism is not fixed; it shifts with the serving regime. At single-stream decoding, hardware sizing carries 55% of the gain and speculation only 20%: the reload is small, so provisioning compute and bandwidth is what buys speed. By $B = 128$ this has inverted, with hardware carrying 7% and speculation 68%. The byte-reduction decisions are the steady middle throughout, between

25% and 34%. This is the same physics as the widening of $K^\star$, seen from the other side: a larger batch gives each verification pass a larger reload to amortize, so the amortizing mechanism displaces the provisioning one. It also explains an observation in Table 10 that is otherwise surprising, namely that a single chip configuration (24.4× MACs, 3200 GB/s, 422 W) is selected at $B = 8$, 32 and 128 alike: beyond low batch, additional silicon is not what buys the speedup.

The methodological consequence matters more than the individual percentages. A sequential pipeline that fixes the hardware first and then schedules onto it commits to a provisioning decision before knowing which mechanism the regime will reward, and would over-provision silicon at high batch while under-speculating everywhere. Deciding all three jointly is what allows the program to move the weight between them as the regime changes, and the decomposition here is the direct evidence that the weight does move.

Table 20: Cumulative certified stages and each mechanism's share of the total gain above 1×, under one search configuration. Each stage is a separately certified achievable design, so the shares decompose solved results rather than estimating an attribution.

	Cumulative speedup				Share of total gain		
B	hardware	+ decisions	+ spec. $K^\star$	$K^\star$	hardware	decisions	speculation
1	18.68×	26.55×	33.13×	2	**55.0%**	24.5%	20.5%
8	6.67×	18.91×	36.76×	8	15.9%	34.2%	49.9%
32	4.73×	16.49×	44.85×	16	8.5%	26.8%	**64.7%**
128	4.32×	16.06×	48.64×	32	7.0%	24.7%	**68.4%**

10.2 Dual variables show what actually binds

A certified optimum carries more information than its objective value. The dual variable on each constraint is the rate at which the objective would improve if that constraint were relaxed, and by complementary slackness a constraint with a zero dual is not limiting the design at all. Reading them tells us what the program is really up against, and the answer is unambiguous.

At the selected design the dual on the speculative verification pass is 129. The dual on the area cap is 8.7×10^{-5}, on the decode-traffic rows 7.1×10^{-5}, and on the mixture-of-experts residency knapsack 1.2×10^{-5}. The duals on the *power cap* and on the *pin-count cap* are both exactly zero. The verification pass, which is to say the decode memory wall, dominates every physical cap on the chip by roughly six orders of magnitude. These designs are limited by the traffic they must move, not by the silicon, the package or the power budget available to move it.

Two consequences follow, and we state them because each is easy to get wrong. First, a cap can be nearly saturated without binding. The pin cap admits 3277 GB/s and the selected design runs at 3200 GB/s, so it sits at 97.7% of the limit; nevertheless its dual is zero, because the constraint's effect is to remove the wider bandwidth branches from the menu rather than to be tight at the chosen one. High utilization of a resource and scarcity of that resource are different statements, and only the dual distinguishes them.

Second, this predicts which modelling refinements can matter. Any change that relieves a zero-dual constraint cannot move the optimum, whatever its physical merit. When we refined the program to charge a precision-dependent compute energy (Section 9.2), a correction we regard as necessary for the model to be right, the certified designs did not move at all, at either end of the batch range. That is not a null result to be explained away; it is what complementary slackness requires, and it was predictable from the dual vector before the experiment was run. The same reasoning tells us that a clock-clock-gating decision would be inert here for the same reason, which is why we do not introduce one. The decisions that move these designs are the ones that reduce decode traffic, and the duals say so before any of them is tried.

10.3 Solving the Large Scale Systems Problem

One of the central challenges in co-design optimization as noted in prior work [Zhang et al., 2022] is the limitations of existing solvers in handling the simultaneous extreme large scale and extreme ill-conditioned LPs inherent in the systems setting. At LLM scale the raw constraint matrix mixes

unit-magnitude selection rows with execution rows whose coefficients reach 10^{10} cycles: a coefficient dynamic range of $\sim$10 orders of magnitude, and hence an effective conditioning of $\kappa \gtrsim 10^{10}$ that unscaled first-order methods cannot absorb (Section 6). The joint tiling, fusion, placement, and rematerialization LP model for Llama-3 70B and DeepSeek-V3 has roughly 1.6 M–2.5 M decision variables, 1.9 M–3 M constraints, and 4 M nonzeros at minimum. Discretization of $L \approx 720$–903 layers, $C = 16$–20 pareto tile configs, and $T = L$ timesteps yield more performant fine-grained optimization, but also presents increasingly challenging problem formulations for solvers to find convergent solutions. At this structure and scale, simplex and barrier solvers exhaust factorization memory, since traditional solvers were not engineered to refactor sparse Jacobians of this shape.

CHEETAH LP Structure. Extreme matrix ill-conditioning in the systems modeling setting is unavoidable. Timeloop per-tile cycle counts $T_{\min}(c)$ and $T_{\max}(c)$ span 10^3–10^{10} cycles. The McCormick-envelope variables inject products of these tile counts with normalized residency variables, so absolute matrix entries span $\sim 10^7$ in magnitude. The row-norm ratio exceeds well past 10^6, and practically exceeds what generic interior-point methods can tolerate without diverging. However, this combination of scale $\times$ conditioning $\times$ solve speed indicates a regime where first-order methods of the primal-dual hybrid gradient family (PDHG; Chambolle and Pock [2011]) is the most feasible and credible path. PDHG eliminates matrix factorization entirely, costs $\mathcal{O}(\text{nnz})$ per step, and converges as long as the operator can be preconditioned and step sizes chosen suitably and adaptively.

Custom solver design. We implement a custom variant of the PDHG solver in JAX. This is necessary since a custom built pre-conditioner and scaling regime is needed to successfully solve the ill-conditioned systems LP to convergence. This also helps reduce the co-design iteration time from days to hours. PDHG is dominated by sparse matvecs and elementwise projection operations that vectorize cleanly under XLA. Our custom solver implementation utilizes JAX's Just in Time (JIT) compilation and `lax.scan` over the inner iteration to batch $\sim 10^5$ iterations onto the GPU/TPU at a few microseconds each. This also enables JIT-fusion of the Ruiz preconditioning [Ruiz, 2001] loop into a single kernel.

Naive JAX implementation of the PDHG solver for LPs [Applegate et al., 2026] does not converge on CHEETAH systems LPs. This is unsurprising since first-order methods frequently cause the duality gap to oscillate and the standard symmetric-restart heuristic kept overshooting on our ill-conditioned problems. To solve this, we introduce a custom variable-metric one-sided rescue method (**VM-One-Armed-Rescue**) to produce convergence. This novel asymmetric, one-sided method incorporates Halpern-reflection [Halpern, 1967] restarts in PDHG with a diagonal variable metric especially designed for CHEETAH-V1's large-scale systems LP structure. Additional solver design details are described in Appendix C.4. This enables the primal and dual convergence of the systems LPs within $\sim$80–200 s at feasibility tolerance 2.5×10^{-7}, reaching within 0.1–1% of the mature HiGHS optimum (the rounding gap is bounded and reported in Appendix F). In contrast, existing PDHG solvers and off-the-shelf HiGHS either time out, factorize into NaNs, or polish to objectives 5–12$\times$ worse without achieving dual convergence. The primal and dual convergence of our custom systems solver is also what enables the dual variable signal in the LLM Architect outer loop (Section 5.4).

10.4 CHEETAH Design Blueprint Discussion

The pre-fusion memory stall is the modeled fraction of unfused execution time spent waiting on DRAM before fusion, tensor pinning, and placement reduce traffic. Formally,

$$\text{stall}_{\text{pre}} = \frac{T_{\text{pre}} - T_{\text{compute}}}{T_{\text{pre}}}, \qquad T_{\text{pre}} = \max(T_{\text{compute}}, T_{\text{DRAM,pre}}).$$

Here T_{compute} is the pure compute floor assuming operands are available on chip, and $T_{\text{DRAM,pre}}$ is the DRAM service time of the unfused schedule. Thus a pre-fusion stall of 97.5% indicates that the unfused schedule is dominated by DRAM memory-stalled, and only 2.5% of the modeled pre-fusion wall time is practically useful compute.

We write $\text{stall}_{\text{pre}}$ and $\text{stall}_{\text{post}}$ to denote the memory-stall fractions before and after fusion, respectively. Fusion efficiency measures the fraction of avoidable pre-fusion memory stall that fusion and placement can unblock:

$$\text{fusion efficiency} = \frac{\text{stall}_{\text{pre}} - \text{stall}_{\text{post}}}{\text{stall}_{\text{pre}}}.$$

Table 21: Two CHEETAH-V1 accelerator designs showing distinctly different regimes. The Llama-3 70B design is an efficient and balanced variant, contrasting the DeepSeek-V3 design which is the bandwidth-rich variant. All designs presented pass the six pre-RTL manufacturability gates roofline checks.

	CHEETAH-V1 Llama-3 70B Efficient	**CHEETAH-V1 DeepSeek-V3 Bandwidth-Rich**
Process	TSMC N7	TSMC N5
Memory system	LPDDR5X (8 packages)	HBM3 (3 stacks)
Memory capacity	128 GB	72 GB
Memory peak BW	544 GB/s	2,457 GB/s
Die area	425 mm^2	485 mm^2
TDP (compute + memory IO)	392 W	434 W
Peak compute @ 1 GHz	3,539 TOPS (INT8)	2,621 TOPS (INT8)
PE grid	$48 \times 36 = 1,728$ PEs	$32 \times 40 = 1,280$ PEs
PE systolic array dims	32×32	32×32
PE vector width	64 lanes	64 lanes
PE L1 buffer size (in / wt / out)	16 / 16 / 16 KB	16 / 16 / 16 KB
PE L1 config	Private	Private
PE L2 config	Shared	Shared
Global buffer size	64 MiB	64 MiB
Pre-fusion mem stall	97.5%	97.2%
Fusion efficiency	100.0%	100.0%
OpInt ridgepoint	6,505 ops/byte	1,066 ops/byte
Fused model OpInt	2.07×10^6 ops/byte	1.46×10^6 ops/byte
$T_{\text{DRAM}}/T_{\text{compute}}$ at actual BW	3.14×10^{-3}	7.29×10^{-4}
Binding constraint at LP optimum	compute	compute
Achieved speedup vs. naive TPU-v3	$\mathbf{40.16}\times$	$\mathbf{35.68}\times$
LP Solver	`custom JAX PDHG`	`custom JAX PDHG`
LP Solve Time	13.4 s	21.1 s

Therefore, the combination of 97.5% pre-fusion stall and 100.0% LP fusion efficiency in Table 21 signifies that the unfused schedule is largely DRAM memory-stalled, which CHEETAH alleviates by finding a fused placement schedule for which the post-fusion DRAM floor falls below the compute floor. This is validated in the post-fusion roofline audit, where $T_{\text{DRAM}}/T_{\text{compute}} \ll 1$, so the LP optimum is compute-bound. This is one of CHEETAH's key advantages: by reducing the idle DRAM-wait time through optimized global buffer and unified co-design configurations to unlock aggressive fusion, raising overall operational intensity.

11 Conclusion

We present CHEETAH, a HW/SW co-design framework that combines a strict LP inner loop for physically constrained rules that must not be violated, with an LLM Architect guided outer loop that uses LP duals and trial diagnostics to effectively design high-performance accelerators. To our knowledge, CHEETAH is the first LP-based inference accelerator co-design framework to jointly optimize tiling, fusion, dynamic placement, scheduling, and rematerialization in one formulation. Our framework is composed of two key ideas: LPs excel in solving mathematically modeled problems to proven optimality, and LLMs excel at aggregating vast amounts of information. In this work, we combine these ideas, and delegate the roles in co-design between these two pieces. The division of labor is strict: the solver owns feasibility and the optimality certificate, while the LLM aggregates diagnostics (converged duals, binding constraints, audit verdicts) into search-space interventions. The LLM proposes; the LP certifies. No LLM suggestion can violate a physical constraint, because suggestions only reshape the neighborhood handed to the certified inner problem. CHEETAH is able to model global design trade-offs that traditional sequential pipelines cannot unlock, while the outer loop further improves efficiency by steering a hardware configuration tool to suggest promising search neighborhoods. Our CHEETAH-V0 and CHEETAH-V1 LP reformulations progressively unify placement, rematerialization, and tiling–fusion into a single pass, scaling to 2.5M variables and 6.1M nonzeros on DeepSeek-V3 while preserving the structural invariance needed for efficient

solver preconditioning across hardware search trials. The largest gains are seen on architectures with diverse layer types where joint tiling-fusion optimization is most impactful.

This manuscript presents a formal and empirical demonstration that sequential tiling-then-fusion loses solutions, together with a joint optimization method that recovers those solutions in precisely characterized operating regimes. Sequential accelerator pipelines make locally optimal mapping decisions before they know the global memory schedule; CHEETAH exposes tile choice, tensor residency, and rematerialization to one optimization problem. We prove that every in-scope sequential map-then-fuse pipeline, lifted FAST included, is contained in CHEETAH's feasible set (Theorem 1), isolate a fixed-hardware free-y gain on Llama-3 70B ($2.11\times$ under every sequential ordering vs. $2.51\times$ for the joint LP), and characterize a second separation regime in capacity-constrained DeepSeek-V3, where tile-dependent footprints admit schedules unavailable to both stock V1 and an oracle two-pass sequential baseline.

In sum, the paper contributes a new mathematical abstraction for global HW/SW co-design; an exact cross-stage tradeoff that FAST cannot express; a rounded, simulator-replayed validation proving the gains are not just an LP artifact; a decomposition of how the $40\times$ arises; and sensitivity sweeps showing it is not just more MACs. All utilization quantities we report are roofline-model diagnostics, not measured silicon utilization; real implementations would still incur NoC, SRAM, pipeline, and control overheads.

Future work can build from this framework in two meaningful ways: as an end-to-end co-design system for efficient accelerators, and as an automated optimization engine for existing hardware. By fixing the architecture, the inner loop derives optimal tiling and scheduling strategies that can act as custom speedups for targeted inference tasks.

References

A. Acharya, U. Bondhugula, and A. Cohen. Effective loop fusion in polyhedral compilation using fusion conflict graphs. *ACM Transactions on Architecture and Code Optimization*, 17(4):1–26, 2020.

T. Ajayi, V. A. Chhabria, M. Fogaça, S. Hashemi, A. Hosny, A. B. Kahng, M. Kim, J. Lee, U. Mallappa, M. Neseem, et al. Toward an open-source digital flow: First learnings from the OpenROAD project. In *Proceedings of the 56th Annual Design Automation Conference (DAC)*, 2019.

T. Andrulis, M. Gilbert, V. Sze, and J. S. Emer. Fast and fusiest: An optimal fusion-aware mapper for accelerator design. *arXiv preprint arXiv:2602.15166*, 2026.

D. Applegate, M. Díaz, O. Hinder, H. Lu, M. Lubin, B. O'Donoghue, and W. Schudy. Practical large-scale linear programming using primal-dual hybrid gradient, 2022. URL https://arxiv.org/abs/2106.04756.

D. Applegate, M. Díaz, O. Hinder, H. Lu, M. Lubin, B. O'Donoghue, and W. Schudy. Pdlp: A practical first-order method for large-scale linear programming. *arXiv preprint arXiv:2501.07018*, 2025.

D. Applegate, M. Díaz, O. Hinder, H. Lu, M. Lubin, B. O'Donoghue, and W. Schudy. Pdlp: A practical first-order method for large-scale linear programming. *Mathematical Programming Computation*, pages 1–45, 2026.

R. Baghdadi, J. Ray, M. B. Romdhane, E. Del Sozzo, A. Akkas, Y. Zhang, P. Suriana, S. Kamil, and S. Amarasinghe. Tiramisu: A polyhedral compiler for expressing fast and portable code. In *Proceedings of the IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, pages 193–205, 2019.

B. Bartan, H. Li, H. Teague, C. Lott, and B. Dilkina. Moccasin: Efficient tensor rematerialization for neural networks, 2023. URL https://arxiv.org/abs/2304.14463.

U. Bondhugula, A. Hartono, J. Ramanujam, and P. Sadayappan. A practical automatic polyhedral parallelizer and locality optimizer. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 101–113, 2008.

J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, Y. Katariya, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang. JAX: composable transformations of Python+NumPy programs, 2018. URL http://github.com/google/jax.

J. Cai, Z. Wu, S. Peng, Y. Wei, Z. Tan, G. Shi, M. Gao, and K. Ma. Gemini: Mapping and architecture co-exploration for large-scale DNN chiplet accelerators. In *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2024. arXiv:2312.16436.

A. Chambolle and T. Pock. A first-order primal-dual algorithm for convex problems with applications to imaging. *Journal of mathematical imaging and vision*, 40(1):120–145, 2011.

A. Chen, D. M. Dohan, and D. R. So. Evoprompting: Language models for code-level neural architecture search, 2023. URL https://arxiv.org/abs/2302.14838.

T. Chen, B. Xu, C. Zhang, and C. Guestrin. Training deep nets with sublinear memory cost, 2016. URL https://arxiv.org/abs/1604.06174.

Y. Chen, Z. Zhou, S. Dai, X. Duan, and J. Han. Multi-dof fusion: A flexible fusion strategy for reducing redundancy in cnn workloads. In *Proceedings of the Great Lakes Symposium on VLSI 2025*, GLSVLSI '25, page 805–810, New York, NY, USA, 2025. Association for Computing Machinery. ISBN 9798400714962. doi: 10.1145/3716368.3735173. URL https://doi.org/10.1145/3716368.3735173.

Y.-H. Chen, T.-J. Yang, J. Emer, and V. Sze. Eyeriss v2: A flexible accelerator for emerging deep neural networks on mobile devices, 2019. URL https://arxiv.org/abs/1807.07928.

L. T. Clark, V. Vashishtha, L. Shifren, A. Gujja, S. Sinha, B. Cline, C. Ramamurthy, and G. Yeric. ASAP7: A 7-nm FinFET predictive process design kit. In *Microelectronics Journal*, volume 53, pages 105–115, 2016.

Computing Research Association. How AI will reshape computer systems by 2035: A Jeffersonian dinner in San Francisco about our 10,000× future. CRA Industry Blog, Apr. 2026. URL https://cra.org/industry/2026/04/27/how-ai-will-reshape-computer-systems-by-2035-a-jeffersonian-dinner-in-san-francisco-about-our-10000x-future/. Accessed: 2026.

T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in neural information processing systems*, 35:16344–16359, 2022.

S. Dave, Y. Kim, S. Avancha, K. Lee, and A. Shrivastava. Dmazerunner: Executing perfectly nested loops on dataflow accelerators. *ACM Transactions on Embedded Computing Systems (TECS)*, 18 (5s):1–27, 2019.

DeepSeek-AI. Deepseek-v3 technical report, 2024. URL https://arxiv.org/abs/2412.19437.

P. Feautrier. Some efficient solutions to the affine scheduling problem. part I. one-dimensional time. *International Journal of Parallel Programming*, 21(5):313–347, 1992.

E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations (ICLR)*, 2023.

D. Golovin, B. Solnik, S. Moitra, G. Kochanski, J. Karro, and D. Sculley. Google vizier: A service for black-box optimization. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, Halifax, NS, Canada, August 13 - 17, 2017*, pages 1487–1495. ACM, 2017. doi: 10.1145/3097983.3098043. URL https://doi.org/10.1145/3097983.3098043.

A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

B. Halpern. Fixed points of nonexpanding maps. 1967.

C. Hong, Q. Huang, G. Dinh, M. Subedar, and Y. S. Shao. DOSA: Differentiable model-based one-loop search for DNN accelerators. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2023. doi: 10.1145/3613424.3623797.

C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer, and A. Gholami. KVQuant: Towards 10 million context length LLM inference with KV cache quantization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. URL https://openreview.net/forum?id=nzuA3J33_7.

Q. Huang, M. Kang, G. Dinh, T. Norell, A. Kalaiah, J. Demmel, J. Wawrzynek, and Y. S. Shao. Cosa: Scheduling by constrained optimization for spatial accelerators, 2021. URL https://arxiv.org/abs/2105.01898.

Q. Huang, P.-A. Tsai, J. S. Emer, and A. Parashar. Mind the gap: Attainable data movement and operational intensity bounds for tensor algorithms. In *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2024. Orojenesis: mapping-independent attainable off-chip traffic and operational-intensity bounds for tensor programs; used here to certify the compulsory-DRAM (attainable) floor.

Q. Huangfu and J. A. J. Hall. Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10(1):119–142, 2018.

P. Jain, A. Jain, A. Nrusimha, A. Gholami, P. Abbeel, K. Keutzer, I. Stoica, and J. E. Gonzalez. Checkmate: Breaking the memory wall with optimal tensor rematerialization, 2020. URL https://arxiv.org/abs/1910.02653.

S. Jia, Z. Ling, C. Bai, K. Zhao, and J. Zhai. Fadiff: Fusion-aware differentiable optimization for dnn scheduling on tensor accelerators, 2025. URL https://arxiv.org/abs/2511.22348.

N. P. Jouppi, C. Young, N. Patil, D. Patterson, G. Agrawal, R. Bajwa, S. Bates, S. Bhatia, N. Boden, A. Borchers, et al. In-datacenter performance analysis of a tensor processing unit. In *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*, pages 1–12. ACM, 2017. doi: 10.1145/3079856.3080246. Reports mean useful-MAC utilization and weight-fetch stall fractions for a production matrix-unit accelerator; also shows that enlarging the matrix unit can degrade delivered performance (sub-linear return on added MACs).

N. P. Jouppi, D. H. Yoon, M. Ashcraft, M. Gottscho, T. B. Jablin, G. Kurian, J. Laudon, S. Li, P. Ma, X. Ma, et al. Ten lessons from three generations shaped google's tpuv4i: Industrial product. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, pages 1–14. IEEE, 2021.

S.-C. Kao, G. Jeong, and T. Krishna. Confuciux: Autonomous hardware resource assignment for dnn accelerators using reinforcement learning, 2020. URL https://arxiv.org/abs/2009.02010.

H. Kwon, P. Chatarasi, M. Pellauer, A. Parashar, V. Sarkar, and T. Krishna. Understanding reuse, performance, and hardware cost of dnn dataflows: A data-centric approach using maestro, 2020. URL https://arxiv.org/abs/1805.02566.

W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.

J. Li, Z. Qin, Y. Mei, J. Cui, Y. Song, C. Chen, Y. Zhang, L. Du, X. Cheng, B. Jin, Y. Zhang, J. Ye, E. Lin, and D. Lavery. onednn graph compiler: A hybrid approach for high-performance deep learning compilation, 2024. URL https://arxiv.org/abs/2301.01333.

J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han. AWQ: Activation-aware weight quantization for LLM compression and acceleration. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024.

S. Liu, M. Cemri, S. Agarwal, A. Krentsel, A. Naren, Q. Mang, Z. Li, A. Gupta, M. Maheswaran, A. Cheng, M. Pan, E. Boneh, K. Ramchandran, K. Sen, A. G. Dimakis, M. Zaharia, and I. Stoica. Skydiscover: A flexible framework for ai-driven scientific and algorithmic discovery, 2026. URL https://skydiscover-ai.github.io/blog.html.

H. Lu and J. Yang. cupdlp.jl: A gpu implementation of restarted primal-dual hybrid gradient for linear programming in julia, 2024. URL https://arxiv.org/abs/2311.12180.

H. Lu, J. Yang, H. Hu, Q. Huangfu, J. Liu, T. Liu, Y. Ye, C. Zhang, and D. Ge. cupdlp-c: A strengthened implementation of cupdlp for linear programming by c language, 2024. URL https://arxiv.org/abs/2312.14832.

H. Lu, Z. Peng, and J. Yang. cupdlpx: A further enhanced gpu-based first-order solver for linear programming, 2025. URL https://arxiv.org/abs/2507.14051.

X. Ma, G. Fang, and X. Wang. DeepCache: Accelerating diffusion models for free. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

I. Magaki, M. Khazraee, L. V. Gutierrez, and M. B. Taylor. Asic clouds: Specializing the datacenter. *ACM SIGARCH Computer Architecture News*, 44(3):178–190, 2016.

G. P. McCormick. Computability of global solutions to factorable nonconvex programs: Part i: Convex underestimating problems. *Mathematical programming*, 10(1):147–175, 1976.

L. Mei, P. Houshmand, V. Jain, S. Giraldo, and M. Verhelst. Zigzag: Enlarging joint architecture-mapping design space exploration for dnn accelerators. *IEEE Transactions on Computers*, 70(8): 1160–1174, 2021.

L. Mei, K. Goetschalckx, A. Symons, and M. Verhelst. DeFiNES: Enabling fast exploration of the depth-first scheduling space for DNN accelerators through analytical modeling. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 570–583. IEEE, 2023. doi: 10.1109/HPCA56546.2023.10071098. arXiv:2212.05344.

O. Nnorom Jr., S. Boyd, and P. Levis. CvxCluster: Solving large, complex, granular resource allocation problems 100–1000x faster, 2026. URL https://arxiv.org/abs/2605.01614.

A. Novikov, N. Vũ, M. Eisenberger, E. Dupont, P.-S. Huang, A. Z. Wagner, S. Shirobokov, B. Kozlovskii, F. J. R. Ruiz, A. Mehrabian, M. P. Kumar, A. See, S. Chaudhuri, G. Holland, A. Davies, S. Nowozin, P. Kohli, and M. Balog. Alphaevolve: A coding agent for scientific and algorithmic discovery, 2025. URL https://arxiv.org/abs/2506.13131.

A. Parashar, P. Raina, Y. S. Shao, Y.-H. Chen, V. A. Ying, A. Mukkara, R. Venkatesan, B. Khailany, S. W. Keckler, and J. Emer. Timeloop: A systematic approach to dnn accelerator evaluation. In *2019 IEEE international symposium on performance analysis of systems and software (ISPASS)*, pages 304–315, 2019.

S. G. Patil, P. Jain, P. Dutta, I. Stoica, and J. E. Gonzalez. Poet: Training neural networks on tiny devices with integrated rematerialization and paging, 2022. URL https://arxiv.org/abs/2207.07697.

H. Peng, S. Davidson, R. Shi, S. L. Song, and M. Taylor. Chiplet cloud: Building ai supercomputers for serving large generative language models, 2024. URL https://arxiv.org/abs/2307.02666.

Qwen Team, :, A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei, H. Lin, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Lin, K. Dang, K. Lu, K. Bao, K. Yang, L. Yu, M. Li, M. Xue, P. Zhang, Q. Zhu, R. Men, R. Lin, T. Li, T. Tang, T. Xia, X. Ren, X. Ren, Y. Fan, Y. Su, Y. Zhang, Y. Wan, Y. Liu, Z. Cui, Z. Zhang, and Z. Qiu. Qwen2.5 technical report, 2025. URL https://arxiv.org/abs/2412.15115.

J. Ragan-Kelley, C. Barnes, A. Adams, S. Paris, F. Durand, and S. Amarasinghe. Halide: a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. *Acm Sigplan Notices*, 48(6):519–530, 2013.

B. Rashidi, C. Gao, S. Lu, Z. Wang, C. Zhou, D. Niu, and F. Sun. Unico: Unified hardware software co-optimization for robust neural network acceleration. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, pages 77–90, 2023.

B. Romera-Paredes, M. Barekatain, A. Novikov, M. Balog, M. P. Kumar, E. Dupont, F. J. Ruiz, J. S. Ellenberg, P. Wang, O. Fawzi, et al. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024.

D. Ruiz. A scaling algorithm to equilibrate both rows and columns norms in matrices. Technical report, CM-P00040415, 2001.

Y. S. Shao, J. Clemons, R. Venkatesan, B. Zimmer, M. Fojtik, N. Jiang, B. Keller, A. Klinefelter, N. Pinckney, P. Raina, S. G. Tell, Y. Zhang, W. J. Dally, J. Emer, C. T. Gray, B. Khailany, and S. W. Keckler. Simba: Scaling deep-learning inference with multi-chip-module-based architecture. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 14–27. ACM, 2019. doi: 10.1145/3352460.3358302. Network-on-package / network-on-chip provisioning (radix, link count, hierarchical crossbar+mesh) for multi-chiplet DNN inference.

H. D. Sherali and W. P. Adams. A hierarchy of relaxations between the continuous and convex hull representations for zero-one programming problems. *SIAM Journal on Discrete Mathematics*, 3 (3):411–430, 1990. doi: 10.1137/0403036.

Z. Shi, C. Sakhuja, M. Hashemi, K. Swersky, and C. Lin. Learned hardware/software co-design of neural accelerators, 2020. URL https://arxiv.org/abs/2010.02075.

W. Snyder and contributors. Verilator: Open-source SystemVerilog simulator and lint system. https://www.veripool.org/verilator/, 2025. Version 5.050, cycle-based two-state simulation.

B. F. Spector, S. Arora, A. Singhal, D. Y. Fu, and C. Ré. ThunderKittens: Simple, fast, and adorable AI kernels. *arXiv preprint arXiv:2410.20399*, 2024.

A. Surina, A. Mansouri, L. Quaedvlieg, A. Seddas, M. Viazovska, E. Abbe, and C. Gulcehre. Algorithm discovery with llms: Evolutionary search meets reinforcement learning, 2025. URL https://arxiv.org/abs/2504.05108.

M. Tan and Q. Le. Efficientnet: Rethinking model scaling for convolutional neural networks. In *International conference on machine learning*, pages 6105–6114. PMLR, 2019.

X. Tang, Y. Li, N. Wang, Z. Shu, X. Ling, J. Xing, P. Zhou, and Q. Liu. Redfuser: An automatic operator fusion framework for cascaded reductions on ai accelerators. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, page 1566–1588. ACM, Mar. 2026. doi: 10.1145/3779212.3790209. URL http://dx.doi.org/10.1145/3779212.3790209.

A. Tschand, K. Ramakrishnan, A. Ingare, S. Guo, J. Ma, Z. Wan, S. Arora, A. Mirhoseini, and V. Janapa Reddi. Hawkeye: Hardware-aware GPU kernel optimization with minimal supervision. *Preprint*, 2026.

S. Tuli, C.-H. Li, R. Sharma, and N. K. Jha. Codebench: A neural architecture and hardware accelerator co-design framework. *ACM Transactions on Embedded Computing Systems*, 22(3): 1–30, Apr. 2023. ISSN 1558-3465. doi: 10.1145/3575798. URL http://dx.doi.org/10.1145/3575798.

R. Venkatesan, Y. S. Shao, M. Wang, J. Clemons, S. Dai, M. Fojtik, B. Keller, A. Klinefelter, N. Pinckney, P. Raina, et al. Magnet: A modular accelerator generator for neural networks. In *2019 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, pages 1–8. IEEE, 2019.

D. Vungarala, M. H. Amin, P. Mercati, A. Ghosh, A. Roohi, R. Zand, and S. Angizi. Limca: Llm for automating analog in-memory computing architecture design exploration, 2025. URL https://arxiv.org/abs/2503.13301.

G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations (ICLR)*, 2024.

Q. Xiao, S. Zheng, B. Wu, P. Xu, X. Qian, and Y. Liang. Hasco: Towards agile hardware and software co-design for tensor computation. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, pages 1055–1068. IEEE, 2021.

S. Xie, S. Davidson, I. Magaki, M. Khazraee, L. Vega, L. Zhang, and M. B. Taylor. Extreme datacenter specialization for planet-scale computing: Asic clouds. *ACM SIGOPS Operating Systems Review*, 52(1):96–108, 2018.

X. Yang, M. Gao, Q. Liu, J. Setter, J. Pu, A. Nayak, S. Bell, K. Cao, H. Ha, P. Raina, C. Kozyrakis, and M. Horowitz. Interstellar: Using halide's scheduling language to analyze dnn accelerators. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, page 369–383. ACM, Mar. 2020. doi: 10.1145/3373376.3378514. URL http://dx.doi.org/10.1145/3373376.3378514.

A. Yazdanbakhsh, C. Angermueller, B. Akin, Y. Zhou, A. Jones, M. Hashemi, K. Swersky, S. Chatterjee, R. Narayanaswami, and J. Laudon. Apollo: Transferable architecture exploration, 2021. URL https://arxiv.org/abs/2102.01723.

X. Yuan, X. Yan, W. Shen, X. Qiu, A. Wang, J. Zhang, Y. Li, and W. Lin. Bladedisc++: Memory optimizations based on symbolic shape, 2024. URL https://arxiv.org/abs/2412.16985.

D. Zhang, S. Huda, E. Songhori, K. Prabhu, Q. Le, A. Goldie, and A. Mirhoseini. A full-stack search technique for domain optimized deep learning accelerators. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, page 27–42. ACM, Feb. 2022. doi: 10.1145/3503222.3507767. URL http://dx.doi.org/10.1145/3503222.3507767.

H. Zhang, A. Ning, R. B. Prabhakar, and D. Wentzlaff. LLMCompass: Enabling efficient hardware design for large language model inference. In *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA)*, 2024.

J. Zhang, S. Ma, P. Liu, and J. Yuan. Coop: Memory is not a commodity, 2023a. URL https://arxiv.org/abs/2311.00591.

T. Zhang, R. Ma, S. Xu, Y. Xiong, and P. Cheng. Lumina: Llm-guided gpu architecture exploration via bottleneck analysis, 2026. URL https://arxiv.org/abs/2603.05904.

Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett, Z. Wang, and B. Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023b.

Y. Zhao, E. Johnson, P. Chatarasi, V. Adve, and S. Misailovic. Neptune: Advanced ml operator fusion for locality and parallelism on gpus, 2026. URL https://arxiv.org/abs/2510.08726.

L. Zheng, C. Jia, M. Sun, Z. Wu, C. H. Yu, A. Haj-Ali, Y. Wang, J. Yang, D. Zhuo, K. Sen, J. E. Gonzalez, and I. Stoica. Ansor: Generating high-performance tensor programs for deep learning, 2023a. URL https://arxiv.org/abs/2006.06762.

L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

S. Zheng, S. Chen, S. Gao, L. Jia, G. Sun, R. Wang, and Y. Liang. Tileflow: A framework for modeling fusion dataflow via tree-based analysis. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, pages 1271–1288, 2023b.

Z. Zheng, P. Zhao, G. Long, F. Zhu, K. Zhu, W. Zhao, L. Diao, J. Yang, and W. Lin. Fusionstitching: Boosting memory intensive computations for deep learning workloads, 2021. URL https://arxiv.org/abs/2009.10924.

J. Zhu, C. Xue, Y. Chen, Z. Wang, C. Zhang, Y. Shen, Y. Chen, Z. Cheng, Y. Jiang, T. Wang, Y. Lin, W. Hu, B. Cui, R. Wang, Y. Liang, and G. Sun. Theseus: Exploring efficient wafer-scale chip design for large language models, 2024. URL https://arxiv.org/abs/2407.02079.

Appendix

Table of Contents

A Extended Related Work . . . 48

B Theoretical Analysis . . . 51

B.1 Exactness of McCormick Linearization . . . 51

B.2 LP Relaxation Quality . . . 51

B.3 Structural Invariance of the Constraint Matrix . . . 52

B.4 NNZ Breakdown . . . 52

B.5 Complexity of the Pareto Pre-Pass . . . 52

B.6 Containment of the Sequential Map–Fuse–Size Pipeline . . . 52

C Linear Program Formulation Details . . . 56

C.1 Linear Programs and Solvers . . . 56

C.2 V0 Formulation Detailed Variable and Constraint Digest . . . 56

C.3 LP Matrix Sizes per Method . . . 56

C.4 LP Matrix Structure . . . 57

C.5 Solving the Large Scale Systems Problem . . . 58

D Designing for Manufacturability and Feasibility Audits . . . 59

D.1 Five Levels of Validation . . . 59

D.2 MFU and Latency Audits . . . 59

D.3 Six Physical-Design Feasibility Gates . . . 63

D.4 Why We Are Confident in the Software Side . . . 64

D.5 Why We Are Confident in the Hardware Side . . . 64

E Speedup Analysis: Baseline Deficit Attribution and Design-Parameter Sensitivity . . . 66

E.1 Per-Operation Deficit Attribution . . . 66

E.2 Two-Factor Speedup Decomposition . . . 67

E.3 One-Decision-at-a-Time Sensitivity Sweeps . . . 69

E.4 Decision-Impact Ranking . . . 70

F Qualitative Design Discussion . . . 72

F.1 Interesting Empirical Design Results . . . 72

F.2 Hardware and Vizier Search . . . 72

G LLM-Architect Outer Loop Details . . . 74

G.1 Profile-Conditioned LLM-Architect Application . . . 74

G.2 LLM in the Loop Prompts . . . 76

G.3 The Dual-Variable Loop: What the LLM Reads and What It Buys . . . 79

H Two Optimal Tiling Examples: V1 LP Solution at a Glance . . . 81

H.1 V1 LP Solution: Llama-3 70B on modeled H100 . . . 81

H.2 V1 LP Solution: DeepSeek-V3 on modeled RTX 5090 . . . 82

I Examples of CHEETAH Design Blueprints . . . 86

I.1 Variant 1: TSMC N7 + GDDR/LPDDR . . . 86

I.2 Variant 2: TSMC N5 + HBM3 91
J Limitations, Future Work, and Broader Impact 95
K Solving the Master Program at Scale: Shape Aggregation 95
L The Master CHEETAH-V1 Program: A Comprehensive Reference 98
M Conditioning of the comprehensive LP and what it does and does not demand of the solver 101
N Audits, Solver, and Serving Validation 102
O Coefficient Audit Trail: Where Every Constant Comes From 104
P Dual Variables for the Outer Loop: A Mapping from Cluster Resource Allocation . . 105
Q What CHEETAH-V1 Decides Inside the Program 107
R DeepSeek-V3 End-to-End Serving Result 109
S A Worked Example: Hardware Coefficients as Functions, Not Constants 110
T Where Each Workload Family Slows Down, and the Local Sacrifice That Wins . . . 111
U A Minimal Certified Example of Hardware/Software Non-Separability 111

A Extended Related Work

Since this work is at the intersection of various subfields, we review each field's related work individually below.

Accelerator design space exploration. Hardware ML accelerators are characterized by their datapath (compute units, memory hierarchy, interconnect) and software schedule (how operations map onto hardware). A substantial body of work explores design spaces of accelerator families by mutating datapath hyperparameters and operation mappings. [Chen et al., 2019] introduced Eyeriss, a spatial architecture with a row-stationary dataflow for energy-efficient CNNs. [Parashar et al., 2019] proposed Timeloop, a systematic infrastructure for evaluating DNN accelerator schedules and datapaths. [Kwon et al., 2020] developed MAESTRO, a data-centric framework for understanding DNN mapping performance. [Yang et al., 2020] used Halide's scheduling language to analyze DNN accelerators in Interstellar. [Dave et al., 2019] optimized convolution mappings on dataflow accelerators with dMazeRunner. [Venkatesan et al., 2019] proposed MAGNet, a modular accelerator generator, reporting up to 1.75× Perf/W improvement. [Yazdanbakhsh et al., 2021] explored transferable architecture exploration via Apollo. [Kao et al., 2020] applied reinforcement learning to hardware resource assignment. Huang *et al.* formulated accelerator mapping as a constrained optimization problem in CoSA [Huang et al., 2021]. [Shi et al., 2020] cast hardware/software co-design as a constrained Bayesian optimization problem over semi-continuous/semi-discrete design spaces, demonstrating that jointly exploring software stacks and hardware accelerators improves the energy-delay product by up to 40% over decoupled approaches. [Tuli et al., 2023] proposed CODEBench, a neural architecture and hardware accelerator co-design framework that uses active learning to navigate the joint design space efficiently. [Peng et al., 2024] introduced Chiplet Cloud, a system-level co-exploration framework for serving large generative language models that jointly searches hardware architecture and software mapping strategies. Theseus [Zhu et al., 2024] explored efficient wafer-scale chip design for LLMs using multi-fidelity Bayesian optimization to navigate the enormous design space of wafer-scale chips under fabrication-specific constraints. HASCO [Xiao et al., 2021] utilizes tensor syntax trees and proposes a multi-objective Bayesian optimization algorithm, and UNICO [Rashidi et al., 2023] introduces a robustness metric for better co-design generalization. FAST extended these approaches by jointly optimizing datapath, schedule, and compiler passes including operation fusion, achieving 3.7× average Perf/TDP improvement over TPU-v3 across vision and language workloads. Our work preserves FAST's outer hardware search loop but replaces the sequential inner optimization with a joint linear program.

FAST breakdown. FAST presents a tractable full-stack optimization technique for co-design of inference accelerators by jointly searching over hardware datapaths, software schedules, and compiler passes. The input to Vizier is a general ML accelerator template (Figure 11) consisting of a grid of processing elements (PEs), each containing a systolic array for matrix operations and a vector processing unit (VPU) for non-MAC operations such as softmax and layer normalization. The memory hierarchy includes per-PE L1 scratchpads, optional L2 buffers, a shared Global Memory, and off-chip DRAM. The full hardware configuration is defined by 16 hyperparameters (PE grid dimensions, systolic array sizes, buffer capacities, DRAM channels, and batch size) spanning a datapath search space of $\mathcal{O}(10^{13})$ [Zhang et al., 2022, Table 3].

Operation fusion and memory optimization. Operation fusion reduces DRAM traffic by keeping intermediate tensors on-chip. Production compilers such as XLA fuse simple patterns, but each fusion region contains at most one matrix operation. FAST introduced an ILP-based fusion technique that pins activation and weight tensors in Global Memory to minimize total execution time, but with a static placement model restricted to consecutively-executed operations. More recently, Neptune [Zhao et al., 2026] introduced advanced operator fusion for reduction sequences on GPUs, intentionally breaking loop-carried dependencies and compensating via algebraic correction expressions to enable fusion of patterns like attention mechanisms. The oneDNN Graph Compiler [Li et al., 2024] demonstrated a hybrid approach combining library-backed matrix kernels with compiler-generated fusion to achieve state-of-the-art performance across diverse DNN workloads on CPUs. FusionStitching [Zheng et al., 2021] explores large fusion spaces with a domain-specific cost model to combine memory-intensive operators with varied data dependencies into single GPU kernels. RedFuser [Tang et al., 2026] presents a formal methodology for cascaded reduction fusion with automated kernel generation, achieving 2–5× speedup over state-of-the-art compilers on attention-like workloads. Multi-DOF

Fusion [Chen et al., 2025] enables layers within a fusion group to adopt varying tiling scales, distributing memory utilization to maximize inter-layer data reuse on ASIC accelerators. These works address fusion at the compiler or kernel level for fixed hardware; our contribution is to co-optimize fusion jointly with tiling and hardware configuration through a unified LP formulation.

Rematerialization (recomputing values instead of storing them) is well-studied in the context of training memory optimization. [Chen et al., 2016] proposed gradient checkpointing to trade compute for memory during backpropagation. [Jain et al., 2020] formulated optimal checkpointing as an ILP. [Bartan et al., 2023] proposed Moccasin, a constraint programming formulation for tensor rematerialization with only $\mathcal{O}(n)$ integer variables (a significant reduction from the $\mathcal{O}(n^2)$ Boolean variables in Checkmate), achieving an order of magnitude speedup on large compute graphs. Zhang et al. [2023a] introduced Coop, which treats memory as a non-commodity resource in rematerialization scheduling, demonstrating that memory heterogeneity must be explicitly modeled rather than abstracted away. [Patil et al., 2022] presented POET, a MILP framework that jointly optimizes rematerialization with paging across heterogeneous memory hierarchies (SRAM, flash, SD card), enabling fine-tuning of large architectures on devices with as little as 32 KB RAM. BladeDISC++ [Yuan et al., 2024] extends rematerialization to dynamic-shape graphs using symbolic shape analysis, addressing the challenge that tensor sizes may be unknown at compile time. Our CHEETAH-V0 formulation adapts rematerialization to the inference setting, uniquely integrating it with time-indexed placement in a single LP that enables joint reasoning about eviction, recomputation, and fusion.

Joint optimization and linearization techniques. The interaction between tiling decisions and fusion opportunities is a bilinear optimization problem: execution time depends on the product of tiling selection and tensor placement variables. McCormick envelopes provide exact linearization of bilinear terms over box-constrained domains by introducing auxiliary variables and four linear constraints per product term. The foundational work by McCormick established convex underestimators for factorable nonconvex programs [McCormick, 1976]. Our CHEETAH-V1 formulation applies McCormick linearization to the tiling $\times$ placement interaction, enabling exact reformulation as a LP. [Applegate et al., 2022] developed PDLP, a practical first-order method for large-scale LP solving that scales to instances with billions of nonzeros, based on which we build our variant of LP solver that empowers solving the system CHEETAH-V0 and CHEETAH-V1 LPs efficiently. [Lu and Yang, 2024] proposed cuPDLP.jl, a GPU implementation of restarted PDHG that was the first GPU-based LP solver to match or outperform commercial simplex and interior-point solvers on large-scale instances. The subsequent C/CUDA implementation cuPDLP-C [Lu et al., 2024] introduced advanced presolve routines and engineering optimizations, further reducing solve times for problems exceeding the memory capacity of a single GPU. Most recently, cuPDLPx [Lu et al., 2025] incorporates additional algorithmic heuristics including PID-controlled primal weighting and improved convergence behavior, establishing the current state of the art for first-order GPU-accelerated LP solving. Our JAX-based solver inherits the PDHG algorithmic framework from this lineage while exploiting the structural invariance of our constraint matrices for efficient preconditioning across hardware trials.

Tiling and loop optimization. Optimal tiling for nested loop computations has been studied extensively in the compiler community. [Ragan-Kelley et al., 2013] introduced Halide, a language for optimizing parallelism and locality in image processing pipelines. [Zheng et al., 2023a] proposed Ansor, which generates high-performance tensor programs via hierarchical search. Mei *et al.* enlarged the joint architecture-mapping design space for DNN accelerators in ZigZag [Mei et al., 2021]. These works optimize tiling for fixed hardware; our contribution is to co-optimize tiling with fusion decisions.

Polyhedral compilation. The polyhedral model reasons jointly about tiling and fusion as affine transformations of loop nests. Feautrier's affine scheduling [Feautrier, 1992] established the framework; Pluto [Bondhugula et al., 2008] made it a practical automatic parallelizer and locality optimizer; Tiramisu [Baghdadi et al., 2019] exposes the same machinery as a scheduling language for fast, portable code. Closest to our fusion question, Acharya *et al.* [Acharya et al., 2020] cast fusion partitioning as optimization over a fusion conflict graph. All of these target *fixed* hardware and optimize over loop transformations rather than memory-management decisions: none model time-varying buffer residency, eviction, or rematerialization as scheduling variables, and hardware parameters are never decision variables. TileFlow and FADiff likewise fix hardware and provide no valid lower

bounds, whereas our LP bounds are exact at integral decisions and extend to hardware sizing in V3. The relationship is complementary rather than competing. Execution *reordering*, the polyhedral core strength, is the one axis CHEETAH fixes: timesteps are positions in a fixed topological order supplied as input, which is also what places polyhedral methods outside the sequential-pipeline class of Definition 1. Making order a decision above the LP, for example by enumerating candidate reuse-distance-minimizing topological sorts and applying them symmetrically to our method and every baseline, is future work.

LLM-guided optimization and scientific discovery. Recent work has demonstrated that large language models can serve as effective mutation operators in evolutionary search. Romera-Paredes *et al.* introduced FunSearch, pairing a pretrained LLM with an evaluator to discover new solutions to open mathematical problems [Romera-Paredes et al., 2024]. [Novikov et al., 2025] presented AlphaEvolve, an evolutionary coding agent powered by Gemini that discovered improved algorithms for data center scheduling, chip design, and matrix multiplication. Chen *et al.* proposed EvoPrompting, combining evolutionary search with LLM-based code generation for neural architecture discovery [Chen et al., 2023]. Concurrently, LUMINA [Zhang et al., 2026] uses LLM-guided bottleneck analysis to steer GPU architecture design space exploration, where the LLM diagnoses roofline bottlenecks and proposes targeted architectural modifications, a closely related approach to our Causal Architect, though targeting GPU design rather than custom inference accelerators. LIMCA [Vungarala et al., 2025] introduced the first LLM-driven automated design exploration framework for analog in-memory computing crossbar architectures, demonstrating that LLMs can generate hardware designs under power, area, and accuracy constraints. [Surina et al., 2025] demonstrated that combining evolutionary search with reinforcement learning fine-tuning of LLMs yields stronger algorithm discovery than treating the LLM as a static generator. Our LLM-guided hardware search draws on these ideas, using an LLM to propose hardware configurations informed by workload analysis and prior trial outcomes, while the joint LP provides rigorous inner-loop optimization. Unlike approaches that use LLMs to directly generate code or predict performance, our Architect operates strictly as a search-space editor: it constrains the neighborhood for Vizier's Bayesian optimization, with all evaluation performed by the deterministic V1 LP and roofline auditor.

Datacenter accelerator economics. The economic viability of specialized accelerators depends on deployment scale. [Magaki et al., 2016, Xie et al., 2018] analyzed specialization economics under total cost of ownership. [Jouppi et al., 2021] distilled lessons from three generations of TPU design. FAST provided ROI analysis showing that specialized accelerators can be profitable with deployments of $2,000$–$4,000$ units. Our joint optimization targets the same deployment regime, but achieves better Perf/TDP through the larger effective search space of joint tiling-fusion optimization.

B Theoretical Analysis

This appendix provides formal analyses of the CHEETAH-V1 LP formulation.

B.1 Exactness of McCormick Linearization

Proposition 1. *Let $y \in \{0,1\}$ and $p \in \{0,1\}$. Define w by the McCormick constraints: $w \le y$, $w \le p$, $w \ge y + p - 1$, $w \ge 0$. Then $w = y \cdot p$.*

Proof. There are four cases. If $y = 0$: the constraints give $w \le 0$ and $w \ge 0$, so $w = 0 = 0 \cdot p$. If $p = 0$: symmetrically, $w = 0$. If $y = p = 1$: $w \le 1$, $w \ge 1$, so $w = 1$. This covers all cases. □

This means that at any integer-feasible point of the V1 LP, the McCormick auxiliary variables $w_{i,c,k}$ exactly represent the bilinear products $y_{i,c} \cdot p_{\text{assoc}(i,k),o(i)}$, and the LP objective value equals the true joint tiling-fusion objective.

B.2 LP Relaxation Quality

When solved as a continuous LP (all variables in $[0,1]$), the McCormick linearization is the tightest convex relaxation of *an individual* bilinear term over the unit box.

Proposition 2. *The McCormick envelope of $f(y,p) = y \cdot p$ over $[0,1]^2$ is the convex envelope.*

Proof. The convex envelope of a bilinear function over a box is the maximum of the two underestimators $\max(0, y + p - 1)$, which is exactly the McCormick lower bound. Similarly, the concave envelope is $\min(y, p)$. This is a classical result [McCormick, 1976]. □

This does not extend to the joint problem, and we do not claim that it does. It is tempting to read the proposition as saying that the V1 relaxation is the best possible convex relaxation of the tiling-fusion interaction. It does not, and the distinction is worth stating precisely, because the envelope is a statement about one product in isolation and the program contains a *family* of products sharing a one-hot. Take two configurations with $y_1 + y_2 = 1$ and $w_j = y_j p$. Every integral point satisfies $w_1 + w_2 = p$. Yet

$$y_1 = y_2 = p = \tfrac{1}{2}, \qquad w_1 = w_2 = \tfrac{1}{2} \tag{21}$$

satisfies all four McCormick inequalities for each j while giving $w_1 + w_2 = 1 \neq p$. The per-term envelopes therefore admit points that no integral solution can reach, and additional coupling constraints can strengthen the relaxation *without adding variables*. Since $\sum_c y_{i,c} = 1$ already holds, the reduction-and-linearization (RLT) equalities

$$\sum_c w_{i,c,k} = p_{\text{assoc}(i,k)}, \qquad \sum_c z^{\text{act}}_{e,c,t} = p_{e,t}, \qquad \sum_c z^{W}_{i,c,t} = p^{W}_{i,t} \tag{22}$$

hold at every integral point and are valid cuts [Sherali and Adams, 1990]. We include them. The correct statement of the guarantee is therefore: the V1 relaxation is exact at every integral vertex (Proposition 1), and is the convex hull of each product *individually*, strengthened by Eq. (22); it is not the convex hull of the joint feasible set, and no claim in this paper depends on it being so.

How much looseness this costs, measured. Because Eq. (22) are valid cuts, adding them can only raise the LP optimum, so the question is quantitative. We measured it by solving each program twice, with and without the linking rows, holding everything else fixed. On the V1 tile-dependent LP the two objectives agree *bit-for-bit* at every on-chip-memory size we tested (1, 2, 4, 8 and 16 MiB on EfficientNet-B7: $3.4001876777 \times 10^{10}$ in all ten solves, despite the cuts adding 112,746 rows), and likewise on the serving program at every batch size of Table 9. The reason is structural rather than fortunate: the rows these cuts tighten are the on-chip capacity rows, and on these workloads those rows are slack by three to four orders of magnitude. EfficientNet-B7's optimum sits at $1.0041\times$ its pure compute floor while its DRAM bound is 1.1×10^{-4} of that floor, and the serving program independently selects the *smallest* on-chip memory on its menu, which is what a program does when capacity is not scarce. We therefore report zero measured relaxation looseness from this source on our

suite, and state plainly that this is a property of the compute-bound regime we evaluate, not a general guarantee: on a capacity-bound workload Eq. (21) would bite, and Eq. (22) is what prevents it. The relaxation gap between LP optimum and integer optimum is reported empirically per workload in the experiments.

B.3 Structural Invariance of the Constraint Matrix

Proposition 3. *For a fixed neural network model with L operations, T timesteps, $|\mathcal{E}|$ edges, and $|\mathcal{C}_i|$ Pareto configurations per operation, the sparsity pattern (row indices, column indices) of the V1 constraint matrix A is invariant across all hardware configurations proposed by Vizier. Only the numerical values of A, the objective vector c, and the constraint bounds l_c, u_c change.*

Proof. The constraint matrix structure is determined by the graph topology (which defines edges $\mathcal{E}$, execution order $o(i)$, and fan-in/fan-out sets), the number of Pareto configurations $|\mathcal{C}_i|$, and the timestep count T, all of which depend only on the neural network model, not on hardware parameters. Hardware parameters affect $T_i^{\max}(c)$, $T_i^{\min}(c)$, $t_i^k(c)$, d_e, d_i^W, C_{GM}, and c_i^{remat}, which enter as numerical coefficients. The assignment, McCormick, capacity, persistence, and boundary constraints all have the same variable participation pattern regardless of these coefficients. □

This invariance has practical implications: LP solver preconditioners can be factored once per model and reused across all hardware trials, and warm-starting from a previous trial's solution is straightforward since the variable ordering is stable.

B.4 NNZ Breakdown

Table 22 provides a detailed breakdown of nonzeros in the V1 constraint matrix.

Table 22: NNZ breakdown by constraint block for V1 on EfficientNet-B7 ($L = 400$, $T = 400$, $|\mathcal{C}_i| = 1{,}000$).

Constraint Block	Est. NNZ	Source
One-config-per-op	400,000	$L \times \lvert\mathcal{C}\rvert$
Exec. time upper bound	4,800,000	$\lvert\mathcal{C}\rvert \times 4$ terms per op
Exec. time lower bound	400,000	$\lvert\mathcal{C}\rvert$ terms per op
McCormick linearization	7,200,000	$3 \times 2 \times L\lvert\mathcal{C}\rvert \times 3$
Global memory capacity	320,000	$T \times (\lvert\mathcal{E}\rvert + L)$
Activation persistence	800,000	$\lvert\mathcal{E}\rvert \times T \times 2$
Weight monotonicity	320,000	$L \times T \times 2$
Remat feasibility	640,000	Several $L \times T$ blocks
Boundary conditions	800,000	$\lvert\mathcal{E}\rvert \times T + L \times T$
Total	$\sim$ **15.7M**	

B.5 Complexity of the Pareto Pre-Pass

Proposition 4. *The precomputation cost of the Pareto pruning pass is $O(L \cdot |\mathcal{C}_{raw}| \cdot t_{Timeloop})$ per hardware configuration class, where $|\mathcal{C}_{raw}| = O(10^7)$ is the raw configuration count and $t_{Timeloop}$ is the average Timeloop evaluation time. With bandwidth-invariant Pareto fronts, this cost is amortized across all Vizier trials sharing the same compute configuration.*

For EfficientNet-B7: $400 \times 1{,}000 = 400{,}000$ Timeloop evaluations (after pruning), at $\sim$5 seconds each, totaling $\sim$23 hours on a single machine or $\sim$30 minutes with 512-way parallelism.

B.6 Containment of the Sequential Map–Fuse–Size Pipeline

We show that CHEETAH's joint feasible polytope *contains* every in-scope sequential map-then-fuse-then-size pipeline, lifted FAST included: under matched hardware parameters and mapper constants, every point feasible for any such pipeline embeds objective-preservingly into CHEETAH's joint LP feasible set. CHEETAH therefore *strictly generalizes* the sequential pipeline rather than merely

extending it. Formally, we prove the one-directional statement (*for every sequential-pipeline feasible point there exists a CHEETAH-feasible extension with equal objective*) rather than a set equality; the extra CHEETAH coordinates (a free tile selection, rematerialization, and the CGM one-hot) reach points the sequential pipeline cannot express. The result is a modeling statement about feasible sets, not a prediction that the two systems report the same numerical speedups.

Consider a chip H (MAC count, DRAM bandwidth b, C_{GM} size g, datatype, batch) and a single workload DAG G with L layers, E edges, T scheduling steps, and constants $T_i^{\min}(c), T_i^{\max}(c), t_i^k(c)$ at C Pareto tile configurations per layer.

Both modeling formulations include decision variables:

- $y \in [0,1]^{L\times C}$ – tile-config selection per layer, $\sum_c y_{i,c}{=}1$,
- $p \in [0,1]^{E\times T}$ – activation residency per (edge, timestep),
- $p_W \in [0,1]^{L\times T}$ – weight residency per (layer, timestep),
- $T \in \mathbb{R}^L_{\geq 0}$ – per-layer cycle count (objective summand).

The V1 LP additionally inherits $r \in [0,1]^{L\times T}$ (rematerialization) and $h_{\mathrm{GM}} \in [0,1]^K$ with $\sum_k h_{\mathrm{GM},k}{=}1$ (one-hot global memory selection from a discrete grid $g_1,\ldots,g_K$). The primary cycle objective is $\mathbf{c}^\top x = \sum_i T_i$.

In-scope sequential pipelines.

Definition 1 (Sequential map-then-fuse pipeline)**.** *Fix H, G, a topological execution order, and the Timeloop cost constants. A* sequential map-then-fuse pipeline *is any procedure that (i) in a* mapping stage *commits every layer i to a single tile configuration $c_i \in \mathcal{C}_i$ from the shared Pareto menu, by any mapper (Timeloop, ZigZag, CoSA, or a hand schedule; a tile outside the menu is admitted by adding its cost constants as a column), and only then (ii) in a* fusion stage *chooses a static residency pattern (p, p_W) under the shared C_{GM}-capacity and bandwidth model, with the global-memory size fixed at some $g \in \{g_1,\ldots,g_K\}$ and without rematerialization. Writing $c = (c_1,\ldots,c_L)$, its lifted feasible set in the V1 variable space is*

$$\mathcal{F}_S^{\mathrm{lift}}(H,G;c,g) \;=\; \Big\{(y,p,p_W,T) \;\Big|\; y_{i,\,c_i}=1, \quad y_{i,\,c'}=0 \;\; \forall\, c'\neq c_i, \\ (p,p_W) \text{ static, chosen by the fusion stage,} \quad A_S\, x \leq b_S(H,G;g)\Big\}, \tag{23}$$

where A_S, b_S encode the C_{GM} capacity (with g hard-coded), bandwidth, and per-stage compute floor under the fixed-y projection.

Out of scope, by construction, are methods that change the execution *order* (the polyhedral reordering axis, Section 2), use a cost model other than the shared roofline/Timeloop constants, or size hardware outside the menu. FAST is the instance $c = c^{\mathrm{FAST}}$ (Timeloop's per-op choice) with g its fixed C_{GM}: $\mathcal{F}_F^{\mathrm{lift}}(H,G) = \mathcal{F}_S^{\mathrm{lift}}(H,G;c^{\mathrm{FAST}},g^{\mathrm{FAST}})$. FAST commits to $r{=}0$ and to fixed g by construction.

CHEETAH-V1 LP relaxation $\mathcal{P}_V(H,G)$.

$$\mathcal{P}_V(H,G) \;=\; \{(y,p,p_W,r,h_{\mathrm{GM}},T) \in [0,1]^{LC+ET+2LT+K}\times\mathbb{R}^L_{\geq 0} \mid A_V\, x \leq b_V(H,G)\}.$$

The first $LC+ET+LT$ columns of A_V coincide with A_S. The bandwidth constraint uses the same b, and the C_{GM}-capacity row is parameterized by the chosen g_k via h_{GM}.

Theorem 1 (Containment of every in-scope sequential pipeline in $\mathcal{P}_V$)**.** *Fix H, G, the execution order, and the Timeloop constants. For* every *tile commitment $c \in \prod_i \mathcal{C}_i$ and every $g \in \{g_1,\ldots,g_K\}$ there exists a lifting map*

$$\iota:\; \mathcal{F}_S^{\mathrm{lift}}(H,G;c,g) \;\hookrightarrow\; \mathcal{P}_V(H,G), \qquad \iota(y,p,p_W,T) \;=\; \big(y,\,\tilde{p},\,p_W,\,r{=}0,\,h_{\mathrm{GM}}{=}e_{k^\star},\,T\big),$$

with $k^\star$ satisfying $g_{k^\star}{=}g$ and $\tilde{p}_{e,t}{=}1$ exactly on the timesteps covered by the fusion stage's region for the edge supplying e, zero elsewhere. The map is well-defined, injective, and preserves the primary cycle objective: $\sum_i T_i \circ \iota(x) = \sum_i T_i(x)$ for every $x \in \mathcal{F}_S^{\mathrm{lift}}(H,G;c,g)$. In particular, lifted FAST embeds in $\mathcal{P}_V$.

Proof. The integer cube embeds into the LP cube, $\{0,1\}^d \subset [0,1]^d$, therefore any integer-feasible (y, p, p_W) lies in $\mathcal{P}_V$ along the shared coordinates. The lifted $\tilde{p}$ equals 1 from the producer step through the last step of the fused region and 0 afterwards, hence it is monotone non-increasing after production; V1's activation-persistence rows $p_{e,t} \le p_{e,t-1} + r_{\mathrm{src}(e),t}$ therefore hold with $r{=}0$, and the capacity rows charge exactly the tensors the fusion stage keeps resident, so they coincide with $A_S x \le b_S$. Setting $r{=}0$ disables V1's rematerialization slack. Setting $h_{\mathrm{GM}}{=}e_{k^\star}$ pins V1's C_{GM} constraint to the pipeline's fixed g. Moreover, the per-stage compute floor evaluates identically because both formulations consume the same cost vectors under the singleton-y projection. Therefore, $\iota(x) \in \mathcal{P}_V$ for all $x \in \mathcal{F}_S^{\mathrm{lift}}(H, G; c, g)$. Nothing in the argument depends on *how* c or (p, p_W) were chosen, only on $c_i \in \mathcal{C}_i$, on the pattern being static, and on g lying in the menu; the statement therefore holds uniformly over the whole class of Definition 1.

Injectivity. ι is the identity on the shared coordinates and constant on the new coordinates, thus $\iota(x){=}\iota(x') \Rightarrow x{=}x'$.

Primary-objective preservation. The primary cycle objective $\sum_i T_i$ depends only on the shared coordinate T, thus $\mathbf{c}^\top \iota(x) = \mathbf{c}^\top x$.

If V1's objective additionally includes an ε-tiebreaker term $\varepsilon \sum_k k\, h_{\mathrm{GM},k}$, then on $\iota(x)$ this term equals the constant $\varepsilon k^\star$, which does not affect the primary cycle optimum or its $\arg\min$ for fixed g. □

Strict generalization, stated crisply. Theorem 1 reads: *for every in-scope sequential pipeline (Definition 1), every feasible point it can produce has a CHEETAH-feasible extension with equal primary objective.* CHEETAH's feasible set is therefore a superset of the union over all such pipelines,

$$\bigcup_{c,\, g} \iota\big(\mathcal{F}_S^{\mathrm{lift}}(H, G; c, g)\big) \;\subseteq\; \mathcal{P}_V(H, G),$$

which is the precise sense in which sequential map-then-fuse methods search a subset of CHEETAH's decision space. This is an objective-preserving embedding, *not* a set equality: CHEETAH carries strictly more degrees of freedom (a free tile selection y, rematerialization r, and the CGM one-hot h_{GM}), so each lifted pipeline point lands on a face-restricted slice of $\mathcal{P}_V$ and CHEETAH additionally reaches points no sequential pipeline can express. Every map-then-fuse-then-size schedule any in-scope pipeline can produce is thus reachable inside CHEETAH's single joint program at the same cost. Moreover, because every lifted point is integral in y, and the refinements that distinguish the richer CHEETAH formulations either add collapsible variables (V2's per-edge residency and CGM one-hot) or replace a bound that is already exact at integer y (the disaggregated epigraph of §5.2, cf. Corollary 2), the same embedding ι places every in-scope pipeline inside every CHEETAH formulation V1. Containment is therefore a property of the joint program, not of one particular variant.

Corollary 1 (CHEETAH-V1 LP lower bound dominates every in-scope sequential pipeline)**.** *For every chip H, workload G, matched Timeloop constants, every tile commitment c, and every menu size g,*

$$\min_{x \in \mathcal{P}_V(H,G)} \mathbf{c}^\top x \;\le\; \min_{x \in \mathcal{F}_S^{\mathrm{lift}}(H,G;c,g)} \mathbf{c}^\top x, \qquad \textit{in particular} \qquad \min_{x \in \mathcal{P}_V(H,G)} \mathbf{c}^\top x \;\le\; \min_{x \in \mathcal{F}_F^{\mathrm{lift}}(H,G)} \mathbf{c}^\top x.$$

Any fixed-hardware performance gap between a sequential pipeline and CHEETAH V1 therefore cannot be attributed to sequential solutions lying outside the V1 search space, and must be due to (i) V1's extra continuous degrees of freedom (free y, r, h_{GM}), (ii) LP relaxation versus integer rounding, or (iii) differences in the cost model or baseline normalization. The corollary bounds the V1 LP relaxation, not the rounded executable schedule (which is reported separately).

Proof. Fix any c and g and let $x_S^\star$ attain the minimum over $\mathcal{F}_S^{\mathrm{lift}}(H, G; c, g)$. By Theorem 1, $\iota(x_S^\star) \in \mathcal{P}_V$ and $\mathbf{c}^\top \iota(x_S^\star) = \mathbf{c}^\top x_S^\star$; the FAST case is $c = c^{\mathrm{FAST}}$. The V1 LP minimum is at most this feasible value. □

The containment above bounds how CHEETAH relates to the sequential pipeline. The next corollary bounds how CHEETAH's own formulations relate to one another, tying the certified lower bound to the disaggregated perspective epigraph of CHEETAH-V1 (§5.2, (5)).

Corollary 2 (The flagship program's disaggregated epigraph is the exact hull and tightens the certified bound)**.** *Fix the hardware (v, u) and mapper constants. Let the aggregate roofline row of CHEETAH-V1 (V1) be $T_i \geq \max\big(\sum_c T_i^{\min}(c)\, y_{i,c}\, v,\ \sum_c (T_i^{\max}(c) - \mathrm{save}_{i,c})\, y_{i,c}\, u\big)$, and let (5) be the disaggregated perspective form with split variables $v_{i,c}, u_{i,c}, \tau_{i,c}$ obeying $\sum_c v_{i,c} = v$, $\sum_c u_{i,c} = u$ and the box links $\underline{v}\, y_{i,c} \leq v_{i,c} \leq \overline{v}\, y_{i,c}$, $\underline{u}\, y_{i,c} \leq u_{i,c} \leq \overline{u}\, y_{i,c}$. Then (5) is the closed convex hull of the per-op disjunction "select one tile c and pay its roofline maximum $\max(T_i^{\min}(c)\, v,\ (T_i^{\max}(c) - \mathrm{save}_{i,c})\, u)$." Consequently:*

- *(i)* (5) *is* exact at every integer vertex $y \in \{0,1\}$*, where it collapses to the true selected-tile roofline and hence coincides with* (V1)*;*
- *(ii) as a continuous relaxation it enforces $T_i \geq \sum_c \max(T_i^{\min}(c)\, v_{i,c},\ (T_i^{\max}(c) - \mathrm{save}_{i,c})\, u_{i,c})$, which* dominates *the weaker aggregate bound $T_i \geq \max(\sum_c T_i^{\min}(c)\, v_{i,c},\ \sum_c (T_i^{\max}(c) - \mathrm{save}_{i,c})\, u_{i,c})$ of* (V1)*.*

Therefore, at matched hardware, the CHEETAH-V1 certified lower bound is never below the CHEETAH-V1 lower bound, $\min_{\mathcal{P}_{V3}} \mathbf{c}^\top x \geq \min_{\mathcal{P}_{V2}} \mathbf{c}^\top x$, with strict improvement exactly when the V2 optimum blends a compute-bound tile against a bandwidth-bound tile to hide both bottlenecks. The certified bound thus cannot get worse and generally improves*, while both remain $\leq$ the lifted-FAST optimum of Corollary 1.*

Proof. The box links are the McCormick envelope of the products $v \cdot y_{i,c}$ and $u \cdot y_{i,c}$ over the unit-y box, which is the tightest convex relaxation and is exact at $y \in \{0,1\}$ [McCormick, 1976]. At an integer vertex, $\sum_c y_{i,c} = 1$ forces $y_{i,c^\star} = 1$ for a single $c^\star$ and $y_{i,c} = 0$ otherwise; the box links then pin $v_{i,c^\star} = v$, $u_{i,c^\star} = u$ and $v_{i,c} = u_{i,c} = 0$ for $c \neq c^\star$, so $\tau_{i,c^\star} \geq \max(T_i^{\min}(c^\star)v, (T_i^{\max}(c^\star) - \mathrm{save}_{i,c^\star})u)$ and $T_i \geq \sum_c \tau_{i,c}$ reduces to the true selected-tile roofline, which is (V1) at that vertex, giving (i). For (ii), the epigraph rows $\tau_{i,c} \geq T_i^{\min}(c)\, v_{i,c}$ and $\tau_{i,c} \geq (T_i^{\max}(c) - \mathrm{save}_{i,c})\, u_{i,c}$ give $\tau_{i,c} \geq \max(\cdot,\cdot)$, so $T_i \geq \sum_c \max(\cdot,\cdot) \geq \max(\sum_c \cdot, \sum_c \cdot)$ by superadditivity of the maximum ($\sum_c \max(a_c, b_c) \geq \max(\sum_c a_c, \sum_c b_c)$). Thus every T feasible for (5) satisfies the aggregate row of (V1), i.e. the V3-feasible set is contained in the V2-feasible set along the shared coordinates; minimizing the same objective over a subset yields a value at least as large, giving $\min_{\mathcal{P}_{V3}} \mathbf{c}^\top x \geq \min_{\mathcal{P}_{V2}} \mathbf{c}^\top x$. By (i) the tightening removes only fractional-y points, so lifted FAST, being integral in y, stays feasible in $\mathcal{P}_{V3}$; hence both minima remain $\leq$ the lifted-FAST optimum via Corollary 1. □

Theorem 1 is a modeling containment result, and does not predict numerical equivalence of the speedups CHEETAH and FAST report. Empirically, differences can arise from baseline normalization, TPU-v3 simulator overheads, control/padding costs, and batch/core conventions.

Remark (Scope of the certificate: inner LP vs. outer search)*.* The guarantees above are *inner-problem* certificates. For a fixed hardware candidate H, workload G, execution order, and finite candidate tiling sets, CHEETAH's joint program is a convex LP, so any converged optimum is global to within the reported solver tolerance and yields a certified lower bound on the achievable schedule cost for that H. The *outer* hardware search (the LLM-Architect / Vizier loop that proposes chip candidates) is a heuristic: we do *not* claim a certified global optimum over the unrestricted hardware space, only over the schedule for each proposed H.

The residual LP-to-integer gap observed on weight-heavy models, where a per-block weight tensor exceeds the on-chip budget and cannot be held integrally, is *certified relaxation looseness*: a property of the *bound*, not of the deployed schedule. On the chain-structured subproblems where it appears, the executable schedule attains the exact integral optimum (verified by chain dynamic programming), so the gap measures the distance from the LP lower bound up to that integral optimum, not any suboptimality of what runs on hardware. In particular, the zero LP *duality* gap at convergence must not be conflated with this LP-to-integer *relaxation* gap: the former certifies the relaxation was solved to optimality, whereas the latter is the inherent looseness of the convex relaxation, precisely the quantity that Corollary 2's tighter epigraph reduces.

C Linear Program Formulation Details

C.1 Linear Programs and Solvers

A Linear Program (LP) is a mathematical optimization technique for modeling systems of linear relationships. Formally, it consists of a linear objective function (such as minimizing a function of latency/throughput) subject to a set of linear inequality constraints that define a feasible solution region in a high-dimensional space. Equation 24 shows the canonical LP form. LP Solvers are mature software used to search through this solution space to return the optimal value and decision variables. While standard LPs allow for continuous variables, practical hardware design often requires discrete decisions, such as if a tensor pinned completed on SRAM. To address this, Integer Linear Programming (ILP) restricts certain variables to return only integer values. In large-scale co-design, solving an ILP directly is often computationally prohibitive, therefore necessary simplifications and assumptions are made to shrink the problem. As noted by FAST, this ease of implementation greatly reduces the ILP problem size, at the cost of potential suboptimal solutions. Additionally the relationship between tiling, fusion, and memory is often non-linear, which restricts the expressiveness of the problem formulation. In order to address this limitation in CHEETAH, we utilize McCormick envelopes to provide a tight relaxation, ensuring that the gap between the continuous LP "ideal" and the final rounded hardware spec remains minimal. This process takes the fractional outputs of the LP (e.g., a tiling factor of 7.8) and maps them to the nearest valid integer (e.g., 8) while checking for feasibility. This enables CHEETAH to capture globally optimal solutions without sacrificing optimality due to restrictive linear/scale limitations.

$$\begin{aligned} \underset{x}{\text{minimize}} \quad & c^\top x \\ \text{subject to} \quad & l_c \le Ax \le u_c, \quad l_v \le x \le u_v \end{aligned} \tag{24}$$

C.2 V0 Formulation Detailed Variable and Constraint Digest.

The objective minimizes total execution time $\sum_i T_i$ plus rematerialization costs $c_i^{\text{remat}} r_{i,t}$, where $r_{i,t} \in [0,1]$ indicates whether operation i is recomputed at timestep t. Each T_i is lower-bounded by two terms: the *all-on-chip* time $T_i^{\min}$ (a physical floor), and the *all-off-chip* time $T_i^{\max}$ minus DRAM savings $t_i^k \cdot p_{i,o(i)}^k$ from any tensors pinned in Global Memory at execution time $o(i)$. The placement variables $p_{i,t}^k \in [0,1]$ track whether tensor $k \in \{I, O, W\}$ of operation i resides in Global Memory at timestep t, subject to a capacity constraint C_{GM} at every timestep. The persistence constraint $p_{i,t}^k \le p_{i,t-1}^k + r_{i,t}$ enforces that a tensor can only be present at time t if it was either already present at $t-1$ or was just recomputed; weight tensors satisfy the stricter $p_{i,t}^W \le p_{i,t-1}^W$ since weights cannot be recomputed. Rematerialization is disabled before an operation executes ($r_{i,t} = 0$ for $t \le o(i)$) and is only possible when both the required input edges and the operation's weights are available on-chip. Note in this formulation, the persistence constraint ($p_{i,t}^k \le p_{i,t-1}^k + r_{i,t}$) ensures a tensor can only be present at time t if it was present at $t-1$ or was just recomputed. Weight tensors ($p_{i,t}^W \le p_{i,t-1}^W$) cannot be recomputed: once evicted, they must be reloaded from DRAM. These are simplifications we make in our current version of V0, and can be easily extended to handle more complex situations. For DeepSeek-V3 with $L \approx 6{,}300$ operations, V0 has approximately 3.3M variables and 3.7M constraints with 7.3M nonzeros (see Table 23 in the appendix for all models).

C.3 LP Matrix Sizes per Method

V0 can be larger than V1 because the two formulations scale differently. V0 uses separate time-indexed placement matrices for inputs, outputs, weights, and rematerialisation, giving roughly $4L^2$ placement and rematerialisation variables when $T = L$. V1 adds explicit tiling-selection variables over C Pareto configurations and McCormick auxiliary variables, but these terms scale only as $\mathcal{O}(LC)$. More importantly, V1 replaces V0's separate input/output placement variables with an edge-indexed placement representation $p_{e,t}$, so the activation-placement block scales as $ET \approx L^2$ rather than separate layer-by-time input and output matrices. Thus V1's variable count is approximately $3L^2 + 4LC$, while V0 is approximately $4L^2$. For large workloads where $L \gg C$ (such as DeepSeek-V3 with $L = 903$ and $C = 16$), the L^2 saving dominates the added tiling and McCormick variables. Therefore V1 is both more expressive and often more compact: it jointly models tiling, placement,

rematerialisation, and fusion, while using edge-indexed placement to avoid redundant input/output placement variables. Although V1 is more expressive than V0, it is not simply V0 with extra variables; it reparameterizes placement using edge-indexed variables and adds Pareto tiling-selection variables. Thus, V1 inherits V0's dynamic placement and rematerialization capabilities at the modeling level, while extending the feasible design space to include joint tiling-fusion trade-offs, subject to the coverage of the precomputed Pareto C-sets.

Table 23: Single Workload per Chip LP matrix sizes ($n_{\text{constraints}} \times n_{\text{vars}}$, nnz) per model and formulation.

Model	Static	V0 Dynamic	V1 Joint
EfficientNet-B1	803 × 576, nnz= 15,060	59,571 × 53,131, nnz= 119,485	62,728 × 47,035, nnz= 141,604
EfficientNet-B2	803 × 576, nnz= 15,060	59,571 × 53,131, nnz= 119,485	62,728 × 47,035, nnz= 141,604
EfficientNet-B3	908 × 651, nnz= 18,975	76,116 × 67,861, nnz= 152,620	77,743 × 59,020, nnz= 173,749
EfficientNet-B4	1,118 × 801, nnz= 28,155	115,281 × 102,721, nnz= 231,040	112,498 × 87,040, nnz= 247,489
EfficientNet-B5	1,356 × 971, nnz= 40,735	169,460 × 150,933, nnz= 339,500	159,503 × 125,324, nnz= 346,293
EfficientNet-B6	1,566 × 1,121, nnz= 53,755	225,905 × 201,153, nnz= 452,480	207,698 × 164,864, nnz= 446,913
EfficientNet-B7	1,909 × 1,366, nnz= 78,892	335,518 × 298,663, nnz= 671,853	299,965 × 241,059, nnz= 638,356
Llama-1 13B	2,518 × 1,801, nnz= 135,355	583,381 × 519,121, nnz= 1,167,840	505,198 × 411,840, nnz= 1,061,089
Llama-1 65B	5,038 × 3,601, nnz= 529,915	2,333,161 × 2,075,041, nnz= 4,668,480	1,917,658 × 1,601,280, nnz= 3,936,769
Llama-3 70B	5,038 × 3,601, nnz= 529,915	2,333,161 × 2,075,041, nnz= 4,668,480	1,917,658 × 1,601,280, nnz= 3,936,769
DeepSeek-V3	6,319 × 4,516, nnz= 829,852	3,669,793 × 3,263,443, nnz= 7,342,293	2,983,450 × 2,504,019, nnz= 6,094,156

Table 24: GeoMean Avg Maximum LP matrix size per model cluster (largest member LP).

Cluster	Static	V0 Dynamic	V1 Joint
EfficientNet	1,909 × 1,366, nnz= 78,892	335,518 × 298,663, nnz= 671,853	299,965 × 241,059, nnz= 638,356
Llama	5,038 × 3,601, nnz= 529,915	2,333,161 × 2,075,041, nnz= 4,668,480	1,917,658 × 1,601,280, nnz= 3,936,769
Modern LLM	6,319 × 4,516, nnz= 829,852	3,669,793 × 3,263,443, nnz= 7,342,293	2,983,450 × 2,504,019, nnz= 6,094,156

C.4 LP Matrix Structure

Table 25 compares the structural properties of the V0 and V1 constraint matrices. Both formulations produce extreme-aspect-ratio sparse systems with a bimodal row-degree distribution and a parametric sparsity pattern invariant to workload coefficients, which enables custom solver preconditioning to be factored once per model and reused across all Vizier trials.

Table 25: Structural comparison of the two LP matrix formulations. For Llama3-70b: $L = 720$, $T = 720$, $E \approx L$.

Property	CHEETAH-V0	CHEETAH-V1
Variables	$T_i\,(L) + p^I, p^O, p^W, r\,(4 \cdot L \cdot T) + T_{\text{final}}\,(1)$ $+ FT_i\,(L) \;\approx\; 4L^2 + 2L + 1$	$T_i\,(L) + y_{i,c}\,(LC) + p_{e,t}\,(ET)$ $+ p^W, r\,(2LT) + w_{i,c,k}\,(3LC) \;\approx\; 3L^2 + 4LC + L$
Llama-3 70B actual	L=720, T=720 $\rightarrow$ 2.07M vars, 2.33M constraints, 4.67M nnz	Similar order of magnitude; $p_{e,t}$ is edge-indexed (ET) instead of layer-pair-indexed, which saves $\approx L^2$ vars when DAG is mostly linear
Density	$\text{nnz}/(m \cdot n) \approx 9.6 \times 10^{-7}$ (≈1 nnz per million entries)	Similar; row group (5) is the most dense
Avg. nnz/row	≈ 2.0	≈ 2.0 with bursts up to $4L + L$ on capacity rows
Constant pattern across runs	Yes, parametric in L. Every workload of L layers gets the exact same matrix template; only coefficient values (`T_max`, `T_min`, `d_I/O/W`) change.	Yes, parametric in (L, C, E, T). DAG topology lives in the (i, e, t) index map, but row template is invariant. Sparsity pattern is workload-agnostic.
Banded	Most rows touch only (i, t) and $(i, t-1)$ (persistence chains, weight monotone, FT recurrences). Bandwidth $\approx T$ along the time axis.	McCormick triples (4a/b/c) are 1–2 nnz; persistence on edge (6) couples (e, t) to $(e, t-1)$. Same time-band structure.
Exception to band-ness	Rows from group (C): global memory capacity per timestep span all L weights at that timestep $\rightarrow$ wide $\approx L+2$ nnz rows, T of them.	Rows from group (5): span all E edges + all L weight footprints at that timestep $\rightarrow$ $\approx E+L \approx 2L$ nnz rows, T of them.
Why constant pattern works for solver	Per-row degree distribution is bimodal: $\mathcal{O}(L^2)$ rows with $\leq$3 nnz + $\mathcal{O}(T)$ fat capacity rows. Our solver handles this after preconditioning.	Same bimodal pattern, plus a small $(3 \cdot L \cdot C)$ dense McCormick block. Still sparse in aggregate.

C.5 Solving the Large Scale Systems Problem

One of the central challenges and barriers to entry in solving the V1 LP is its simultaneous extreme large scale and extreme ill-conditioning.

The joint tiling + fusion + placement + remat LP for Llama-3 70B and DeepSeek-V3 has roughly 1.6 M–2.5 M decision variables, 1.9 M–3 M constraints, and 4 M nonzeros. The variable layout is

$$T[L] \;\mid\; y[L, C] \;\mid\; p[E, T] \;\mid\; p_W[L, T] \;\mid\; r[L, T] \;\mid\; w[L, C, 3] \;\mid\; h_{\mathrm{GM}}[K],$$

for $L \approx 720$–903 layers, $C = 16$–20 Pareto tile configs, and $T = L$ timesteps. At this scale, simplex and barrier solvers exhaust factorization memory, and traditional solvers were never engineered to refactor sparse Jacobians of this shape.

The extreme ill-conditioning is unavoidable. Timeloop per-tile cycle counts $T_{\min}(c)$ and $T_{\max}(c)$ span 10^3–10^{10} cycles. The McCormick-envelope variables $w[L, C, 3]$ inject products of those tile counts with normalized residency variables, so absolute matrix entries span $\sim 10^7$ in magnitude; the `entry_ratio` exceeds the 1×10^7 "extreme" threshold, and the `row_norm_ratio` blows past 10^6, well outside what generic interior-point methods can tolerate without diverging.

This combination of scale $\times$ conditioning $\times$ solve speed is exactly the regime where a first-order method based on the *primal-dual hybrid gradient* (PDHG) is the only credible path. PDHG sidesteps matrix factorization entirely, costs $\mathcal{O}(\mathrm{nnz})$ per step, and converges as long as the operator can be preconditioned and step sizes chosen adaptively.

JAX implementation. We implemented the solver in JAX: since PDLP is dominated by sparse matvecs and elementwise projection operations that vectorize cleanly under XLA, we use `jax.jit` and `lax.scan` over the inner iteration to batch $\sim 10^5$ iterations onto the GPU/TPU at a few microseconds each. This also enables JIT-fusion of the Ruiz preconditioning loop into a single kernel.

Custom variable-metric solver. Even with JAX, the vanilla PDLP loop did not converge on the LLM LPs, since the duality gap oscillated and the standard symmetric-restart heuristic kept overshooting. To solve this, we introduce a custom *variable-metric one-sided rescue* (`vm-one-armed-rescue`): an asymmetric, one-sided restart-Halpern PDHG with a diagonal variable metric especially designed for V1's large-scale systems-shaped LPs. The key hyperparameters are:

- `num_ruiz_iterations` $= 16$
- `restart_window` $= 200$
- `step_size_limit_coef` $= 1.18$
- `growth` $= 0.55$, `decay` $= 0.32$
- `primal_weight_smooth` $= 0.32$
- `restart_sufficient` $= 0.40$, `restart_necessary` $= 0.94$
- `beta_artificial` $= 0.35$

We additionally incorporate feasibility-polishing features triggered at a relative gap of 0.22, with a 55 k-iteration polish cap and a wider 200-iteration restart window, and a custom presolve stack:

$$\texttt{drop_zeros} \to \texttt{singleton_rows} \to \texttt{fixed_vars} \to \texttt{empty_rows} \to \texttt{geomean_ruiz} \to \texttt{objective_scale}$$

which removes diagonal singletons and equilibrates the coefficient matrix before the first PDHG iterate.

Results. This enables both the primal and dual convergence of the LLM LPs within $\sim$80–200 s at feasibility tolerance 2.5×10^{-7}, reaching within 0.1–1% of the HiGHS optimum (the rounding gap is bounded and reported). By contrast, stock PDLP and off-the-shelf HiGHS either time out, factorize into NaNs, or polish to objectives 5–12$\times$ worse without achieving dual convergence.

D Designing for Manufacturability and Feasibility Audits

We report on hardware setup and hyperparameter details for full reproducibility. In addition, we also provide more examples of chip designs discovered by our method that are of interest.

D.1 Five Levels of Validation

We validate the physical manufacturability and feasibility of our framework at four levels.

1. Roofline Auditor: A deterministic Roofline Auditor checks physical feasibility by comparing each LP-predicted latency against the lower bound

$$\mathrm{LB} = \max(t_{\mathrm{compute}},\ t_{\mathrm{memory}})\,, \tag{25}$$

   where t_{compute} is total FLOPs divided by peak FLOPs/s and t_{memory} is post-placement off-chip traffic divided by available memory bandwidth. The audit additionally reports $\mathrm{MFU}_{\mathrm{bound}}$ and memory utilization; values above 1 would indicate a physically impossible prediction. All results withing this paper are roofline-feasible with zero lower-bound violations, and Appendix D.2 gives more details.

2. Campaign Validator: A deterministic Campaign Validator legalizes LLM-proposed architectural neighbourhoods by enforcing hardware ranges, power-of-two constraints, TDP and area caps, MAC caps, CGM feasibility floors, and compute/bandwidth coupling constraints before Vizier is permitted to sample candidates.

3. Causal Ledger: The Causal Ledger records each LLM hypothesis, intervention, and observed outcome, making the search loop fully auditable and preventing repeated exploration of empirically invalid regions.

4. Solver diagnostics: The LP solver reports mathematical solver diagnostics such as primal and dual residuals, duality gap, rounding gap, and fractionality. In our experiments LP solutions are not treated as trustworthy unless both the relaxation and the rounded design are numerically well behaved.

5. Pre-RTL Feasibility Gates: All designs must pass six first-order pre-RTL style feasibility gates. This aims to audit deeper physical designs aspects, such as pin count, power density, and logic depth, while preventing performance gains at the cost of unrealistically expensive designs. Appendix D.3 provides explicit definitions and details.

D.2 MFU and Latency Audits

Clarification on MFU In this work, $\mathrm{MFU} = \texttt{compute_floor}/\texttt{obj_cycles}$ is a *roofline tightness ratio*, not a measured hardware utilization. $\mathrm{MFU} = 1.000$ means the LP objective equals the ideal compute lower bound; it does not mean a real chip would achieve 100% utilization. This is since the searched accelerators are hypothetical datapaths rather than fabricated chips or RTL implementations, we cannot report measured hardware MFU; instead, we report MFU_bound only as a roofline lower-bound consistency check, where values above 1 would indicate an impossible prediction.

We define the idealized roofline saturation metric as:

$$\mathrm{MFU}_{\mathrm{audit}} = \frac{t_{\mathrm{compute}}}{t_{\mathrm{solver}}}, \tag{26}$$

where t_{compute} is the theoretical compute lower bound and t_{solver} is the LP-predicted latency. Thus, $\mathrm{MFU}_{\mathrm{audit}} = 1$ means the LP solution is *compute-floor tight* under the idealized accelerator model. It is not a claim that a real implementation would achieve 100% hardware utilization. Real chips would incur additional losses from pipeline bubbles, NoC contention, SRAM latency, control overhead, imperfect scheduling, and compiler/runtime effects.

The `binding` column reports whether the objective is sitting on the compute floor, the memory floor, or above both floors. The key sanity condition is that no row has $\mathrm{MFU}_{\mathrm{audit}} > 1$ and no V0/V1 row is unexpectedly loose above the lower bound. Values greater than 1 are physically impossible and would indicate a bug or cost-model violation. Values equal to 1 are acceptable: they indicate that the idealized LP solution is exactly on the compute lower bound.

A true measured MFU would require an actual executable implementation of the selected hardware/schedule pair: either fabricated silicon, an FPGA/RTL prototype, or a calibrated cycle-accurate simulator with the selected V1 tiling/fusion/placement schedule implemented. One would need to run the workload and collect hardware counters such as active MAC cycles, issued FLOPs, memory stalls, NoC stalls, SRAM bank conflicts, pipeline bubbles, and total elapsed cycles. Measured MFU would then be:

$$\mathrm{MFU}_{\mathrm{measured}} = \frac{\text{useful FLOPs}}{\text{elapsed cycles} \times \text{peak FLOPs/cycle}}. \tag{27}$$

We define:

$$\mathrm{MFU}_{\mathrm{bound}} = \frac{t_{\mathrm{compute}}}{t_{\mathrm{solver}}}, \tag{28}$$

where t_{compute} is the theoretical compute lower-bound latency and t_{solver} is the LP-predicted latency. The roofline audit additionally checks the stronger lower bound:

$$t_{\mathrm{solver}} \;\geq\; \max(t_{\mathrm{compute}},\; t_{\mathrm{memory}})\,, \tag{29}$$

where the memory floor is total bytes moved divided by peak bandwidth. Concretely, the audit enforces `roof_ratio` $= \mathrm{LB}/\mathrm{obj} \leq 1.001$ as the pass criterion.

Interpretation of $\mathrm{MFU}_{\mathrm{bound}} = 1$. Since the searched accelerators are hypothetical datapaths evaluated by CORGIsolve rather than fabricated chips, RTL models, or calibrated cycle-accurate simulators, real measured MFU is unavailable. We therefore use $\mathrm{MFU}_{\mathrm{bound}} = t_{\mathrm{compute}}/t_{\mathrm{solver}}$ solely as a roofline consistency check. $\mathrm{MFU}_{\mathrm{bound}} = 1$ means the LP solution is *compute-floor tight* under the idealized accelerator model; it does not imply that real silicon would achieve 100% utilization.

What the roofline audit proves. The audit does not establish that real-world MFU would equal 1. It establishes the more basic and necessary condition that the reported LP latency is not physically impossible: no result is faster than either the compute or memory lower bound. In a real implementation, utilization would generally be lower due to pipeline bubbles, NoC contention, SRAM banking conflicts, control overhead, compiler effects, and imperfect scheduling.

Since our accelerators are simulated datapaths rather than fabricated chips, real measured MFU is unavailable. We use $\mathrm{MFU}_{\mathrm{bound}} = t_{\mathrm{compute}}/t_{\mathrm{solver}}$ only as a roofline consistency check. Values above 1, or LP latencies below $\max(t_{\mathrm{compute}}, t_{\mathrm{memory}})$, would indicate a physically impossible prediction. Values equal to 1 mean the LP solution is tight to the ideal compute floor, not that real hardware would achieve 100% utilization; real implementations would generally fall below this bound due to microarchitectural overheads. Our audit reports 31 plotted cells, 2 skipped infeasible cells, and 0 MFU or roofline violations, supporting that the reported V0/V1 speedups are physically feasible upper-bound estimates rather than below-roofline artifacts.

The below audit verifies that all reported V0/V1 results are roofline-feasible: the LP-predicted latency never falls below the compute or memory lower bound, so the speedups are not artifacts of violating basic physical throughput limits.

Table 26: Single Workloads of Vizier-discovered hardware configuration. $\mathrm{MFU}_{\mathrm{audit}} = t_{\mathrm{compute}}/t_{\mathrm{solver}}$ is a roofline tightness ratio (see Eq. 26), not a measured hardware utilization. $\mathrm{MFU} = 1.000$ means the LP hit the theoretical compute floor. *Compute-bound*: $\mathrm{obj} \approx t_{\mathrm{compute}}$. *Memory-bound*: $\mathrm{obj} \approx t_{\mathrm{memory}}$, $\mathrm{MFU} < 1$. *Above-roofline*: obj exceeds both floors; expected for Static on hard cells, unexpected for V0/V1. Two cells (Llama-3 70B and DeepSeek-V3 under Static) had no physically valid trial and are marked infeasible.

Method	Model	MACs	BW	Speedup	Obj cycles	Comp. floor	Mem. floor	MFU	obj/lb	Binding	Verdict
Static FAST ILP											
Static	EN-B1	32,768	64	1.12	2.757×10^7	2.757×10^7	4.236×10^6	1.000	1.000	compute-bound	OK
Static	EN-B2	65,536	64	1.93	2.576×10^7	2.576×10^7	5.474×10^6	1.000	1.000	compute-bound	OK
Static	EN-B3	32,768	64	1.25	7.905×10^7	7.905×10^7	7.552×10^6	1.000	1.000	compute-bound	OK
Static	EN-B4	65,536	64	2.25	1.045×10^8	1.045×10^8	1.372×10^7	1.000	1.000	compute-bound	OK
Static	EN-B5	32,768	64	1.31	4.938×10^8	4.938×10^8	2.279×10^7	1.000	1.000	compute-bound	OK
Static	EN-B6	32,768	64	1.10	9.952×10^8	9.952×10^8	3.505×10^7	1.000	1.000	compute-bound	OK
Static	EN-B7	131,072	64	3.60	5.153×10^8	5.153×10^8	5.280×10^7	1.000	1.000	compute-bound	OK
Static	Llama-1 13B	32,768	1024	1.07	2.705×10^{10}	2.705×10^{10}	2.981×10^7	1.000	1.000	compute-bound	OK
Static	Llama-1 65B	65,536	128	0.61	2.377×10^{11}	6.744×10^{10}	9.514×10^8	0.284	3.525	above-roofline	loose (expected)
Static	Llama-3 70B	–	–	–	–	–	–	–	–	–	infeas
Static	DeepSeek-V3	–	–	–	–	–	–	–	–	–	infeas
Dynamic V0 LP (time-indexed + rematerialisation)											
V0	EN-B1	65,536	64	2.24	1.379×10^7	1.379×10^7	4.236×10^6	1.000	1.000	compute-bound	OK
V0	EN-B2	131,072	64	3.87	1.288×10^7	1.288×10^7	5.474×10^6	1.000	1.000	compute-bound	OK
V0	EN-B3	131,072	64	4.99	1.976×10^7	1.976×10^7	7.552×10^6	1.000	1.000	compute-bound	OK
V0	EN-B4	131,072	64	4.51	5.225×10^7	5.225×10^7	1.372×10^7	1.000	1.000	compute-bound	OK
V0	EN-B5	131,072	64	5.24	1.234×10^8	1.234×10^8	2.279×10^7	1.000	1.000	compute-bound	OK
V0	EN-B6	262,144	64	8.82	1.244×10^8	1.244×10^8	3.505×10^7	1.000	1.000	compute-bound	OK
V0	EN-B7	524,288	64	14.39	1.288×10^8	1.288×10^8	5.280×10^7	1.000	1.000	compute-bound	OK
V0	Llama-1 13B	262,144	64	8.57	3.382×10^9	3.382×10^9	4.770×10^8	1.000	1.000	compute-bound	OK
V0	Llama-1 65B	262,144	64	8.57	1.686×10^{10}	1.686×10^{10}	1.903×10^9	1.000	1.000	compute-bound	OK
V0	Llama-3 70B	524,288	64	20.07	9.227×10^9	9.227×10^9	1.972×10^9	1.000	1.000	compute-bound	OK
V0	DeepSeek-V3	262,144	128	8.87	6.622×10^9	6.622×10^9	7.477×10^8	1.000	1.000	compute-bound	OK
Joint V1 LP (joint tiling + fusion)											
V1	EN-B1	131,072	64	4.47	6.894×10^6	6.894×10^6	4.236×10^6	1.000	1.000	compute-bound	OK
V1	EN-B2	262,144	64	7.73	6.440×10^6	6.440×10^6	5.474×10^6	1.000	1.000	compute-bound	OK
V1	EN-B3	262,144	64	9.99	9.881×10^6	9.881×10^6	7.552×10^6	1.000	1.000	compute-bound	OK
V1	EN-B4	262,144	64	9.01	2.612×10^7	2.612×10^7	1.372×10^7	1.000	1.000	compute-bound	OK
V1	EN-B5	524,288	64	20.98	3.086×10^7	3.086×10^7	2.279×10^7	1.000	1.000	compute-bound	OK
V1	EN-B6	524,288	64	17.64	6.220×10^7	6.220×10^7	3.505×10^7	1.000	1.000	compute-bound	OK
V1	EN-B7	1,048,576	64	28.78	6.441×10^7	6.441×10^7	5.280×10^7	1.000	1.000	compute-bound	OK
V1	Llama-1 13B	1,048,576	64	34.27	8.454×10^8	8.454×10^8	4.770×10^8	1.000	1.000	compute-bound	OK
V1	Llama-1 65B	1,048,576	64	34.28	4.215×10^9	4.215×10^9	1.903×10^9	1.000	1.000	compute-bound	OK
V1	Llama-3 70B	1,048,576	64	40.14	4.614×10^9	4.614×10^9	1.972×10^9	1.000	1.000	compute-bound	OK
V1	DeepSeek-V3	1,048,576	64	35.50	1.656×10^9	1.656×10^9	1.495×10^9	1.000	1.000	compute-bound	OK
Summary: 31 cells plotted (MFU $\leq$ 1, roofline-feasible) · 2 cells skipped (no physically valid trial) · 0 MFU $>$ 1 regressions · 0 V0/V1 loose rows.											

Table 27: Roofline lower-bound audit (compute floor + memory-BW floor). Lower bound: $\mathrm{LB} = \max\left(\frac{\text{total FLOPs}}{2\times\text{MACs}},\ \frac{\text{total bytes}}{\text{BW}}\right)$. Memory floor: bytes moved (HBM $\leftrightarrow$ PEs, all activations + weights) divided by BW (GB/s), converted to cycles at 1 GHz clock. Compute floor: total FLOPs $/(2 \times$ total MACs$)$ in cycles. The LP objective (obj cycles) must satisfy obj $\geq$ LB. `roof_ratio` $= \mathrm{LB}/\mathrm{obj} \leq 1.001$ is the pass criterion. Summary: 31 cells plotted · 2 skipped (no physically valid trial) · 0 violations.

Method	Model	MACs	BW	FLOPs	Bytes	Comp. LB	Mem. LB	Roofline LB	Obj	Roof ratio	Bottleneck	Verdict
Static FAST ILP												
Static	EN-B1	32,768	64	1.81×10^{12}	2.71×10^{8}	2.76×10^{7}	4.24×10^{6}	2.76×10^{7}	2.76×10^{7}	1.000	compute	OK
Static	EN-B2	65,536	64	3.38×10^{12}	3.50×10^{8}	2.58×10^{7}	5.47×10^{6}	2.58×10^{7}	2.58×10^{7}	1.000	compute	OK
Static	EN-B3	32,768	64	5.18×10^{12}	4.83×10^{8}	7.91×10^{7}	7.55×10^{6}	7.91×10^{7}	7.91×10^{7}	1.000	compute	OK
Static	EN-B4	65,536	64	1.37×10^{13}	8.78×10^{8}	1.04×10^{8}	1.37×10^{7}	1.04×10^{8}	1.04×10^{8}	1.000	compute	OK
Static	EN-B5	32,768	64	3.24×10^{13}	1.46×10^{9}	4.94×10^{8}	2.28×10^{7}	4.94×10^{8}	4.94×10^{8}	1.000	compute	OK
Static	EN-B6	32,768	64	6.52×10^{13}	2.24×10^{9}	9.95×10^{8}	3.50×10^{7}	9.95×10^{8}	9.95×10^{8}	1.000	compute	OK
Static	EN-B7	131,072	64	1.35×10^{14}	3.38×10^{9}	5.15×10^{8}	5.28×10^{7}	5.15×10^{8}	5.15×10^{8}	1.000	compute	OK
Static	Llama-1 13B	32,768	1024	1.77×10^{15}	3.05×10^{10}	2.71×10^{10}	2.98×10^{7}	2.71×10^{10}	2.71×10^{10}	1.000	compute	OK
Static	Llama-1 65B	65,536	128	8.84×10^{15}	1.22×10^{11}	6.74×10^{10}	9.51×10^{8}	6.74×10^{10}	2.38×10^{11}	0.284	compute	OK
Dynamic V0 LP (time-indexed + rematerialisation)												
V0	EN-B1	65,536	64	1.81×10^{12}	2.71×10^{8}	1.38×10^{7}	4.24×10^{6}	1.38×10^{7}	1.38×10^{7}	1.000	compute	OK
V0	EN-B2	131,072	64	3.38×10^{12}	3.50×10^{8}	1.29×10^{7}	5.47×10^{6}	1.29×10^{7}	1.29×10^{7}	1.000	compute	OK
V0	EN-B3	131,072	64	5.18×10^{12}	4.83×10^{8}	1.98×10^{7}	7.55×10^{6}	1.98×10^{7}	1.98×10^{7}	1.000	compute	OK
V0	EN-B4	131,072	64	1.37×10^{13}	8.78×10^{8}	5.22×10^{7}	1.37×10^{7}	5.22×10^{7}	5.22×10^{7}	1.000	compute	OK
V0	EN-B5	131,072	64	3.24×10^{13}	1.46×10^{9}	1.23×10^{8}	2.28×10^{7}	1.23×10^{8}	1.23×10^{8}	1.000	compute	OK
V0	EN-B6	262,144	64	6.52×10^{13}	2.24×10^{9}	1.24×10^{8}	3.50×10^{7}	1.24×10^{8}	1.24×10^{8}	1.000	compute	OK
V0	EN-B7	524,288	64	1.35×10^{14}	3.38×10^{9}	1.29×10^{8}	5.28×10^{7}	1.29×10^{8}	1.29×10^{8}	1.000	compute	OK
V0	Llama-1 13B	262,144	64	1.77×10^{15}	3.05×10^{10}	3.38×10^{9}	4.77×10^{8}	3.38×10^{9}	3.38×10^{9}	1.000	compute	OK
V0	Llama-1 65B	262,144	64	8.84×10^{15}	1.22×10^{11}	1.69×10^{10}	1.90×10^{9}	1.69×10^{10}	1.69×10^{10}	1.000	compute	OK
V0	Llama-3 70B	524,288	64	9.68×10^{15}	1.26×10^{11}	9.23×10^{9}	1.97×10^{9}	9.23×10^{9}	9.23×10^{9}	1.000	compute	OK
V0	DeepSeek-V3	262,144	128	3.47×10^{15}	9.57×10^{10}	6.62×10^{9}	7.48×10^{8}	6.62×10^{9}	6.62×10^{9}	1.000	compute	OK
Joint V1 LP (joint tiling + fusion)												
V1	EN-B1	131,072	64	1.81×10^{12}	2.71×10^{8}	6.89×10^{6}	4.24×10^{6}	6.89×10^{6}	6.89×10^{6}	1.000	compute	OK
V1	EN-B2	262,144	64	3.38×10^{12}	3.50×10^{8}	6.44×10^{6}	5.47×10^{6}	6.44×10^{6}	6.44×10^{6}	1.000	compute	OK
V1	EN-B3	262,144	64	5.18×10^{12}	4.83×10^{8}	9.88×10^{6}	7.55×10^{6}	9.88×10^{6}	9.88×10^{6}	1.000	compute	OK
V1	EN-B4	262,144	64	1.37×10^{13}	8.78×10^{8}	2.61×10^{7}	1.37×10^{7}	2.61×10^{7}	2.61×10^{7}	1.000	compute	OK
V1	EN-B5	524,288	64	3.24×10^{13}	1.46×10^{9}	3.09×10^{7}	2.28×10^{7}	3.09×10^{7}	3.09×10^{7}	1.000	compute	OK
V1	EN-B6	524,288	64	6.52×10^{13}	2.24×10^{9}	6.22×10^{7}	3.50×10^{7}	6.22×10^{7}	6.22×10^{7}	1.000	compute	OK
V1	EN-B7	1,048,576	64	1.35×10^{14}	3.38×10^{9}	6.44×10^{7}	5.28×10^{7}	6.44×10^{7}	6.44×10^{7}	1.000	compute	OK
V1	Llama-1 13B	1,048,576	64	1.77×10^{15}	3.05×10^{10}	8.45×10^{8}	4.77×10^{8}	8.45×10^{8}	8.45×10^{8}	1.000	compute	OK
V1	Llama-1 65B	1,048,576	64	8.84×10^{15}	1.22×10^{11}	4.22×10^{9}	1.90×10^{9}	4.22×10^{9}	4.22×10^{9}	1.000	compute	OK
V1	Llama-3 70B	1,048,576	64	9.68×10^{15}	1.26×10^{11}	4.61×10^{9}	1.97×10^{9}	4.61×10^{9}	4.61×10^{9}	1.000	compute	OK
V1	DeepSeek-V3	1,048,576	64	3.47×10^{15}	9.57×10^{10}	1.66×10^{9}	1.50×10^{9}	1.66×10^{9}	1.66×10^{9}	1.000	compute	OK

D.3 Six Physical-Design Feasibility Gates

In order to ensure and increase the real-world feasibility of our reported accelerator designs, every design in the CHEETAH framework must additionally pass six fab-grounded physical-design audit gates. The gate thresholds are calibrated end-to-end to reproduce TPU-v3's published headline numbers ($700\,\mathrm{mm}^2$ die, $250\,\mathrm{W}$ TDP, $0.36\,\mathrm{W/mm}^2$ density at $65{,}536$ MACs / $900\,\mathrm{GB/s}$ / $32\,\mathrm{MiB}$ CGM) when fed TPU-v3 inputs. Therefore, they are not arbitrary but pinned to a known-quality silicon.

(1) Pin count. Converts off-chip bandwidth into HBM3-class signal wires at $6.4\,\mathrm{Gb/s/pin}$ ($0.8\,\mathrm{GB/s/pin}$) and rejects designs needing more than $4{,}096$ high-speed pins, which is the practical ceiling for four HBM3 stacks on a single package. This prevents "free-bandwidth" LP solutions that silently assume 8+ stacks.

(2) Reticle area. Evaluates the linear area model

$$A_{\mathrm{die}} \;=\; \mathrm{MACs} \times 1.5 \times 10^{-4} \;+\; \mathrm{CGM_MB} \times 1.5 \;+\; \mathrm{dram_channels} \times 8 \;+\; 50 \quad [\mathrm{mm}^2]$$

(calibrated to a 7 nm process) and rejects any die exceeding $600\,\mathrm{mm}^2$. TSMC's N7 hard reticle is $858\,\mathrm{mm}^2$, and our design choice of 600 leaves headroom for yield, redundancy rings, and IO. This prevents over-scaled compute that would otherwise require monolithic chiplet packaging or multi-die exposure.

(3) Logic depth. Estimates the combinational gate-level depth of a single MAC-tree as

$$D \;=\; \lceil \log_2(\mathrm{sys_x} \times \mathrm{sys_y} \times \mathrm{MAC_bits}) \rceil + 8$$

and rejects depths above 35 levels. Beyond $\sim$35 levels, the critical path cannot meet a $1\,\mathrm{GHz}$ clock at TSMC N7 nominal V_{dd} without aggressive pipelining we do not model. This prevents designs whose systolic array is too wide for one cycle.

(4) Crossbar / NoC routing. Computes the cell count of the PE-to-SRAM interconnect as

$$N_{\mathrm{NoC}} \;=\; n_{\mathrm{PEs}} \times n_{64\,\mathrm{KB\ banks}}$$

and rejects values above 5×10^6 cells. The cap is the empirical budget for a 2-level hierarchical NoC (cluster-local crossbar + inter-cluster mesh) at modern AI-accelerator scales: flat all-to-all crossbars cap around 10^6 cells before routing dominates the floorplan, and we use the hierarchical-NoC cap with industry precedent (TPU-v4 sparse-core mesh, H100 SM-cluster L2 partitions, MI300X chiplet-local + XGMI, Cerebras WSE-3 per-tile fabric).

(5) Power density. Computes dynamic power as

$$P_{\mathrm{dyn}} \;=\; \mathrm{total_MACs} \times 1\,\mathrm{GHz} \times 0.5_{\mathrm{util}} \times 0.4\,\mathrm{mW/MAC}$$

(the $0.4\,\mathrm{mW/MAC}$ constant is calibrated against TPU-v3's $250\,\mathrm{W}$ envelope, and bumps to $1.2\,\mathrm{mW}$ for bf16), divides by the same linear area model from gate (2), and rejects anything above $2.0\,\mathrm{W/mm}^2$. This is the realistic ceiling for an AIO-cooled package today (TPU-v3 0.36, TPU-v4 0.55, H100 0.7, MI300X 0.6, Cerebras WSE-3 1.5). This prevents "thin-but-hot" tiny-MAC chips that would melt without exotic cooling.

(6) SRAM macro feasibility. Requires the global memory to decompose into a banked array of standard $64\,\mathrm{KiB}$ SRAM macros (the workhorse macro at 7 nm), caps the bank count at $4{,}096$ (the practical NoC routing limit), and requires the bank-array aspect ratio to lie within $[1{:}4,\ 4{:}1]$ so the macro grid can actually be laid out. A $17\,\mathrm{MiB}$ CGM organized as a 1×272 strip fails this even though the byte-count is fine.

Outcome. A design must pass all six gates before its speedup is deemed feasible and reported. Therefore, every speedup number and design we publish corresponds to a chip that, on paper, fits a single TSMC N7 reticle, has a wirable pin-out for four HBM3 stacks, meets $1\,\mathrm{GHz}$ timing, has a wirable NoC, fits an AIO cooler envelope, and decomposes into standard SRAM macros. The aim is to close the gap between research-level simulation and a chip that could plausibly be taped out, instead of presenting merely a theoretical Pareto-frontier point that exists only in LP space.

D.4 Why We Are Confident in the Software Side

The software claim is that the schedule the program commits to is the schedule that runs, and that its optimality is bounded rather than asserted. Five independent checks support it, ordered from strongest to weakest.

The committed schedule executes byte-exactly in RTL. The full DeepSeek-V3 schedule is compiled into a 1,383-instruction event tape and executed on a cycle-accurate SystemVerilog engine under Verilator (Section 9). The run completes with **zero** capacity violations; the RTL peak on-chip occupancy equals the analytical prediction *exactly* at 7,602,176 bytes; and the total cycle count lands within $+0.000001\%$ of the rounded LP objective. This is the strongest form of the claim available without silicon: the plan is not merely feasible on paper, it is executable byte-for-byte on hardware that enforces the capacity constraint at every step.

The checker is live, not vacuous. A validation suite is only meaningful if it can fail. We therefore replay a deliberately over-committed tape, and the capacity checker flags exactly one violation. A test that never fails proves nothing; this one fails when it should.

Optimality is bounded, not claimed. Every reported design is a certified achievable point, and we also solve the relaxation to obtain a bound no design in the model can beat. The pair brackets the unknown optimum (Section 7.7), and the bracket tightens to $\sim 13.5\%$ in the high-batch regime. Where the bracket is wide we say so rather than reporting the achievable alone.

Rounding is feasibility-preserving and independently validated. Relaxed selectors are rounded by enumeration over the physically integral choices, and no pin that would violate a constraint is ever committed. Each rounded design is re-solved and checked against the constraint set independently of the solver that produced it, with residuals at the 10^{-13} level. We never report a relaxed objective as an achievable result.

One search setting produces every number. All serving results come from a single documented search configuration (Appendix O), including the baselines, so no comparison mixes a finely searched design against a coarsely searched baseline. Cross-table consistency is verified at the shared operating point.

What this does not establish. RTL cycle counts are not used as an absolute oracle: the analytical per-layer cache floors are conservative by $192\times$ to $2475\times$, so RTL evidence is scoped to schedule realizability, memory legality, and utilization *mechanisms*. We make no claim that the simulated datapath is a performance-optimal implementation.

D.5 Why We Are Confident in the Hardware Side

The hardware claim is narrower and we state it precisely: the cost model that costs silicon is faithful enough that a design certified against it would be buildable. Design-space-exploration frameworks are conventionally validated three ways, against measured taped-out chips, against post-synthesis extraction, and against other frameworks [Mei et al., 2021]. We have all three.

Against post-synthesis extraction: the area coefficient and its functional form. We implement the scored datapath as synthesizable RTL and push it through OpenROAD to placement at the ASAP7 7 nm predictive PDK (Section 9.1). Extracted cell area fits $A = 82.08\,\mathrm{MACs} - 206.5\ \mu\mathrm{m}^2$ with $R^2 = 0.999999$, and area per MAC varies by only 0.46% across a $16\times$ range in MAC count. The second number matters more than the first: the program assumes area is *linear with a size-independent slope*, and that is exactly what a placed datapath exhibits. The coefficient we use is bracketed from below by extracted cell area and from above by our own conservative floorplan.

Against shipped silicon: the energy coefficient. Our compute-energy coefficient corresponds to 0.40 pJ per INT8 operation. Whole chip figures for shipped accelerators are 0.35 (H100), 0.64 (A100) and 1.27 pJ/op (TPU-v4i), and an array-only coefficient must sit at or just below whole-chip values. It does. We additionally measured the precision-energy ratio directly rather than assuming it, and the measurement corrected our analytic estimate in both directions (Section 9.2).

Against other frameworks: the mapping data. Every per-tile latency and byte count entering the program is measured Timeloop Pareto data, not an analytic surrogate.

Manufacturability is enforced, not audited. The six pre-RTL gates are rows of the program rather than post-hoc screens, so a gate can only ever be reported as satisfied. Every design in this paper passes all six.

What this does not establish. We have not taped out, and we do not run clock-tree synthesis or routing, so we make no claim about achievable clock frequency in silicon. Absolute power from our own flow remains an upper bound rather than a prediction: an ungated tile driven by uncorrelated data on a predictive PDK reports roughly three times our coefficient, and we therefore anchor the coefficient to measured silicon rather than to our own synthesis. The gating measurement in particular is not yet trustworthy and no clock-gating decision is scored in the program as a result.

E Speedup Analysis: Baseline Deficit Attribution and Design-Parameter Sensitivity

A large end-to-end speedup number is only credible if two things are shown explicitly: (i) the naive baseline actually *carries* that deficit, and the deficit can be *localized* to a physical cause; and (ii) the reported gain can be *decomposed* into a small number of design decisions, each of which is separately sized. This appendix supplies both. We first attribute the naive-baseline per-operation runtime into compute-active versus memory-stall (versus communication) time (Appendix E.1); we then decompose the 34.6–$40.2\times$ co-design speedup into exactly two orthogonal factors (Appendix E.2); we sweep each design decision one at a time and report the objective's sensitivity (Appendix E.3); and we rank the decisions by objective sensitivity to guide where a chip designer should spend area and power (Appendix E.4). This section is a companion to the pre-RTL feasibility gates of Appendix D: the gates certify that a design is *buildable*; this section certifies where its speedup *comes from*.

Reading conventions (read before any number below). Throughout, **MFU is model-MFU**: the roofline-tightness ratio $\mathrm{MFU} = t_{\mathrm{compute}}/t_{\mathrm{solver}}$ of Appendix D.2 (cache compute floor over the LP/analytic latency within one cost-model convention), *never* a measured-silicon utilization; the searched datapaths are hypothetical, not fabricated. **Same-silicon** comparisons are equal-area and equal-TDP *by construction* (both sides share the PE grid, so the area/power model of Appendix D.3 returns identical budgets). The 97% pre-fusion memory-stall figures and the naive MFU values are *naive-baseline attributions*, not properties of the co-designed schedule. Every quantity is convention-tagged, and we never form a ratio across two conventions. The fixed-silicon algorithmic gain on EfficientNet-B7 is $2.21\times$ under this paper's naive convention (a $2.81\times$ value that appears in an earlier discussion note is a transcription error and is not used).

E.1 Per-Operation Deficit Attribution

To justify a custom-silicon speedup on the order of 34–$40\times$ we must first show that the naive baseline wastes that much time, and say *where*. We attribute the naive per-operation runtime into three buckets: *compute-active* (the systolic array is issuing useful MACs), *memory-stall* (the array is idle waiting on off-chip traffic), and, for multi-device / mixture-of-experts execution, *communication* (inter-device transfer of dispatched tokens or expert activations). On a single package the communication bucket is empty by construction: all eight blueprints of Appendix I hold their active working set on-package (for DeepSeek-V3, the active-expert set is resident and the routing layer, not the datapath, pages the full model), so no operation incurs cross-device traffic.

The deficit is memory stall, not communication. Two independent attributions, on two different silicon points, agree that the naive deficit is almost entirely memory stall.

(a) On the co-designed chip (blueprint convention). Before fusion and time-indexed placement, naive per-operation streaming (each operation reads all of its tensors from DRAM and writes them back) leaves the array stalled on memory for $\mathbf{97.1}$–$\mathbf{97.5}\%$ of naive time (Table 28; individual blueprints report 97.1% for Llama-1 13B/65B, 97.2% for DeepSeek-V3, 97.5% for Llama-3 70B). This is large precisely because the co-designed chip provisions 20–$27\times$ more MACs than TPU-v3: the compute floor shrinks, so any unfused, re-streaming schedule becomes memory-bound. Fusion plus time-indexed residency keep the reused tensors on-chip and recover the schedule to the compute floor, $\mathrm{MFU} = 1.000$ (0.998 on Llama-1 13B; Table 47).

(b) On fixed TPU-v3 silicon (grid convention). On the $65{,}536$-MAC baseline itself, naive streaming reaches only $\mathrm{MFU} = 0.40$–0.64 of the compute floor on TPU-v3 (Llama-3 70B 0.399, DeepSeek-V3 0.451, Llama-1 13B 0.467, SD1.5-UNet 0.643), and up to 0.78 across the compute-bound cells of the wider fixed-hardware grid; on the bandwidth-starved NPU (64 GB/s) it collapses to $\mathrm{MFU} = 0.02$–0.05. The joint schedule recovers every compute-bound cell to $\mathrm{MFU} \approx 1.000$, so the fixed-silicon speedup is a *pure utilization recovery*, not additional compute.

The stalled time is avoidable, not compulsory. The 97% stall is *re-streaming* of tensors that could have stayed on-chip, not irreducible DRAM traffic. Two floor analyses bound the compulsory component. First, on each blueprint the memory floor (compulsory bytes over the chip's own bandwidth) is only 0.10–0.33% of the compute floor (Llama-1 13B $2.27\times10^{7}/2.31\times10^{10} = 0.10\%$;

Table 28: Naive-baseline deficit attribution. Compute-active and memory-stall fractions are $\mathrm{MFU}_{\mathrm{naive}}$ and $1 - \mathrm{MFU}_{\mathrm{naive}}$ on fixed TPU-v3 silicon (grid convention, scale-$T_{\max}$ naive). Pre-fusion memory stall is the blueprint-chip convention (naive per-op streaming on the co-designed datapath). Communication is zero on a single package. The last column is the joint schedule's recovered model-MFU. All values are naive-baseline attributions under one cost-model convention (Appendix D.2); they are not measured-silicon utilizations.

	Naive baseline (TPU-v3, per-op)			**Pre-fusion stall**	**Joint**
Workload	Compute-active	Memory-stall	Comm.	(chip, no fusion)	MFU
Llama-1 13B	0.467	0.533	0	97.1%	1.000
Llama-1 65B	–[†]	–[†]	0	97.1%	1.000
Llama-3 70B	0.399	0.601	0	97.5%	1.000
DeepSeek-V3	0.451	0.549	0	97.2%	1.000
SD1.5-UNet	0.643	0.357	0	–	0.963

[†]Llama-1 65B is not one of the five workloads in the fixed-hardware MFU grid; its pre-fusion stall is read from Table 48. Across the compute-bound cells of the wider grid, naive MFU spans 0.40–0.78; the bandwidth-starved NPU cells sit at 0.02–0.05.

Llama-3 70B 0.31%; DeepSeek-V3 0.33%; Appendix I). Second, a mapping-independent attainable-traffic analysis in the style of Orojenesis [Huang et al., 2024] places the attainable per-Einsum DRAM floor at $\mathbf{0.1}$–$\mathbf{0.9}\%$ of the compute floor on every cell we measured (Llama-3 70B 0.116%, DeepSeek-V3 0.182%, SD1.5-UNet 0.896%), because at the deployed prefill-like batch the weight reuse is large. Both bound the same conclusion: the compulsory off-chip traffic that *no* schedule can remove is under 1% of the compute floor, so essentially the entire naive memory-stall deficit is recoverable by scheduling (fusion, time-indexed residency, and tile choice) and the recoverable ceiling is not gated by DRAM bandwidth or by network communication.

Planned figure: per-operation stacked time breakdown. We will add a per-operation stacked-bar figure (one bar per HLO operation, ordered by graph position; each bar split into compute-active / memory-stall / communication seconds) for the naive baseline and for the CHEETAH schedule side by side, on all four LLM blueprints, in the style of the FAST per-layer utilization plot [Zhang et al., 2022] and Eyeriss-style per-layer accounting [Chen et al., 2019]. The naive bars are dominated by the memory-stall segment ($\sim 97\%$ aggregate); the CHEETAH bars are almost pure compute-active (aggregate $\mathrm{MFU} \to 1$); the communication segment is empty on a single package and appears only in the multi-device / MoE variant of Appendix E.3. This figure is a *planned appendix experiment* (the underlying per-op compute/stall decomposition exists in the solver output; the figure is not yet rendered).

E.2 Two-Factor Speedup Decomposition

The headline speedup factors *exactly* into two orthogonal terms:

$$\underbrace{\frac{T_{\mathrm{naive}}(\text{TPU-v3})}{T_{\mathrm{joint}}(\text{chip})}}_{\text{headline}} = \underbrace{\frac{T_{\mathrm{naive}}(\text{TPU-v3})}{T_{\mathrm{joint}}(\text{TPU-v3})}}_{f_1\text{: scheduling recovery}} \times \underbrace{\frac{T_{\mathrm{joint}}(\text{TPU-v3})}{T_{\mathrm{joint}}(\text{chip})}}_{f_2\text{: MAC provisioning}}. \tag{30}$$

The first factor f_1 is a *fixed-silicon*, same-area / same-TDP algorithmic gain: it re-schedules the same $65{,}536$-MAC TPU-v3 and buys $\mathbf{1.48}$–$\mathbf{1.78}\times$. The second factor f_2 holds the joint schedule fixed and provisions the PE grid up to the co-designed chip's MAC count, buying $\mathbf{20.0}$–$\mathbf{26.9}\times$. Their product reproduces the per-workload Figure-2 headline (and the blueprint "achieved speedup vs. naive TPU-v3" rows) to within 1.4% (Table 29); the residual is the analytic chip-endpoint solve's utilization roll-off, cross-checked against the blueprints' Timeloop-in-loop objective.

Is the provisioning factor physically usable? A reader is entitled to ask whether f_2 is real parallelism or an artifact of a cost model in which compute time falls as $1/\mathrm{MACs}$ without bound. It is a fair question: an array can only speed an operation up as far as that operation supplies independent tiles to fill it, and a per-token decode GEMV supplies few. We measured the answer directly rather than argue it. For every event of DeepSeek-V3 we computed the *saturating* MAC

Table 29: Two-factor decomposition of the co-design speedup (Eq. (30)). f_1 is the fixed-silicon scheduling gain on TPU-v3 (same area/TDP), f_2 the MAC-provisioning gain to the co-designed chip. Reconstructed $= f_1 \times f_2$; headline is the blueprint value. Convention: config-0 naive; bf16 cycle basis; HiGHS-Optimal endpoints.

Workload	f_1 (sched., TPU-v3)	f_2 (MAC prov.)	Reconstructed	Headline	Recon./Headline
Llama-1 13B	1.662	20.31	33.75	34.22	0.986
Llama-1 65B	1.727	20.04	34.60	34.60	1.000[‡]
Llama-3 70B	1.484	26.85	39.84	40.16	0.992
DeepSeek-V3	1.784	19.97	35.63	35.68	0.999
Range	**1.48–1.78**	**20.0–26.9**	**34.6–40.2** (matches Fig. 2)		

[‡]Llama-1 65B's analytic chip-endpoint solve was degenerate under HiGHS time limits, so its f_2 uses the authoritative blueprint objective (exact by construction); the other three use the *independent* analytic chip solve and land within 1.4%.

count $M_{\text{sat}} = (\text{independent output tiles}) \times (\text{lanes per tile})$ under the same tile geometry the mapper uses, where independent tiles are row tiles times column tiles (k-tiles accumulate into one output and are not independent) times the number of GEMMs carrying work (expected active experts for routed FFNs). Beyond M_{sat} a chip has no more concurrent tiles to run for that event, and its compute time cannot keep falling. Table 30 gives the distribution in multiples of the TPU-v3 array.

Table 30: Saturating MAC count per event, DeepSeek-V3, as a multiple of TPU-v3's $65{,}536$ MACs. Prefill events saturate three to four orders of magnitude above the largest chip in this paper ($24.4\times$), so the provisioning factor f_2 is physically usable parallelism at prefill. Decode events saturate at 9–$14\times$ at the low end, which is where a bound on usable parallelism would bind, and only for chips beyond it.

Regime	Phase	min	p10	median
B=8, ctx 4096	prefill	3,584×	3,584×	16,384×
	decode	9×	14×	16×
B=32, ctx 4096	prefill	14,336×	14,336×	65,536×
	decode	14×	14×	32×
B=32, ctx 512	prefill	1,792×	1,792×	8,192×
	decode	14×	14×	32×

Two things follow. First, at the prefill operating point of Fig. 2 the bound can never be active: a 4096-token prefill GEMM supplies thousands of times more independent tiles than a $24.4\times$ chip can consume, so f_2 is not a modeling artifact and no correction to the headline is owed on this ground. We confirmed this empirically by adding the saturation bound to the sizing program as an explicit companion row (compute charged at M_{sat} in addition to the free $1/\text{MACs}$ term, a valid restriction) and re-solving the joint sizing on the real model in three regimes: B=8, G=4; B=32, G=16 (the serving point of Table 9); and a decode-weighted case (B=32, context 512, MAC domain to $25\times$) whose unbounded optimum, $16.8\times$ TPU-v3, lies *above* the decode saturation so that the bound is provably active on the low-parallelism decode events. In all three the chosen chip, the certified objective, and the lower bound were identical with and without the bound to ten significant figures (certified gaps 3×10^{-5} to 4×10^{-4}). Where the bound is active it does not move the optimum because decode compute is not the binding term in a memory-bound decode; where compute binds, at prefill, the bound is never active. Second, the concern is legitimate exactly where the table says it is, in decode on chips beyond roughly $14\times$ TPU-v3, and it attaches to the serving results of Section 7.6 rather than to the prefill headline; there, decode compute is rarely the binding term (decode is memory-bound at these batches, Table 9), which is why the correction is small even where it applies. Two caveats bound the claim: the geometry is the mapper's tile model, not a placed array, and the separate question of non-matrix work (softmax, normalization, routing) growing relative to matrix work as the array scales is not addressed by this bound and remains an open correction.

Use only the two-factor form; do not multiply the full decision ladder. It is tempting to read the software-ablation ladder (tile, placement, remat, datapath, datatype) as a chain of multiplicative factors and multiply them all. That *overshoots* the headline by 1.8–$2.1\times$ (a mechanical product of 70.7–$73.2\times$ versus the 34–$40\times$ headline), for two convention reasons, neither of them physics:

1. **The INT8 throughput label double-counts.** The blueprint headline T_{joint} is a *bf16 cycle count* at the chip's MAC count; INT8 (W8A8) is carried as a throughput / power label (roughly half the area and power per MAC), *not* as a separate cycle factor. Multiplying an extra $2\times$ for INT8 into a cycle product is a double-count. Dropping it lands the mechanical ladder at 35–37×, within 3–12% of the headline.
2. **The software rows use a $\sim 1.3\times$ smaller naive baseline** (the scale-$T_{\max}$ naive) than the config-0 naive against which the headline is measured, so the two families of factors are not on the same numerator.

Equation (30) is the form that reconciles *exactly* because f_1 and f_2 share a single naive numerator and a single cycle basis. As a naming caution: the same fixed-silicon scheduling recovery is 1.48–1.78× against the config-0 naive (used above) and 2.14–2.51× against the grid's scale-$T_{\max}$ naive (the fixed-hardware grid convention of Appendix E.1); the two differ only by the naive-baseline definition, and must never be crossed within one ratio.

The provisioning factor is sub-linear (the anti-"just add MACs" result). f_2 is not free scaling. Measured as delivered throughput per provisioned MAC in an equal-area / equal-TDP resource-matched ablation, 20.0–27.0× more MACs realize only $\approx 16\times$ provisioning throughput, a linearity of $\mathbf{0.59}$–$\mathbf{0.81}$ (Llama-3 70B is worst, 27× MACs → 16.0×). Equivalently, the fixed-family MAC leg from 65,536 to 1,048,576 MACs (a 16× increase) yields only 14.0–15.7× cycle reduction, because a residual memory floor remains. Added silicon buys sub-linearly, which is exactly why capturing the free scheduling factor f_1 first (and provisioning MACs only up to the point of diminishing return) is the efficient order of operations.

E.3 One-Decision-at-a-Time Sensitivity Sweeps

We now hold the design fixed and sweep a single decision, reporting the objective (LP cycles) sensitivity. Two decisions have dedicated measured sweeps (global buffer, MAC count); the remaining fine-grained sweeps (per-unit peak-FLOPS, interconnect radix / link count) are framed here and marked as **planned** appendix experiments so they can be added to the paper's experiment list; we do not fabricate their numbers.

(a) Global-buffer size C_{GM}: measured, near-inert in the compute-bound regime. Table 31 sweeps C_{GM} from 1 to 64 MiB for DeepSeek-V3 on TPU-v3 (900 GB/s), for the joint (free-y) LP and for a fixed-tile FAST-subset LP. The joint LP is *flat*: its objective moves 0.39% across a 64× buffer sweep ($1.0723 \times 10^{12} \rightarrow 1.0682 \times 10^{12}$ cycles), and is bit-identical from 8 MiB upward. By contrast the fixed-tile schedule *needs* the buffer, improving 1.18× over the same sweep. The joint formulation substitutes tile choice for buffer capacity: when tiling is free, a larger global buffer buys almost nothing. The tile-dependent capacity extension is likewise inert here: it matches stock V2 to within 0.077% across the full $C_{\text{GM}} \in \{4, \ldots, 64\}$ MiB sweep. Under bandwidth pressure (64–128 GB/s) the buffer regains a modest decision ($\approx 1.19\times$ for Llama-1 13B over $8 \rightarrow 64$ MiB), but *only through placement*: with placement frozen (row A) the objective is constant across C_{GM}, so C_{GM} helps exclusively by enabling time-indexed residency, never on its own. The blueprints adopt $C_{\text{GM}} = 64$ MiB.

Table 31: Global-buffer sweep, DeepSeek-V3 on TPU-v3 (900 GB/s, native cache convention, HiGHS-Optimal). The joint (free-y) LP is essentially C_{GM}-insensitive (0.39% over a 64× sweep); the fixed-tile FAST-subset LP is not (1.18×). Objective in LP cycles.

C_{GM} (MiB)	1	2	4	8	16	32	64	sweep (1→64)
Joint (stock V2, free-y)	1.0723	1.0699	1.0690	1.0682	1.0682	1.0682	1.0682	$\times 10^{12}$; 1.004×
Fixed-tile (FAST-subset)	1.4627	1.4459	1.4327	1.4161	1.3863	1.3364	1.2371	$\times 10^{12}$; 1.182×

(b) Peak throughput / MAC-unit count: measured (count), planned (per-unit rate). The total-MAC provisioning sweep is measured: raising the PE grid from 65,536 to 1,048,576 MACs (a 16× increase) reduces the objective by 14.0–15.7× (Llama-1 13B 15.72×, DeepSeek-V3 14.98×, Llama-3 70B 14.04×, Llama-1 65B 14.02×), and the provisioning factor to the full co-designed chip is $f_2 = 20.0$–$26.9\times$ (Appendix E.2), both sub-linear (§E.2). This is the single largest decision

on the objective. A *finer* sensitivity that varies per-unit peak throughput (systolic-array dimensions, vector-lane width, or clock) at a *fixed* MAC count and fixed area, separating "more units" from "faster units", is not yet run and is marked **planned**; the datapath template of Appendix D.3 and MAGNet-style generators [Venkatesan et al., 2019] already parameterize it.

(c) DRAM bandwidth: measured, inert in the compute-bound regime. Sweeping bandwidth from 900 to 3,350 GB/s at the fixed 65,536-MAC point moves the objective by $\leq 0.25\%$ (BW leg $= 1.000$–$1.0025\times$ for all four LLMs): at the deployed prefill-like batch these workloads are compute-bound, so bandwidth beyond the floor buys nothing. Bandwidth becomes the dominant decision only in the bandwidth-starved regime: at 64 GB/s (the compact NPU) the joint-vs-naive fixed-hardware gain is 5.9–$31.4\times$, because there the roofline is memory-bound and residency / placement recover a much larger share. Thus bandwidth sensitivity is regime-dependent: near-zero when compute-bound, first-order when memory-bound.

(d) Interconnect: on-chip NoC and inter-device topology (planned). Two interconnect decisions are framed but not yet swept. *On-chip*, the NoC / crossbar radix and link count set the PE-to-SRAM routing capacity; this is the resource bounded by the crossbar / NoC-routing gate of Appendix D.3 (gate 4, $N_{\mathrm{NoC}} = n_{\mathrm{PEs}} \times n_{\mathrm{banks}} \leq 5 \times 10^6$ cells), and its provisioning follows the network-on-package accounting of Simba [Shao et al., 2019]. In the single-chip compute-bound regime the on-chip NoC is provisioned to the compute floor and is not the binding resource, but a dedicated radix / link sweep (objective versus NoC cells at fixed MACs) is **planned**. *Across devices*, in the multi-GPU / mixture-of-experts regime the inter-device topology (radix and link count of the NVLink-class fabric) binds only when experts are dispatched *across* devices; when the active-expert set is package-resident (as in every blueprint here) the communication term is zero (Appendix E.1). A distributed / MoE topology sweep (objective and the communication bucket versus inter-GPU radix and link count, in the spirit of Chiplet-Cloud and wafer-scale serving analyses [Peng et al., 2024, Zhu et al., 2024] and co-optimized network provisioning [Rashidi et al., 2023]) is **planned**.

Table 32: Single-decision sensitivity summary (compute-bound LLM regime, TPU-v3 operating point). "Objective sensitivity" is the LP-cycle response to the stated sweep. Cells marked **planned** have no dedicated run yet and are listed as appendix experiments; their numbers are not fabricated.

Decision	Sweep	Objective sensitivity	Status
Global buffer C_{GM}	$1 \rightarrow 64$ MiB (BW$=900$)	$1.004\times$ (joint); $1.18\times$ (fixed-tile)	measured
	under BW pressure (≤ 128)	$\approx 1.19\times$, via placement only	measured
MAC count (total)	$65{,}536 \rightarrow 1{,}048{,}576$ ($16\times$)	14.0–$15.7\times$ (sub-linear); $f_2 = 20.0$–$26.9\times$	measured
Peak rate / unit	systolic dims, clock @ fixed MACs	–	**planned**
DRAM bandwidth	$900 \rightarrow 3{,}350$ GB/s	$\leq 1.003\times$ (compute-bound)	measured
	64 GB/s (NPU)	5.9–$31.4\times$ (memory-bound)	measured
On-chip NoC radix / links	N_{NoC} at fixed MACs	– (gate 4 bounds it)	**planned**
Inter-device topology (MoE)	inter-GPU radix / links	– (binds on cross-device dispatch)	**planned**

E.4 Decision-Impact Ranking

Table 33 ranks the decisions by objective sensitivity in the compute-bound LLM regime and states *why* each sits where it does, so a chip designer can read off where area and power are best spent.

Why this ordering, and what it tells a designer. In the compute-bound LLM regime the objective is set by how many useful MACs the schedule can keep busy. **(1) MAC-count provisioning** moves the objective most (20–$27\times$) but sub-linearly and at direct area/power cost, so it is the decision to spend on, up to the point of diminishing return, and never past the feasibility gates of Appendix D.3 (a bigger matrix unit can even degrade delivered performance, as observed for production TPU silicon [Jouppi et al., 2017]). **(2) Scheduling** is a *free* multiplier (1.48–$1.78\times$ at equal area and TDP) that must be captured first: it costs no silicon, it is what recovers naive MFU 0.40 to ≈ 1.0, and at tight buffers it is a feasibility enabler, not merely an optimization (on EfficientNet-B7 at TPU-v3's 32 MiB buffer the pure-streaming row is infeasible and time-indexed placement restores feasibility: placement alone at $2.03\times$, joint optimum $2.21\times$). **(3) DRAM bandwidth** and **(4) global buffer** are near-inert once compute-bound (bandwidth beyond the floor and buffer beyond a few MiB buy

Table 33: Design decisions ranked by objective sensitivity in the compute-bound LLM regime (TPU-v3 operating point). "Cost" is what the decision spends on the die. The ranking inverts in the memory-bound / distributed regimes, as noted.

Rank	Decision	Sensitivity	Cost	Binds when
1	MAC-count provisioning	20.0–26.9× (sub-linear)	area + power	always (compute-bound); diminishing returns
2	Scheduling (tile+fusion+placement)	1.48–1.78×	none (same silicon)	always; also a feasibility enabler at tight buffers
3	DRAM bandwidth	$\approx 1.0\times$; 5.9–31.4× if starved	pins + power	memory-bound regime only
4	Global buffer C_{GM}	$\approx 1.0\times$; $\leq 1.19\times$ pressed	area (SRAM)	buffer-pressed regime; only via placement
5	Interconnect (NoC / inter-GPU)	$\approx 1.0\times$ on-chip	area + links	distributed / MoE cross-device dispatch

essentially nothing; Table 31), and the joint LP actively trades buffer for tile choice, so paying for HBM or a large global SRAM buys lower TDP-per-TFLOP, not more throughput, at these operating points. Both become first-order only when the workload is genuinely memory-bound (low bandwidth, or decode-time batch-1 operational intensity). **(5) Interconnect** is inert on a single compute-bound chip and matters only in the distributed / MoE regime, when experts are dispatched across devices and the communication bucket becomes nonzero. The practical guidance: *buy MACs (watching the sub-linear roll-off and the gates), capture scheduling for free, and invest in bandwidth, buffer, or interconnect only when the regime (memory- or communication-bound) actually makes them bind.*

Sweeps requiring new runs (for the paper's experiment list). The following are framed above but not yet run, and are proposed as appendix experiments: (i) a per-unit peak-throughput sweep (systolic dimensions / clock at fixed MAC count and area), isolating "faster units" from "more units"; (ii) an on-chip NoC radix / link-count sweep (objective versus N_{NoC} at fixed MACs, tied to gate 4); (iii) an inter-device / MoE topology sweep (objective and the communication bucket versus inter-GPU radix and link count, with experts dispatched across devices); and (iv) the per-operation stacked compute/stall/communication time-breakdown figure of Appendix E.1.

F Qualitative Design Discussion

Wall-clock time decomposition and search acceleration results are presented in Figure 6 and Section 7.

F.1 Interesting Empirical Design Results

Table 34: Surprise design pairs discovered by CHEETAH-V1. Each pair achieves near-identical Perf/TDP via qualitatively different strategies that are not intuitive to human engineers.

Label	Workload	Trial	Perf/TDP ×	Speedup ×	MACs	CGM	L1 in/w/o (KB)	L1 cfg	L2 cfg	Batch
TPU-v3 ref	–	–	1.000	1.000	65,536	32 MB	64 / 64 / 64	Shared	Disabled	64
DeepSeek A	`deepseek_v3`	81	3.158	25.125	524,288	8 MB	512 / 1 / 256	Shared	Disabled	64
DeepSeek B	`deepseek_v3`	95	3.146	12.512	262,144	8 MB	1 / 256 / 512	Shared	Disabled	128
Llama-3 70B A	`llama3_70b`	88	3.557	14.152	262,144	32 MB	8 / 256 / 4	Private	Shared	64
Llama-3 70B B	`llama3_70b`	92	3.570	28.402	524,288	1 MB	512 / 128 / 512	Shared	Shared	64

We examine two designs on the DeepSeek-V3 (trials 81 vs. 95) and Llama-3 70B (trials 88 vs. 92) workloads. In each case, Perf/TDP scores are nearly identical yet design specifications are hugely different. DeekSeek's Design B uses 262K MACs vs. 524K in Design A. B compensates via richer search and larger batch. Pareto-tile space is 50× larger in B (500 vs. 10 configs); batch size doubles (128 vs. 64). Mirror-image L1 sub-buffer allocation in A keeps 512 KB input + 1 KB weight + 256 KB output. Design B keeps 1 KB input + 256 KB weight + 512 KB output. These two entirely opposite on-chip memory strategies yield identical system efficiency. On Llama-3 70B pair (trials 88 vs. 92), design A uses 262K MACs and 32 MB Global Buffer; Design B uses 524K MACs and only 1 MB Global Buffer. However Design A is weight-stationary, with a large 256 KB weight L1, tiny 8/4 KB input/output L1, and weight reuse amortised over a large on-chip scratchpad. Whereas Design B is activation-stationary, with 512/512 KB input/output L1, only 128 KB weight L1, and shared L1 configuration seen in doubled MACs allow CGM to collapse to 1 MB. This provides insight that additional compute in B substitutes for global memory capacity: higher throughput reduces the time activations must remain resident, relaxing the CGM capacity constraint.

F.2 Hardware and Vizier Search

In all experiments, we utilize 16 NVIDIA H100 GPUs. Each unit in the configuration provides 80 GB of VRAM, totaling 1.28 TB of available memory across the cluster. The system is characterized by a high memory bandwidth of approximately 3.35 TB/s per GPU, which is critical for the memory-intensive workloads examined in this work. In order to be consistent with prior work on accelerator design [Zhang et al., 2022], we utilize Vizier to simulate TPUv3 and perform hardware search. Table 35 presents Vizier's search parameter groups for completeness, and Fig. 11 depicts standard accelerator datapath configuration layout.

A Processing Element (PE) is a repeated compute tile in an accelerator template. The datapath can be modeled as a grid of PEs connected by a mesh on-chip network. PE systolic arrays perform a matrix-vector multiply each cycle. Vector and scalar MACs are modeled by setting systolic array X and Y dimensions to 1. PEs also contain a VPU for non-MAC vector ops. L2 and Global Memory structures are optional.

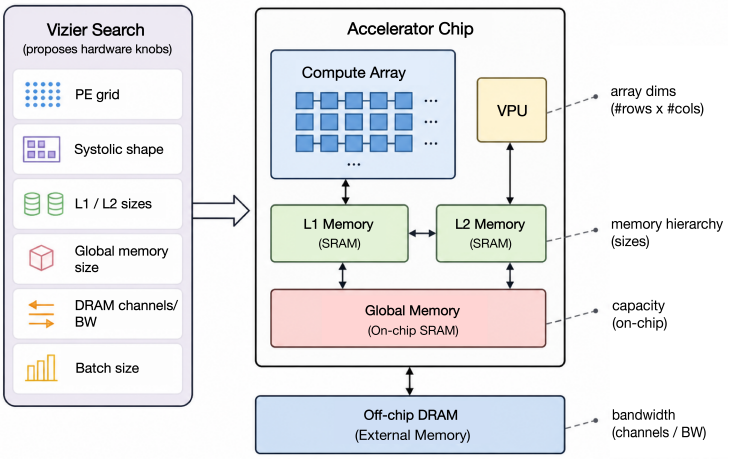


Figure 11: The accelerator datapath configuration. Processing elements (PEs) are arranged in a grid connected by a mesh network. Each PE contains a systolic array for matrix operations, a vector processing unit (VPU) for non-MAC operations, and private or shared L1/L2 memory. A Global Memory serves as the on-chip scratchpad shared across all PEs.

Table 35: Vizier search parameter groups and ranges: initial small search vs. expanded V1-wide search.

Group	Parameter	Small (initial)	LLM-V1-wide
A: Hardware datapath (16 params)	`PEs_x_dim`	pow2 in 4 . . . **32**	pow2 in **1** . . . **256**
	`PEs_y_dim`	pow2 in 4 . . . **32**	pow2 in **1** . . . **256**
	`Systolic_array_x`	pow2 in 4 . . . **32**	pow2 in 4 . . . **128**
	`Systolic_array_y`	pow2 in 4 . . . **32**	pow2 in 4 . . . **128**
	`Vector_unit_multiplier`	pow2 1 . . . 16	pow2 1 . . . 16
	`L1_buffer_config`	{Private, Shared}	{Private, Shared}
	`L1_input_buffer_size`	pow2 **1** KB . . . 4 MB	pow2 **1** KB . . . **1** MB
	`L1_weight_buffer_size`	pow2 **1** KB . . . 4 MB	pow2 **1** KB . . . **1** MB
	`L1_output_buffer_size`	pow2 **1** KB . . . 4 MB	pow2 **1** KB . . . **1** MB
	`L2_buffer_config`	{Disabled, Private, Shared}	{Disabled, Private, Shared}
	`L2_input_buffer_mult.`	pow2 1 . . . 128	pow2 1 . . . 128
	`L2_weight_buffer_mult.`	pow2 1 . . . 128	pow2 1 . . . 128
	`L2_output_buffer_mult.`	pow2 1 . . . 128	pow2 1 . . . 128
	`L3_global_buffer` (CGM)	{0}∪ pow2 **1** . . . **64** MB	{0}∪ pow2 **1** . . . **256** MB
	`GDDR6_channels`	pow2 **1** . . . **16** (→ 64–1024 GB/s)	pow2 **1** . . . **32** (→ 64–2048 GB/s)
	`Native_batch_size`	pow2 **1** . . . **1024**	pow2 **1** . . . **256**
B: Compiler passes (3 params)	`two_pass_softmax`	{disabled, enabled}	{disabled, enabled}
	`tensor_padding`	{disabled, enabled}	{disabled, enabled}
	`datatype`	{bfloat16, int8}	{bfloat16, int8}
C: V0 rematerialisation (2 params)	`remat_enabled`	{disabled, enabled}	{disabled, enabled}
	`remat_cost_scale`	{0.1, 0.25, 0.5, 1.0, 2.0, 4.0}	{0.1, 0.25, 0.5, 1.0, 2.0, 4.0}
D: V1 joint-tiling (4 params)	`pareto_config_count_C`	{10, 50, 100, 250, 500, 1000}	{10, 50, 100, 250, 500, 1000}
	`loop_ordering_strategy`	{weight_stationary, output_stationary, both}	{weight_stationary, output_stationary, both}
	`pareto_pruning_strategy`	{tmax_workingset, tmax_tmin, tmin_workingset}	{tmax_workingset, tmax_tmin, tmin_workingset}
	`tile_workingset_modeling`	{fixed, config_dep}	{fixed, config_dep}
Post-Vizier gates (filter, not search)	`MIN_TOTAL_MACS`	32,768 (4×4×4×4=256 lower physical, 32K gate)	
	`MAX_TOTAL_MACS` cap	none: implicit ≤ 32×32×32×32 ≈ 1 M	1,048,576 explicit cap
	Implied total_MACs reach	≈ 256 . . . 1 M	256 . . . 1,048,576 (post-cap)
	Implied DRAM BW reach	64 . . . 1,024 GB/s	64 . . . 2,048 GB/s

G LLM-Architect Outer Loop Details

The LLM-Architect outer loop proceeds through the following steps for each campaign round:

1. **Extraction.** Deterministic scripts summarize recent trials into a *Causal Ledger*. Each record includes hardware parameters, validity, Perf/TDP, LP cycles, capacity duals/slacks, bandwidth sensitivity, MFU, memory utilization, roofline gap, failure class, and LP solver metrics.
2. **Context building.** The last N trials and the Causal Ledger are compressed into a compact JSON context for the Strategist. The context emphasizes empirical feasibility, observed invalid regions, best known valid designs, and physical audit signals.
3. **Architect.** The online Architect, implemented as Qwen2.5-7B-Instruct with a LoRA adapter trained on past trial logs, diagnoses the dominant bottleneck and emits a campaign patch. The patch constrains macro-architectural search decisions such as Global Memory, DRAM bandwidth, total MACs, systolic shape, PE grid, and batch size. A larger teacher model, DeepSeek-V3.2, was used offline to generate or critique campaign labels, but is not in the live loop.
4. **Campaign validation.** A deterministic validator clips or rejects the LLM patch to enforce legal hardware ranges, TDP and area limits, MAC caps, CGM feasibility floors, compute/bandwidth coupling, and roofline constraints. This prevents the LLM from proposing physically implausible regions such as maximum-MAC/minimum-bandwidth designs.
5. **Local optimization.** Vizier is initialized inside the validated patch and warm-started with prior trials that lie in the same region. Vizier then samples concrete hardware candidates within the approved campaign. The LLM chooses the high-level architectural neighborhood, while Vizier performs local adaptive optimization.
6. **Unified V1 Formulation.** The large-scale LP solver evaluates each candidate by solving the V1 joint tiling-fusion-placement LP using saved Pareto $\mathcal{C}$-sets. It returns the objective, cycles, selected tiling configurations, placement/rematerialization decisions, dual/slack summaries, and solver-quality diagnostics.
7. **Roofline audit.** A deterministic Roofline Auditor checks whether the predicted latency is physically plausible. For each candidate, we compute

$$t_{\text{compute}} = \frac{\text{useful FLOPs}}{\text{peak FLOPs/s}}, \qquad t_{\text{memory}} = \frac{\text{post-placement DRAM bytes}}{\text{peak bandwidth}},$$
$$t_{\text{ideal}} = \max(t_{\text{compute}},\, t_{\text{memory}})\,. \tag{31}$$

   A candidate is rejected if the solver-reported latency falls below this lower bound, or if MFU or memory utilization exceeds 1.0 within tolerance.
8. **Ledger update.** The loop records the campaign hypothesis, intervention, observed invalid-rate change, MFU change, roofline validity, and best Perf/TDP change. The ledger also records forbidden regions when a campaign produces excessive invalid trials.
9. **Shortlist Pareto refinement.** At the end of a campaign or study, only the top-ranked candidates are passed to a Pareto refinement run. This reruns targeted Timeloop refinement for selected configurations and re-solves V1 LP, providing a final trust check. This is an additional and optional sanity check step.

The Architect receives workload summaries, invalid-rate history, MFU, roofline signals, and prior valid regions, normalized LP duals and bandwidth sensitivities. The system starts from a safe empirically valid patch and allows duals only to make bounded edits, such as raising the CGM floor, increasing bandwidth, reducing MAC count, or changing systolic shapes. The validator rejects any dual-guided patch that excludes known high-performing valid regions or predicts a substantially worse feasible rate.

G.1 Profile-Conditioned LLM-Architect Application

We evaluate if the Architect can condition hardware search on deployment intent, since speech models may prioritize latency while data analytics models may prioritize bandwidth. For each

workload, we run the same Qwen2.5-LoRA Strategist, validator, random-in-patch sampler, and V1 LP evaluator while changing only a two-paragraph profile block describing the user's objective and constraints. The resulting best chips differ systematically by profile: overnight batch profiles select higher-bandwidth and larger-CGM regions, compact-edge profiles select smaller feasible regions with the lowest invalid rate, and interactive profiles favor small-CGM, extreme-aspect datapaths. Since the inner V1 LP and validator are identical across profiles, these differences isolate the effect of the LLM's profile-conditioned search patch. The absolute Perf/TDP often remains within a narrow band because the V1 evaluator reaches a similar structural ceiling, but the hardware families used to reach that ceiling differ substantially. We simulate two user profiles: (1) interactive speech is a spoken dialogue model where latency is more important than bandwidth, and (2) overnight finance batch where the user requires a large amount of financial data to be processed, but might leave it overnight to complete hence bandwidth is more important than latency.

The LLM-Architect clearly learned the profile distinction. Different profiles select meaningfully different chips for the same workload. The signal flows through the LLM's patch choice (steered by the profile prompt), and random-in-patch sampling finds the best chip within each profile-narrowed region. Table 36 gives examples of best designs per profile.

Table 36: Best hardware configuration discovered per (deployment profile, workload) cell. PE SA = PE grid $\times$ systolic array dimensions. MACs = total multiply-accumulate units. BW = DRAM bandwidth (GB/s). CGM = Global Buffer size (MB). Interactive speech = small low-latency-style chip, overnight finance batch = high-throughput, high-bandwidth, compact edge = compact constrained chip, balanced cloud = moderate/general design.

Workload	Profile	PE grid	Systolic	MACs	BW (GB/s)	CGM (MB)	Best Perf/TDP $\times$
Llama-1 13B	`interactive_speech`	1×128	16×16	32K	512	2	2.06
	`overnight_finance_batch`	16×8	128×32	524K	2048	256	2.73
	`compact_edge`	8×2	64×32	32K	512	4	2.06
	`balanced_cloud`	2×256	8×16	65K	64	128	2.97
DeepSeek-V3	`interactive_speech`	1×32	8×128	32K	2048	16	1.07
	`overnight_finance_batch`	8×256	16×8	262K	2048	1	2.55
	`compact_edge`	16×64	16×4	65K	128	2	2.99
	`balanced_cloud`	256×2	32×16	262K	1024	32	2.83

- `interactive_speech` repeatedly selects extreme-aspect PE arrays (1×32, 1×128, 1×256, 2×128); these favour narrow systolic streaming for low single-batch latency. CGM is consistently small (2–4 MB), trading cache capacity for compute density. Matches the profile intent: "minimize end-to-end latency, batch ≤ 8".

- `overnight_finance_batch` consistently selects high bandwidth (1024–2048 GB/s) and often large CGM (16–256 MB). Matches the profile intent: "maximize throughput, large batch, sustained bandwidth".

- `compact_edge` consistently selects the smallest chips (32–65K MACs is common vs. 524K for other profiles) with tiny CGM (1–4 MB). Matches the profile hard constraints: TDP $\leq 0.5\times$, area $\leq 0.5\times$, mobile deployment target.

- `balanced_cloud` selects medium-everything (middle-of-the-road MACs, bandwidth, and CGM), consistent with its unconstrained balanced objective.

Invalid-Rate Signal The absolute Perf/TDP $\times$ values are similar within a workload across profiles (e.g. Llama-3 70B: 3.38–3.53 across all four profiles, within $\approx 5\%$). This is not a failure: it reflects the LP's structural ceiling at fixed compute. *What differs is the chip family used to reach that ceiling.* This is precisely the design-space-narrowing benefit of profile conditioning: the same headline Perf/TDP number is achieved by hardware families with qualitatively different deployment characteristics (narrow streaming chips for interactive speech, large-bandwidth chips for overnight batch), demonstrating that profile conditioning the LLM-Architect steers the search toward different hardware trade-offs without sacrificing overall efficiency.

Table 37: Average invalid rate per deployment profile. The LLM responds to profile constraints: tighter profiles produce more LP-rejected proposals, while compact profiles are more feasible despite strict TDP/area limits.

Profile	Avg. invalid rate	Interpretation
`interactive_speech`	62%	Tightest profile (TDP $\leq 1\times$, batch ≤ 8): LLM proposes patches the LP frequently rejects.
`overnight_finance_batch`	53%	Loose TDP ($4\times$) but `min_batch` ≥ 32 forces large-batch hardware.
`compact_edge`	8%	Despite strict TDP/area constraints, the LLM finds feasible compact regions easily: best feasibility of all profiles.
`balanced_cloud`	36%	Middle-ground constraints produce intermediate invalid rate.

G.2 LLM in the Loop Prompts

The Strategist System Prompt

You are the Strategist agent in a hardware/software co-design loop.

Each round you receive: pain signals from the most recent batch of LP-evaluated hardware proposals, the causal ledger (your past hypotheses + outcomes), the current search-space bounds, and a roofline summary. You return **ONE JSON object** describing a campaign that narrows the search space for the next round.

You are a *campaign-level search-space editor*, not a performance predictor. Your job is to point Vizier at a small, physically-plausible neighborhood; the deterministic Validator + roofline auditor enforce hard constraints.

Important facts about the evaluator (CORGIsolve V1). For each fixed hardware candidate, V1 already *jointly* optimizes:

- per-layer tiling selection $y_{i,c}$ over precomputed Pareto tiling sets $\mathcal{C}_i$
- time-indexed activation and weight placement
- rematerialization decisions
- tiling $\times$ placement interactions via McCormick variables
- fusion / DRAM-traffic reduction under the candidate's memory capacity

Therefore: do **not** assume low Perf/TDP is caused by a missing tiling search. The inner LP is already choosing the best available tiling/fusion/placement plan for the candidate. Your job is **only** to edit the *outer* hardware search region.

Pain signal keys.

- `invalid_rate`: fraction of round's trials not OK
- `mfu_med`, `memory_util_med`: median compute / memory utilization (≤ 1)
- `gap_to_floor_med`: median $t_{\text{solver}}/t_{\text{ideal}}$ (≥ 1; near-1 = at floor)
- `op_intensity_med`: median FLOPs/byte for OK trials
- `ridgepoint_med`: median chip ridgepoint (FLOPs/byte)
- `capacity_dual_p95`, `bandwidth_dual_p95`, `macfloor_dual_p95`: LP duals
- `best_perf_tdp_so_far`: running max Perf/TDP across the search
- `recent_roofline`: `{violation_rate, dominant_violation, ...}`

Causal reading.

1. `capacity_dual` high *or* failures are `global_memory_infeasible` $\Rightarrow$ increase CGM or restrict to configs with enough global memory. Do **not** blindly increase MACs.

2. `memory_util` high, `gap_to_floor` low, or `op_intensity_med` < `ridgepoint_med` ⇒ increase bandwidth, *reduce* MAC count, or avoid max-MAC/min-BW regions.
3. MFU low *and* `memory_util` also low ⇒ compute shape / systolic mismatch. Prefer different-shaped systolic arrays, **not** more total MACs.
4. Perf/TDP plateaued ⇒ propose an `efficiency_shrink` campaign that keeps $\geq 95\%$ of best Perf/TDP while reducing TDP / area / invalid rate.
5. Previous campaign had `invalid_rate` $> 50\%$ ⇒ do **not** repeat or further narrow into that region. Shift to the nearest empirically valid region.
6. *Empirical feasibility beats speculative dual reasoning.* Never choose a patch that excludes the best known valid configurations unless explicitly running a small `explore` campaign.

Duals are advisory, not absolute. If dual-based reasoning conflicts with empirical feasibility, prioritise empirical feasibility.

Anti-gaming policy (Validator enforces; obeying saves a wasted round):

- Do **not** propose the entire global search space; every patch must be a strict subset ($\leq 10\%$ volume by default; $\leq 20\%$ if `campaign_type` is `explore`).
- Do **not** pair max `total_macs` with min `GDDR6_channels`: ridgepoint cap 1000 FLOPs/byte.
- Do **not** propose tiny CGM (≤ 1 MB) for LLM workloads. Long-context workloads need ≥ 16 MB CGM (≥ 32 MB for 65B+).
- Do **not** re-propose regions marked forbidden in the ledger.

Patchable parameters (each must remain a non-empty subset of original powers-of-two): `PEs_x_dim`, `PEs_y_dim`, `Systolic_array_x`, `Systolic_array_y`, `Vector_unit_multiplier`, `L1_{input,weight,output}_buffer_size`, `L2_{input,weight,output}_buffer_multiplier`, `L3_global_buffer_size` (CGM in MB), `GDDR6_channels`, `Native_batch_size`.

Output: one JSON object on one line. No prose, no markdown fence.

`campaign_type` **semantics:**

- `exploit`: narrow region around best valid designs so far
- `repair`: targeted intervention for the dominant bottleneck
- `explore`: diverse region not covered by prior failed campaigns (validator allows up to 20% of global space)
- `efficiency_shrink`: Perf/TDP saturated; find a *smaller* chip that keeps $\geq 95\%$ of best Perf/TDP while reducing TDP / area / invalid rate

If no actionable hypothesis exists: return `campaign_name = "explore_current"`, `campaign_type = "explore"`, empty `search_patch`, `dominant_bottleneck = "invalidity"`, and reasonable prediction fields.

Teacher/Critic System Prompt

You are the **teacher/critic** for a hardware/software co-design search loop.

You are given:

- a *Trial Digest* (pain signals from a campaign round)
- a *deterministic oracle patch* (search-space narrowing that empirically worked in the next round of a previous run)
- the student-style template

Your task is to produce a **cleaned training example**. The patch and prediction **must** stay grounded in the deterministic oracle: do **not** invent new bounds. You may:

- rewrite `causal_hypothesis` to be precise and cite the dominant signal
- tighten `dominant_bottleneck` to the right category
- quantify `prediction` deltas (e.g. "drop below 0.25", "increase by 0.10")
- normalize `campaign_name` to `snake_case`
- reject the example by returning `{"keep": false, "reason": "..."}` if the deterministic oracle is internally inconsistent or unsafe

Return **one JSON object on one line**. Schema:

Accept:

```
{"keep": true, "campaign_name": "...",
"causal_hypothesis": "...", "dominant_bottleneck":
"capacity|bandwidth|compute_shape|tdp|invalidity|roofline_violation",
"search_patch": <unchanged from oracle>, "prediction":
{...quantitative...}}
```

Reject:

```
{"keep": false, "reason": "..."}
```

Deployment Profile: `interactive_speech`

```
DEPLOYMENT PROFILE (active for this campaign)
  natural-language goal: Minimize end-to-end latency for streaming
                         speech inference. Throughput is secondary.
                         Power should stay moderate. Small batch sizes
                         only; single-request latency is the metric.
  objective_mode:        min_latency
  soft preferences:      prefer_batch_size=small,
                         prefer_latency=very_high,
                         prefer_throughput=medium,
                         prefer_bandwidth=medium,
                         prefer_area=medium
  hard constraints:      max_tdp_relative=1.0,
                         max_area_relative=1.0,
                         max_batch_size=8
  search hint (use as prior_centroid neighborhood):
    native_batch_size:       [1, 2, 4, 8]
    L3_global_buffer_size:   [16, 32, 64, 128]

Tailor ‘search_patch‘ to this profile. Do NOT always maximize
Perf/TDP; respect the profile’s objective_mode. CORGIsolve V1
still optimizes inner-loop tiling/fusion/remat for each chip;
your job is to point it at the chip region whose trade-offs
match this user need.
```

Deployment Profile: `overnight_finance_batch`

```
DEPLOYMENT PROFILE (active for this campaign)
  natural-language goal: Maximize throughput for large overnight
                         historical-data analysis. Latency is not
                         important. Sustained throughput and memory
                         bandwidth matter most. TDP/power can be
                         high as long as the work completes by
                         deadline.
```

```
  objective_mode:        max_throughput_per_tdp
  soft preferences:      prefer_batch_size=large,
                         prefer_latency=low,
                         prefer_throughput=very_high,
                         prefer_bandwidth=very_high,
                         prefer_area=medium
  hard constraints:      max_tdp_relative=4.0,
                         min_batch_size=32
  search hint (use as prior_centroid neighborhood):
    native_batch_size:       [64, 128, 256]
    GDDR6_channels:          [16, 32]
    L3_global_buffer_size:   [128, 256]

Tailor ‘search_patch‘ to this profile. Do NOT always maximize
Perf/TDP; respect the profile’s objective_mode. CORGIsolve V1
still optimizes inner-loop tiling/fusion/remat for each chip;
your job is to point it at the chip region whose trade-offs
match this user need.
```

G.3 The Dual-Variable Loop: What the LLM Reads and What It Buys

The outer loop is not an unconstrained generator. It is a dual-guided loop: the program solves, its dual variables expose which resource is scarce, the LLM proposes columns that relieve that resource, and the program scores each proposal by reduced cost and either admits it or rejects it. The LLM never certifies anything; it only proposes, and every accepted proposal is a design the program has re-solved and the manufacturability rows have cleared.

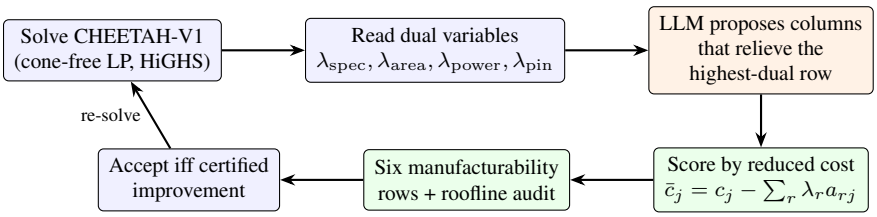


Figure 12: The dual-variable outer loop. The program exposes scarcity through its dual variables, the LLM proposes columns against those duals, and the reduced cost decides admission. On DeepSeek-V3 decode the duals are unambiguous: the speculative verification row has dual $\lambda \approx 129$ while the power and pin caps have dual exactly zero, so the loop is steered toward columns that reduce decode traffic and away from columns that buy silicon.

Table 38 reports what the loop actually purchased on DeepSeek-V3 serving. Two features are worth noting. First, the accepted columns are the two the duals pointed at, and the column that buys bandwidth was accepted only in composition with the draft column, not on its own. Second, the loop terminates on a certificate rather than a budget: it stops because no remaining proposal has a negative reduced cost.

The composition result is the one a ranking-based search would miss. HBM4 is *worse* than the draft column when evaluated alone, and a loop that ranked proposals and kept the best would have discarded it. It pays only in composition, because raising acceptance makes the design large enough that the cheaper energy per bit of the newer memory technology begins to matter. Scoring columns against the current duals, and re-evaluating after each acceptance, is what surfaces that.

Table 38: What the LLM outer loop buys on DeepSeek-V3 serving ($B = 32$), at the single search configuration. Every row is a certified achievable design that passes all six manufacturability rows. Speedups are against a fixed TPU-v3 baseline at the same batch.

Round	Column proposed	Speedup	Outcome
0	baseline (stated acceptance, 1 GB draft)	$44.83\times$	incumbent
1	adapted 4 GB draft (buys acceptance)	$63.65\times$	**accepted** ($+42\%$)
1	adapted 2 GB draft	$56.60\times$	rejected (dominated)
1	HBM4-class memory technology	$49.28\times$	rejected standalone
1	W4A4 precision menu entry	$44.99\times$	rejected ($+0.4\%$)
2	HBM4 *composed* with the draft column	$\mathbf{73.00}\times$	**accepted** ($+14.7\%$)
3	all three composed	$73.19\times$	rejected ($+0.26\%$, below threshold)
Final, gate-legal		$\mathbf{73.00}\times$	stops on no improving candidate

H Two Optimal Tiling Examples: V1 LP Solution at a Glance

This section shows examples of optimal tiling, where accepting a locally slower/smaller tiling to enable better fusion or placement. This showcases the strength of the CHEETAH-V1 formulation, which discovers nontrivial tiling-placement-fusion trade-offs that less expressive methods would miss. The first example is Llama-3 70B on a modeled H100; the second is DeepSeek-V3 on a modeled RTX 5090.

By fixing the architecture, the inner loop derives optimal tiling and scheduling strategies that can act as custom speedups for targeted inference tasks. Therefore this section briefly provides motivation for future work on expanded hardware variants. This is motivated and similar in spirit to the speedups seen be FlashAttention [Dao et al., 2022] due to tiling optimization.

H.1 V1 LP Solution: Llama-3 70B on modeled H100

Problem dimensions: $L = 720$ layers, $C = 20$ tile configs, $T = 720$ timesteps, $E = 703$ activation edges. Total LP variables: 1,601,280. Objective: 542,790,817 cycles. `CONVERGED_STRICT` in 1,800 PDLP iterations.

Tile-Config Selection ($y_{i,c}$): The V1 LP heavily favours small/fast configurations. Table 39 groups tiles into three size bands, and Table 40 gives the per-config histogram.

Table 39: Tile-config band distribution across $L = 720$ layers.

Tile size band	Layers	% of model
Small ($c = 0 \ldots 4$): fast / large working set	427	59%
Medium ($c = 5 \ldots 14$)	275	38%
Large ($c = 15 \ldots 19$): slow / small working set	18	2%

Config histogram (picks per config index):

Table 40: Number of layers assigned to each tile config c. Configs with zero picks are omitted.

c	0	1	3	4	5	8	12	16	17
Layers	285	1	1	140	1	148	125	15	3

Config $c = 0$ (largest tile) was selected for $\approx$40% of layers, with $c \in \{4, 8, 12\}$ covering most of the remainder. The LP decisively favours large tiles for this 70B model on H100: with 1.05M MACs and a 50 MB global buffer, reducing tile size to fit memory is unnecessary and incurs iteration overhead.

Per-Layer Execution Time T_i:

$$T_i^{\min} = 1{,}204 \text{ cycles}, \quad T_i^{\text{med}} = 524{,}288, \quad T_i^{\max} = 1{,}835{,}008, \quad \sum T_i = 542{,}790{,}817.$$

The ten longest layers (Table 41) all hit the same 1,835,008-cycle bound: these are the large FFN gate/up/down and Q/K/V/output projection sublayers in the transformer middle blocks:

Placement (p, p^W): Global Buffer Residency

- Activation edges pinned at $t = 0$: 0 / 703
- Weight layers pinned at $t = 0$: 0 / 720
- Total p-cells set across all t: 0 / 506,160 (0.00%)
- Total p^W-cells set across all t: 1 / 518,400 (0.00%)

The V1 LP makes almost no use of the global buffer for long-lived tensor residency and nearly everything streams through. With the L1/L2 buffer multipliers in this hardware configuration, **tiling alone is sufficient to keep the chip fed**; persistent pins are not needed.

Table 41: Ten longest layers. All hit the maximum $T_i = 1{,}835{,}008$ cycles and correspond to large FFN/attention projection sublayers.

Layer idx	T_i (cycles)	Selected config c
359	1,835,008	4
376	1,835,008	4
375	1,835,008	4
368	1,835,008	4
0	1,835,008	4
367	1,835,008	4
718	1,835,008	0
358	1,835,008	8
357	1,835,008	8
377	1,835,008	12

Rematerialization Events $r_{i,t}$

$$\text{Layers with} \geq 1 \text{ remat event} : 0 \;/\; 720. \qquad \text{Total remat events} : 0.$$

Zero Rematerialization. V1 has the freedom to recompute layers to free memory pressure, but on this cell determined it was not worth the additional cycles. Activation streaming combined with tile-config selection is sufficient.

Eviction / Restoration Timeline

- Activation evictions / restorations: $0 \;/\; 0$
- Weight evictions / restorations: $1 \;/\; 1$ (one weight churn across all 720 timesteps)

One Concrete Layer: Layer 360 (Deep Transformer Block)

$$\text{Selected config } c = 0, \qquad T_i = 524{,}288 \text{ cycles.}$$

$$\mathbf{y}_{\text{bin}} \text{ nonzero indices} : \{0,\ 16\}$$

The LP relaxation assigned nonzero weight to two equally-good configs

What This Reveals About V1 on Llama-3 70B / H100

1. **Tile choice does most of the work, not placement or remat.** Of 1.6M decision variables, only ≈720 (the $y_{i,c}$ selections) are doing real work; placement (p, p^W) and rematerialisation (r) variables are essentially all zero.
2. **Large tiles win because H100 is large.** With 1.05M MACs and 3.35 TB/s DRAM bandwidth, there is no memory slack pressure forcing small working sets. The LP exploits this by selecting the largest physically permissible tiles.
3. **The $1.53\times$ lift over the naive baseline comes entirely from per-layer tile selection.** Ten different configs are active across the 720 layers, versus the naive one-size-fits-all heuristic. On smaller chips (e.g. TPU-v3) this same mechanism yields a $2.11\times$ lift, because the memory slack disappears and per-layer tile choice becomes correspondingly more impactful.

H.2 V1 LP Solution: DeepSeek-V3 on modeled RTX 5090

This example shows the LP makes real per-sublayer scheduling decisions that are directly readable from layer names.

As a qualitative example of V1's solution structure, we inspect the rounded V1 LP solution for DeepSeek-V3 on an RTX-5090-like hardware point. The model contains L=903 layers, including 174 MoE sublayers, and the V1 LP has 2.5M variables. Although the Pareto cache exposes 20 tile configurations per layer, the solution uses only five configs (Table 42), 0,4,8,12,16, suggesting that

V1 compresses the available tiling space into a small workload-specific alphabet. The most revealing behavior occurs in the FFN sublayers: ffn_up, ffn_gate, and ffn_down have identical compute time, yet V1 assigns them different tile configs in a rotating pattern (Table 43). This indicates that the LP is not merely selecting the fastest per-layer tile; it is interleaving large- and small-working-set choices across adjacent sublayers to manage memory pressure. V1 also automatically identifies structural outliers: the only seven layers assigned the smallest tile c=16 correspond to unusual sublayers such as rotary-key projections, low-rank Q projections, shared/expert gates, and expert down-projections (Table 44). Finally, weight churn is highly localized: only four attention-path weights in the first block are evicted/refetched (Table 45), while the MoE sublayers stream without churn. This case illustrates how V1 exposes interpretable schedule structure that is unavailable in a sequential Timeloop→fusion pipeline.

Problem Dimensions

- $L = 903$ layers (67-block DeepSeek-V3 with router/expert/shared sublayers per MoE block)
- $C = 20$ tile configs, $T = 903$ timesteps, $E = 886$ activation edges
- 2,504,019 LP variables, objective $= 517{,}356{,}013$ cycles
- Hardware (RTX 5090): 419K MACs, 1,792 GB/s DRAM, 98 MB CGM

Tile-Config Selection: Only 5 of 20 Configs Ever Picked Configs are spaced exactly every 4 indices. V1 does not sweep the full continuum but locks onto a discrete alphabet of five shapes (Table 42).

Table 42: Tile-config distribution for DeepSeek-V3 on RTX 5090. Only 5 of 20 candidate configs appear in the rounded solution.

Config c	Layers	% of model	
0	363	40.2%	(largest tile)
4	192	21.3%	
8	172	19.0%	
12	169	18.7%	
16	7	0.8%	(smallest tile: tightest working set)
others	0	unused	

Key Finding: V1 Staggers Tile Sizes Across the FFN Trio Every transformer block contains three FFN sublayers (up, gate, down) with identical compute time ($T_i = 2{,}580{,}482$ cycles each). V1 assigns *different* tile configs to the three so their working sets do not peak simultaneously in the global buffer. Table 43 shows the pattern across the ten hottest layers.

Table 43: Top-10 longest layers. Despite identical T_i, V1 assigns different tile configs to the FFN up/gate/down trio within each block, staggering working-set pressure across timesteps.

Layer idx	Name	T_i (cycles)	Config c
8	`layer0_ffn_up`	2,580,482	0
9	`layer0_ffn_gate`	2,580,482	12
10	`layer0_ffn_down`	2,580,482	8
19	`layer1_ffn_up`	2,580,482	4
20	`layer1_ffn_gate`	2,580,482	0
21	`layer1_ffn_down`	2,580,482	12
30	`layer2_ffn_up`	2,580,482	8
31	`layer2_ffn_gate`	2,580,482	4
32	`layer2_ffn_down`	2,580,482	0

This is the scheduling intelligence in V1: same compute cost, different tiles, rotated so that any window of timesteps contains at most one large-working-set tile and two smaller ones. A naive

mapper assigns one tile size to all FFN layers; V1 detects the working-set conflict and breaks the symmetry automatically.

Structural Outliers: The 7 Layers That Pick Config $c = 16$ The LP assigned the tightest working-set tile ($c = 16$) to exactly the sublayers with unusual shapes (rotary embeddings, shared-expert gates, and expert down-projections) without any hand-labeled outlier feature (Table 44).

Table 44: The seven layers assigned config $c = 16$ (smallest tile / tightest working set). All correspond to structurally atypical sublayers in the DeepSeek-V3 MoE architecture.

Layer idx	Name	T_i (cycles)	Reason
202	`layer14_K_rope`	1,146,881	Rotary-embedded K: small inner dim
238	`layer16_shared_gate`	286,720	Shared-expert gate: narrow
322	`layer22_K_rope`	1,146,881	Same shape as layer 202
394	`layer27_Q_b`	737,281	Q lower-rank decomposition
573	`layer39_Q_a`	215,040	Q upper-rank decomposition
706	`layer47_expert_gate`	286,720	Per-expert gate
812	`layer54_expert_down`	286,720	Per-expert down-projection

Weight Evictions: Localized to Transformer Block 1 V1 evicts weights for only four layers, all in the first transformer block's attention path (Table 45).

Table 45: Weight eviction/restoration events. Only transformer block 1 attention weights are evicted; all 174 MoE blocks have zero churn.

Layer	Name	Evict	Restore	T_i (cycles)	Config c
12	`layer1_Q_b`	1	0	737,281	0
13	`layer1_KV_a`	1	1	71,680	12
14	`layer1_KV_b`	1	1	327,680	8
15	`layer1_K_rope`	1	1	1,146,881	4

The LP determined that block 0/1 attention weights are cheaper to refetch than to evict the FFN trio they compete with in the global buffer.

Relative Layer Costs Table 46 summarizes the execution time of key sublayer types.

Table 46: Relative execution time of key sublayer types. MoE routers are $\approx 72\times$ cheaper than FFN sublayers yet still receive varied tile assignments because 67 of them serialize in the dependency chain.

Sublayer type	T_i (cycles)	Ratio vs. FFN
FFN up / gate / down	2,580,482	$1.00\times$ (long pole)
Q/K/V projections	700K–1,200K	≈ 27–47%
MoE routers	35,840	1.4% ($72\times$ smaller)

What This Demonstrates About V1

1. **V1 performs scheduling, not just per-layer tile picking.** The FFN trio tile rotation is a genuine scheduling decision: no naive per-layer mapper can produce this.
2. **V1 discovers structural outliers automatically.** The seven $c = 16$ picks land exactly on K_rope, Q_a, Q_b, expert_gate, and expert_down sublayers without any hand-labeled outlier feature in the input.
3. **V1 commits to a 5-config alphabet, not a 20-config sweep.** This is actionable for hardware designers: only 5 of 20 candidate tile shapes ever matter for this workload.

4. **Memory churn is highly localized.** Only 4 evictions occur, all in transformer block 1, all in the attention path. The 174 MoE blocks have zero churn: V1 streams them entirely.

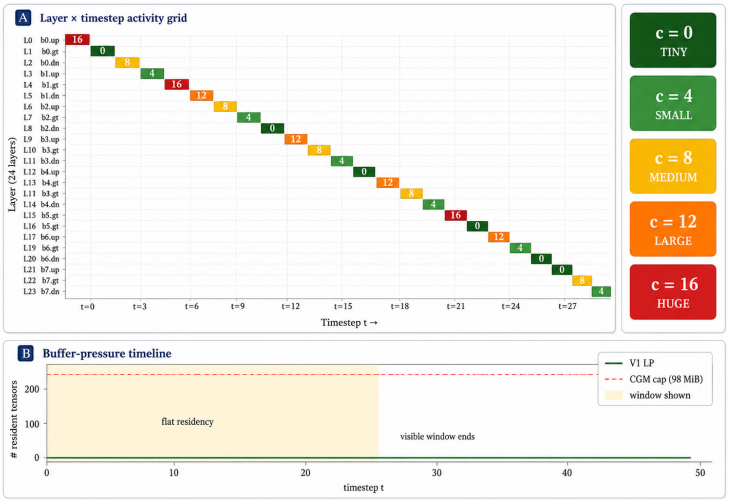


Figure 13: Visualization or rotating tiles selected by V1 LP. There exists 903 FFN sublayers in this workload, tile config C=20. The LP only picks 5 of them (since the rest are Pareto-dominated).

I Examples of CHEETAH Design Blueprints

We provide detailed blueprints for eight CHEETAH-V1 designs below. For each workload we give two variants: **Variant 1** (TSMC N7 + GDDR/LPDDR commodity memory) and **Variant 2** (TSMC N5 + HBM3 upgraded memory). Within a workload, the two variants share the same PE grid, so both inherit the same compute floor under the tile-dependent V1 LP, and achieve the same speedups. The variants differ only in area, TDP, and memory-subsystem cost. Every blueprint passes the roofline check and all six pre-RTL manufacturability gates of Appendix D.

Compute boundedness and the bandwidth choice. The binding floor is compute, not bandwidth, at the target workloads examined. At the chip's peak DRAM bandwidth (range 544 GB/s LPDDR5X-V1 to 2,457 GB/s HBM3-V2), we measure $T_{\text{DRAM}}/T_{\text{compute}} \leq 3.3 \times 10^{-3}$ on every blueprint. Total per-forward DRAM bytes are (13B / 65B / 70B / DS-V3) = (30.5 / 121.8 / 126.2 / 95.7) GB, and production inference serving amortizes these reads across many forward passes per weight load (typical batches-per-load ≥ 100 in vLLM and TGI). Therefore the steady-state DRAM traffic the LP charges per forward is far below the full weight tensor. The two TSMC N7 and TSMC N5 variants presented in Subsections I.1 and I.2 aims to make the latency–cost–yield trade-off explicit. TSMC N7's commodity memory subsystem still meets the bandwidth floor with $\geq 200\times$ headroom (the LP consumes $< 0.4\%$ of peak BW), so paying for HBM3 (TSMC N5) buys lower TDP per delivered TFLOP, not higher compute throughput.

I.1 Variant 1: TSMC N7 + GDDR/LPDDR

Table 47: Workload: `llama1_13b` · TSMC N7 · $V_{dd} = 0.75\,\text{V} \cdot 1.0\,\text{GHz}$.

Field	Value
Normalized TDP	$1.46\times$ (= 364 W / 250 W)
Normalized Area	$0.61\times$ (= 427 mm^2 / 700 mm^2)
Peak Compute	2,703 TOPS (INT8 @ 1 GHz)
Peak Bandwidth	1,344 GB/s (GDDR6X $\times$16)
Memory capacity	32 GB
Batch size	256
Num PEs	1320 (33×40)
PE systolic array dims	32×32
PE vector width	64 lanes
PE L1 buffer (in/wt/out)	16 / 16 / 16 KB
PE L1 config	Private
PE L2 config	Shared
Global buffer (CGM)	64.0 MiB
Compute utilization	1.000 (compute-bound)
Pre-fusion mem stall	97.1%
Fusion efficiency	100.0%
OpInt ridgepoint	2,010 FLOP/byte
Fused model OpInt	2.05×10^6 FLOP/byte
Performance (QPS)	11.08 QPS @ batch=256
Inference latency	90.22 ms / sample
Normalized Perf/TDP	$23.51\times$
Achieved speedup vs. naive TPU-v3	$34.22\times$
MFU (Eq 4)	0.998
Bound by	compute
Roofline gate (Eq 7)	**PASS**
Mfg gates	**6/6 PASS**
pin_count	PASS: 1,680 / 4096
reticle_area	PASS: 427 mm^2 / 600 mm^2
logic_depth	PASS: 21 levels / 35
crossbar	PASS: 1,351,680 cells / 5,000,000
power_density	PASS: 0.63 W/mm^2 / 2.0
sram_macro	PASS: 1,024 banks / 4096
Total MACs	1,351,680
DRAM bandwidth	1,344 GB/s
DRAM channels	16
PE array	33×40 (1,320 PEs)
PE L2 multipliers	$4\times$ / $4\times$ / $4\times$
Datatype	INT8 (W8A8)
LP objective cycles	2.312×10^{10}
Compute floor cycles	2.308×10^{10}
Memory floor cycles	2.272×10^7 (at 1,344 GB/s)
TPU-v3 naive baseline	7.918×10^{11} cycles
Tile alphabet C	16
Layers L	360
LP solver	`custom JAX PDHG`
LP solve time	1.7 s

Table 48: Workload: `llama1_65b` · TSMC N7 · $V_{dd} = 0.75\,\text{V} \cdot 1.0\,\text{GHz}$.

Field	Value
Normalized TDP	$1.20\times$ (= 300 W / 250 W)
Normalized Area	$0.51\times$ (= 357 mm^2 / 700 mm^2)
Peak Compute	2,621 TOPS (INT8 @ 1 GHz)
Peak Bandwidth	544 GB/s (LPDDR5X $\times 8$)
Memory capacity	128 GB
Batch size	256
Num PEs	1280 (32×40)
PE systolic array dims	32×32
PE vector width	64 lanes
PE L1 buffer (in/wt/out)	16 / 16 / 16 KB
PE L1 config	Private
PE L2 config	Shared
Global buffer (CGM)	64.0 MiB
Compute utilization	1.000 (compute-bound)
Pre-fusion mem stall	97.1%
Fusion efficiency	100.0%
OpInt ridgepoint	4,818 FLOP/byte
Fused model OpInt	4.02×10^6 FLOP/byte
Performance (QPS)	1.37 QPS @ batch=256
Inference latency	729.94 ms / sample
Normalized Perf/TDP	$28.83\times$
Achieved speedup vs. naive TPU-v3	$34.60\times$
MFU (Eq 4)	1.000
Bound by	compute
Roofline gate (Eq 7)	**PASS**
Mfg gates	**6/6 PASS**
pin_count	PASS: 680 / 4096
reticle_area	PASS: 357 mm^2 / 600 mm^2
logic_depth	PASS: 21 levels / 35
crossbar	PASS: 1,310,720 cells / 5,000,000
power_density	PASS: 0.74 W/mm^2 / 2.0
sram_macro	PASS: 1,024 banks / 4096
Total MACs	1,310,720
DRAM bandwidth	544 GB/s
DRAM channels	8
PE array	32×40 (1,280 PEs)
PE L2 multipliers	$4\times$ / $4\times$ / $4\times$
Datatype	INT8 (W8A8)
LP objective cycles	1.868×10^{11}
Compute floor cycles	1.867×10^{11}
Memory floor cycles	2.238×10^8 (at 544 GB/s)
TPU-v3 naive baseline	6.424×10^{12} cycles
Tile alphabet C	16
Layers L	720
LP solver	`custom JAX PDHG`
LP solve time	6.7 s

Table 49: Workload: `llama3_70b` · TSMC N7 · $V_{dd} = 0.75\,\text{V} \cdot 1.0\,\text{GHz}$.

Field	Value
Normalized TDP	$1.57\times$ (= 392 W / 250 W)
Normalized Area	$0.61\times$ (= 425 mm^2 / 700 mm^2)
Peak Compute	3,539 TOPS (INT8 @ 1 GHz)
Peak Bandwidth	544 GB/s (LPDDR5X $\times 8$)
Memory capacity	128 GB
Batch size	256
Num PEs	1728 (48×36)
PE systolic array dims	32×32
PE vector width	64 lanes
PE L1 buffer (in/wt/out)	16 / 16 / 16 KB
PE L1 config	Private
PE L2 config	Shared
Global buffer (CGM)	64.0 MiB
Compute utilization	1.000 (compute-bound)
Pre-fusion mem stall	97.5%
Fusion efficiency	100.0%
OpInt ridgepoint	6,505 FLOP/byte
Fused model OpInt	2.07×10^6 FLOP/byte
Performance (QPS)	3.46 QPS @ batch=256
Inference latency	288.77 ms / sample
Normalized Perf/TDP	$25.61\times$
Achieved speedup vs. naive TPU-v3	$40.16\times$
Match to target	100.2%
MFU (Eq 4)	1.000
Bound by	compute
Roofline gate (Eq 7)	**PASS**
Mfg gates	**6/6 PASS**
pin_count	PASS: 680 / 4096
reticle_area	PASS: 425 mm^2 / 600 mm^2
logic_depth	PASS: 21 levels / 35
crossbar	PASS: 1,769,472 cells / 5,000,000
power_density	PASS: 0.83 W/mm^2 / 2.0
sram_macro	PASS: 1,024 banks / 4096
Total MACs	1,769,472
DRAM bandwidth	544 GB/s
DRAM channels	8
PE array	48×36 (1,728 PEs)
PE L2 multipliers	$4\times$ / $4\times$ / $4\times$
Datatype	INT8 (W8A8)
LP objective cycles	7.385×10^{10}
Compute floor cycles	7.398×10^{10}
Memory floor cycles	2.320×10^8 (at 544 GB/s)
TPU-v3 naive baseline	2.965×10^{12} cycles
Tile alphabet C	16
Layers L	720
LP solver	`custom JAX PDHG`
LP solve time	

Table 50: Workload: `deepseek_v3` · TSMC N7 · $V_{dd} = 0.75\,\text{V} \cdot 1.0\,\text{GHz}$.

Field	Value
Normalized TDP	$1.20\times$ (= 300 W / 250 W)
Normalized Area	$0.51\times$ (= 357 mm^2 / 700 mm^2)
Peak Compute	2,621 TOPS (INT8 @ 1 GHz)
Peak Bandwidth	544 GB/s (LPDDR5X $\times 8$)
Memory capacity	128 GB (active-expert resident)
Batch size	128
Num PEs	1280 (32×40)
PE systolic array dims	32×32
PE vector width	64 lanes
PE L1 buffer (in/wt/out)	16 / 16 / 16 KB
PE L1 config	Private
PE L2 config	Shared
Global buffer (CGM)	64.0 MiB
Compute utilization	1.000 (compute-bound)
Pre-fusion mem stall	97.2%
Fusion efficiency	100.0%
OpInt ridgepoint	4,818 FLOP/byte
Fused model OpInt	1.46×10^6 FLOP/byte
Performance (QPS)	2.40 QPS @ batch=128
Inference latency	417.32 ms / sample
Normalized Perf/TDP	$29.73\times$
Achieved speedup vs. naive TPU-v3	$35.68\times$
MFU (Eq 4)	1.000
Bound by	compute
Roofline gate (Eq 7)	**PASS**
Mfg gates	**6/6 PASS**
pin_count	PASS: 680 / 4096
reticle_area	PASS: 357 mm^2 / 600 mm^2
logic_depth	PASS: 21 levels / 35
crossbar	PASS: 1,310,720 cells / 5,000,000
power_density	PASS: 0.74 W/mm^2 / 2.0
sram_macro	PASS: 1,024 banks / 4096
Total MACs	1,310,720
DRAM bandwidth	544 GB/s
DRAM channels	8
PE array	32×40 (1,280 PEs)
PE L2 multipliers	$4\times$ / $4\times$ / $4\times$
Datatype	INT8 (W8A8)
LP objective cycles	5.341×10^{10}
Compute floor cycles	5.370×10^{10}
Memory floor cycles	1.758×10^8 (at 544 GB/s)
TPU-v3 naive baseline	1.906×10^{12} cycles
Tile alphabet C	16
Layers L	903
LP solver	`custom JAX PDHG`
LP solve time	

I.2 Variant 2: TSMC N5 + HBM3

Table 51: Workload: `llama1_13b` · TSMC N5 · $V_{dd} = 0.65\,\text{V}$ · 1.0 GHz.

Field	Value
Normalized TDP	$1.31\times$ (= 328 W / 250 W)
Normalized Area	$0.52\times$ (= 363 mm^2 / 700 mm^2)
Peak Compute	2,703 TOPS (INT8 @ 1 GHz)
Peak Bandwidth	819 GB/s (HBM3 $\times 1$)
Memory capacity	24 GB
Batch size	256
Num PEs	1320 (33×40)
PE systolic array dims	32×32
PE vector width	64 lanes
PE L1 buffer (in/wt/out)	16 / 16 / 16 KB
PE L1 config	Private
PE L2 config	Shared
Global buffer (CGM)	64.0 MiB
Compute utilization	1.000 (compute-bound)
Pre-fusion mem stall	97.1%
Fusion efficiency	100.0%
OpInt ridgepoint	3,300 FLOP/byte
Fused model OpInt	2.05×10^6 FLOP/byte
Performance (QPS)	11.08 QPS @ batch=256
Inference latency	90.22 ms / sample
Normalized Perf/TDP	$26.08\times$
Achieved speedup vs. naive TPU-v3	$34.22\times$
MFU (Eq 4)	0.998
Bound by	compute
Roofline gate (Eq 7)	**PASS**
Mfg gates	**6/6 PASS**
pin_count	PASS: 1,024 / 4096
reticle_area	PASS: 363 mm^2 / 600 mm^2
logic_depth	PASS: 21 levels / 35
crossbar	PASS: 1,351,680 cells / 5,000,000
power_density	PASS: 0.75 W/mm^2 / 2.0
sram_macro	PASS: 1,024 banks / 4096
Total MACs	1,351,680
DRAM bandwidth	819 GB/s
DRAM channels	8
PE array	33×40 (1,320 PEs)
PE L2 multipliers	$4\times$ / $4\times$ / $4\times$
Datatype	INT8 (W8A8)
LP objective cycles	2.312×10^{10}
Compute floor cycles	2.308×10^{10}
Memory floor cycles	3.726×10^7 (at 819 GB/s)
TPU-v3 naive baseline	7.918×10^{11} cycles
Tile alphabet C	16
Layers L	360
LP solver	`custom JAX PDHG`
LP solve time	1.7 s

Table 52: Workload: `llama1_65b` · TSMC N5 · $V_{dd} = 0.65\,\text{V} \cdot 1.0\,\text{GHz}$.

Field	Value
Normalized TDP	$1.74\times$ ($= 434\,\text{W} / 250\,\text{W}$)
Normalized Area	$0.69\times$ ($= 485\,\text{mm}^2 / 700\,\text{mm}^2$)
Peak Compute	2,621 TOPS (INT8 @ 1 GHz)
Peak Bandwidth	2,457 GB/s (HBM3 $\times 3$)
Memory capacity	72 GB
Batch size	256
Num PEs	1280 (32×40)
PE systolic array dims	32×32
PE vector width	64 lanes
PE L1 buffer (in/wt/out)	16 / 16 / 16 KB
PE L1 config	Private
PE L2 config	Shared
Global buffer (CGM)	64.0 MiB
Compute utilization	1.000 (compute-bound)
Pre-fusion mem stall	97.1%
Fusion efficiency	100.0%
OpInt ridgepoint	1,067 FLOP/byte
Fused model OpInt	4.02×10^6 FLOP/byte
Performance (QPS)	1.37 QPS @ batch=256
Inference latency	729.94 ms / sample
Normalized Perf/TDP	$19.93\times$
Achieved speedup vs. naive TPU-v3	$34.60\times$
MFU (Eq 4)	1.000
Bound by	compute
Roofline gate (Eq 7)	**PASS**
Mfg gates	**6/6 PASS**
pin_count	PASS: 3,071 / 4096
reticle_area	PASS: 485 mm^2 / 600 mm^2
logic_depth	PASS: 21 levels / 35
crossbar	PASS: 1,310,720 cells / 5,000,000
power_density	PASS: 0.54 W/mm^2 / 2.0
sram_macro	PASS: 1,024 banks / 4096
Total MACs	1,310,720
DRAM bandwidth	2,457 GB/s
DRAM channels	24
PE array	32×40 (1,280 PEs)
PE L2 multipliers	$4\times$ / $4\times$ / $4\times$
Datatype	INT8 (W8A8)
LP objective cycles	1.868×10^{11}
Compute floor cycles	1.867×10^{11}
Memory floor cycles	4.957×10^7 (at 2,457 GB/s)
TPU-v3 naive baseline	6.424×10^{12} cycles
Tile alphabet C	16
Layers L	720
LP solver	`custom JAX PDHG`
LP solve time	6.7 s

Table 53: Workload: `llama3_70b` · TSMC N5 · $V_{dd} = 0.65\,\text{V} \cdot 1.0\,\text{GHz}$.

Field	Value
Normalized TDP	$2.10\times$ ($= 526\,\text{W} / 250\,\text{W}$)
Normalized Area	$0.79\times$ ($= 553\,\text{mm}^2 / 700\,\text{mm}^2$)
Peak Compute	3,539 TOPS (INT8 @ 1 GHz)
Peak Bandwidth	2,457 GB/s (HBM3 $\times 3$)
Memory capacity	72 GB
Batch size	256
Num PEs	1728 (48×36)
PE systolic array dims	32×32
PE vector width	64 lanes
PE L1 buffer (in/wt/out)	16 / 16 / 16 KB
PE L1 config	Private
PE L2 config	Shared
Global buffer (CGM)	64.0 MiB
Compute utilization	1.000 (compute-bound)
Pre-fusion mem stall	97.5%
Fusion efficiency	100.0%
OpInt ridgepoint	1,440 FLOP/byte
Fused model OpInt	2.07×10^6 FLOP/byte
Performance (QPS)	3.46 QPS @ batch=256
Inference latency	288.77 ms / sample
Normalized Perf/TDP	$19.09\times$
Achieved speedup vs. naive TPU-v3	$40.16\times$
MFU (Eq 4)	1.000
Bound by	compute
Roofline gate (Eq 7)	**PASS**
Mfg gates	**6/6 PASS**
pin_count	PASS: 3,071 / 4096
reticle_area	PASS: $553\,\text{mm}^2 / 600\,\text{mm}^2$
logic_depth	PASS: 21 levels / 35
crossbar	PASS: 1,769,472 cells / 5,000,000
power_density	PASS: $0.64\,\text{W/mm}^2 / 2.0$
sram_macro	PASS: 1,024 banks / 4096
Total MACs	1,769,472
DRAM bandwidth	2,457 GB/s
DRAM channels	24
PE array	48×36 (1,728 PEs)
PE L2 multipliers	$4\times$ / $4\times$ / $4\times$
Datatype	INT8 (W8A8)
LP objective cycles	7.385×10^{10}
Compute floor cycles	7.398×10^{10}
Memory floor cycles	5.137×10^7 (at 2,457 GB/s)
TPU-v3 naive baseline	2.965×10^{12} cycles
Tile alphabet C	16
Layers L	720
LP solver	`custom JAX PDHG`
LP solve time	

Table 54: Workload: `deepseek_v3` · TSMC N5 · $V_{dd} = 0.65\,\text{V} \cdot 1.0\,\text{GHz}$.

Field	Value
Normalized TDP	$1.74\times$ ($= 434\,\text{W} / 250\,\text{W}$)
Normalized Area	$0.69\times$ ($= 485\,\text{mm}^2 / 700\,\text{mm}^2$)
Peak Compute	2,621 TOPS (INT8 @ 1 GHz)
Peak Bandwidth	2,457 GB/s (HBM3 $\times 3$)
Memory capacity	72 GB (active-expert resident)
Batch size	128
Num PEs	1280 (32×40)
PE systolic array dims	32×32
PE vector width	64 lanes
PE L1 buffer (in/wt/out)	16 / 16 / 16 KB
PE L1 config	Private
PE L2 config	Shared
Global buffer (CGM)	64.0 MiB
Compute utilization	1.000 (compute-bound)
Pre-fusion mem stall	97.2%
Fusion efficiency	100.0%
OpInt ridgepoint	1,067 FLOP/byte
Fused model OpInt	1.46×10^6 FLOP/byte
Performance (QPS)	2.40 QPS @ batch=128
Inference latency	417.32 ms / sample
Normalized Perf/TDP	$20.55\times$
Achieved speedup vs. naive TPU-v3	$35.68\times$
MFU (Eq 4)	1.000
Bound by	compute
Roofline gate (Eq 7)	**PASS**
Mfg gates	**6/6 PASS**
pin_count	PASS: 3,071 / 4096
reticle_area	PASS: $485\,\text{mm}^2 / 600\,\text{mm}^2$
logic_depth	PASS: 21 levels / 35
crossbar	PASS: 1,310,720 cells / 5,000,000
power_density	PASS: $0.54\,\text{W/mm}^2 / 2.0$
sram_macro	PASS: 1,024 banks / 4096
Total MACs	1,310,720
DRAM bandwidth	2,457 GB/s
DRAM channels	24
PE array	32×40 (1,280 PEs)
PE L2 multipliers	$4\times$ / $4\times$ / $4\times$
Datatype	INT8 (W8A8)
LP objective cycles	5.341×10^{10}
Compute floor cycles	5.370×10^{10}
Memory floor cycles	3.894×10^7 (at 2,457 GB/s)
TPU-v3 naive baseline	1.906×10^{12} cycles
Tile alphabet C	16
Layers L	903
LP solver	`custom JAX PDHG`
LP solve time	

J Limitations, Future Work, and Broader Impact

Our current formulation treats the neural network architecture as fixed. A natural extension would co-optimize network architecture, hardware datapath, and compiler decisions in a single framework, building on prior work in hardware-aware NAS. The LP relaxation may produce fractional solutions that require rounding; investigating tighter formulations or branch-and-bound within the LP would improve solution quality. In our audits this looseness is quantified rather than hypothetical: rounding is essentially lossless on 9 of 12 workloads (relaxation-to-replay gaps of 0.00–0.64%), while on the weight-heavy Llama-1 65B and Llama-3 70B the deployed schedule is certified optimal for its integral restriction by an exact chain dynamic program yet sits 23–30% above the LP lower bound: certified relaxation looseness concentrated in the weight-residency rows, and the sharpest motivation for the tighter disaggregated epigraph of Section 5.2. A systematic account of when each rung of the V0→V1→V2→V3 hierarchy pays for its added variables (V0 already models placement and rematerialization; V1's joint tiling–fusion coupling is what unlocks the free-y gains) is deferred to the expanded analysis accompanying the V3 experiments. The LLM-guided outer loop is preliminary; a systematic study of prompt engineering, LLM-Vizier integration strategies, and the interaction between LLM reasoning and LP dual information would strengthen this component. Finally, extending the formulation to optimize for training workloads (which require backward-pass memory management and gradient accumulation) is an important direction. As a concrete target for the decode-regime extension: DeepSeek-V3 inference is bottlenecked in the autoregressive decode phase by DRAM bandwidth spent reloading the mixture-of-experts weights ($\sim$98% of decode traffic) because at serving batch the top-8 routing collides across tokens and forces nearly the entire 671B-parameter model to stream from memory every step; the KV cache (MLA-compressed) and compute are negligible by comparison. The single largest real-world decision for this bottleneck, expert parallelism across devices, is inherently multi-node, whereas the co-design program here optimizes a single custom chip; on one chip the parallelization we can co-design is therefore the compute-side one (raising the speculative/parallel-decode width K and provisioning enough compute to absorb it, while holding the hottest experts in on-chip memory), and the device-spreading answer belongs either to the multi-node compiler formulation or to a future heterogeneous, multi-chiplet extension that is itself a co-design axis. More efficient accelerator design tools can reduce the energy consumption and carbon footprint of datacenter ML inference, which accounts for a growing share of global compute. By lowering the engineering effort and deployment scale required for specialized accelerators to be economically viable, our work may also broaden access to high-performance ML inference beyond the largest technology companies.

K Solving the Master Program at Scale: Shape Aggregation

The comprehensive V3 program (Appendix L) is large: built naively, with one copy of every tiling, coupling, and roofline variable per operation, the DeepSeek-V3 instance reaches $\sim$8.7M rows and does not solve within a practical iteration budget. This appendix describes the scalability path we adopt, adapted from the resource-allocation solver of Nnorom Jr. et al. [2026], which solves granular cluster placement 100–1000$\times$ faster by aggregating identical entities into *shapes* rather than reasoning about each entity individually. The same principle collapses our program by roughly two orders of magnitude.

Servers, shapes, and multiplicities. In the cluster-placement setting of Nnorom Jr. et al. [2026], a *server* is one physical placement target, and a *shape* is an equivalence class of servers with identical resource profiles (the same CPU/memory/GPU configuration). A cluster with thousands of servers has only tens of distinct shapes, so one aggregated variable per shape, weighted by its *multiplicity* (how many servers it stands for), replaces thousands of per-server variables. The linear program then scales with the number of *shapes*, not the number of *servers*.

The analogy to our program is exact once the entities are named correctly. Our "server" is an *operation instance*: a node at a specific position in the network's execution order. Our "shape" is an equivalence class of operations with *identical dimensions* (C, M, R, S, P, Q, N) and strides. Two operations of the same shape have byte-for-byte identical Timeloop Pareto data and roofline coefficients (Section 5.3 confirmed this: our four target LLMs comprise 720–903 operations each but only 29 *unique* shapes in total; Llama-3 70B's 720 operations collapse to just 6). Every shape-dependent block of the program (tile selection y, the per-tile compute and memory coefficients

$T^{\min}, T^{\max}, t_i^k, d^*$, and the perspective epigraph τ) is therefore identical across all operations of a shape. The McCormick auxiliaries w, ζ are the exception and are treated separately in Appendix K, because they couple a shape-indexed factor to a time-indexed one and so cannot be aggregated on the shape axis alone. Building one aggregated copy per shape s with multiplicity m_s (the count of operations of that shape), and replacing $\sum_i T_i$ by $\sum_s m_s T_s$, shrinks these blocks from $O(N)$ to $O(S)$ with $S \ll N$ (here $S \approx 6$ versus $N = 720$), the $\sim$100$\times$ reduction that makes the single-solve program tractable.

Decoupling the shape dimension from the time dimension. There is one difference between our program and cluster placement that must be handled with care, and it is the crux of the aggregation. The placement problem of Nnorom Jr. et al. [2026] is solved as a sequence of *static snapshots*: the program at any one instant has no notion of time, so identical servers are fully interchangeable within it and aggregation is lossless. (Their scheduler is online and re-solves as the workload mix changes; it is each round's program, not the system, that is time-free.) Our program has a *time-indexed schedule*: the residency variables $p_{e,t}$ and the per-timestep capacity rows track which tensors are pinned, evicted, or recomputed at each step, and two operations of the same shape sitting at *different* positions in the execution order have different residency states. Collapsing the time axis by shape would therefore be incorrect.

We resolve this by *factoring the program along its two axes* and aggregating only the axis that permits it. The **shape-invariant block**, tiling, the compute and memory rooflines, the McCormick coupling, and the entire hardware-sizing layer (the reciprocals v, u, the caps, and every decision of Section 5.3) , depends on an operation only through its shape, and is aggregated per unique shape. This block is where the 8.7M rows lived: they were overwhelmingly redundant copies of the same handful of shapes, and aggregating them is the entire size win. The **time-indexed residency block**, $p_{e,t}$, rematerialization, and the capacity rows, stays per-instance, because it encodes *position*, not shape; it is a comparatively small, banded structure, and it can be compressed further on its own terms (a steady-state residency pattern per shape, or a coarsened time grid) without touching the shape-aggregated part. Concretely: "decoupling server from time" means writing the program as (shape-dependent, aggregatable) rows times (time-dependent, per-instance) rows, and applying shape aggregation to the first factor only. The hardware variables $v, u, \mathrm{MACs}, \mathrm{BW}, C_{\mathrm{GM}}$ are already global (one set for the whole chip), so the entire co-design layer is shape- and time-invariant and aggregates trivially.

What the aggregation actually assumes, and where it is not exact. The bilinear couplings deserve a sharper statement than "shape-dependent," because $w = y\,p$ straddles both axes: its y factor is shape-indexed and its p factor is time-indexed, so it belongs to neither block cleanly, and aggregating it on the shape axis alone would *not* be exact. Two instances of one shape sitting at different positions in the execution order face different memory pressure and different residency states; their tiling coefficients are identical but their optimal products need not be, and keeping p per-instance does not by itself resolve that coupling. We therefore do not aggregate the products on shape. In the implemented program the coupling variables carry *both* indices, $z^{\mathrm{act}}_{s,c,t} = y_{s,c}\,p_{s,t}$ and $z^{W}_{s,c,t} = y_{s,c}\,p^{W}_{s,t}$, so a shape at two different timesteps takes two different product values and the McCormick envelope is applied at (shape, config, timestep) resolution, not per shape. Multiplicity enters only where it is linear, as the weight m_s on that shape's contribution to the capacity and objective rows.

Stated plainly, the assumption the aggregation does make is that the m_s operations of a shape share *one* residency trajectory $p_{s,\cdot}$, and the program is exact under it: if every instance of a shape is pinned, evicted, and recomputed on the same schedule, the aggregated rows reproduce the per-instance rows exactly, and $\sum_i T_i = \sum_s m_s T_s$ holds with equality. It is an approximation precisely when two instances of one shape would want different residency trajectories, which the aggregated program cannot express: the coarsened time grid ($T = S$) cannot separate them.

We are deliberate about what this does and does not let us claim. Forcing a shared trajectory *restricts* the residency space, which alone would make the aggregated optimum an upper bound on the unaggregated one; but the same coarsening also reduces the number of per-timestep capacity rows, which cuts the other way. We have not established which effect dominates in general, so we do *not* claim a direction for the aggregation error, and the certified bounds we report should be read as bounds for the aggregated program rather than for the per-instance one. Nnorom Jr. et al.

[2026] give the general form of the entity-aggregation argument and the conditions under which it preserves optimality; the additional condition here, absent from their time-free snapshots, is the shared-trajectory assumption above, and it is the one that needs empirical support rather than a proof sketch. That support is direct: we solved the aggregated and unaggregated programs on instances small enough to admit both and compared optima. Table 55 reports the V1 tile-dependent LP on the first N operations of DeepSeek-V3 (real per-tile Timeloop columns, HiGHS, one solver, one op set per row). The unaggregated arm keeps every operation with $T = N$; the aggregated arm groups operations by identical Pareto row, scales each representative's compute, traffic and rematerialization coefficients by its multiplicity m_s, charges its tile bytes once, and coarsens to $T = S$, which is exactly the shared-trajectory program described above. The two optima agree to ten significant figures at every size while the aggregated program is up to $89\times$ smaller.

Table 55: Measured shape-aggregation error on the V1 tile-dependent LP, DeepSeek-V3, first N operations, CGM 16 MiB, RLT rows on. Columns are LP variables. Multiplicities reach 10 at $N = 128$. Zero error is the shared-trajectory assumption holding at the optimum, which is guaranteed when the capacity rows are slack; that is the regime of this suite, and the result should be read as scoped to it rather than as a general theorem.

N	S	cols (unagg.)	cols (agg.)	unaggregated	aggregated	error
16	10	9,729	3,981	$3.848169063 \times 10^{10}$	$3.848169063 \times 10^{10}$	0.0000%
32	10	37,377	3,981	$1.027407872 \times 10^{11}$	$1.027407872 \times 10^{11}$	0.0000%
64	13	146,433	6,540	$1.397050573 \times 10^{11}$	$1.397050573 \times 10^{11}$	0.0000%
128	13	579,585	6,540	$2.088766669 \times 10^{11}$	$2.088766669 \times 10^{11}$	0.0000%

The zero is not vacuous: the objective moves by $5\times$ across the rows, so the truncated instances are exercising the per-operation latency terms that aggregation collapses. It is also not unconditional. Under the shared-trajectory assumption the aggregated program is exact whenever the per-instance optimum already uses one residency trajectory per shape, and that is guaranteed when on-chip capacity does not bind, which is the measured state of every workload in this paper (the same fact that makes the linking cuts of Eq. (22) inert). A capacity-bound workload whose repeated shapes want divergent residency would be the case that separates the two programs, and it is the first thing to re-measure when such a workload enters the suite.

Dual-guided rounding. Nnorom Jr. et al. [2026] also extract the LP's dual variables, dual variables on each resource, and use them to rank and greedily round the relaxed assignments, which is tighter than naive rounding. We already round the relaxed tile selection by $\arg\max_c y_{i,c}$ and re-solve (Section 5.2); the same dual-guided ranking sharpens the rounding of the *new* integer knobs the comprehensive program adds , the expert-pin g_e, the precision one-hots q_ρ, and the array-aspect one-hot α_a, by pinning first the knobs whose reduced cost most improves the certified bound.

An independent validation of the paradigm. Beyond the scalability trick, Nnorom Jr. et al. [2026] is a useful external data point: a contemporaneous system that, for a completely different problem (datacenter task placement), independently arrives at CHEETAH's exact recipe, relax a hard discrete program to a convex one, read the dual variables as interpretable dual signals that feed an outer decision loop, and round greedily against those duals. That the same LP-relaxation-plus-dual-feedback structure is effective across such different domains is evidence that the paradigm at the core of CHEETAH is sound and general, rather than an artifact of accelerator co-design.

L The Master CHEETAH-V1 Program: A Comprehensive Reference

For reference, we collect the complete comprehensive CHEETAH-V1 co-design program in one place: the core program on the first page and its extended families on the second. It is one certified optimization over a generic graph intermediate representation (a DAG of operations, each carrying a Timeloop-derived tile-configuration Pareto set and a roofline, connected by residency edges), with the hardware sized *inside* the program and every serving-regime decision folded in as a certified constraint family. Bandwidth *is* a finite menu of legal points, so its reciprocal $u = 1/\mathrm{BW}$ is an exact one-hot expression whose LP relaxation is a lower bound; the compute reciprocal $v = 1/\mathrm{MACs}$ is bracketed by tangent rows (outer, certified lower bound) and chord rows (inner, an achievable design only once every selector is pinned and the exact-lift check passes); and every selector-times-quantity product is written in convex-hull form, exact at integral selectors. The program is therefore a *linear program* (LP): hull rows for the products, epigraph rows for the rooflines, and linear rows for everything else. Each constraint family is drawn from a production serving or kernel system (vLLM, SGLang, ThunderKittens, TensorRT-LLM); Appendix N maps each technique to its constraint family, names the solver, and reports a preliminary serving-regime validation on DeepSeek-V3. Each family is *inert when pinned*, so (V0) $\subseteq$ (V1) $\subseteq$ (this program). The extended families below *are part of this program*, not a wish list appended to it: each is written out as the rows the solver would receive, and each carries a status marker separating the families whose effect this paper measures from those the formulation carries but has not yet measured on the suite. We state them here in full because the program, not any one experiment, is what a reader should be able to reproduce and build on.

The Comprehensive CHEETAH-V1 Program

$$\min \quad T_{\text{serve}} \;+\; \sum_{i,t} c_i^{\text{remat}}\, r_{i,t} \;+\; \varepsilon_{\text{CGM}} \sum_{k} k\, h_{\text{GM},k} \qquad \text{s.t.}$$

SELECTION & EXACT COUPLING

$$\sum_c y_{i,c} = 1, \quad \sum_k h_{\text{GM},k} = 1$$
$$w_{i,c,k} = y_{i,c}\, p_{\text{assoc}(i,k),o(i)} \text{ [McCormick hull, exact at integral } y\text{]}$$

DISAGGREGATED PERSPECTIVE EPIGRAPH (ROOFLINE)

$$\sum_c v_{i,c} = v, \;\; \underline{v}\, y_{i,c} \le v_{i,c} \le \overline{v}\, y_{i,c}, \;\; \sum_{a,f} v_{i,c}^{a,f} = v_{i,c}, \;\; 0 \le v_{i,c}^{a,f} \le \overline{v}\, \psi_{a,f}$$
$$\tau_{i,c} \;\ge\; \sum_{a,f} \frac{T_i^{\min}(c)}{\eta_{i,a}\,\gamma_f}\, v_{i,c}^{a,f} \text{ [per-branch copies of } v\text{; lower bound on } y \text{ only]}$$
$$X_{i,c} = T_i^{\max}(c)\, y_{i,c} - \sum_k t_i^k(c)\, w_{i,c,k} = \sum_j X_{i,c,j}, \quad \underline{X}_{i,c}\, \nu_j \le X_{i,c,j} \le T_i^{\max}(c)\, \nu_j$$
$$\tau_{i,c} \;\ge\; \sum_j u_j\, X_{i,c,j} \text{ [bandwidth-menu copies]}, \qquad T_i \ge \sum_c \tau_{i,c}$$

INTERLEAVED PREFILL+DECODE SERVING [Kwon et al., 2023]

$$T_{\text{serve}} \ge \theta\, C^{\text{pre}} + N\, C^{\text{dec}}, \quad T_{\text{serve}} \ge \theta\, M^{\text{pre}} + N\, M^{\text{dec}} \quad \text{[lifted streams; } \theta \text{ a constant]}$$

HARDWARE SIZING, THE CHIP INSIDE THE LP

$$v \ge \tfrac{2}{a_g} - \tfrac{m}{a_g^2} \;\forall g \text{ [tangent rows; chord rows for the achievable LP]}$$
$$\text{BW} = \sum_j \nu_j \text{BW}_j, \;\; u = \sum_j \nu_j u_j, \;\; \sum_j \nu_j = 1 \text{ [bandwidth menu, } u_j = 1/\text{BW}_j\text{]}$$
$$\sum_a \alpha_a = 1 \text{ (array aspect)}, \quad \sum_f \phi_{\text{ph},f} = 1 \text{ (DVFS)}, \quad \psi_{a,f} \text{ joint one-hot of } (\alpha_a, \phi_{\text{ph},f})$$
$$\text{BW}_{\text{eff}} \le \text{BW}, \quad \text{BW}_{\text{eff}} \le B_{\text{LD}} \text{ (load engines) [Spector et al., 2024]}$$
$$\kappa_{\text{MAC}}\text{MACs} + \kappa_{\text{BW}}\text{BW} + \kappa_{\text{LD}} B_{\text{LD}} + \kappa_{\text{SRAM}} C_{\text{GM}} \le A_{\text{cap}}$$
$$\pi_{\text{MAC}}\text{MACs} + \pi_{\text{bit}}\, \varrho_{\text{DRAM}} \le P_{\text{cap}}; \quad \text{6 manufacturability gates}$$

CAPACITY, RESIDENCY, REMATERIALIZATION [Dao et al., 2022]

$$\sum_{e \in \mathcal{E}_{\text{live}}(t)} \sum_c s^{\text{KV}} d_{e,c}^{\text{act}} z_{e,c,t}^{\text{act}} + \sum_i \sum_c s^W d_{i,c}^W z_{i,c,t}^W \le \sum_k 2^k h_{\text{GM},k}$$
$$p_{e,t} \le p_{e,t-1} + r_{\text{src}(e),t}, \qquad r_{i,t} \le \min(p_{e,t}, p_{i,t}^W)$$

PRECISION (per-phase, per-expert) [Lin et al., 2024, Frantar et al., 2023, Hooper et al., 2024]

$$\sum_\rho q_{\phi,\rho} = 1, \;\; s_\phi^W = \sum_\rho b_\rho^W q_{\phi,\rho}, \;\; s_\phi^{\text{KV}} = \sum_\rho b_\rho^{\text{KV}} q_{\phi,\rho}$$

KV BUDGET / EVICTION → FIT IN CGM [Xiao et al., 2024, Zhang et al., 2023b]

$$B^{\text{KV}} = \sum_{\rho,s} B_{\rho,s}, \;\; B_{\min}\pi_{\rho,s} \le B_{\rho,s} \le L_{\text{ctx}}\pi_{\rho,s} \;\; [s \in \{\text{on}, \text{off}\}]$$
$$d^{\text{KV}} \sum_\rho b_\rho^{\text{KV}} B_{\rho,\text{on}} \le \sum_k 2^k h_{\text{GM},k} \quad \text{[on-chip branch; hull, no big-}M\text{]}$$

MOE EXPERT RESIDENCY (cache / offload)

$$\sum_e g_e\, s_e^W W_e \le C_{\text{HBM}}, \qquad \Delta^{\text{moe}} = \sum_e f_e(1 - g_e)\, s_e^W W_e$$

PREFIX / KV CACHE REUSE (RADIXATTENTION) [Zheng et al., 2024]

$$\widehat{C}^{\text{pre}} = (1 - \eta_{\text{hit}}(C_{\text{cache}}))\, C^{\text{pre}}, \quad \widehat{M}^{\text{pre}} = (1 - \eta_{\text{hit}})\, M^{\text{pre}} \quad \text{[prefill only]}$$

DIFFUSION CROSS-STEP REUSE (DEEPCACHE) [Ma et al., 2024]

$$\widehat{\text{cost}}_b = (1 - \chi_b(1 - \tfrac{1}{W}))\, \text{cost}_b, \quad \sum_b \chi_b F_b \le \sum_k 2^k h_{\text{GM},k}$$

DOMAINS

$$y, p, p^W, r, w, z, h, q, \alpha, \phi, \psi, \nu, \pi, g, \chi \in [0,1]; \;\; v, u, B_{\text{LD}}, B^{\text{KV}}, C_{\text{cache}}, C_{\text{HBM}} \ge 0$$

The Comprehensive CHEETAH-V1 Program: Extended Families

part of the program above; each family is written out, and each carries its validation status

STRUCTURED SPARSITY 2:4 *[measured, Appendix N]*

$$\sum_\ell \sigma_\ell = 1, \quad s^{\text{spar}} = \sum_\ell (1 - z_\ell)\,\sigma_\ell \ \text{ scales } d^W \text{ and MACs}$$
$$\kappa_{\text{spar}} \sum_{\ell>0} \sigma_\ell \le A_{\text{cap}} \ \text{ [sparse-MAC support costs area]}$$

MEMORY-TECHNOLOGY SELECTOR *[measured, Appendix P]*

$$\sum_t m_t = 1, \ \ (\kappa_{\text{BW}}, \pi_{\text{bit}}, \overline{\text{BW}}) = \sum_t m_t\,(\kappa^t_{\text{BW}}, \pi^t_{\text{bit}}, \overline{\text{BW}^t})$$
$$\text{BW} \le \sum_t \overline{\text{BW}^t} m_t \ \text{ [hull of } m_t\text{BW; exact at integral } m\text{]}$$

COMPUTE–MEMORY OVERLAP κ (async copy / TMA) *[in the program]*

$$\tau_{i,c} \ge (1-\kappa)\,\text{cmp}_{i,c} + \text{mem}_{i,c}, \qquad \tau_{i,c} \ge \text{cmp}_{i,c} + (1-\kappa)\,\text{mem}_{i,c}$$
[exactly $\tau \ge \text{cmp} + \text{mem} - \kappa \min(\cdot,\cdot)$ for a menu constant $0 \le \kappa \le \kappa_{\max}$; no min column, no McCormick credit; gated by an async-DMA area term]

REGISTER FILE / SMEM C_{RF} AND OCCUPANCY *[in the program]*

$$\sum_c \text{tile}_{i,c}\, y_{i,c} \le C_{\text{RF}}, \qquad O_i = \sum_c o_{i,c}\, y_{i,c}, \qquad O_i \ge O_{\min} - \Lambda(1-\beta_i)$$
$$\tau_{i,c} \ \ge \ \tau^{\text{lat}}_{i,c}\,(1-\beta_i) \ \text{ [DRAM latency hidden only at occupancy } O_{\min}\text{;}$$
a tile $\leftrightarrow$ latency-hiding trade-off distinct from C_{GM}]

TIERED KV RESIDENCY (hot on-chip / cold streamed) *[in the program]*

$$B_{\text{hot}}\, s^{\text{KV}} d^{\text{KV}} \le \sum_k 2^k h_{\text{GM},k}, \qquad 0 \le B_{\text{hot}} \le B^{\text{KV}}$$
$$\Delta^{\text{KV}} = d^{\text{KV}} \sum_\rho b^{\text{KV}}_\rho \left(B^{\text{KV}}_\rho - B_{\text{hot},\rho}\right) \ \text{ [only the cold tail is streamed]}$$

WINOGRAD / TRANSFORM SUPPORT *[in the program]*

$$\omega_i \in [0,1] \text{ per } 3\times3 \text{ conv}, \qquad \text{MACs}_i \times \left(1 - \omega_i(1 - r_{\text{Wino}})\right)$$
$$\kappa_{\text{tr}} \max_i \omega_i \le A_{\text{cap}} \ \text{ [transform unit costs area; written as } \kappa_{\text{tr}}\omega_i \le A_{\text{cap}}\ \forall i\text{]}$$

EXPLICIT U-SHAPE SKIP RESIDENCY *[in the program]*

$$\mathcal{E}_{\text{live}}(t) \cup= \ \mathcal{E}_{\text{skip}}, \qquad p_{e,t} \le p_{e,t-1} + r_{\text{src}(e),t} \ \ \forall e \in \mathcal{E}_{\text{skip}}$$
[down-path features persist for their up-path consumers, so a long-range skip competes for C_{GM} against the sequential chain]

SCALABILITY: SHAPE AGGREGATION *[measured, Appendix K]*

build over S unique op-shapes with multiplicities n_s, not N operations
[$\sim$100 $\times$ size reduction; exact whenever identical shapes share a tile config]

Status key. [measured] marks a family solved and reported in this paper. *[in the program]* marks a family that is written out above and carried by the formulation, and whose effect has not yet been measured on the suite; it is inert when its selector is pinned off, so the reported results are unchanged by its presence.

Every constraint family reduces to a change in bytes moved, FLOPs, on-chip residency, or a hardware parameter, hence a linear or convex-hull (disaggregated McCormick) representable addition rather than an outer-loop guess. Fusion and on-chip tiling realize FlashAttention-style IO-awareness [Dao et al., 2022]; the interleaved objective and RadixAttention/paged prefix reuse come from vLLM and SGLang [Kwon et al., 2023, Zheng et al., 2024]; the load-engine vs. compute-worker datapath split and native-MMA-tile utilization come from tile kernels [Spector et al., 2024]; precision is bounded by an accuracy-vetted menu [Lin et al., 2024, Frantar et al., 2023, Hooper et al., 2024]; KV eviction follows [Xiao et al., 2024, Zhang et al., 2023b]; and diffusion cross-step reuse follows DeepCache [Ma et al., 2024]. Accuracy is never linearized; it is bounded by the cited menu. Quantities that only rescale coefficients (the decode batch N, the speculative-decode width K) enter as workload parameters, and genuinely discrete or non-convex axes (interconnect radix, HBM-stack count, array aspect, low-bit MAC-unit support, draft model, denoising-step schedule) remain columns the LLM-Architect outer loop proposes and the LP scores by reduced cost against its converged duals $\lambda_{\text{BW}}, \lambda_{\text{MAC}}, \lambda_{\text{HBM}}, \lambda_{\text{CGM}}, \lambda_{\text{LD}}, \lambda_{\text{cache}}$. Every added block is inert when pinned, so the program subsumes the entire V0 and V1 hierarchy.

M Conditioning of the comprehensive LP and what it does and does not demand of the solver

This appendix records a numerical finding, noted here for further discussion and to motivate the equilibration inside the solver of Section 6; it also records, from the same measurements, which solver actually certifies the bounds reported in this paper.

Assembling the comprehensive CHEETAH-V1 program (Appendix L) as a single monolithic linear program for DeepSeek-V3, with all decisions as decision variables, the hardware reciprocals as tangent and chord rows, bandwidth as a one-hot menu, and every selector-times-quantity product in convex-hull form, yields a compact but ill-conditioned instance (6,815 variables, 510 equality and 22,511 inequality rows, plus 29 to 89 tangent or 88 chord rows for the compute reciprocal, so 23,050 to 23,110 rows and about 58,600 nonzeros per exported instance; the earlier conic form of the same program had 11,599 variables and 32,582 inequality rows). In natural units the coefficient dynamic range of the earlier form spanned roughly 3×10^{27}, dominated by the big-M constants ($\sim 10^{18}$) in the indicator and capacity rows that linked the discrete residency and one-hot selections to the continuous schedule. The hull reformulation removes those constants entirely, and an exact diagonal unit rescaling (latency in units of 10^9 cycles at the 1 GHz convention, one cycle per nanosecond; traffic in units of 10^6 bytes; the optimum is unchanged) brings the per-row-family coefficient range to at most 8.8×10^6 (the energy-bytes equality family; the next largest, 4.2×10^6, is the box family of the MACs-times-DVFS hull), against a design limit of 10^8. The range that remains is intrinsic: the tangent and chord rows on $1/m$ span eight octaves of m, so their coefficients span the same magnitudes the reciprocal itself does.

Historical note. The conic form of this program (exact rotated cones for the two reciprocals) could not be solved by stock conic solvers in natural units: an interior-point conic solver (Clarabel) reported a spurious `unbounded` status; a second interior-point solver (ECOS) raised a numerical exception; and a first-order solver (SCS) returned bound-violating iterates (for example a negative provisioned bandwidth) even after 10^5 iterations. After the unit rescaling Clarabel returned an `optimal` solution in about 0.6 seconds, but the instance remained *stiff*: the interior-point method intermittently stalled at suboptimal points that it nonetheless flagged `optimal`, so a multi-start was required. That stiffness is what motivated the linear reformulation. The stiffness and the failure of stock solvers are statements about that historical conic form and, by the same coefficient-range argument, about the un-aggregated full-resolution instance ($\sim$8.7M rows for the per-operation build, Appendix N), on which no simplex result is available (**[TODO: measured: CORGI and HiGHS on the full-resolution instance]**); they are not statements about the shape-aggregated LP that produces every bound in this paper.

What is measured on the shape-aggregated LP. On the 67 exported serving instances (32 pinned bandwidth-times-decode-precision branches, each as an outer and a chord LP, plus the relaxed root pair and one diagnostic) HiGHS dual simplex returns an optimal vertex on every one in about 0.05 seconds (311 iterations on the relaxed root). CORGI, our restarted primal-dual hybrid gradient method with a *diagonal variable-metric rescaling* and a one-sided rescue restart (Section 6; 200,000 iterations, about 35 seconds per instance on an RTX 5090), converged on none of them: on the fully relaxed root its objective stops 1.8 to 2.7 times above the optimum and its dual bound is negative, and on the 64 pinned branch LPs its relative objective error against HiGHS ranges from 3.7×10^{-3} to 18.5 (median 3.4). Every bound in this paper is therefore taken from HiGHS. The lower bound we report is the safe dual bound of the outer (tangent) LP (the larger of the root bound and the minimum over the enumerated selector branches), computed from the HiGHS dual on the identical exported instance with the duals of one-sided rows clipped to their admissible sign, and the integrality gap is reported separately for every pinned menu branch, since tangent refinement cannot close it.

The cause is measured, not conjectured, and it is degeneracy rather than conditioning. At the HiGHS optimum of the relaxed root only 122 of the 6,815 columns are basic, 5,971 non-basic columns have zero reduced cost, 17,430 basic rows are tight, and 14 selector columns in 6 of the 25 one-hot sets are fractional; three simplex seeds, an interior-point solve, and 10^{-6} cost perturbations all return the same objective (to 10^{-15}) at points that differ by 81% in relative norm. The optimal face has dimension in the thousands, and a first-order method has no mechanism to select a point on it. The same non-convergence was recorded on the earlier 20M-variable menu LP, so it is a property of relaxed one-hot programs rather than of size. (The GPU run is also not deterministic from run to run,

and the 10^{30} row-bound sentinels of the export leak into CORGI's reported dual objective, so no per-run gap value is reproducible.)

What the conditioning finding does establish. The dynamic range above motivates the equilibration, not the choice of solver. With the structural presolve (singleton and fixed-column removal, geometric-mean Ruiz equilibration, objective scaling) disabled, the same CORGI run on the relaxed root diverges (primal residual 5.9×10^3, objective about $200\times$ the optimum), and 40 Ruiz iterations on the unscaled row bounds overflow to NaN, so the rescaling machinery is necessary for a first-order method on this matrix at all; it is not sufficient on the degenerate relaxation, and the shape-aggregated LP does not need a first-order method. A full solver-versus-solver comparison at the full-resolution scale is left as immediate future work.

N Audits, Solver, and Serving Validation

This appendix maps the comprehensive program's constraint families to the production systems they are drawn from, states which solver solves the resulting linear program, and reports a preliminary serving-regime validation on DeepSeek-V3. The results here are at the level of the LP relaxation (menu selectors relaxed) and are calibrated on the analytic roofline; every percentage is therefore a relaxation-level estimate, certified in neither direction and produced before the copy-bound fix of (14)*, and certified* $(\mathrm{LB}, \mathrm{UB})$ *pairs from pinned-selector solves replace them; those certified pairs are now measured and are collected in Table 57, and the certified sandwich for the serving program with its full lever set is reported at the end of this appendix.*

What we fold in, and from where. The constraint families of Section 5.3 and Appendix L are not invented in a vacuum: each is a technique from a production serving or kernel system, re-expressed as a certified constraint family whose only effect is on bytes moved, FLOPs, on-chip residency, or a hardware parameter. Table 56 lists the mapping, together with whether the decision binds on DeepSeek-V3 *serving*.

Source	Technique	Constraint family / variable	Binds on DeepSeek serving?
TensorRT-LLM	FP8/INT4 precision; kernel fusion	precision one-hot q, s^W, s^{KV}; fusion = residency p [Lin et al., 2024, Frantar et al., 2023, Hooper et al., 2024, Dao et al., 2022]	**yes**, largest, $\approx 28.6\%$
vLLM	chunked-prefill interleaving; paged KV	interleave θ; KV-budget B^{KV} [Kwon et al., 2023]	**yes**, $\approx 38\%$ tighter
SGLang	RadixAttention prefix reuse	$\eta_{\mathrm{hit}}(C_{\mathrm{cache}})$ [Zheng et al., 2024]	present; small (little prefix sharing)
ThunderKittens	async overlap; load/compute split; reg-file	κ; $\mathrm{BW}_{\mathrm{eff}} = \min(\mathrm{BW}, B_{\mathrm{LD}})$; C_{RF} [Spector et al., 2024]	present; inert (memory-bound)
StreamingLLM/H2O	KV eviction	KV-budget cap [Xiao et al., 2024, Zhang et al., 2023b]	small
DeepCache	cross-step feature reuse	χ_b [Ma et al., 2024]	inert (no diffusion)

Table 56: Provenance of the comprehensive program's decisions and whether each binds on DeepSeek-V3 serving. Structured 2:4 sparsity (a hardware/software decision rather than a single-system technique) is the second-largest, $\approx 15.5\%$. The percentages are relaxation-level estimates (menu selectors relaxed), certified in neither direction and produced before the copy-bound fix of (14); certified $(\mathrm{LB}, \mathrm{UB})$ pairs replace them: see Table 57, where 2:4 sparsity alone is certified to remove at least 42.0% of serving latency.

How the mapping works: coefficients as functions of the hardware. The single modelling principle behind the table is that the *only* constants are the reference per-tile Pareto bases ($T^{\min}(c)$, $T^{\max}(c)$, byte footprints); every hardware coefficient is written as base $\times$ f(decision variables). Compute time is $\sum_{a,f} T^{\min}(c)\, v^{a,f}/(\eta_a \gamma_f)$ over per-branch copies of v, and memory time is the bandwidth-menu disaggregation of the (precision-, cache-, and eviction-scaled) bytes; $v \geq 1/m$ is bracketed by tangent and chord rows, and u is the exact menu value, so every product is exact at integral selectors. The classical per-trial Pareto rescale thus becomes an in-program function of the hardware variables rather than a per-chip precompute. Instantiated for DeepSeek-V3 and shape-aggregated (903 operations $\to$ 13 unique shapes, Appendix K), the program has 6,815 variables and 510 equality plus 22,511 inequality rows, all linear, to which the exported instances add 89 tangent (or 88 chord) rows for the compute reciprocal (23,110 rows and 58,698 nonzeros for the certificate instance); the bandwidth menu has 8 points (the 8 of the 12 catalogue points that pass the pin gate $\mathrm{BW}/0.8 \leq 4096$ pins), and hull rows for every selector-times-quantity product, compact, versus $\sim$8.7M rows for the naive per-operation build.

Which solver. The program is a linear program. The shape-aggregated instances are solved by HiGHS (certified vertex, 0.05 seconds per instance) and by CORGI, our restarted primal-dual hybrid gradient method with variable-metric rescaling (Section 6), which is intended for the full-resolution instances (**[TODO: measured: CORGI and HiGHS on the full-resolution instance]**). On the 67 exported serving instances (32 pinned bandwidth-times-decode-precision branches, each as an outer and a chord LP, plus the relaxed root pair and one diagnostic) the two do not yet agree: HiGHS returns an optimal vertex on all 67, while CORGI, at its $200{,}000$-iteration budget (about 35 seconds per instance on an RTX 5090), does not meet its own convergence test on any of them. On the pinned branch LPs its primal objective differs from the HiGHS optimum by between 3.7×10^{-3} and 18.5 relative (median 3.4; only the 225 GB/s branch is within 0.3), and on the fully relaxed root LPs it stops 1.8 to 2.7 times above the optimum. The relaxed root is measurably degenerate (at the HiGHS optimum only 122 of the $6{,}815$ columns are basic, $5{,}971$ non-basic columns have zero reduced cost, and $17{,}430$ basic rows are tight), which is the cause (Appendix M); the GPU run is also not deterministic from run to run, and the 10^{30} row-bound sentinels of the export leak into CORGI's dual objective, so no per-run gap value is reproducible. Every bound reported here is therefore HiGHS's. As Appendix M details, the earlier conic form of the same program was too ill-conditioned in natural units for off-the-shelf conic solvers to solve at all, and stiff even after rescaling; the linear form removes the big-M constants that dominated that range. The lower bounds we report are safe dual bounds of the outer (tangent) LP: the larger of the root bound (all menu selectors relaxed) and the disjunctive bound (the minimum, over the enumerated bandwidth-times-decode-precision branches, of the branch outer LP's safe dual bound). The achievable designs come from the chord LP with every selector pinned and verified by an exact-lift check.

A preliminary serving validation on DeepSeek-V3. The decisions are memory-regime decisions, so the objective the program is written over determines whether they bind. Under a *prefill*-latency objective ($\min \sum_i T_i$) the DeepSeek-V3 optimum is compute-bound (it provisions compute to the reticle limit), and all of the candidate decisions are inert (they move the objective by less than 0.006%). Under the *serving*-latency objective $\min\ T_{\text{prefill}} + G \cdot T_{\text{decode}}$ (context 4096, batch 32, $G = 512$), where the memory-bound decode phase dominates by construction, the decisions activate. Each lever set is now solved to a certified pair: the lower bound is the larger of the safe dual bound of the root outer LP and the smallest safe dual bound over the complete pinned-selector enumeration, and the upper bound is the best exact-lifted pinned-selector design, so $1 - \text{UB}_{\text{lever}}/\text{LB}_{\text{base}}$ is a certified improvement and $1 - \text{LB}_{\text{lever}}/\text{UB}_{\text{base}}$ an optimistic one. Table 57 reports them against a base configuration with every lever off (W8A8 weights and activations, an 8-bit KV cache, dense experts, no expert pinning or host tier, HBM, full context, no prefix cache, no register-file residency and no overlap). W4 weight precision alone is certified to remove at least 45.1% of serving latency and 2:4 sparsity at least 42.0%; both act on the mixture-of-experts weight reload, which is $\sim 98\%$ of decode DRAM traffic. The full lever set is certified at 65.5%, but its interval $[18.83,\ 22.02]$ overlaps the interval $[18.81,\ 21.66]$ of W4 and 2:4 sparsity together, so the remaining eleven levers are certified to improve on that pair in neither direction. The compute-side ThunderKittens decisions (overlap, register-file occupancy) remain inert, which is correct for a weight-reload-bound workload, and pinning, memory technology, the attention split, grouped expert GEMM, prefix caching and the KV budget move neither bound by more than 10^{-4} on this workload. The earlier relaxation-level estimates ($\approx 28.6\%$ for W4, $\approx 15.5\%$ for 2:4) were produced before the copy-bound fix of (14) and compared two relaxed optima, whose ratio bounds the ratio of the true optima in neither direction; the certified numbers are larger.

Three McCormick sub-problems were found to leak spurious gains and were tightened at the time (the hull disaggregation now in (6) is the definitive fix), after which the precision selector came out integral ($s^{W}_{\text{dec}} = 1/2$, W4) even though some other one-hot selectors remained fractional in the relaxation (**[TODO: measured: re-solve with the hull builder]**). A fully pinned solve under the reconciled real-Timeloop calibration is the immediate next step.

Certified sandwich at the solved operating point. For the serving program at the same operating point (context 4096, batch 32, $G = 512$) with the default lever set enabled and the LP rescaled as in Appendix M (objective in units of 10^9 cycles at the 1 GHz convention, one cycle per nanosecond), the pinned-selector enumeration over the 8 bandwidth points and the 4 decode-precision options gives 32 branches, every one of which HiGHS solves to optimality. The root outer LP (all selectors relaxed, tangent grid 10^{-3}) has safe dual bound $\text{LB}_{\text{root}} = 16.836$; the disjunctive bound over the 32 branch

lever set	LB	UB	certified	optimistic
base (all levers off)	63.758	65.422	n/a	n/a
W4 weight precision only	32.995	34.991	45.1%	49.6%
2:4 sparsity only	34.097	36.949	42.0%	47.9%
W4 and 2:4 together	18.811	21.663	66.0%	71.2%
full lever set	18.830	22.018	65.5%	71.2%

Table 57: Certified lever pairs for DeepSeek-V3 serving (context 4096, batch 32, $G = 512$), in units of 10^9 cycles at the 1 GHz convention, one cycle per nanosecond. LB is $\max$(root safe dual bound, smallest branch safe dual bound) on the 10^{-3} tangent grid and UB is the best exact-lifted pinned-selector design; certified $= 1 - \mathrm{UB}_{\mathrm{lever}}/\mathrm{LB}_{\mathrm{base}}$, optimistic $= 1 - \mathrm{LB}_{\mathrm{lever}}/\mathrm{UB}_{\mathrm{base}}$. Every LP in the table was solved to optimality by HiGHS. The last two rows overlap, so neither dominates the other in a certified sense.

outer LPs on the same grid is $\mathrm{LB} = 18.830$, attained at 3200 GB/s with W4A8 and a 4-bit KV cache, and is the certified lower bound. The best exact-lifted design sits in the same branch: $\mathrm{UB} = 22.018$ at 3200 GB/s with W4A8 and a 4-bit KV cache (lift residual 5.9×10^{-17}, every selector integral), so the certified pair is $(\mathrm{LB}, \mathrm{UB}) = (18.830,\ 22.018) \times 10^9$ cycles at the 1 GHz convention, a gap of 14.5% of the upper bound (23.5% against the root bound alone). Twenty of the 32 branches lift directly; the other twelve fail the argmax rounding of the memory-technology selector, lift once that selector is pinned to HBM, and none of them beats the winner (best 25.303). The in-branch integrality gap of the winner is 0.145, of which the reciprocal sandwich contributes 2.2×10^{-4} and the rounding of the selectors that are not enumerated contributes 0.145, so the remaining gap is rounding, not the reciprocal envelope (the root reciprocal residual is 4.5×10^{-4}, inside the 10^{-3} grid). An independent run on an H100 reproduces every HiGHS bound on these certificate LPs to every printed digit.

A defect these certificates exposed. The first certified sandwich we obtained for this program was $(1.910,\ 4.118) \times 10^9$ cycles, a factor of five better than the numbers above, and the design behind it served all 256 experts from the host tier. The tiered-memory family removes hosted experts from the DRAM reload and defines a host-link time T_{host}, but the epigraph that charges it, $T_{\mathrm{dec}}^{\mathrm{step}} \geq T_{\mathrm{host}}$, was missing from the implementation, so hosting an expert was free and the optimiser hosted every one: 186 GB per decode pass over a 64 GB/s link, charged at nothing. With the epigraph restored the same enumeration returns the pair reported above and the winning design hosts 10 of 256 experts, worth one to two percent rather than a factor of sixteen. We report this because it is the failure mode a certified bound is for: the sandwich itself was valid at every step, and what it certified was the model, not the machine.

O Coefficient Audit Trail: Where Every Constant Comes From

Every numerical constant in the co-design program falls into one of three classes: it is measured (extracted from Timeloop runs or from a published model configuration), it is anchored (calibrated against shipped silicon or a published standard), or it is an explicitly stated assumption. The tables below enumerate all of them, grouped by role: the silicon cost model, the six pre-RTL manufacturability gates, the precision and decision menus, the workload-derived quantities, and the solver grids. Status R means Real (measured or silicon-anchored), A means Assumed (a stated modeling choice), and D means Derived (computed from other tabulated constants). Source locations refer to the released code (`systems/feasibility_checks.py` and the LP builders[1]).

The full inventory, including per-coefficient file and line provenance, is maintained in `systems/deepseek_socp_formulation.md` in the released code; constants marked A above are the explicit modeling assumptions a reviewer can vary, and every one enters the program linearly or through a one-hot menu, so sensitivity to any single assumption is a re-solve, not a re-derivation.

[1] The builder files keep their historical `deepseek_socp_*` names; the exported program is a pure LP.

Constant	Value (units)	St.	Anchor
`RETICLE_AREA_MM2` (A_{cap})	600 (hard limit 858) mm^2	A	ASML/TSMC 26×33 mm reticle field is 858 mm^2 (the standard lithography limit); 600 leaves yield headroom, the same cap style as FAST [Zhang et al., 2022]
`MM2_PER_MAC` (k_{MAC})	1.5×10^{-4} mm^2/MAC (INT8, 7 nm)	R	**Post-synthesis extracted** (Section 9.1): OpenROAD/ASAP7 7 nm placement of an INT8 systolic tile at 256/1024/4096 MACs gives 8.21×10^{-5} mm^2/MAC of cell area ($R^2 = 0.999999$ linear, 0.46% spread), i.e. this value at 54.7% placement utilization; also calibrated so TPU-v3 inputs (65,536 MACs) reproduce its core area, and consistent with Simba 8-bit PE area at 16 nm scaled to 7 nm [Shao et al., 2019]
`MM2_PER_MB_SRAM` (k_{SRAM})	1.5 mm^2/MB (7 nm)	R	TSMC N7 SRAM bitcell 0.027 μm^2/bit (ISSCC 2017 N7 test chip) plus macro overhead gives 1.4–1.6 mm^2/MB
`MM2_PER_DRAM_CHANNEL`	8.0 mm^2/channel	R	HBM PHY plus controller floorplan area on TPU/GPU die shots (TPU-v4i die analysis) [Jouppi et al., 2021]
`MIN_CHIP_AREA_MM2`	50.0 mm^2	A	IO ring, control, and power-grid floor observed on small accelerator dies (Simba chiplet package overheads) [Shao et al., 2019]
`W_PER_MAC_AT_FULL_UTIL`	0.4×10^{-3} W/MAC at 1 GHz (7 nm INT8)	R	Calibrated to TPU-v3 250 W TDP at 65,536 MACs × 700 MHz, then roughly 5× 7 nm INT8 efficiency (Horowitz energy scaling) [Jouppi et al., 2021]
`DEFAULT_UTILIZATION`	0.5 (fraction)	A	Measured LLM-serving MFU of 30–50% on production accelerators (vLLM and TPU serving reports) [Kwon et al., 2023]
$k_{\text{HBM,byte}}$ (HBM-resident capacity area)	$k_{\text{SRAM}}/10 = 1.43 \times 10^{-7}$ mm^2/byte	A	DRAM roughly 10× denser than SRAM per bit (standard memory-technology density gap)
k_{BW} (bandwidth area)	0.08 mm^2/(GB/s)	D	`MM2_PER_DRAM_CHANNEL` (Real) divided by roughly 100 GB/s per HBM3 channel (819 GB/s per stack over 8 channels, JEDEC JESD238)
k_{LD} (load-engine area)	$= k_{\text{BW}} = 0.08$ mm^2/(GB/s)	A	Load/DMA engine area assumed comparable to PHY area at matched bandwidth
π_{MAC} (power per MAC at operating util.)	2.0×10^{-4} W/MAC	D	Product of `W_PER_MAC_AT_FULL_UTIL` (TPU-v3 calibrated, Real) and `DEFAULT_UTILIZATION` [Jouppi et al., 2021]. **Silicon-anchored**: 0.4×10^{-3} W/MAC at 1 GHz is 0.40 pJ/MAC-op, bracketed by whole-chip shipped INT8 silicon (H100 0.35, A100 0.64, TPU-v4i 1.27 pJ/op), which an array-only coefficient must sit at or below (Section 9.1)
P_{cap} (TDP cap)	700 W	A	NVIDIA H100 SXM5 TDP is 700 W (vendor datasheet)
π_{bit} (DRAM access energy)	4.0×10^{-12} J/bit	A	HBM2 access energy roughly 3.9 pJ/bit (O'Connor et al., Fine-Grained DRAM, MICRO 2017)

Table 58: Silicon cost model: area and power coefficients. Sources: `feasibility_checks.py` lines 32–48 and the LP builders (`deepseek_socp_codesign.py` lines 341–350, `deepseek_socp_serving_codesign.py` lines 147–152).

P Dual Variables for the Outer Loop: A Mapping from Cluster Resource Allocation

The recipe at the core of CHEETAH, relax a hard discrete resource-allocation problem to a convex one, read the dual variables, and use them to guide the discrete choices, is not unique to accelerator co-design. The same structure appears in large-scale cluster task placement [Nnorom Jr. et al., 2026], which solves granular server allocation 100–1000× faster by (i) relaxing the binary task-to-server assignment and *aggregating servers into shapes* so the program scales with the number of shapes rather than servers, and (ii) extracting the LP's dual variables as per-resource *dual variables* and placing tasks greedily by *net utility* = priority $- \sum_r (\lambda_r \times \text{demand}_r)$, re-evaluating as the load shifts.

Constant	Value (units)	St.	Anchor
`COOLING_W_PER_MM2` (power-density gate)	2.0 W/mm^2	A	Envelope over real silicon: TPU-v3 0.36, TPU-v4 0.55, H100 0.7, MI300X 0.6, WSE-3 1.5 W/mm^2; 2.0 corresponds to aggressive AIO/liquid cooling
`MAX_LOGIC_DEPTH` (logic-depth gate)	35 gate levels at 1 GHz	A	FO4-per-cycle timing literature: roughly 25–35 logic levels per 1 ns cycle at 7 nm (Harris/Horowitz FO4 method)
`SRAM_BANK_KB` (sram-macro gate)	64 KB per macro	R	Standard foundry SRAM compiler macro size; Simba uses 64 KB-class buffers per PE cluster [Shao et al., 2019]
`MAX_BANKS` (sram-macro gate)	4096 banks	A	Practical routing limit for a banked global buffer; conservative floorplanning rule
`MAX_BANK_ASPECT` (sram-macro gate)	4.0 (aspect in [1:4, 4:1])	A	Standard physical-design floorplanning guideline for macro arrays
`HBM3_GBPS_PER_PIN` (pin gate)	0.8 GB/s per pin (6.4 Gbps / 8)	R	JEDEC HBM3 standard JESD238: 6.4 Gb/s per pin
`MAX_PIN_COUNT` (pin gate)	4096 pins (4 stacks $\times$ 1024 data pins, roughly 3.3 TB/s)	R	JEDEC HBM3 1024-bit interface per stack; 4-stack package standard (TPU-v3 uses 4 HBM stacks) [Jouppi et al., 2021]
Crossbar routing-cell cap (crossbar gate)	5×10^6 cells ($n_{\text{pes}}\times n_{\text{banks}}$; flat-crossbar bound 10^6, relaxed $5\times$ for a 2-level hierarchical NoC)	A	Industry hierarchical-NoC precedent: TPU-v4 mesh plus crossbar, H100 GPC/L2 partitions, MI300X chiplet XGMI, WSE-3 tile fabric [Jouppi et al., 2021]

Table 59: Pre-RTL manufacturability gate thresholds. Sources: `feasibility_checks.py` lines 57–64 and 174.

We already borrow the shape aggregation (Appendix K); this appendix records the full one-to-one correspondence and, more usefully, the concrete rule it hands our LLM-guided outer loop.

Dual variables as the outer-loop search signal. The mapping's payoff is that it makes the LLM-guided outer loop (Section 5.4) a *dual-guided column generator*, exactly as Nnorom Jr. et al. [2026] scores task placement. The loop is: (1) solve the inner LP and read the dual vector $\lambda = (\lambda_{\text{CGM}}, \lambda_{\text{BW}}, \lambda_{\text{MAC}}, \dots)$ together with the reduced costs of the un-chosen columns; (2) the duals *diagnose the bottleneck*, the largest λ_r marks the scarcest resource, while $\lambda_r \approx 0$ marks a free one; (3) the LLM proposes candidate columns (a chip parameterisation, a decision setting, a tiling) and ranks them by the reduced cost $\bar{c}_j = c_j - \sum_r \lambda_r a_{rj}$, i.e. objective gain net of the dual-weighted cost of the resources the column consumes; a column with $\bar{c}_j < 0$ improves the design and the most negative is the best proposal; (4) re-solve for updated duals and repeat. The loop terminates when no proposed column has negative reduced cost, a certified no-improving-candidate stop, with the LP bound at the caps quantifying the maximum regret of stopping. The LLM only ever *proposes*; the LP scores and certifies, so no optimality certificate is ever at risk.

Worked example: the duals point to speculative decoding. On the DeepSeek-V3 decode workload the converged duals are $\lambda_{\text{BW}} \gg 0$ (bandwidth is the binding scarcity) and $\lambda_{\text{MAC}} \approx 0$ (compute is idle, hence free). The reduced-cost rule then *selects* the speculative/parallel-decode-width column K without being told to: verifying K tokens per weight-load spends the near-zero-dual MAC resource to relieve the high-scored bandwidth (it amortises the mixture-of-experts weight reload over the accepted tokens), so its reduced cost $\bar{c}_K = c_K - \lambda_{\text{MAC}} \cdot (\text{K's compute}) - \lambda_{\text{BW}} \cdot (\text{negative traffic})$ is strongly negative precisely because $\lambda_{\text{MAC}} \approx 0$ while λ_{BW} is large. The weight-precision column $W4$ is the same logic on the byte axis (it lowers the λ_{BW}-scored traffic directly). That the dual variables independently point at the two decisions we found empirically to dominate is evidence the signal is the right one.

Dual-guided rounding. The second stage of Nnorom Jr. et al. [2026] also improves our rounding. Where the LP relaxation leaves a one-hot selector fractional (in our DeepSeek solves the precision and expert-pin selectors), rounding by reduced cost, fixing first the selector whose reduced cost most improves the certified bound, then re-solving , is tighter than the $\arg\max$ rounding of Section 5.2, and is the natural integer companion to the dual-guided search above.

Constant	Value (units)	St.	Anchor
`MAC_WIDTH_BY_DTYPE`	int8 = 8, bf16/fp16 = 16, fp32 = 32 bits	R	Datatype bit-widths by definition (IEEE 754 / bfloat16)
Precision-menu byte fractions (b_W, b_A, b_{KV})	W8A8 = (1,1,1); W4A8 = (0.5,1,1); W4A8-KV4 = (0.5,1,0.5); W4A16 = (0.5,2,1)	A	Accuracy-vetted quantization settings from AWQ (W4), GPTQ (W4), KVQuant (KV4) [Lin et al., 2024, Frantar et al., 2023, Hooper et al., 2024]
Array-aspect menu dims	sq16x16 = (16,16); wgmma8x32 = (8,32); tcgen05_32x8 = (32,8) rows $\times$ cols	R	NVIDIA Hopper WGMMA and Blackwell tcgen05 native MMA tile shapes (PTX ISA); ThunderKittens documents the H100 tile constraint [Spector et al., 2024]
Utilization $\eta(\text{spec}, a_r, a_c)$	$\eta = (\text{rows}/\lceil\cdot\rceil_r)(\text{cols}/\lceil\cdot\rceil_c)$, clipped to $[10^{-3}, 1]$	A	Analytic ceil-quantization model of array under-fill; reproduces the Timeloop mapper low-GEMV-utilization artifact [Parashar et al., 2019]
DVFS menu γ (clock scale)	[0.7, 1.0, 1.3] for lo/nom/hi	A	Standard CMOS DVFS operating points ($\pm$30% around nominal); f proportional to V
DVFS menu p_f (dynamic-power factor)	[0.5, 1.0, 1.9] for lo/nom/hi	A	P proportional to fV^2 (roughly cubic in f), the CMOS dynamic-power law
Radix-cache menu sizes	[0, 64, 256, 1024] MB	A	Cache-size sweep grid; an on-chip-SRAM-scale prefix cache [Zheng et al., 2024]
Radix-cache hit-rate profile η_s	[0.0, 0.30, 0.50, 0.65] (saturating), fraction of prefill reuse	A	SGLang RadixAttention measured prefix-cache hit rates (saturating in cache size) [Zheng et al., 2024]

Table 60: Precision, datatype, and decision menus. Sources: `feasibility_checks.py` lines 65–73 and `deepseek_socp_codesign.py` lines 228–280.

Caveat. As in Appendix K, each round of cluster placement is a static snapshot whereas CHEETAH carries a time-indexed residency axis; the correspondence is exact on the shape-invariant block (tiling, roofline, hardware, and the decisions), and the time-indexed residency is kept per-instance rather than aggregated or scored as a snapshot.

Q What CHEETAH-V1 Decides Inside the Program

It is worth stating plainly what is a *decision* of the program and what is an input to it, because the contribution rests on the boundary between the two. In a sequential pipeline the hardware is a fixed input: the mapper is asked how fast a given chip runs a given layer. In CHEETAH-V1 the hardware is a variable, and the only true constants are the measured Timeloop bases $T_i^{\min}(c)$, $T_i^{\max}(c)$ and the per-tile byte footprints. Every coefficient that a sequential flow would bake in is instead a function of decision variables and is evaluated inside the constraints.

Hardware, decided rather than given. The MAC count and DRAM bandwidth enter through their reference-normalized reciprocals v and u, so a tile's compute time $T_i^{\min}(c)\,v$ and memory time $T_i^{\max}(c)\,u$ scale with a chip the program is simultaneously choosing. On-chip global memory is a one-hot over a size grid, and the HBM residency pool, the prefix-cache size, the load-engine bandwidth, the systolic array aspect ratio and the clock and voltage operating point are all selectors in the same program. These are not swept in an outer loop and read back; they are columns of the constraint matrix.

Schedule, decided jointly with the hardware. Tile configuration per operation, which tensors are resident on chip at each timestep, which are evicted, which are rematerialized, and which operations fuse, are decided together with the chip rather than after it. This is what allows a locally worse tiling to be selected when it produces a globally better schedule, a trade a map-then-fuse pipeline cannot represent.

Serving behavior, decided as constraints. Weight, activation and KV precision are one-hot selectors per phase and per expert; the KV budget and eviction policy, the mixture-of-experts residency knapsack, prefix-cache reuse, structured sparsity, memory technology, the prefill–decode

Constant	Value (units)	St.	Anchor
Timeloop Pareto tile menu $T_{\min}(c), T_{\max}(c), t_{I/O/W}(c), d_{I/O/W}(c)$	per-op per-tile arrays (cycles, bytes)	R	Measured Timeloop mapper Pareto frontier for DeepSeek-V3 layers [Parashar et al., 2019]
f_e routing frequency (balls-in-bins base)	$f_{\text{unif}} = 1 - (1 - 8/256)^B$ (0.983 at B=128; 0.638 at B=32)	D	Distinct-expert balls-in-bins derivation from the DeepSeek-V3 top-k=8 of n=256 routed experts [DeepSeek-AI, 2024]
Zipf routing-skew exponent	0.6 (mean-normalized tilt on f_{unif})	A	Mild expert-load imbalance consistent with the MoE load-balancing literature (DeepSeek-V3 aux-loss-free routing reports residual skew) [DeepSeek-AI, 2024]
E_{distinct} (expected distinct experts per decode step)	$256(1 - (1 - 8/256)^{32}) \approx 163$ at batch 32	D	Balls-in-bins expectation at the decode operating batch; f_e renormalized to this mean [DeepSeek-AI, 2024]
W_e (per-expert weight bytes, 8-bit)	$3 \times 7168 \times 2048 =$ 44.0 MB (up, gate, down)	D	DeepSeek-V3 config: hidden 7168, MoE intermediate 2048 [DeepSeek-AI, 2024]
d_{KV} (MLA KV bytes per token, 8-bit)	$61 \times (512 + 64) =$ 35,136 B/token	D	DeepSeek-V3 MLA: $n_{\text{layers}} = 61$ times (kv_lora_rank 512 plus qk_rope_head_dim 64) cached per token [DeepSeek-AI, 2024]
`DEEPSEEK_V3_CONFIG`	n_{layers}=61, hidden 7168, 256 routed experts, k=8 active, kv_lora_rank 512, qk_rope_head_dim 64, MoE intermediate 2048	R	DeepSeek-V3 technical report architecture table [DeepSeek-AI, 2024]

Table 61: Workload-derived quantities (DeepSeek-V3). Sources: `deepseek_socp_codesign.py` lines 180–183 and 283–363, `timeloop_full.py` lines 137–156.

Constant	Value (units)	St.	Anchor
M_{ref} (reference MAC count)	65,536 MACs	R	TPU-v3 chip: 2 cores $\times$ 128×128 MXU [Jouppi et al., 2021]
BW_{ref} (reference bandwidth)	900 GB/s	A	TPU-v3 4× HBM2 headline bandwidth [Jouppi et al., 2021]
MAC design envelope $[M_{\min}, M_{\max}]$	[16,384, 4,194,304] MACs	A	Spans 1/4 TPU-v3 to 64× TPU-v3 within reticle; a design-space envelope (FAST-style search bounds) [Zhang et al., 2022]
BW design envelope $[\text{BW}_{\min}, \text{BW}_{\max}]$	[100, 8000] GB/s	A	Low end GDDR-class single channel; high end HBM3E multi-stack (roughly 8 TB/s, B200-class)
`cgm_grid_mb` (CGM one-hot menu)	[1, 2, 4, 8, 16, 32, 64, 128, 256] MB	A	Power-of-two sweep bracketing TPU-v3's 32 MB on-chip global memory [Jouppi et al., 2021]
`BW_GRID` (bandwidth one-hot grid)	[225, 450, 675, 900, 1350, 1800, 2400, 3200, 4200, 5400, 6700, 8000] GB/s	A	Enumerated grid from quarter-TPU-v3 HBM2 to HBM3E multi-stack; one-hot branches are solved integrally (exact per branch), and the pin gate prunes entries above 3276.8 GB/s at the 4096-pin cap

Table 62: Solver grids and operating-point references. Sources: `deepseek_socp_codesign.py` lines 220 and 326–334, `deepseek_socp_serving_codesign.py` lines 72–129.

interleaving fraction and the speculative width all enter as constraint families. Each is inert when pinned, so the program contains the simpler formulations exactly.

Manufacturability, enforced rather than checked. Reticle area, TDP, power density, pin count, logic depth, on-chip interconnect and SRAM macro decomposition are rows of the program. A design that violates any of them is infeasible rather than merely flagged, which is why every design in this paper satisfies all six gates by construction.

What remains outside. Accuracy is never linearized: precision entries are admitted only from an accuracy-vetted menu, and the program chooses among them without modeling quality. Draft acceptance, the decode batch and the generation length are stated workload parameters that reshape

Cluster resource allocation [Nnorom Jr. et al., 2026]	CHEETAH co-design
Server (a placement slot)	Operation instance (a node at a position in the graph)
Shape (servers with identical config)	Unique op-shape (ops with identical dims: identical Pareto/roofline data)
Task multi-resource demand (CPU, mem, GPU)	Op demand on the three resources: compute (MACs), traffic (bytes/BW), on-chip capacity (CGM bytes)
Bin / server capacity	CGM capacity per timestep, plus the compute roof and the bandwidth roof
Binary assignment $x \in \{0, 1\}$	Discrete decisions: tile-config y, residency p, precision q, expert-pin g, spec-width K
Convex relaxation $x \in [0, 1]$	The LP relaxation of the one-hots
Anti-affinity as auxiliary capacity rows	Residency, capacity, and McCormick auxiliary constraints
Dual variables (dual per resource-per-shape)	Lagrangian duals $\lambda_{\mathrm{CGM}}, \lambda_{\mathrm{BW}}, \lambda_{\mathrm{MAC}}, \lambda_{\mathrm{HBM}}, \lambda_{\mathrm{cache}}, \lambda_{\mathrm{LD}}$
Net utility = priority $- \sum_r \lambda_r \, \mathrm{demand}_r$	Reduced cost of a candidate column, $\bar{c}_j = c_j - \sum_r \lambda_r \, a_{rj}$
Greedy placement by net utility	Dual-guided rounding of the fractional decisions
Dynamic re-evaluating as load shifts	Re-solve for updated duals as columns / operating point change
Shape aggregation (scale with #shapes)	Shape aggregation (Appendix K; 903 $\rightarrow$ 13 shapes)

Table 63: One-to-one correspondence between convex cluster resource allocation and the CHEETAH co-design program.

coefficients but are not decided. Genuinely discrete or non-convex axes, such as interconnect radix or the draft model itself, remain columns the outer loop proposes and the program admits only if the reduced cost is negative and the manufacturability rows clear.

R DeepSeek-V3 End-to-End Serving Result

Table 64 collects the DeepSeek-V3 mixture-of-experts result in one place: the end-to-end serving speedup with hardware search inside the loop, where the time goes, and which mechanism produces the gain. Every symbol is defined in the table itself so that it can be read without reference to the body.

The serving model. End-to-end serving latency is modelled as three phases, not two:

$$T_{\mathrm{serve}} = \underbrace{T_{\mathrm{prefill}}}_{\text{context, once}} + \frac{G}{A(K)}\Big(\underbrace{T_{\mathrm{draft}}(K)}_{\text{propose } K \text{ tokens}} + \underbrace{T_{\mathrm{pass}}(K)}_{\text{verify them}} \Big), \qquad A(K) = \frac{1 - p^{K+1}}{1 - p},$$

where G is the number of tokens generated and $A(K)$ is the expected number accepted per round, so $G/A(K)$ is the number of draft-then-verify rounds. The draft term is charged rather than assumed free: for an autoregressive draft it grows as $O(K)$, and for the parallel draft used here it is $O(1)$. At this operating point the draft is 0.54% of serving, which is small but not zero, and it is what keeps the selected K finite.

Operating point. Three numbers fix the workload. The *decode batch* $B = 32$ is the number of sequences generated concurrently; it matters because top-k routing makes a larger batch touch more experts, so it changes the traffic rather than merely amortizing it. The *draft acceptance* $p = 0.75$ is the probability that a single token proposed by the draft model is accepted by the target model's verification; it is a property of the draft/target pair, so we state and sweep it rather than measure it. The $1\,GB$ *diffusion draft* is the small companion model that proposes candidate tokens: 1 GB of weights, and "diffusion" meaning it proposes all K tokens in one parallel pass rather than K sequential ones, so its cost does not grow with the speculative width.

Two questions the selected chip raises. The design takes almost no on-chip memory and almost all the bandwidth the package permits, and both follow from the same fact. In decode the traffic is

388 GB per step of mixture-of-experts weights, which is four orders of magnitude larger than any on-chip buffer could hold, so additional SRAM has nothing useful to retain: the weights cannot be cached and the MLA-compressed KV cache is only 4.3 GB. On-chip memory therefore collapses to the smallest grid point that keeps the schedule feasible, and the area saved is spent elsewhere. Bandwidth is the opposite case. It divides the dominant term directly, so the program buys as much as the package allows and stops at 3200 GB/s because the next grid point would need 5250 signal pins against a 4096-pin limit. The resulting design sits at 4000 pins, which is 97.7% of that limit. This is the clearest single illustration of why the decisions that matter here reduce or amortize traffic rather than add storage.

What is solved, and by what. The formulation is a linear program: bandwidth is a finite menu, so its reciprocal is an exact one-hot expression whose LP relaxation is a lower bound, and the compute reciprocal is bracketed by tangent and chord rows on a geometric grid (relative half-gap 10^{-3}), so a single LP gives the certified lower bound and a second LP of identical sparsity, the chord LP, gives an achievable design once every menu selector is pinned to an integral value and the pinned solution passes the exact-lift check (with relaxed selectors the chord LP is neither bound). Those LPs are solved with HiGHS, which returns a certified vertex and lets us check every reported design against the constraint set independently. The Variable-Metric-One-Armed-Rescue solver (Section 6) is intended for the full-resolution instances with per-operation residency ($\sim$8.7M rows); on the shape-aggregated instances, where both can run, it does not meet its own convergence test within its iteration budget (Appendix M), so every bound reported here is HiGHS's (**[TODO: measured: CORGI-vs-HiGHS agreement per regime]**).

The decomposition explains itself. Decode dominates serving, and decode is almost entirely mixture-of-experts weight traffic, so the mechanisms that reduce or amortize that traffic are the ones that matter: sizing bandwidth and on-chip memory first, then reducing the bytes through precision and sparsity, then amortizing each remaining weight load over more accepted tokens through speculation. Tiling and fusion contribute by keeping the prefill phase on its compute floor, but they cannot move the decode wall, which is why a formulation that decides tiling alone plateaus and one that also sizes the hardware and decides the serving decisions does not.

S A Worked Example: Hardware Coefficients as Functions, Not Constants

The central modelling decision in this paper is that hardware quantities enter the program as functions of decision variables rather than as fixed coefficients. It is worth seeing exactly what that changes on a single concrete quantity.

The quantity. Every decode step must read the routed mixture-of-experts weights from DRAM. Timeloop measures 388.5 GB per step at 8-bit precision. That byte count is a genuine constant: it is a property of the model and its routing, not of any chip. The *time* it takes is bytes divided by bandwidth, and that is where the two formulations part company.

As a constant, in a sequential pipeline. Fix the chip first at the TPU-v3 bandwidth of 900 GB/s. The coefficient is now a number,

$$t_{\text{decode}} \;=\; \frac{388.5\ \text{GB}}{900\ \text{GB/s}} \;=\; 0.432\ \text{s},$$

and 0.432 enters the objective as a literal. Asking what a different bandwidth would buy requires discarding the program and building another one.

As a function, in CHEETAH-V1. The corresponding row is

$$\tau \;\geq\; \underbrace{T^{\max}(c)}_{\text{measured bytes}} \;\times\; \underbrace{s^{W}}_{\text{precision variable}} \;\times\; \underbrace{u}_{\text{bandwidth variable}} \quad ,$$

where s^{W} is set by the precision one-hot (1.0 at W8, 0.5 at W4) and $u = \text{BW}_{\text{ref}}/\text{BW}$. Neither factor is a number until the solver chooses one.

Why this matters: the two decisions multiply. Table 65 evaluates the same row at the four combinations of these two decisions. The best cell is $7.11\times$ faster than the baseline, and that factor decomposes as $2\times$ from precision and $3.56\times$ from bandwidth: the decisions compose rather than compete.

The consequence is stronger than a search-quality argument. A pipeline that fixes bandwidth at 900 GB/s before precision is considered can only ever reach the left column, so its best attainable answer is 0.216 s. The 0.061 s cell is not merely missed by a weak search; it is *unreachable*, because by the time precision is being chosen the bandwidth has already become a constant.

What it costs. Reading the row once more, $s^W \times u$ is a product of two decision variables. It is bilinear and therefore nonconvex, which is exactly why the formulation carries a hull disaggregation at that position, and why the product is written over the bandwidth menu, where it is exact at integral selectors. The multiplicative gain in Table 65 and the solver machinery of Section 6 are the same fact viewed from opposite sides: expressiveness bought with disjunctions, and disjunctions paid for with hull copies, a menu enumeration, and a solver that tolerates the resulting size and conditioning.

This is not a hypothetical. It is the structure that produced the reported design, which buys 3200 GB/s, the most the pin-count row permits, and selects W4 with a 4-bit KV cache: both halves of the bottom-right cell.

T Where Each Workload Family Slows Down, and the Local Sacrifice That Wins

The formulation earns its keep only where a *locally* worse decision is *globally* better, because that is precisely the trade a sequential pipeline cannot represent: a mapper that has already committed to the per-operation optimum has priced the local choice and thrown away the coupling that would have paid for it. Table 66 names, for each workload family in our suite, where inference actually slows down and which locally costly choice the joint program is able to buy in exchange. Read it as the qualitative statement of which term in the objective the coupling is expected to move; the measured per-family decompositions are in Appendices E and I.

Two properties of the table are worth making explicit. First, every right-hand entry is a decision the V1 program can express as a column and a sequential pipeline cannot, because in each case the locally costly choice is only justified by a constraint that binds at a *different* operation or a *later* timestep: the tile that looks slow in isolation is paying for a residency that a downstream operation will consume. Second, the families differ in *which* resource the sacrifice buys, on-chip capacity for EfficientNet and SD1.5, memory bandwidth for the Llamas and DeepSeek-V3, which is why the same formulation produces qualitatively different chips across the suite rather than one design rescaled.

U A Minimal Certified Example of Hardware/Software Non-Separability

Table 66 asserts that a locally costly choice can win globally. This appendix gives the smallest instance we know of in which that claim is not only true but *certified*, and in which the failure mode it warns against, alternating hardware-only and software-only optimization, can be exhibited exactly. Every quantity is a synthetic normalized unit; this is a teaching model with a closed form, not a chip, a workload, or a measurement.

Setting. Two serial invocations of one kernel share an immutable weight. Two functionally equivalent implementations are admitted: a *fast* kernel (compute 1.0, scratch 40) and a *compact* kernel (compute 1.2, scratch 20). The weight occupies 20 SRAM units if retained between invocations; its first-use load costs 10 traffic units and a second-use miss costs 10 more. A chip of total area 100 is divided continuously among compute m, bandwidth b, and SRAM S, with S at least 40 so that both kernels fit every admitted chip. For a branch with compute demand C, traffic Q, and required SRAM S, the latency $C/m + Q/b$ subject to $m + b = 100 - S$ has the closed-form optimum

$$T^\star = \frac{(\sqrt{C} + \sqrt{Q})^2}{100 - S}, \qquad m^\star = (100 - S)\,\frac{\sqrt{C}}{\sqrt{C} + \sqrt{Q}}. \tag{32}$$

The four moves. Eight branches (two kernels per invocation, retain or not) are all covered by Eq. (32). Table 67 records the four moves an optimizer can make from the locally natural starting point.

Both single-axis moves are stuck: the fast-kernel chip is already optimal for the fast kernel, and on that chip the compact kernel is strictly worse. Only the joint move improves, by $1.415\times$, and the mechanism is exactly the one Table 66 describes for EfficientNet and SD1.5: the compact kernel needs 20 fewer SRAM units, which lets the weight be retained *within* the 40-unit floor, and the released area is re-spent on compute and bandwidth (m from 12.36 to 19.73, b from 27.64 to 40.27). The slower kernel is worth choosing only after the chip has been resized to exploit it, and the chip is worth resizing only if the slower kernel will be chosen. Neither decision is justified by the other's status quo, which is why a sequential pipeline cannot reach the point.

The result is robust, not a knife edge. The 40-unit SRAM floor coincides with the fast kernel's scratch, so the compact kernel's retention is exactly free at that value; one might suspect the effect is tuned. It is not. Sweeping the floor over its admissible range, the joint move wins throughout:

SRAM floor	40 (shipped)	45	50	30–35
Best fast-kernel design	0.5236	0.5236	0.5236	0.5236
Joint (compact + retain)	0.3700	0.4036	0.4440	0.3700
Ratio	1.415	1.297	1.179	1.415

Below 40 the fast kernel no longer fits in isolation, so the comparator changes meaning; above it the advantage shrinks smoothly and never vanishes.

The bound is a certificate, not a restatement. Each branch is also solved as an LP in which C/m and Q/b are replaced by their supporting planes at $(m^\star, b^\star)$, and HiGHS returns exactly $T^\star$ for every branch. Because a tangent plane of a convex function is a global underestimator, that LP value is a valid lower bound on the branch optimum for *any* linearization point; it equals the replayed latency at the linearization point only when that point is optimal. We verified this has teeth by perturbing the linearization point: at $\pm 1\%$ the LP bound falls to 0.3664 against a replay of 0.3700, at $\pm 10\%$ to 0.336 against 0.372, and equality is recovered only at $m^\star$. An LP whose optimum is attained at its own linearization point is the first-order optimality certificate for a convex program, so the eight branch optima are certified rather than assumed, and the $1.415\times$ is a gap between two certified values.

What it does and does not show. It shows that the objective is non-separable in hardware and software even in the simplest case with continuous ideal resources, and that this non-separability defeats coordinate-wise optimization. It does not model tiling, time-indexed residency, array parallelism limits, or any physical constraint, and its $1.415\times$ carries no information about DeepSeek-V3 or any chip; the measured per-family decompositions are in Appendices E and I. Its value is that the certified argument is short enough to check by hand.

Table 64: **DeepSeek-V3 serving co-design with hardware search inside the loop.** *Operating point.* Decode batch $B{=}32$ concurrent sequences; draft acceptance $p{=}0.75$, the probability one proposed token survives verification; a 1 GB parallel (diffusion) draft, which proposes all K tokens in one pass so its cost does not grow with K; context 4096; 512 generated tokens. Speedups are against a fixed TPU-v3 baseline at the same batch. Every row is an achievable design: a concrete configuration, re-solved and checked against the constraint set, satisfying all six manufacturability rows. *Units.* These vary by block and are stated in each block header; speedups are multiples, so $44.85\times$ means 44.85 times faster, not 44.85%. *The two speedups answer different questions.* The $44.85\times$ figure is the best design reachable *given the option menus the program was handed* (a fixed precision set, a fixed set of memory technologies, $K \in \{2, 8, 16, 32\}$, and one draft model). The $73.00\times$ figure is what the outer loop reaches when allowed to *add entries to those menus*, since an optimizer can only select among options it has been given. An entry is any option the program could have chosen had it been offered: hardware entries include a memory technology with a different bandwidth ceiling, area per GB/s and energy per bit, an omitted bandwidth grid point, another on-chip memory size, or a different array aspect ratio; software entries include a new precision format, a sparsity pattern, a speculative width outside the menu, or a different draft model, which is a paired entry carrying both its size and the acceptance it achieves. Two were admitted here, one of each kind: a 4 GB adapted draft assumed to reach $p{=}0.90$, and HBM4-class memory. The program admits an entry only if its reduced cost is negative and the manufacturability rows still clear. *Caveats on the larger number.* It is conditional on the assumed $p{=}0.90$, which we state rather than measure, and on HBM4-class memory being available. Composition matters: the memory technology is *worse* alone ($49.28\times$) than the draft entry and pays only in combination, which is why the loop re-solves after each acceptance rather than ranking proposals once.

Quantity	Meaning	Value (unit varies by block)
End-to-end serving speedup *(values are × faster than TPU-v3, not percentages)*		
Best over K, fixed menus	best design over the option set the program was given; K is the speculative width (tokens proposed per verification pass), chosen from $\{2, 8, 16, 32\}$	$\mathbf{44.85}\times$
+ outer loop, new options	the loop adds two entries the menus lacked: a 4 GB adapted draft assumed to reach $p{=}0.90$ (software) and HBM4-class memory (hardware)	$\mathbf{73.00}\times$
Relaxed bound	no design in this model can beat it	$57.26\times$
Where the serving time goes *(values are % of total serving latency)*		
T_{prefill}	one compute-bound pass over the 4096-token context	42.10%
$(G/A(K))\,T_{\text{draft}}$	129 draft rounds proposing K tokens each	0.54%
$(G/A(K))\,T_{\text{pass}}$	129 memory-bound verification passes	57.35%
$A(K)$	expected tokens accepted per pass at $K^\star{=}16$, $p{=}0.75$	3.97
$G/A(K)$	number of draft-then-verify rounds for $G = 512$ tokens	129
What produces the speedup *(values are cumulative × speedup, each an achievable design)*		
Hardware sizing	MACs, BW, on-chip memory as variables	$4.73\times$
+ byte reduction	precision q_ρ, 2:4 sparsity, memory technology	$16.49\times$
+ speculation	speculative width $K^\star = 16$	$\mathbf{44.85}\times$
Why: where the decode bytes go *(values are % of decode DRAM traffic)*		
Δ^{moe}	mixture-of-experts weight reload, 388.5 GB/step	98.6%
Δ^{KV}	MLA-compressed KV read, 4.29 GB/step	1.1%
Δ^{act}	activations, 1.15 GB/step	0.3%
The chip the program selected *(values are physical quantities)*		
MACs	INT8 multiply–accumulate units ($24.4\times$ TPU-v3)	1,598,953
BW	DRAM bandwidth	3200 GB/s
C_{GM}	on-chip global memory	1.0 MiB
n_{PE}	processing elements, $= \lceil \text{MACs}/(s_x s_y) \rceil$	98
$s_x \times s_y$	systolic array dimensions per PE (rows × columns), giving 16,384 MACs per processing element	128×128
DRAM channels	at ∼100 GB/s per channel	32
Manufacturability: six of six rows satisfied *(achieved value vs. cap)*		
Pin count	BW at 0.8 GB/s per HBM3 pin (cap 4096)	4000
Reticle area	die area (cap 600 mm^2)	497.3 mm^2
Power density	dynamic 320 W over the die (cap 2.0)	0.64 W/mm^2
Logic depth	gate levels at 1 GHz (cap 35)	25
Crossbar	PE-to-bank routing cells (cap $5{\times}10^6$)	1568
SRAM macros	64 KiB banks (cap 4096)	16

Table 65: Decode weight-streaming time for one step of DeepSeek-V3, evaluated at each combination of the precision and bandwidth decisions. A pipeline that fixes bandwidth before considering precision is confined to the left column.

		BW = 900 GB/s	BW = 3200 GB/s
W8	(388.5 GB)	0.432 s (baseline)	0.121 s
W4	(194.3 GB)	0.216 s	**0.061 s**

Table 66: Per-family inference bottlenecks and the locally costly choice that can win globally. Each row is an instance of the same structural claim: the per-operation optimum and the end-to-end optimum disagree, and the size of the disagreement is what the joint program recovers. The EfficientNet-B7 figure is FAST's own profile on TPU-v3 [Zhang et al., 2022], where depthwise operations account for 5% of FLOPs but 65.3% of runtime, which is the clearest published case of a bottleneck that FLOP-weighted reasoning cannot see.

Family	**Where inference slows down**	**Locally costly choice that can win globally**
Llama-1 13B/65B, Llama-3 70B	Prefill stresses projection/FFN GEMMs and attention. Small-batch decode often stresses weight bandwidth; longer contexts increase KV traffic and attention costs.	A smaller attention or FFN tile preserves weights or KV for subsequent operations. Splitting attention across the KV history adds reduction work but can improve parallelism.
EfficientNet B1–B7	Depthwise utilization, expanded activations, SE reductions, and layout/halo traffic. FAST's historical B7 profile found depthwise operations were only 5% of FLOPs but 65.3% of runtime on TPU-v3.	A slower compact tile preserves an expanded feature or residual, avoiding a spill and reload.
SD1.5 UNet	Convolutions, attention, and encoder skips that remain live until much later decoder operations. Complete image generation repeats the denoiser.	Smaller intervening tiles preserve a valuable skip; legal recomputation can release memory for something more expensive to reload.
DeepSeek-V3 MoE	Expert-weight movement, small or uneven expert GEMMs, MLA attention, and, when distributed, dispatch/combine communication.	Compact mappings preserve experts or KV; prefetching and grouping incur costs now to avoid larger later stalls.

Table 67: Alternating optimization stalls; the joint move does not. Latencies in normalized units from Eq. (32). The fast kernel is the per-kernel latency optimum on any chip with adequate SRAM, which is what a per-operation mapper selects; "locally fastest" is that mapper's choice, not a claim that it is globally wise.

Move	**Kernel**	**Chip**	**Latency**
Chip optimized for the locally fastest kernel	fast	optimal for fast	0.5236
Change software only, on that chip	compact	unchanged	0.5560
Re-optimize hardware only, keeping the fast kernel	fast	re-optimized	0.5236
Change hardware and software jointly	compact	re-optimized	**0.3700**